

VI

Wallet Guide

The Zcash wallet layer: keys, transactions, and light-client synchronisation

Marek
m@rek.onl

Abstract

This volume of *The Zcash Arboretum* states the protocols that a Zcash wallet runs: the derivation of its keys and addresses from one seed; the discovery of its notes from the chain through an untrusted light-client server, and what that server learns; and the construction, authorisation and submission of its payments. Each protocol is stated as a definition, construction or rule, and its correctness, security or privacy guarantee as a lemma, proposition or theorem under named assumptions. The subject is the wallet of a user of the Orchard protocol who holds funds in the Ironwood pool, the sealed Orchard pool, the Sapling pool and the transparent pool; the light-client service is an abstract query and response interface, and no implementation is described. The volume assumes the *Math Guide*, the *Crypto Guide*, the *Halo 2 Guide* and the *Consensus Guide*, and cites the *Ironwood Guide* for every construction of the shielded protocol. It is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals are authoritative.

Contents

1	Introduction	6
1.1	Scope and status	6
1.2	Parties, pools, and classifications	6
2	Hierarchical key derivation (ZIP 32)	8
2.1	Wallet seeds	8
2.2	Hardened-only derivation	9
2.3	The standard account path	11
2.4	Orchard key components of an account	13
2.5	Internal keys: change and auto-shielding	15
2.6	Valid account keys	19
2.7	The PRFexpand lead-byte registry	20
3	Addresses and viewing keys (ZIP 316)	22
3.1	Diversifier indices	22
3.2	Raw encodings	26
3.3	Unified addresses	27
3.4	String encoding: F4Jumble and Bech32m	29
3.5	Unified viewing keys	33
3.6	Deriving unified addresses	35
3.7	Revision 2 containers	37
3.8	Recipient addresses	39
4	Compact blocks (ZIP 307)	39
4.1	The compact block and the compact transaction	40
4.2	Compact outputs, spends, and Actions	41
4.3	Compact trial decryption	43
4.4	Note positions and tree sizes	45
5	The light-client service	47
5.1	Architecture and authority	47
5.2	The query interface	50
6	Note discovery	53
6.1	The account birthday	53
6.2	Scanning keys	54
6.3	Receipt and spend detection	57
6.4	Scan order	58
6.5	The cost of scanning	62

7	Commitment-tree synchronisation	63
7.1	Witness maintenance by linear scanning	63
7.2	Subtree roots and the cap	64
7.3	Frontiers as scan seeds	66
7.4	The information a wallet retains	67
7.5	Witness readiness	69
7.6	Reorganisation detection and recovery	71
8	Leakage to the light-client server	75
8.1	Payment-detection privacy	75
8.2	Key independence of queries	77
8.3	Leakage of range queries	80
8.4	Leakage of transaction queries	82
8.5	The submission channel	83
8.6	Summary of server observables	84
9	Anchors, confirmations, and expiry	85
9.1	The confirmation policy	85
9.2	Target and anchor heights	87
9.3	Expiry (ZIP 203)	88
9.4	The transaction record and locked notes	89
9.5	Spendable notes	91
10	Fees (ZIP 317)	93
10.1	The conventional fee	93
10.2	The fee on padded counts	95
10.3	Relay, mempool, and block template	97
11	Transaction construction	98
11.1	The transaction plan	98
11.2	Payments to pools	101
11.3	TEX transfers and shielding transactions	103
11.4	Input selection	104
11.5	Change	106
11.6	Outgoing viewing keys	108
11.7	Padding, dummies, and shuffling	109
11.8	Authorisation of a transaction	111
12	Partially created transactions (PCZT)	114
12.1	The PCZT and its requirements	114
12.2	Roles and the modification state	116
12.3	Fields	117
12.4	Encoding	121
12.5	PCZT v2: deferred anchors and the Ironwood bundle	123

12.6 Construction and IO finalisation	124
12.7 Signing	126
12.8 Proving	129
12.9 Combining, redaction, and privacy	130
12.10 Spend finalisation and extraction	132
13 The transaction lifecycle	133
13.1 Submission and the transaction identifier	133
13.2 Status tracking and terminal states	136
14 Payment requests (ZIP 321)	139
14.1 The URI grammar	139
14.2 Parameter semantics	142

1 Introduction

This section fixes the subject of the volume, its status and the vocabulary that every later section uses.

1.1 Scope and status

This volume states the protocols by which a wallet, in the sense of “Parties, pools, and classifications” (§1.2), performs three tasks. It derives and holds keys and addresses: hierarchical key derivation (ZIP 32), and unified addresses and unified viewing keys (ZIP 316). It discovers and tracks its notes: the compact-block format (ZIP 307) as an abstract message format, trial decryption, nullifier tracking, the retention of the note commitment tree information that its witnesses need, and what an untrusted light-client server learns. It pays: anchors and confirmation depths (ZIP 315), expiry (ZIP 203), the conventional fee (ZIP 317), transaction construction (note selection, change, dummy Actions), partially created transactions (ZIP 374), the transaction lifecycle from submission to a terminal state, and payment requests (ZIP 321).

The subject is the wallet of a user of the Orchard protocol who holds funds in the Ironwood pool, the sealed Orchard pool, the Sapling pool and the transparent pool. Sapling constructions and transparent scripts are cited to the protocol specification and to the BIPs where a wallet protocol needs them, and are never constructed; constructions of the shielded protocol are cited to the *Ironwood Guide* by section title, and are never re-derived. The light-client service is an abstract query and response interface, and tree storage is the information that a wallet retains; no implementation is described. The volume is non-normative: the protocol specification and the ZIPs are authoritative.

The volume describes the protocols as specified for NU7, whose deployment ZIP, ZIP 259, has status Draft.

ZIP 307 specifies the compact format for the Sapling pool only: a compact block of height, block hash and transactions, and a compact transaction of index, transaction identifier, and compact Sapling spends and outputs. The parent hash, the tree sizes, the compact Actions of both Orchard-protocol pools, the transparent part and pool selection are designed but unspecified in the sense of the *Ironwood Guide*, Definition “Epistemic classes” (“Scope and status”), and the volume states them abstractly in “Compact blocks (ZIP 307)” (§4).

Rules of the Ironwood pool and of NU6.3 are cited to the NU6.3 edition of the protocol specification, labelled “NU6.3 proposal”, whose section numbers equal those of the default edition; the text on use_ysk for protocol specification, §“Orchard Key Components”, is in no edition and is cited to ZIP 2005, “Changes to the Protocol Specification”, directly.

Table 1 gives the status, and only the status, of each ZIP that the volume cites. Every later citation of these ZIPs relies on the table for status. ZIP 258 activates ZIP 2005 with NU6.3 (ZIP 258, “ZIP 2005 activation”).

1.2 Parties, pools, and classifications

A *wallet* is a party that holds a seed and the keys derived from it, and no copy of the chain. A *light-client server* is a party that answers the queries of a wallet from its own view of the best chain, the

Table 1. Status of the ZIPs cited, as given in the preamble of each ZIP.

ZIP	Title	Status
32	Shielded Hierarchical Deterministic Wallets	Final
48	Transparent Multisig Wallets	Draft
173	Bech32 Format	Final
203	Transaction Expiry	Final
211	Disabling Addition of New Value to the Sprout Chain Value Pool	Final
212	Allow Recipient to Derive Ephemeral Secret from Note Plaintext	Final
218	25-second Block Target Spacing	Draft
225	Version 5 Transaction Format	Final
227	Issuance of Zcash Shielded Assets	Draft
229	Version 6 Transaction Format	Draft
231	Memo Bundles	Draft
235	Remove 60% of Transaction Fees From Circulation	Draft
244	Transaction Identifier Non-Malleability	Final
248	Extensible Transaction Format	Draft
256	Deployment of Consensus Bug Fixes Between NU6.1 and NU6.2	Final
258	Deployment of the NU6.3 Network Upgrade	Draft
259	Deployment of the NU7 Network Upgrade	Draft
302	Standardized Memo Field Format	Draft
307	Light Client Protocol for Payment Detection	Draft
312	FROST for Spend Authorization Multisignatures	Draft
314	Privacy upgrades to the Zcash light client protocol	Reserved
315	Best Practices for Wallet Implementations	Draft
316	Unified Addresses and Unified Viewing Keys	Revision 0 Active; Revision 1 Withdrawn; Revision 2 Draft
317	Proportional Transfer Fee Mechanism	Revision 0 Active; Revisions 1 and 2 Draft
318	Orchard to Ironwood Migration	Draft
320	Defining an Address Type to which funds can only be sent from Transparent Addresses	Active
321	Payment Request URIs	Active
326	NU6.3 Consequences for Wallets	Draft
374	Partially Created Zcash Transaction Format	Revision 0 Draft
401	Addressing Mempool Denial-of-Service	Active
2003	Disallow version 4 transactions	Draft
2005	Ironwood Quantum Recoverability	Proposed

best valid block chain of the *Consensus Guide*, “Work, and the best chain” (Definition “Best valid block chain”). The *network* is the set of the nodes that relay transactions and blocks and of the miners that produce blocks. The trust placed in the server is not fixed here; it is an assumption of “Architecture and authority” (§5.1).

A wallet handles funds in four value pools. The first is the *transparent pool*. The second is the *Sapling pool* (protocol specification, §“Chain Value Pool Balances”), whose constructions are cited, never constructed. The third and fourth are the *Ironwood pool* and the sealed *Orchard pool*, the two value pools of the Orchard protocol (*Ironwood Guide*, “The Orchard protocol and its two pools”, Definitions “Orchard protocol” and “Orchard pool, Ironwood pool”; ZIP 229, Terminology); each has its own note commitment tree, its own nullifier set and its own chain value pool balance. The rules that seal the Orchard pool are cited there (ZIP 258, “Consensus rules from NU6.3 activation”), and are stated for wallets in “Payments to pools” (§11.2).

Heights are counted as in the *Ironwood Guide*, “Scope and status” (Definition “Block chain, transaction, consensus rules”), the genesis block having height 0. The *tip* T is the height of the last block of the wallet’s view of the best chain. Every other height (the target height, the anchor height, the expiry height) is defined where it is first used.

Definition 1.1 (Classes of claim). The classes *specified* and *designed but unspecified* are those of the *Ironwood Guide*, Definition “Epistemic classes” (“Scope and status”), applied here to wallet rules as well as to claims about the deployed protocol. A wallet rule is *specified* when normative text of a ZIP or of the protocol specification fixes it, the status of the ZIP being that of Table 1. A wallet rule is *designed but unspecified* when wallets are designed to follow it and no normative text fixes it. This volume adds a third class: a question is an *open problem* when neither a ZIP nor a stated design settles it.

Every design-stage statement of the volume carries one class of Definition 1.1, and every deployed rule is cited at its use to the section of the protocol specification or of the ZIP that fixes it. A specified rule keeps the normative level of its source (MUST, SHOULD, MAY).

2 Hierarchical key derivation (ZIP 32)

This section derives every key of a wallet from one seed. It fixes the seed and its fingerprint, the hardened-only derivation of Orchard spending keys and the standard account path, and the passage from an account spending key to the key components that the *Ironwood Guide* constructs from a spending key, including the alternative branch of ZIP 2005. It then constructs the internal keys of an account and proves the separations between the two scopes, bounds the probability that an account key is invalid, and closes with the domains of key derivation.

2.1 Wallet seeds

Definition 2.1 (Wallet seed). A *wallet seed* is a byte string S with $32 \leq \text{length}(S) \leq 252$, where $\text{length}(S)$ is its length in bytes (MUST). The method that generates S MUST give it at least 256 bits of entropy; the requirement extends to any mnemonic or input randomness from which

S is obtained (ZIP 32, “Specification: Wallet seeds”). Every key of every account of the wallet is derived from S : the account spending key in §2.3, and what recovery needs beyond S in §2.4.

Every key of every account is a deterministic function of S , so a seed space of lower entropy is searched once for all accounts, and across many wallets as a multi-target search that matches the first derived addresses or viewing keys against a set of observed ones (ZIP 32, “Rationale for the 256-bit entropy requirement”).

Construction 2.2 (Seed fingerprint). For a wallet seed S , the *seed fingerprint* is

$$\text{SeedFP}(S) := \text{BLAKE2b-256}(\text{Zcash_HD_Seed_FP}, [\text{length}(S)] \parallel S),$$

unkeyed BLAKE2b with a 32-byte output and the 16-byte personalisation shown, over the single length byte of S followed by S ; the length fits one byte because $\text{length}(S) \leq 252$. It MAY be used to identify the seed of a hierarchical deterministic wallet. Its string form is Bech32m (BIP 350) with human-readable part `zip32seedfp` and the 32 fingerprint bytes as data; no short tag is defined (ZIP 32, “Seed Fingerprints”). A partially created transaction carries the fingerprint together with a derivation path, so that a signing party locates its key (§12.3).

A partially created transaction relies on collision resistance of BLAKE2b-256 under this personalisation (*Crypto Guide*, §“Security notions: preimage, second-preimage, and collision resistance”, Definition “Collision resistance”); Lemma 2.5 additionally models it as a random oracle, so that the fingerprint reveals nothing of the keys. The fingerprint of a full viewing key is constructed with the key components, in §2.4.

2.2 Hardened-only derivation

ZIP 32 instantiates its hardened-only derivation for Orchard with the master-key domain `ZcashIP320rchaind`, a 16-byte BLAKE2b personalisation that SHOULD be disjoint from every other BLAKE2b domain separator of Zcash, and the child-key domain `0x81`, a lead byte of PRFexpand (ZIP 32, “Specification: Hardened-only key derivation”, “Instantiation”, and “Specification: Orchard key derivation”). An *Orchard extended spending key* is a pair (sk, c) of 32-byte strings, where sk is a spending key in the sense of the *Ironwood Guide*, §“The spending key and the spend-side secrets”, and c is the *chain code* (ZIP 32, “Orchard extended keys”). Only hardened derivation is defined for Orchard: child derivation fails at every index $i < 2^{31}$. ZIP 32 defines no extended full viewing key for Orchard; a full viewing key has no child-derivation capability for which a chain code would serve. For $0 \leq i < 2^{31}$ write $i' := i + 2^{31}$.

Construction 2.3 (Orchard master key generation). For a wallet seed S (Definition 2.1), let

$$I := \text{BLAKE2b-512}(\text{ZcashIP320rchaind}, S),$$

unkeyed BLAKE2b with a 64-byte output, and let $sk_m := I[0..32)$ and $c_m := I[32..64)$, the first and the last 32 bytes. The *master extended spending key* is $m_{\text{Orchard}} := (sk_m, c_m) =: \text{MKG}(S)$ (ZIP 32, “Hardened-only master key generation” and “Orchard master key generation”).

Construction 2.4 (Orchard child key derivation). For a parent extended spending key $(\text{sk}_{\text{par}}, \text{c}_{\text{par}})$ and an index $i \in \{0, \dots, 2^{32} - 1\}$: if $i < 2^{31}$, derivation fails; otherwise

$$I := \text{PRFexpand}_{\text{c}_{\text{par}}} ([0x81] \parallel \text{sk}_{\text{par}} \parallel \text{l2LEOSP}_{32}(i)),$$

a 64-byte string, and $\text{CKD}((\text{sk}_{\text{par}}, \text{c}_{\text{par}}), i) := (I[0..32], I[32..64])$. Here $\text{l2LEOSP}_{32}(i)$ is the 4-byte little-endian encoding of i , and $\text{PRFexpand}_k(t) = \text{BLAKE2b-512}(\text{Zcash_ExpandSeed}, k \parallel t)$ is the *Ironwood Guide*, §“Pseudorandom functions and field reductions”, Construction “Expansion function” (ZIP 32, “Hardened-only child key derivation” and “Orchard child key derivation”). The parent chain code is the key of the expansion; the parent spending key is message data.

Lemma 2.5 (Pseudorandomness of derived spending keys). *Assume the following.*

- (i) *BLAKE2b-512 under the personalisation ZcashIP320rchain and BLAKE2b-256 under the personalisation Zcash_HD_Seed_FP are modelled as random oracles (Crypto Guide, §“The random oracle model”, Definition “Random oracle”), independent of each other and of every other personalised use of BLAKE2b (Crypto Guide, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”), and of every other derivation from S that is modelled as a random oracle.*
- (ii) *The seed S has min-entropy at least 256 bits, the reading of the entropy requirement of Definition 2.1 that the lemma uses.*
- (iii) *The Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”) holds.*

Let N be a fixed finite set of distinct nodes of the Orchard derivation tree rooted at m_{Orchard} , each reached by Construction 2.4 along hardened indices, and let T be the subtree spanned by the paths from the root to the nodes of N . Then the spending keys of the nodes of N , the chain codes of those nodes of N that are not a proper ancestor of another node of N , and $\text{SeedFP}(S)$ are jointly computationally indistinguishable from independent uniform 32-byte strings. For an adversary making q random-oracle queries, the distinguishing advantage is at most $q \cdot 2^{-256}$ plus one PRF advantage for each node of T whose chain code keys a derivation in T ; there are at most as many such nodes as T has edges.

Proof. A hybrid argument (Crypto Guide, §“The hybrid argument”, Lemma “Hybrid lemma”). Game 0 is the real experiment. Game 1 answers every oracle query whose input is determined by S (the master query of Construction 2.3, the input of SeedFP and the inputs of the other modelled derivations) by fresh uniform strings, independent of S . The two games differ only if the adversary itself queries such an input. In Game 1 its view is independent of S until that happens, so each of its q queries does so with probability at most 2^{-256} by hypothesis (ii), and the games differ by at most $q \cdot 2^{-256}$. In Game 1 the pair $(\text{sk}_m, \text{c}_m)$ and $\text{SeedFP}(S)$ are independent uniform strings.

Then visit the nodes of T that have a child in T , each parent before its children. At such a node v the chain code c_v is, by the earlier hops, a uniform string that is used only as a PRFexpand key

and is not released, because v is a proper ancestor of a node of N . The hop replaces PRFexpand_{c_v} by a uniformly random function. The children of v in T have distinct indices, hence distinct inputs $[0x81] \parallel \text{sk}_v \parallel \text{I2LEOSP}_{32}(i)$, so their 64-byte outputs become fresh uniform strings (*Ironwood Guide*, §“Pseudorandom functions and field reductions”, Lemma “Independence of domain-separated expansions”, first conclusion, applied to a uniform key; its statistical-distance clause is not used). The hop costs the PRF advantage of a reduction that samples every other node itself and forwards the queries of v to its oracle. In the last game every released value is a fresh uniform string; the chain codes of proper ancestors, which are not fresh relative to their children, are exactly those the lemma does not release. $\square$

Remark 2.6 (The spending key as a leaf capability). In Construction 2.4 the parent chain code keys the expansion and the parent spending key is message data, so deriving children requires the pair (sk, c) . A 32-byte spending key alone derives no child: by Lemma 2.5, without c the children are pseudorandom (ZIP 32, “Orchard child key derivation”). A wallet that keeps sk_{acct} and not c_{acct} keeps no power to derive keys below the account.

2.3 The standard account path

Definition 2.7 (Account spending key). For a wallet seed S , a coin type t and an account index $a \in \{0, \dots, 2^{31} - 1\}$,

$$(\text{sk}_{\text{acct}}, c_{\text{acct}}) := \text{CKD}(\text{CKD}(\text{CKD}(\text{MKG}(S), 32'), t'), a'),$$

the path $m_{\text{Orchard}} / \textit{purpose}' / \textit{coin_type}' / \textit{account}'$. The purpose index is $32' = 0x80000020$, following BIP 43; the coin type is the SLIP 44 value, 133 on Mainnet and 1 on Testnet, every test network sharing the value 1; accounts are numbered sequentially from 0, as in BIP 44. All three levels are hardened. The *account spending key* is sk_{acct} ; no construction of this volume uses c_{acct} (Remark 2.6). A wallet implementing Orchard ZIP 32 derivation **MUST** support this path for every account in $\{0, \dots, 2^{31} - 1\}$, **MUST** support generating the default payment address, at diversifier index 0, of every account it supports, and **MAY** generate further diversified addresses of an account (ZIP 32, “Key path levels” and “Orchard key path”).

The path ends at the account. Addresses are not path children: an account has 2^{88} diversified addresses per scope, the images of the diversifier indices (*Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”). There is no change level. A shielded output does not show its address on chain, so returning change to the originating address is indistinguishable from using a change address; the wallet separates change from received funds by the internal scope below the account (§2.5; ZIP 32, “Key path levels”). Figure 1 shows the derivation from the seed to the addresses of both scopes.

Example 2.8 (The account path of a fixed seed). Let S be the 32 bytes $0x00, 0x01, \dots, 0x1f$, on Mainnet (coin type 133), account 0. The pairs (sk, c) along the path, in hexadecimal, are:

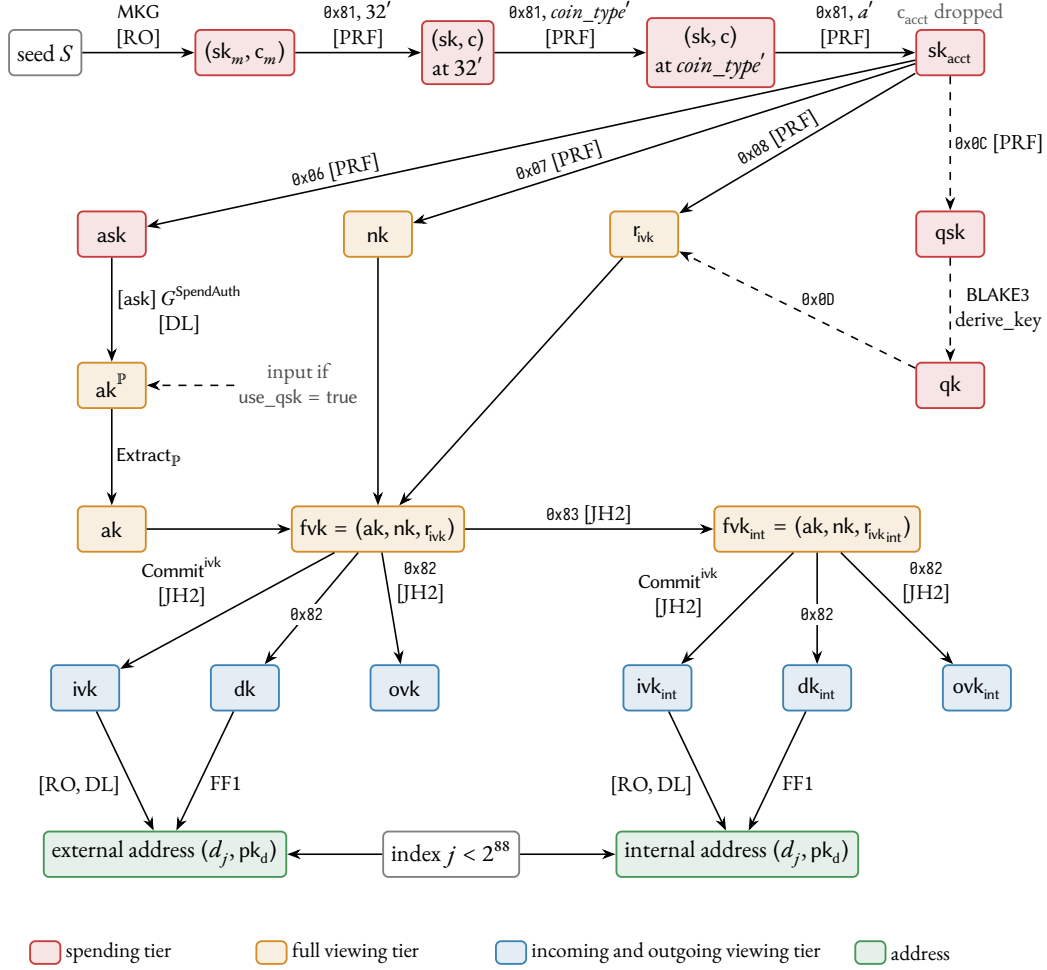


Figure 1. Derivation of the keys of one account, from the seed S through the master key and the hardened levels $32'$, $coin_type'$ and a' to the account spending key sk_{acct} , whose chain code is dropped; then the key components, the full viewing key fvk , the internal full viewing key fvk_{int} , the incoming and outgoing viewing keys of each scope, and the addresses at each index j . Solid edges are those of a key with $use_qsk = false$; dashed edges are the $use_qsk = true$ sources of ak^P , an input, and of r_{ivk} , through qsk and qk . Each derivation edge carries its expansion byte or operation; unlabelled edges are projections into a tuple or inputs. Each one-way edge also carries the assumption under which it is one-way: PRF, the *Ironwood Guide*’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”); RO, a random oracle (BLAKE2b under ZcashIP320rchain for MKG, GroupHash for the diversified base); DL, its Assumption “Discrete logarithms on Pallas”; JH2, the joint hiding assumption of both scopes stated in §2.5. Colours give the lowest key tier that computes a node. The separations the figure depicts are proved in §2.5.

Node	Value
m	sk 7eee3c1017870990a3dd6891b82f80be8976c1e7dc20d60817a5e88e8b2cd4b8 c ab8b7a00509ef20e469b5292b61d474b7cfcfb1657924cda720250ae40526677
$m/32'$	sk 93af0a87c2e4704d8825e145557c6a5d4dcbc9016170e276a2080a2dc9eb0f87 c 6e1e808d410d6d1f84e62e67c8b60e80e5ee4de5021d575c0433a625d609c45c
$m/32' / 133'$	sk 149fcee594a2a03de277ab919f22611d0ef6aa8e8e9a0bef32f01a0bcb0e8c4c c 07d13fdeccd21da16b467f8e785edc32f74ceeb541bede0394647f10fdb7114f
$m/32' / 133' / 0'$	sk b67d8d87cab9189500afb45dbca9f92c924c1de9d9ee1451be4a78313cb223b4 c 7ff0a6392e144d4c8f45bca21fb7827b508446ac069890aefbebbe388b7a84c0

The last spending key is sk_{acct} , and it is valid in the sense of §2.6; the last chain code is c_{acct} , which no construction uses. The master spending key agrees with the first Orchard test vector of ZIP 32, “Test Vectors”.

2.4 Orchard key components of an account

The *Ironwood Guide* constructs from a spending key sk the components ask , nk and r_{ivk} (§“The spending key and the spend-side secrets”), the spend validating key ak^P and its coordinate ak (same subsection), the full viewing key $fvk = (ak, nk, r_{\text{ivk}})$, the incoming viewing key scalar $ivk = \text{Commit}_{\text{fvk}}^{\text{ivk}}(ak, nk)$, and (dk, ovk) as the two halves of

$$\text{PRFexpand}_{\text{LE}_{256}(r_{\text{ivk}})}([0x82] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))$$

(§“Viewing keys”), and the diversified addresses (§“Diversified addresses”). These constructions are the $\text{use_qsk} = \text{false}$ branch (*Ironwood Guide*, §“The spending key and the spend-side secrets”, Remark “Scope of the key derivations”; protocol specification, §“Orchard Key Components”). They are cited, not restated; this subsection supplies $sk := sk_{\text{acct}}$ and the $\text{use_qsk} = \text{true}$ branch.

Lemma 2.9 (Account keys as uniform spending keys). *Let R be a result of the Ironwood Guide about a key generated with $\text{use_qsk} = \text{false}$, stated as a bound on the probability of a win event W in a game G whose challenger receives the output sk of key generation (*Ironwood Guide*, §“The spending key and the spend-side secrets”, Definition “Spending key and key generation”), runs in time polynomial given sk , and decides W . Let G_{acct} be G with $sk := sk_{\text{acct}}$ at a fixed account path, the adversary A also receiving $\text{SeedFP}(S)$ and the spending keys at a fixed finite set of other account paths, and let V be the event that sk_{acct} passes the discard conditions of that Definition (made exact in §2.6). Let $\mathcal{B}$ be the adversary of G that samples those extra inputs as independent uniform 32-byte strings, answers the random-oracle queries of A by lazy sampling, and runs A . Then*

$$|\Pr[W \text{ in } G_{\text{acct}} \mid V] - \Pr[\mathcal{B} \text{ wins } G]| \leq \frac{2\delta}{1 - \rho - \delta},$$

*where δ is the advantage bound of Lemma 2.5 for the distinguisher of the proof and ρ is the probability that a single draw from a uniform sk is discarded, negligible by the *Ironwood Guide*’s Lemma “Rejection in key generation” (§“Viewing keys”); the bound assumes $\rho + \delta < 1$. Every key-side result of the *Ironwood Guide* therefore holds for sk_{acct} up to this loss.*

Proof. The key sk_{acct} is only computationally close to uniform, so the statistical-distance clause of the *Ironwood Guide*’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”) does not apply; the proof is a reduction. Both V and W

are decided efficiently by the challenger. A distinguisher for Lemma 2.5 receives either the real values (sk_{acct} , the other spending keys and $\text{SeedFP}(S)$) or independent uniform strings, runs the challenger and $\mathcal{A}$ on them, and outputs 1 exactly on $W \wedge V$; a second distinguisher outputs 1 exactly on V . Write $a := \Pr[W \wedge V]$ and $v := \Pr[V]$ for the real values, and a_u, v_u for the uniform ones. Then $|a - a_u| \leq \delta$ and $|v - v_u| \leq \delta$, with $v_u = 1 - \rho$. With uniform inputs, the extra inputs are independent of the key and distributed as $\mathcal{B}$ samples them. Key generation outputs a single uniform draw conditioned on acceptance, so $\Pr[\mathcal{B} \text{ wins } G] = a_u/v_u$. Hence

$$\left| \frac{a}{v} - \frac{a_u}{v_u} \right| \leq \frac{|a - a_u|}{v} + \frac{a_u}{v_u} \cdot \frac{|v_u - v|}{v} \leq \frac{2\delta}{v} \leq \frac{2\delta}{1 - \rho - \delta},$$

using $a_u \leq v_u$ and $v \geq v_u - \delta$. $\square$

Construction 2.10 (Key components under `use_qsk`). The flag `use_qsk` $\in \{\text{false}, \text{true}\}$ is chosen at key generation. For a spending key `sk`, in both branches

$$\text{nk} := \text{ToBase}(\text{PRFexpand}_{\text{sk}}([\text{0x07}])).$$

1. If `use_qsk` = `false`: `ask`, ak^{P} , `ak` and r_{ivk} are those of the *Ironwood Guide*, and `qsk` = `qk` = $\perp$.
2. If `use_qsk` = `true`: a spend validating key $\text{ak}^{\text{P}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$ with even y -coordinate, that is, with bit 255 of its star encoding equal to 0, is an input, obtained by a method outside this construction, for example a directly generated `ask` with $\text{ak}^{\text{P}} = [\text{ask}] G^{\text{SpendAuth}}$, or FROST distributed key generation. Then

$$\text{ak} := \text{Extract}_{\text{p}}(\text{ak}^{\text{P}}),$$

$$\text{qsk} := \text{truncate}_{32}(\text{PRFexpand}_{\text{sk}}([\text{0x0C}])),$$

$$\text{qk} := \text{BLAKE3.derive_key}(\text{Zcash ZIP 2005 qk-derivation v1}, \text{qsk}, 32),$$

$$\text{r}_{\text{ivk}} := \text{ToScalar}(\text{PRFexpand}_{\text{qk}}([\text{0x0D}] \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}))),$$

where truncate_{32} keeps the first 32 bytes and

$$\text{BLAKE3.derive_key}(\text{context}, \text{material}, \text{length})$$

is the function that ZIP 2005 specifies, cited to it and not constructed; no property of it is assumed.

The components `ivk`, `dk`, `ovk` and every address are computed from $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ by the *Ironwood Guide*'s constructions in both branches, so they depend on `use_qsk` (ZIP 2005, “Changes to the Protocol Specification”, §4.2.3 “Orchard Key Components”, and “Usage with FROST”). No edition of the protocol specification carries this branch.

Wallet rules. A wallet MUST generate every key of one ZIP 32 account with the same value of `use_qsk` (ZIP 326, “Wallet key-generation restrictions”). The rule of the same section on Orchard-pool funds and `use_qsk` = `true` keys is stated once, with routing (§11.2). The value of `use_qsk` is not part of the recovery information, so a wallet restoring from a seed scans with both values (ZIP 326, “Scanning when restoring from seed”; §6.2).

Class (Definition 1.1): how an account whose keys are to be recoverable from the seed with `use_qsk` = `true` derives ak^{P} from sk_{acct} is fixed by no ZIP; it is designed but unspecified.

Remark 2.11 (Scope of the key results). Lemma 2.9 and every later key-side result of this volume (separations, hiding, validity bounds) is claimed for $\text{use_qsk} = \text{false}$ only. For $\text{use_qsk} = \text{true}$ the trapdoor r_{ivk} is an expansion keyed by qk , and a result would need BLAKE3 in its key-derivation mode modelled as a PRF, which is not assumed, and a hypothesis on the source of ak^{P} . Constructive statements, which state what is computed from what, hold in both branches.

Proposition 2.12 (Key recoverability). *For $\text{use_qsk} = \text{false}$, every spending, full viewing, incoming viewing and outgoing viewing key and every address of an account is a deterministic function of the seed, the coin type, the account index, the scope and the diversifier index; the seed with the account index recovers all of them. For $\text{use_qsk} = \text{true}$ the same holds of nk , qsk and qk , and, given ak , of r_{ivk} , ivk , dk , ovk and every address; recovery needs ak in addition, and, to spend, the key material that signs under ak , held collectively by the signers under FROST. In both cases the wallet must know the account’s value of use_qsk or try both. The function is partial: it is undefined at an account index whose key is invalid (§2.6).*

Proof. Composition of Constructions 2.3 and 2.4 and Definition 2.7, which map the seed to sk_{acct} ; of Construction 2.10, which maps sk_{acct} and use_qsk , together with ak^{P} when $\text{use_qsk} = \text{true}$, to $(\text{ak}, \text{nk}, r_{\text{ivk}})$; and of the *Ironwood Guide*’s constructions from $(\text{ak}, \text{nk}, r_{\text{ivk}})$ (§“Viewing keys” and §“Diversified addresses”). The keys of the internal scope are a function of fvk (§2.5), and the composition only adds that function. ZIP 2005, “Changes to the Protocol Specification”, §4.2.3 “Orchard Key Components”, notes, and “Usage with FROST” state the same requirements for recovery. $\square$

Construction 2.13 (Full viewing key fingerprint and tag). For a full viewing key $(\text{ak}, \text{nk}, r_{\text{ivk}})$ of either branch,

$$\text{FVFP}(\text{ak}, \text{nk}, r_{\text{ivk}}) := \text{BLAKE2b-256}(\text{Zcash0rchainFVFP}, \\ \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}) \parallel \text{I2LEOSP}_{256}(r_{\text{ivk}})),$$

over the 96-byte string shown, which is the raw encoding of an Orchard full viewing key (protocol specification, §“Orchard Raw Full Viewing Keys”). It MAY be used to identify the key uniquely; its uses rely on collision resistance, cited as for Construction 2.2; Lemma 2.24 models it as a random oracle. The *tag* is $\text{FVFP}[0..4)$, its first 4 bytes; it serves key lookup and MUST NOT be assumed unique, and nothing is claimed of it (ZIP 32, “Orchard Full Viewing Key Fingerprints and Tags”). In ZIP 32’s encoding of an extended spending key, a key names its parent by this tag, the master key’s parent tag being four zero bytes (ZIP 32, “Orchard extended spending keys”).

2.5 Internal keys: change and auto-shielding

With no change level, the separation between externally visible receipt and wallet-internal operations runs below the account. The internal operations are change and *auto-shielding*, the automatic transfer of received transparent funds to the most recent shielded pool (ZIP 315, “Terminology”; its rules in §11.3). The derivation applies to keys at the account level (ZIP 32, “Orchard internal key derivation”).

Construction 2.14 (Internal full viewing key). For a full viewing key $\text{fvk} = (\text{ak}, \text{nk}, r_{\text{ivk}})$ let $K := \text{LE}_{256}(r_{\text{ivk}})$, written $\text{I2LEBSP}_{256}(r_{\text{ivk}})$ in ZIP 32, and

$$r_{\text{ivk}_{\text{int}}} := \text{ToScalar}(\text{PRFexpand}_K([\text{0x83}] \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}))) \in \mathbb{F}_{p_{\text{Vesta}}}.$$

The *internal full viewing key* is $\text{fvk}_{\text{int}} := (\text{ak}, \text{nk}, r_{\text{ivk}_{\text{int}}})$. The internal keys are $\text{ivk}_{\text{int}} := \text{Commit}_{r_{\text{ivk}_{\text{int}}}}^{\text{ivk}}(\text{ak}, \text{nk})$ and $(\text{dk}_{\text{int}}, \text{ovk}_{\text{int}})$, the two halves of the 0x82 expansion keyed by $\text{LE}_{256}(r_{\text{ivk}_{\text{int}}})$ over the same message. The input of the 0x83 expansion has the key and message body of the 0x82 expansion and another lead byte. The internal expanded spending key is $(\text{ask}, \text{nk}, r_{\text{ivk}_{\text{int}}})$; no 32-byte internal spending key exists (ZIP 32, “Orchard internal key derivation”; protocol specification, §“Orchard Key Components”).

Hence $\text{ak}_{\text{int}} = \text{ak}$ and $\text{nk}_{\text{int}} = \text{nk}$, as the protocol specification, §“Orchard Key Components”, notes: spend authorisation and nullifier derivation serve both scopes, and ivk , dk and ovk are derived again from $r_{\text{ivk}_{\text{int}}}$. The construction does not force the keys of the two scopes to differ; their equality is an event of negligible probability (Proposition 2.17), not an excluded case. For Sapling, ZIP 32 keeps the spend authorising key and derives again the key that enters the proof and the nullifier, the outgoing viewing key and the diversifier key, so the two scopes share spend authority and differ in the key that enters the proof and the nullifier (ZIP 32, “Sapling internal key derivation”); this is cited, not constructed. Payments are sent from an account and a scope, not from an address: no outgoing viewing key per diversified address exists (ZIP 32, “Orchard diversifier and OVK derivation”; ZIP 316, “Usage of Outgoing Viewing Keys”).

Proposition 2.15 (Capability of a full viewing key). *From fvk alone one computes, for both scopes, ivk , dk , ovk and the address at every diversifier index $j \in \{0, \dots, 2^{88} - 1\}$. For a key with $\text{use_qsk} = \text{false}$, no efficient algorithm given fvk outputs ask , and no efficient algorithm given the keys of one account outputs the spending key of another account, except with negligible probability. Recovering the index and scope of a given address is stated with the index (§3.1).*

Proof. Computation: fvk_{int} is a function of fvk (Construction 2.14), and each address is a function of (dk, ivk) and j (*Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”). No ask : the *Ironwood Guide*, §“Key capabilities and scope”, Proposition “Capability separation”, part (a), under its Assumptions “Pseudorandomness of PRF expansion” and “Discrete logarithms on Pallas”, transferred to sk_{acct} by Lemma 2.9. No other account: the spending key of another account lies at a distinct path, and only hardened derivation exists; by Lemma 2.5 the two keys are jointly indistinguishable from independent uniform strings, so an algorithm given every key of this account outputs the other key with probability at most 2^{-256} plus the advantage bound of that lemma. $\square$

Assumption 2.16 (Joint hiding of the viewing keys of both scopes). For keys with $\text{use_qsk} = \text{false}$: for every $(\text{ak}, \text{nk}) \in \{1, \dots, p_{\text{Pallas}} - 1\} \times \mathbb{F}_{p_{\text{Pallas}}}$, given to the distinguisher, and r_{ivk} uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, the six keys $(\text{ivk}, \text{dk}, \text{ovk}, \text{ivk}_{\text{int}}, \text{dk}_{\text{int}}, \text{ovk}_{\text{int}})$, computed by the *Ironwood Guide*’s constructions and Construction 2.14, are computationally indistinguishable from

$$(\text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}), u_1, u_2, \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{ak}, \text{nk}), u_3, u_4),$$

where r and r' are uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and $u_1, \dots, u_4$ are uniform 32-byte strings, all independent. It extends the *Ironwood Guide*'s Assumption "Joint hiding of the viewing keys" (§"Viewing keys"), which covers the external triple only, to the $\theta x83$ row, whose key $\text{LE}_{256}(r_{\text{ivk}})$ is also the $\text{Commit}^{\text{ivk}}$ trapdoor of the external ivk and so lies outside the hypothesis of the *Ironwood Guide*'s Lemma "Independence of domain-separated expansions" (§"Pseudorandom functions and field reductions") (Remark "Hiding of ivk and the double use of r_{ivk} " (§"Viewing keys")); the PRF assumption does not give it (protocol specification, §"Orchard Key Components", note on r_{ivk} as trapdoor and key). It is applied after Lemma 2.9, averaging over the key-generation distribution of (ak, nk) .

Proposition 2.17 (Separation of scopes under trial decryption). *Let an account key have $\text{use_qsk} = \text{false}$, with external and internal incoming viewing keys outside $\{0, \perp\}$. An output honestly addressed to an internal-scope address of the account is accepted under the account's external ivk only with negligible probability, under the *Ironwood Guide*'s Assumptions "Discrete logarithms on Pallas" (§"Sinsemilla: hash, commitment, and short forms") and "GroupHash as a random oracle" (§"GroupHash, domain separation, and nothing-up-my-sleeve generators"), Assumption 2.16 and the hypotheses of Lemma 2.9. Consequently an external incoming viewing key reveals no change or auto-shielding output. Since fvk_{int} is computable from fvk , a full viewing key reveals both scopes, and the boundary between the scopes lies at the incoming viewing tier.*

Proof. Acceptance under a key ivk derives the transmission key again as $[\text{ivk}] g_d$ from the decrypted diversifier and checks the note commitment of the derived note against cmx (*Ironwood Guide*, §"Trial decryption and note acceptance", Construction "Acceptance of an Ironwood-pool output", check (iv); for an Orchard-pool output the check of the protocol specification, §"Decryption using an Incoming Viewing Key (Sapling and Orchard)"). The honest output commits to a note n with transmission key $[\text{ivk}_{\text{int}}] g_d$. If the note n' derived under ivk_{ext} differs from n , acceptance exhibits two distinct notes with one extracted commitment, which happens with negligible probability (*Ironwood Guide*, §"Note commitments", Proposition "Hiding and binding of note commitments", part (b)). If $n' = n$, then $[\text{ivk}_{\text{ext}}] g_d = [\text{ivk}_{\text{int}}] g_d$; since $g_d \neq \mathcal{O}$ in a group of prime order and both scalars lie in $\{1, \dots, p_{\text{Pallas}} - 1\}$ with $p_{\text{Pallas}} < p_{\text{Vesta}}$, this forces $\text{ivk}_{\text{ext}} = \text{ivk}_{\text{int}}$ (*Ironwood Guide*, §"Viewing keys"). By Lemma 2.9 and Assumption 2.16 the pair is replaced by $(\text{Commit}_r^{\text{ivk}}(\text{ak}, \text{nk}), \text{Commit}_{r'}^{\text{ivk}}(\text{ak}, \text{nk}))$ with independent uniform r and r' , at the cost of the distinguishing advantage, since equality is efficiently decidable. By the *Ironwood Guide*'s Lemma "Distribution of ivk" (§"Viewing keys"), the second value equals any fixed value with probability at most $2/p_{\text{Vesta}}$, independently of the first, so the two are equal with probability at most $2/p_{\text{Vesta}}$. $\square$

Construction 2.18 (Transparent outgoing viewing keys). A P2PKH (pay-to-public-key-hash) output pays to the hash of one secp256k1 public key (*Crypto Guide*, §"The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"). P2PKH keys have no outgoing viewing key, yet auto-shielding outputs need one. For the BIP 32 account-level extended public key (c, pk) of a transparent account, at path $m/44'/\text{coin_type}'/\text{account}'$ (BIP 44), let

$$I_{\text{ovk}} := \text{PRFexpand}_c([\theta x D0] \parallel \text{ser}_p(\text{pk})),$$

with the chain code c used as a 32-byte key and $\text{ser}_p(\text{pk})$ the 33-byte compressed encoding of BIP 32; then $\text{ovk}_{\text{external}} := I_{\text{ovk}}[0..32]$ and $\text{ovk}_{\text{internal}} := I_{\text{ovk}}[32..64]$ (ZIP 316, “Deriving Internal Keys”). The external transparent incoming viewing key is the extended public key of the non-hardened child 0 of (c, pk) and the internal one that of the non-hardened child 1 (ZIP 316, “Deriving a UIVK from a UFVK”; BIP 44), each obtained by BIP 32 public-to-public derivation, which is HMAC-SHA512 keyed by c (BIP 32, “Public parent key $\rightarrow$ public child key”; *Crypto Guide*, §“PRFs from hash functions: length extension and keyed BLAKE2”).

Assumption 2.19 (Hiding of the transparent account key). For an honestly generated account-level extended public key (c, pk) , the extended public keys of its non-hardened children 0 and 1 and the two halves of I_{ovk} are jointly computationally indistinguishable from two independently generated extended public keys, each a uniform 32-byte chain code with the public key of a uniform secret key, and two independent uniform 32-byte strings. The chain code c keys both HMAC-SHA512 in BIP 32 non-hardened derivation and PRFexpand in the $0x00$ row, so this is not an instance of the *Ironwood Guide*’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”); it is assumed.

Proposition 2.20 (Capability separation of internal keys). *The following hold, where “derives” means outputs the key with non-negligible probability.*

- (i) *A holder of a full viewing key derives the external and internal incoming viewing keys and the external and internal outgoing viewing keys.*
- (ii) *A holder of the external incoming viewing key derives neither the internal incoming viewing key nor any outgoing viewing key.*
- (iii) *A holder of the external outgoing viewing key derives neither the internal outgoing viewing key nor any incoming viewing key.*

For Orchard the statements hold for keys with $\text{use_qsk} = \text{false}$ under Lemma 2.9 and Assumption 2.16; for Sapling they are the properties that ZIP 316, “Deriving Internal Keys”, states for the construction of ZIP 32, “Sapling internal key derivation”, and are cited, not proved; for transparent P2PKH keys under Assumption 2.19 (ZIP 316, “Deriving Internal Keys”).

Proof. (i) Constructive: Construction 2.14 and the *Ironwood Guide*’s constructions; for transparent keys, Construction 2.18 from (c, pk) .

(ii) and (iii), Orchard: by Lemma 2.9 and Assumption 2.16 the six keys are replaced by the ideal distribution, at the cost of the distinguishing advantage, since success is decided by comparing the output with the key. In the ideal distribution every key not computed from the held one is independent of it: either a uniform 32-byte string, output with probability 2^{-256} , or a $\text{Commit}^{\text{ivk}}$ value under an independent trapdoor, output with probability at most $2/p_{\text{Vesta}}$ (*Ironwood Guide*, §“Viewing keys”, Lemma “Distribution of ivk”). Transparent: the same argument in the ideal distribution of Assumption 2.19, where an independently generated extended public key has a uniform chain code, output with probability 2^{-256} . Sapling: cited to ZIP 316, “Deriving Internal

Keys”, which asserts them from the one-wayness of PRFexpand and CRH^{ivk} ; not proved here. $\square$

2.6 Valid account keys

Definition 2.21 (Valid account key). For $\text{use_qsk} = \text{false}$, an account spending key sk_{acct} is *valid* if and only if $\text{ask} \neq 0$, $\text{ivk} \notin \{0, \perp\}$ and $\text{ivk}_{\text{int}} \notin \{0, \perp\}$. The first two conditions are discard conditions of the *Ironwood Guide*, §“The spending key and the spend-side secrets”, Definition “Spending key and key generation”; the third is the specification’s condition on the internal key of Construction 2.14, which that Definition names without constructing (protocol specification, §“Orchard Key Components”; ZIP 32, “Orchard internal key derivation”). ZIP 2005 amends the derivation for $\text{use_qsk} = \text{true}$ (“Changes to the Protocol Specification”, §4.2.3), for which this volume states no validity condition. Only keys a wallet uses need be valid; intermediate path nodes are never expanded into components.

Remark 2.22 (Derivation failure). ZIP 32 notes that a child spending key yields an invalid external or internal full viewing key “with small probability” and prescribes no wallet behaviour (“Orchard child key derivation”). The specification’s instruction to discard the key and repeat with a new sk applies to a freshly random key, not to a fixed path, where nothing is drawn again: an account index whose key is invalid has no key. Class (Definition 1.1): the wallet’s handling of such an index is an open problem; Proposition 2.23 bounds its probability.

Proposition 2.23 (Probability of an invalid account key). *Model the Sinsemilla generators, the points $S(j)$ for $0 \leq j < 2^{10}$ and the initial point $Q := Q(\text{z.cash:Orchard-CommitIvk-M})$, as independent uniform points of the Pallas group, independent of every other quantity (Ironwood Guide, §“GroupHash, domain separation, and nothing-up-my-sleeve generators”, Assumption “GroupHash as a random oracle”). Let*

$$\varepsilon := \frac{3[2^{512}/p_{\text{Vesta}}]}{2^{512}} + \frac{408}{p_{\text{Vesta}}} < \frac{412}{p_{\text{Vesta}}} = 2^{-245.31}.$$

For $\text{use_qsk} = \text{false}$ and a fixed account path, $\Pr[\text{sk}_{\text{acct}} \text{ is invalid}] \leq \varepsilon + \delta$, where δ is the sum of the advantages of efficient reductions, each deciding validity, against Lemma 2.5, the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”) (rows 0x06, 0x07 and 0x08) and Assumption 2.16 (row 0x83), plus the statistical distance of ToScalar from uniform, below 2^{-257} (Ironwood Guide, §“Pseudorandom functions and field reductions”). For the n accounts a wallet uses, $\Pr[\text{some key is invalid}] \leq n\varepsilon + \delta_n$, where δ_n is the corresponding sum for the n paths jointly; the union runs over the keys used, not over path nodes. The bound averages over generators modelled as random and is not a frequency for the fixed deployed generators.

Proof. Replace sk_{acct} by a uniform string (Lemma 2.5); the 512-bit outputs of rows 0x06, 0x07 and 0x08 by independent uniform strings (the *Ironwood Guide*’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”), first conclusion); r_{ivk} by a uniform element of $\mathbb{F}_{p_{\text{Vesta}}}$ (the reduction distance); and, on the event $\text{ask} \neq 0$, the pair $(\text{ivk}, \text{ivk}_{\text{int}})$

by $(\text{Commit}_r^{\text{ivk}}(\text{ak}, \text{nk}), \text{Commit}_{r'}^{\text{ivk}}(\text{ak}, \text{nk}))$ with independent uniform r and r' (Assumption 2.16). Each replacement changes the probability of invalidity by at most one term of δ , since validity is efficiently decidable. Then, by the union bound (*Math Guide*, §“The union bound and a birthday calculation”, Theorem “Union bound / Boole’s inequality”), the key is invalid only on one of the following events.

- (a) $\text{ask} = 0$. The sign normalisation preserves 0, so this happens exactly when a uniform 512-bit integer is divisible by p_{Vesta} , with probability $\lfloor 2^{512}/p_{\text{Vesta}} \rfloor / 2^{512}$ (*Math Guide*, §“Uniform sampling and the bias of modular reduction”).
- (b) The Sinsemilla hash point $\hat{M}$ of the message $\text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})$ is $\perp$. The message has 510 bits, 51 chunks of 10 bits, so the hash performs 102 incomplete additions, two per round, $\text{Acc}_i := (\text{Acc}_{i-1} + S(m_i)) + \text{Acc}_{i-1}$ (protocol specification, §“Sinsemilla Hash Function”; *Ironwood Guide*, §“Sinsemilla: hash, commitment, and short forms”, Construction “Sinsemilla hash on Pallas”). An incomplete addition $A + B$ of points is exceptional exactly when $A = \mathcal{O}$, $B = \mathcal{O}$, $A = B$ or $A = -B$. At the first exceptional addition every accumulator so far equals its formal value, an integer combination of Q and the $S(j)$, so one of four linear relations among the generators holds. Each relation is nontrivial: it either states that the single generator $S(m_i)$ is $\mathcal{O}$, or gives Q a coefficient 2^t or 2^{t+1} , non-zero modulo the odd prime p_{Vesta} . The message depends only on (ask, nk) and $G^{\text{SpendAuth}}$, which are independent of Q and the $S(j)$, so each relation holds with probability $1/p_{\text{Vesta}}$, and the 102 additions cost at most $408/p_{\text{Vesta}}$. Both scopes commit to the same message, so this event is counted once.
- (c) $\text{ivk} = 0$ with $\hat{M} \neq \perp$. The addition of $[r]H_D$ is complete, and $\text{ivk} = 0$ exactly when $\hat{M} + [r]H_D = \mathcal{O}$ (*Ironwood Guide*, §“Fields, groups, and encodings”, Lemma “Extract vanishes only at the identity”), which, since $H_D \neq \mathcal{O}$, holds for a single value of r : probability $1/p_{\text{Vesta}} \leq \lfloor 2^{512}/p_{\text{Vesta}} \rfloor / 2^{512}$.
- (d) $\text{ivk}_{\text{int}} = 0$ with $\hat{M} \neq \perp$: likewise, under the independent r' , with the same bound.

The sum is ε . Negligibility alone is the *Ironwood Guide*’s Lemma “Rejection in key generation” (§“Viewing keys”). For n accounts, apply the replacements to the n paths jointly and the union bound over the n keys; path nodes that are not account keys are never expanded into components and do not enter the union. $\square$

2.7 The PRFexpand lead-byte registry

Domain separation of PRFexpand is per key: under one key, distinct lead bytes give jointly pseudorandom outputs (*Ironwood Guide*, §“Pseudorandom functions and field reductions”, Lemma “Independence of domain-separated expansions”), while expansions under different keys may reuse a byte. Byte conventions: the key that ZIP 32 writes $\text{l2LEBSP}_{256}(\text{r}_{\text{ivk}})$ is $\text{LE}_{256}(\text{r}_{\text{ivk}})$; the bytes of ak are $\text{l2LEOSP}_{256}(\text{ak})$ of the field element $\text{ak} = \text{Extract}_p(\text{ak}^P)$, which for the sign-normalised ak^P equals the byte form LEBS2OSP_{256} of its star encoding, bit 255 being 0 (*Ironwood Guide*, §“The spending key and the spend-side secrets”); the key nk enters as $\text{l2LEOSP}_{256}(\text{nk})$ (protocol specification, §“Orchard Key Components”).

Table 2. Domains of key derivation. Upper part: each expansion $\text{PRFexpand}_{\text{key}}(b \parallel \text{message})$ that this volume constructs, with its lead byte b , key, message and output; the rows 0x0C and 0x0D are those of ZIP 2005. Lower part: the 16-byte BLAKE2b personalisations of key derivation, with the function, output and constructing environment of each.

Byte	Key	Message	Output	Constructed in
0x81	c_{par}	$\text{sk}_{\text{par}} \parallel \text{I2LEOSP}_{32}(i)$	(sk_i, c_i)	Constr. 2.4
0x83	$\text{LE}_{256}(\text{r}_{\text{ivk}})$	$\text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk})$	$\text{r}_{\text{ivkint}}(\text{ToScalar})$	Constr. 2.14
0x0C	sk	ε (empty)	$\text{qsk}(\text{truncate}_{32})$	Constr. 2.10
0x0D	qk	$\text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk})$	r_{ivk} if $\text{use_qsk} = \text{true}$ (ToScalar)	Constr. 2.10
0xD0	transparent chain code c	$\text{ser}_p(\text{pk})$	$\text{ovk}_{\text{external}} \parallel \text{ovk}_{\text{internal}}$	Constr. 2.18
Personalisation	Function		Output	Constructed in
ZcashIP320rchaind	BLAKE2b-512 of S		(sk_m, c_m)	Constr. 2.3
Zcash_HD_Seed_FP	BLAKE2b-256 of $[\text{length}(S)] \parallel S$		$\text{SeedFP}(S)$	Constr. 2.2
Zcash0rchaindFVFP	BLAKE2b-256 of the 96-byte encoding of fvk		FVFP	Constr. 2.13

Table 2 lists the expansions and personalisations that this volume constructs. The expansions of the key components and of notes are the rows of the *Ironwood Guide*’s Construction “Domain bytes of PRF expansion” (§“Pseudorandom functions and field reductions”) and are not restated. The list of expansion bytes that the protocol specification maintains, as amended by ZIP 2005, is §“Pseudo Random Functions” (ZIP 2005, “Changes to the Protocol Specification”, §4.1.2); bytes this volume does not use are not listed.

Lemma 2.24 (Independence of the key-derivation domains). *Under the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” (§“Pseudorandom functions and field reductions”), with BLAKE2b under distinct personalisations modelled as independent random oracles, the expansions of rows 0x81 and 0x0C, whose keys (a chain code of the Orchard tree, and sk_{acct}) are pseudorandom by Lemma 2.5 and enter no function other than PRFexpand , and the three personalised BLAKE2b functions of Table 2 are jointly indistinguishable from independent random functions. The lemma excludes the other rows, each outside the hypothesis of the Ironwood Guide’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”): the row 0x83, whose key is also the $\text{Commit}^{\text{ivk}}$ trapdoor, is covered by Assumption 2.16; the row 0xD0, whose key also keys HMAC-SHA512 , by Assumption 2.19; the row 0x0D, keyed by the BLAKE3 output qk of which nothing is assumed (Remark 2.11), carries no claim.*

Proof. For each key, the *Ironwood Guide*’s Lemma “Independence of domain-separated expansions” (§“Pseudorandom functions and field reductions”), applied after the hybrid of Lemma 2.5 has made the keys uniform; this is a reduction, as in Lemma 2.9, not the statistical-distance clause of that lemma. For the three personalisations, the *Crypto Guide*’s Proposition “Domain separation yields independent oracles” (§“Domain separation and personalisation”): they are pairwise

distinct 16-byte strings, and the two BLAKE2b-256 uses and the BLAKE2b-512 use differ also in the output length, a parameter of BLAKE2b. The proof of Lemma 2.5 uses the same two citations directly. $\square$

3 Addresses and viewing keys (ZIP 316)

This section passes from the keys of an account to the addresses and viewing keys that a wallet hands out. It defines the diversifier index and the recovery of an index from an address, the raw encodings of receivers and keys, the unified container with its jumbled string encoding and its viewing-key form, the unified address at an index of a viewing key, the extensions of Revision 2 of ZIP 316, and the recipient addresses a sender handles.

3.1 Diversifier indices

The address layer meets two requirements.

- (R1) Each payee receives an address unlinkable to every other address of the same holder, while all of them are spendable by one key and recoverable from one seed (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, security requirements; ZIP 32; ZIP 316, “Rationale for address derivation”). Each shielded protocol meets (R1) with diversified addresses.
- (R2) Transparent, Sapling and Orchard receivers coexist, each with its own format. A recipient publishes one string, from which the sender selects the most preferred receiver type it supports, and whose characters a sender can check (ZIP 316, “Motivation” and “Requirements”). ZIP 316 meets (R2) above the protocols, with unified containers (§3.3) and their jumbled string encoding (§3.4).

The *Ironwood Guide* constructs the Orchard key tiers, the diversifier map and the diversified base (§“Viewing keys” and §“Diversified addresses”). This subsection adds the diversifier index, shared by Sapling and Orchard and used by transparent derivation.

The Orchard address at index j of an incoming viewing key (dk, ivk) is that of the *Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”, cited and not re-derived: $d := \text{FF1-AES256}_{dk}(\text{LE}_{88}(j))$, $g_d := \text{DiversifyHash}^{\text{Orchard}}(d)$, which is $\text{GroupHash}(z.\text{cash}:\text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$ unless that is the identity and the fallback point $\text{GroupHash}(z.\text{cash}:\text{Orchard-gd}, \epsilon)$ otherwise, and $pk_d := [ivk] g_d$; the address is (d, pk_d) . For each scope $s \in \{\text{ext}, \text{int}\}$ of an account (Construction 2.14) the pair (dk_s, ovk_s) is the first and the second 32 bytes of

$$\text{PRFexpand}_{\text{LE}_{256}(\mathbf{r}_{ivk_s})}([\text{0x82}] \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}))$$

(*Ironwood Guide*, §“Viewing keys”), and ivk_s is the incoming viewing key of the scope. The address at index 0 is the *default address* of the scope (protocol specification, §“Orchard Key Components”; ZIP 32, “Orchard diversifier and OVK derivation”).

Definition 3.1 (Diversifier index). For a 32-byte diversifier key dk and an index $j \in \{0, \dots, 2^{88} - 1\}$, the *diversifier at index j* is

$$d_j := \text{FF1-AES256.Encrypt}(dk, "", \text{LE}_{88}(j)) \in \{0, 1\}^{88},$$

the FF1 mode of NIST SP 800-38G over AES-256 with key dk , radix 2, length 88 and the empty tweak (*Crypto Guide*, §“Format-preserving encryption and FF1”, Construction “The FF1 mode of NIST SP 800-38G”; protocol specification, §“Pseudo Random Permutations”). The map is the same in both shielded protocols; only the key differs.

1. For Orchard, $dk = dk_s$ of the scope s , as above.
2. For Sapling, dk is the diversifier-key component of the account’s Sapling extended spending key: $dk_m = \text{truncate}_{32}(\text{PRFexpand}_{sk_m}([0x10]))$ at the master key and $dk_i = \text{truncate}_{32}(\text{PRFexpand}_{I_L}([0x16] \parallel dk_{par}))$ for a hardened child, with I_L the left half of the child’s derivation output (ZIP 32, “Sapling master key generation” and “Sapling child key derivation”); Sapling key derivation is cited and not constructed.

The diversifier d_j , and the index j , are *valid* for a protocol if and only if that protocol’s `DiversifyHash` does not return $\perp$ at d_j .

3. Orchard: $\text{DiversifyHash}^{\text{Orchard}}$ is total, so every index is valid, a scope has exactly 2^{88} addresses, and the default diversifier is d_0 (ZIP 32, “Orchard diversifier and OVK derivation”).
4. Sapling:

$$\text{DiversifyHash}^{\text{Sapling}}(d) := \text{GroupHash}_{\text{URS}}^{J(r)^*}(\text{Zcash_gd}, \text{LEBS2OSP}_{88}(d)),$$

a hash into the points of order r_j of the Jubjub curve that returns $\perp$ on some inputs (protocol specification, §“ $\text{DiversifyHash}^{\text{Sapling}}$ and $\text{DiversifyHash}^{\text{Orchard}}$ Hash Functions”, §“Jubjub” and §“Group Hash into Jubjub”, cited and not constructed). Approximately half of all indices are valid (ZIP 32, “Sapling diversifier derivation”); the probability under the specification’s random-oracle model is derived in §3.6. The default Sapling diversifier is d_j for the least valid j (same section).

Totality of $\text{DiversifyHash}^{\text{Orchard}}$ removes the search for a valid index. The Sapling failure rate matters when one index must serve several protocols at once (§3.6).

Index-choice rules, specified (protocol specification, §“Orchard Key Components” and §“Sapling Key Components”, notes). Orchard diversifier indices SHOULD NOT be chosen at random; ZIP 32 specifies their use in hierarchical deterministic wallets. Address generators MAY encode information in the diversifier index, recoverable by a holder of dk (Lemma 3.3). For Sapling, generators MAY encode information in the diversifier itself, and it is RECOMMENDED to encode it in the ZIP 32 diversifier index instead, so that only a holder of dk reads it.

Proposition 3.2 (Distinct diversifiers). (a) For every diversifier key dk the map $j \mapsto d_j$ is a bijection from $\{0, \dots, 2^{88} - 1\}$ onto $\{0, 1\}^{88}$. In particular distinct indices give distinct diversifiers, and a wallet issues the indices $0, 1, 2, \dots$ over the whole index space without a repeated diversifier.

(b) If instead k diversifiers are drawn independently and uniformly from $\{0, 1\}^{88}$, the probability P that two coincide satisfies

$$1 - e^{-k(k-1)/2^{89}} \leq P \leq \frac{k(k-1)}{2^{89}},$$

so a collision becomes likely once k is of order $2^{44} = \sqrt{2^{88}}$; at $k = 2^{44}$ the bounds are 0.393 and 0.500.

Proof. (a) The map LE_{88} is a bijection from $\{0, \dots, 2^{88} - 1\}$ onto $\{0, 1\}^{88}$, and FF1-AES256 under a fixed key and tweak is a bijection of $\{0, 1\}^{88}$ (*Crypto Guide*, §“Format-preserving encryption and FF1”, Proposition “FF1 is a keyed bijection on $\mathcal{X}$ ”); the composition is a bijection. (b) The *Math Guide*, §“The union bound and a birthday calculation”, Proposition “Birthday bound”, with $N = 2^{88}$. $\square$

The specification notes that an 88-bit diversifier is not unguessable at the 128-bit security level and that randomly chosen diversifiers collide near 2^{44} choices; the keyed permutation removes the collisions, provided indices do not repeat (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, notes). One further reason for a keyed permutation is ZIP 32’s: it makes the diversifier sequence of each account pseudorandom and uncorrelated with that of any other account, so issued diversifiers do not reveal how many addresses have been generated (ZIP 32, “Sapling diversifier derivation” and “Orchard diversifier and OVK derivation”). The *Ironwood Guide* supplies Assumption “FF1-AES256 as a pseudorandom permutation” (§“Diversified addresses”) and the unlinkability proposition only.

In Orchard, dk_s is derived from the full viewing key of scope s , so a holder of the account’s Orchard full viewing key computes dk_s for both scopes, hence the index of any diversifier of the account (Lemma 3.3; ZIP 32, “Orchard diversifier and OVK derivation”). In Sapling the same capability belongs to the extended full viewing key, of which dk is a component. A holder of the incoming viewing key (dk, ivk) of one scope has this capability for that scope only.

Lemma 3.3 (Index recovery and ownership). Let (d, pk_d) be an Orchard address and (dk, ivk) the incoming viewing key of some scope.

- (i) The value $j := LE_{88}^{-1}(FF1\text{-AES256.Decrypt}(dk, "", d))$ is defined for every d and every dk , and $d_j = d$. Decryption alone therefore proves nothing about ownership.
- (ii) The address (d, pk_d) is the address of (dk, ivk) at some index if and only if $pk_d = [ivk] \text{DiversifyHash}^{\text{Orchard}}(d)$; the index is then unique and equals j .
- (iii) A holder of an account’s full viewing key decides whether (d, pk_d) belongs to the account, and at which scope and index, by applying (ii) with $(dk_{\text{ext}}, ivk_{\text{ext}})$ and then with

$(dk_{\text{int}}, ivk_{\text{int}})$. Both tests succeed only if $ivk_{\text{ext}} = ivk_{\text{int}}$, which has negligible probability for keys with $\text{use_qsk} = \text{false}$ under Assumption 2.16 and the hypotheses of Lemma 2.9.

The Sapling case of (i) and (ii) is the same with $\text{DiversifyHash}^{\text{Sapling}}$, the test of (ii) failing when it returns $\perp$. For (iii), both tests succeed only if the two Sapling incoming viewing keys are equal; this volume does not bound that probability for Sapling.

Proof. (i) FF1 decryption is the two-sided inverse of encryption under every key and tweak (*Crypto Guide*, §“Format-preserving encryption and FF1”, Proposition “FF1 is a keyed bijection on $\mathcal{X}$ ”), and LE_{88} is a bijection. (ii) If (d, pk_d) is the address at index j' , then $d = d_{j'}$, so $j' = j$ by (i) and Proposition 3.2 (a), and $pk_d = [ivk] \text{DiversifyHash}^{\text{Orchard}}(d)$ by the *Ironwood Guide*’s Construction “Diversified address” (§“Diversified addresses”). Conversely, if $pk_d = [ivk] \text{DiversifyHash}^{\text{Orchard}}(d)$, the address at index j is $(d_j, [ivk] \text{DiversifyHash}^{\text{Orchard}}(d_j)) = (d, pk_d)$. (iii) If both tests succeed, then $[ivk_{\text{ext}}] g_d = [ivk_{\text{int}}] g_d$ with $g_d \neq \mathcal{O}$ in a group of prime order p_{Vesta} , and both scalars lie in $\{1, \dots, p_{\text{Pallas}} - 1\}$ with $p_{\text{Pallas}} < p_{\text{Vesta}}$, so $ivk_{\text{ext}} = ivk_{\text{int}}$. Its probability is bounded as in the proof of Proposition 2.17: after the replacement of Assumption 2.16, at most $2/p_{\text{Vesta}}$ by the *Ironwood Guide*’s Lemma “Distribution of ivk ” (§“Viewing keys”), plus the distinguishing advantage, equality being efficiently decidable. $\square$

Unlinkability of the addresses of one key is cited with its assumptions. For Orchard it is the *Ironwood Guide*, §“Diversified addresses”, Proposition “Unlinkability of diversified addresses”: for two keys generated independently with $\text{use_qsk} = \text{false}$, an adversary that holds ak , nk and ovk of both keys and chooses indices j_0, j_1 and $j' \notin \{j_0, j_1\}$ distinguishes which key produced the third address with negligible advantage, conditioned on the three diversifiers being pairwise distinct. Its assumptions are the *Ironwood Guide*’s “Pseudorandomness of PRF expansion”, “Joint hiding of the viewing keys”, “FF1-AES256 as a pseudorandom permutation”, “GroupHash as a random oracle”, “DDH on Pallas” and “Discrete logarithms on Pallas”. Account keys are admitted as such keys by Lemma 2.9, the win event being decidable by the challenger; the addresses of one key at distinct indices have distinct diversifiers by Proposition 3.2 (a). For Sapling the protocol specification argues the same property under DDH on the prime-order subgroup of Jubjub, with $\text{GroupHash}^{(r)*}$ modelled as a random oracle, through IK-CPA key privacy of ElGamal (§“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, notes; *Crypto Guide*, §“Key privacy”). This volume does not prove the Sapling case.

Remark 3.4 (Chosen-diversifier oracle). Suppose that the holder of an incoming viewing key gives an adversary the address of that key at any diversifier the adversary chooses. The adversary then breaks unlinkability outright: given an address (d', pk_d') , it requests the address at d' and compares the returned transmission key with pk_d' . The two are equal if and only if the key is the same, the base $g_{d'}$ being a point other than the identity in a group of prime order. No other privacy property is affected. Implementations SHOULD avoid providing such a chosen-diversifier oracle (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, notes). An interface returning the address at a requested *index* gives no chosen diversifier (Lemma 3.5), but it returns the issued addresses themselves: it links every address whose index the requester queries, and under sequential issuance (Proposition 3.2) small indices reach every issued address.

Lemma 3.5 (Index queries give no chosen diversifier). *Let dk be either diversifier key of an Orchard key with $use_qsk = \text{false}$. Under Assumption 2.16, the hypotheses of Lemma 2.9, and the Ironwood Guide’s Assumption “FF1-AES256 as a pseudorandom permutation” (§“Diversified addresses”), an adversary without dk that obtains the diversifiers at q indices of its choice obtains a given d , fixed independently of dk , with probability at most $q/2^{88}$ plus its distinguishing advantages against the two assumptions and, for an account key, the loss of Lemma 2.9. This volume does not prove the Sapling case.*

Proof. Replace dk by a uniform 32-byte string independent of the other keys; by Assumption 2.16, applied after Lemma 2.9, the probability changes by at most that distinguishing advantage and the loss of that Lemma, the win event being decidable. Then replace FF1-AES256 keyed by dk with a uniform permutation of $\{0, 1\}^{88}$; the probability changes by at most the distinguishing advantage. Repeated indices add nothing, so let the adversary query $k \leq q$ distinct indices. Each image is uniform on the values not yet returned, so the i -th image is the first to equal d with probability

$$\prod_{j=1}^{i-1} \frac{2^{88} - j}{2^{88} - j + 1} \cdot \frac{1}{2^{88} - i + 1} = 2^{-88};$$

summing over $i \leq k$, d is returned with probability $k \cdot 2^{-88} \leq q \cdot 2^{-88}$. $\square$

3.2 Raw encodings

The items of unified containers carry the byte encodings below. Each is stated with the validity condition that a decoder applies.

Definition 3.6 (Raw receiver and key encodings). (a) An Orchard raw payment address has 43 bytes, $\text{LEBS2OSP}_{88}(d) \parallel \text{pk}_d^*$, where $\text{pk}_d^* = \text{LEBS2OSP}_{256}(\text{repr}_p(\text{pk}_d))$ is the 32-byte compressed encoding of the Pallas point pk_d (Ironwood Guide, §“Fields, groups, and encodings”). A decoder MUST treat the address as invalid if abst_p returns $\perp$ or the identity (protocol specification, §“Orchard Raw Payment Addresses”).

(b) A Sapling raw payment address has 43 bytes,

$$\text{LEBS2OSP}_{88}(d) \parallel \text{LEBS2OSP}_{256}(\text{repr}_j(\text{pk}_d)),$$

with the compressed Jubjub encoding. It MUST be considered invalid if abst_j returns $\perp$, if the encoding of pk_d is non-canonical, or if pk_d is not a point of order n_j , the identity included (protocol specification, §“Sapling Payment Addresses”).

(c) An Orchard raw full viewing key has 96 bytes,

$$\text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk}) \parallel \text{I2LEOSP}_{256}(\text{r}_{\text{vk}}).$$

It MUST be considered invalid if ak , nk or r_{vk} is not the canonical encoding of an element of its field ($\mathbb{F}_{P_{\text{Pallas}}}$, $\mathbb{F}_{P_{\text{Pallas}}}$, $\mathbb{F}_{P_{\text{Vesta}}}$), if ak is not the x -coordinate of a point of $\mathcal{E}(\mathbb{F}_{P_{\text{Pallas}}})$ other than the identity, or if the external or the internal incoming viewing key derived from it is 0 or $\perp$ (protocol specification, §“Orchard Raw Full Viewing Keys”).

- (d) An *Orchard raw incoming viewing key* has 64 bytes, $dk \parallel I2LEOSP_{256}(ivk)$. It MUST be considered invalid unless $ivk \in \{1, \dots, p_{\text{Pallas}} - 1\}$ (protocol specification, §“Orchard Raw Incoming Viewing Keys”).
- (e) String forms. No Bech32 or Bech32m encoding exists for an individual Orchard payment address, incoming viewing key or full viewing key; the unified encodings of §3.4 are their only strings. A Sapling payment address keeps a standalone Bech32 string, with human-readable part `zs` on Mainnet and `ztestsapling` on Testnet (same sections).
- (f) Transparent receivers. A *P2PKH* (pay-to-public-key-hash) address pays to the 20-byte hash160 of a 33-byte compressed ECDSA validating key; a *P2SH* (pay-to-script-hash) address pays to the 20-byte hash160 of a redeem script that the spender reveals and satisfies (*Crypto Guide*, §“The transparent layer’s hashes”). Their raw encoding prefixes two version bytes to the hash and is written in Base58Check (protocol specification, §“Transparent Addresses”). Inside a unified container a transparent receiver is the bare 20-byte hash, without the two version bytes (ZIP 316, “Requirements for both Unified Addresses and Unified Viewing Keys”).

One Orchard raw address serves both pools of the Orchard protocol (§3.3).

3.3 Unified addresses

ZIP 316 defines Revision 0, the format of this subsection and of §3.4, and Revision 2, which adds metadata items, splits the address prefix into a shielded-only and a transparent-enabled form, and renames the viewing-key prefixes (§3.7); Revision 1 is withdrawn (ZIP 316, “Revisions”). The revision of a container is read from its human-readable part. The status of ZIP 316 is that of Table 1; everything Revision 2 adds is specified in the sense of Definition 1.1.

Definition 3.7 (Unified container). For $n \in \{0, \dots, 2^{64} - 1\}$, $\text{compactSize}(n)$ is the variable-length integer of Bitcoin that ZIP 316 cites:

$$\text{compactSize}(n) := \begin{cases} [n] & \text{if } n < 253, \\ [0xFD] \parallel I2LEOSP_{16}(n) & \text{if } 253 \leq n < 2^{16}, \\ [0xFE] \parallel I2LEOSP_{32}(n) & \text{if } 2^{16} \leq n < 2^{32}, \\ [0xFF] \parallel I2LEOSP_{64}(n) & \text{otherwise,} \end{cases}$$

the range of n fixing the form. An *item* is a pair (τ, v) of a *typecode* $\tau \in \mathbb{N}$ and a byte string v . The *raw encoding* of a container with items $(\tau_1, v_1), \dots, (\tau_k, v_k)$, where $\tau_1 < \dots < \tau_k$, is

$$\text{raw} := \parallel_{i=1}^k \text{compactSize}(\tau_i) \parallel \text{compactSize}(\text{length}(v_i)) \parallel v_i.$$

Both τ_i and $\text{length}(v_i)$ MUST be at most $0x2000000$; since $0x2000000 \in [2^{16}, 2^{32})$, each takes at most 5 bytes and an item header at most 10.

Typecodes $0x00$ and $0x01$ are *transparent*, $0x02$ and $0x03$ *shielded*, and $0xC0$ to $0xFC$ designate *metadata items*, items that are neither receivers nor viewing keys (§3.7). A *unified address* (UA)

is a container whose non-metadata items are receivers, in the raw encodings of Definition 3.6 under the typecodes of Table 3. A *unified full viewing key* (UFVK) and a *unified incoming viewing key* (UIVK) are containers whose non-metadata items are full, respectively incoming, viewing keys (§3.5); UVK denotes either. No typecode exists for Sprout.

Validity. Every container holds at least one non-metadata item. A Revision 0 UA, and a Revision 0 UVK, holds at least one shielded item. A Revision 2 shielded-only UA holds no transparent item; a Revision 2 transparent-enabled UA may consist of a single P2PKH receiver, and a Revision 2 UVK need hold no shielded item. A Consumer MUST reject a container

- (i) that violates the containment rule of its revision;
- (ii) in which a typecode repeats, or which holds both $0x00$ and $0x01$;
- (iii) which holds only metadata items;
- (iv) whose items are not in ascending typecode order;
- (v) which has bytes after the last item;
- (vi) any of whose items fails the validation of its encoding (Definition 3.6; §3.5).

A Consumer MUST ignore items with typecodes it does not recognise, with the exception stated in §3.7; a Consumer therefore parses a container that holds unrecognised items.

Same-account rule. Within one UA or UVK, all receivers, full viewing keys and incoming viewing keys derived hierarchically SHOULD belong to the same account at the ZIP 32 or BIP 44 account level, or, for ZIP 48 and FROST UVKs, to one set of signers able to generate independent addresses for that spending authority. FROST key material MUST be derived according to ZIP 312 and ZIP 2005, in particular ZIP 2005, “Usage with FROST”.

Sources: ZIP 316, “Encoding of Unified Addresses”, “Requirements for both Unified Addresses and Unified Viewing Keys” and “Receivers”.

The ascending order makes the raw encoding a function of the set of items, so a representation that discards the order reproduces the same encoding, provided it retains unrecognised items (ZIP 316, “Rationale for Item ordering”). This order is not the priority order below.

Receiver selection. The Sender of a payment to a UA MUST use the receiver of the most preferred type that it supports, by the Priority List Orchard ($0x03$), Sapling ($0x02$), then P2SH ($0x01$) or P2PKH ($0x00$); an unrecognised item is treated as absent, with the exception of §3.7. A wallet MAY let its users configure which receiver types it can send to, and MUST NOT let them change the order of the Priority List except by opting into an experiment, whose specification MAY add to the list and SHOULD keep the preference for more recent shielded protocols (ZIP 316, “Encoding of Unified Addresses”, “Transfers” and “Experimental Usage”). The Orchard receiver belongs to the Orchard protocol, not to a pool: one ivk trial-decrypts both Orchard-pool and Ironwood-pool note ciphertexts (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”), and the Ironwood pool changes no address structure or encoding (ZIP 229, “Abstract”). The pool in which a payment to an Orchard receiver is made is fixed by the routing rule of §11.2, which carries the prohibition of ZIP 326, “Wallet key-generation restrictions”, on paying external

Table 3. Typecodes and item lengths of Revision 0, in bytes. In Revision 0 Producers MUST NOT include typecode $0x01$ in a UFVK or UIVK, and Consumers MUST treat it as unrecognised there. The Priority List of preference is $0x03$, then $0x02$, then $0x01$ or $0x00$ (at most one of the two is present), the reverse of the encoding order. Receiver lengths are those of Definition 3.6; the viewing-key lengths are those of §3.5: $65 = 32 + 33$ (chain code and compressed key), 128 for (ak, nk, ovk, dk), 64 for (dk, ivk), and 96 for (ak, nk, r_{ivk}) (ZIP 316, “Encoding of Unified Addresses” and “Encoding of Unified Full/Incoming Viewing Keys”).

Typecode	Item	Receiver	UFVK item	UIVK item
$0x00$	transparent P2PKH	20	65	65
$0x01$	transparent P2SH	20	—	—
$0x02$	Sapling	43	128	64
$0x03$	Orchard	43	96	64

receivers in the Orchard pool; the Priority List is unchanged.

3.4 String encoding: F4Jumble and Bech32m

The Bech32m checksum does not cascade: a user who checks only the beginning and the end of a long string can miss an adversarial substitution of an inner receiver. ZIP 316 therefore passes the whole payload through F4Jumble, an unkeyed four-round Feistel construction that approximates a random permutation of the entire byte string, so that every output character depends on every input byte. It states two goals (ZIP 316, “Jumbling”).

- (i) *Near second-preimage resistance.* Given q honestly generated UAs or UVKs, an adversary does not produce a valid one that it controls, knowing at least one constituent private key, whose string matches one of the targets on a chosen set of characters, for concreteness the first n and the last m .
- (ii) *Nonmalleability.* Without any private key, an adversary does not produce a valid UA or UVK usefully different from a target, for example with its shielded receiver removed, that matches the target on the checked characters.

Generic attacks bound what is achievable: about $w c^{n+m}/q$ for (i) and w/p for (ii), where w is the work to produce and verify one UA or UVK, c the size of the character set and p the probability that a random decoding is valid and useful (ZIP 316, “Discussion”). Jumbling is not authentication. F4Jumble is unkeyed, so anyone can jumble and checksum a wholly substituted address, and an address is still obtained or verified through a trusted channel; jumbling raises only the cost of partial-match and malleation attacks against a genuine string.

Construction 3.8 (F4Jumble). Let $\ell_H := 64$ and $\ell_{\text{MAX}} := (2^{16} + 1) \ell_H = 4194368$. For $i \in \{0, 1\}$ and byte strings u , with the first argument of BLAKE2b the personalisation,

$$H_i(u) := \text{BLAKE2b}-(8\ell_L)(\text{UA_F4Jumble_H} \parallel [i, 0, 0], u),$$

$$G_i(u) := \text{the first } \ell_R \text{ bytes of } \bigparallel_{j=0}^{\lceil \ell_R/\ell_H \rceil - 1} \text{BLAKE2b-512}(\text{UA_F4Jumble_G} \parallel [i] \parallel 12\text{LEOSP}_{16}(j), u),$$

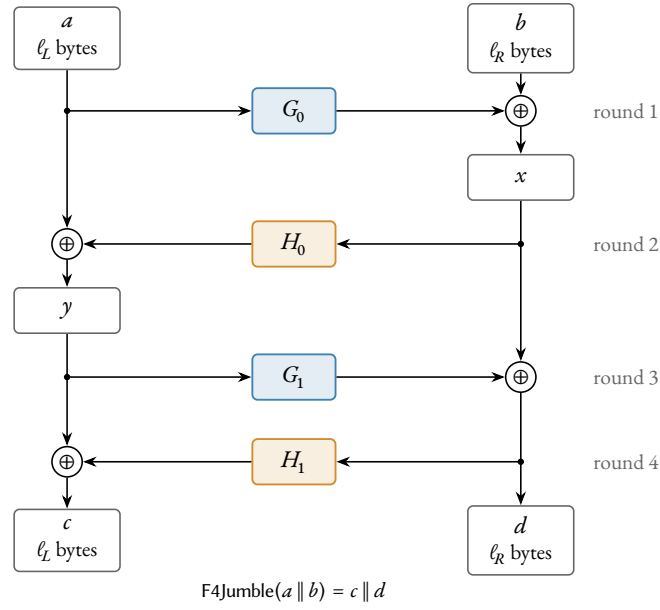


Figure 2. The four rounds of F4Jumble (Construction 3.8) on the unequal halves a (ℓ_L bytes) and b (ℓ_R bytes). Each round replaces one half by its XOR with a function of the other half, which it leaves unchanged: G_0 of a into b gives x , H_0 of x into a gives y , G_1 of y into x gives d , and H_1 of d into y gives c . Blue boxes are the extendable-output functions G_i , with ℓ_R output bytes; amber boxes are the hashes H_i , with ℓ_L output bytes. The names are those of the cells of Example 3.11. The structure is the content of Proposition 3.9.

an ℓ_L -byte hash and an extendable-output function in the sense of the *Crypto Guide*, §“Hash functions and the random oracle model”, Definition “Extendable-output function”, for the lengths ℓ_L and ℓ_R below. For a byte string M of length ℓ_M with $38 \leq \ell_M \leq \ell_{\text{MAX}}$, let

$$\ell_L := \min(\ell_H, \lfloor \ell_M/2 \rfloor), \quad \ell_R := \ell_M - \ell_L,$$

split $M = a \parallel b$ with $\text{length}(a) = \ell_L$, and let

$$\begin{aligned} x &:= b \oplus G_0(a), & y &:= a \oplus H_0(x), \\ d &:= x \oplus G_1(y), & c &:= y \oplus H_1(d), \end{aligned}$$

and $\text{F4Jumble}(M) := c \parallel d$ (ZIP 316, “Solution”). Figure 2 shows the four rounds.

Each personalisation has 16 bytes: 13 + 3 for H_i and 13 + 1 + 2 for G_i . They are pairwise distinct, differing at byte 12 between the H and the G family and at bytes 13 to 15 within a family, and distinct from the three personalisations of key derivation in Table 2 and from that of PRFexpand, Zcash_ExpandSeed. Being of one length and distinct, they are prefix-free tags, so BLAKE2b under these personalisations is modelled as independent random oracles (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”).

Proposition 3.9 (F4Jumble is a permutation). *For every ℓ_M with $38 \leq \ell_M \leq \ell_{\text{MAX}}$, F4Jumble is well defined and is a bijection of the ℓ_M -byte strings. Its inverse maps $c \parallel d$, with $\text{length}(c) = \ell_L$, to*

$a \parallel b$ computed by

$$y := c \oplus H_1(d), \quad x := d \oplus G_1(y), \quad a := y \oplus H_0(x), \quad b := x \oplus G_0(a).$$

Proof. Well defined. From $\ell_M \geq 38$, $\ell_L \geq 19$; from $\ell_L \leq 64$, the output length $8\ell_L$ of H_i is admissible for BLAKE2b. If $\ell_M \leq 128$ then $\ell_R \leq 64$ and one block of G_i suffices. Otherwise $\ell_L = 64$ and $\ell_R = \ell_M - 64 \leq 2^{16} \cdot 64$, so $\lceil \ell_R / \ell_H \rceil \leq 2^{16}$ and every counter j fits l2LEOSP_{16} ; this is the reason for $\ell_{\text{MAX}} = (2^{16} + 1) \ell_H$.

Bijection. Each of the four steps replaces one half by its XOR with a function of the other half, which it leaves unchanged, and $r = p \oplus q$ implies $p = r \oplus q$. Each step is therefore inverted by recomputing the same function of the unchanged half, and the composition of the four inverses in reverse order is the stated map, a two-sided inverse. This is the argument of the *Crypto Guide*, §“Format-preserving encryption and FF1”, Proposition “Feistel is a bijection regardless of the round functions”. Unequal halves change nothing, since G_i maps into ℓ_R bytes and H_i into ℓ_L bytes, the lengths of the halves they are XORed into. $\square$

ZIP 316, “Heuristic analysis”, motivates the number of rounds.

Construction 3.10 (Unified string encoding). Human-readable parts of Revision 0: u for a UA, $u\text{view}$ for a UFVK and $u\text{ivk}$ for a UIVK on Mainnet; on Testnet test is appended ($u\text{test}$, $u\text{viewtest}$, $u\text{ivktest}$). Let raw be the raw encoding of Definition 3.7, hrp the human-readable part and padding the human-readable part in US-ASCII right-padded with zero bytes to 16 bytes.

1. Producer. It MUST ensure $\text{length}(\text{raw}) \leq \ell_{\text{MAX}} - 16 = 4194352$, and outputs

$$\text{Bech32m}(\text{hrp}, \text{F4Jumble}(\text{raw} \parallel \text{padding})),$$

the Bech32m encoding of BIP 350 with its length restriction ignored.

2. Consumer. It MUST decode as follows, rejecting the string at the first failure:

- (1) decode with Bech32m, rejecting an incorrect checksum;
- (2) reject a decoded byte string shorter than 38 or longer than ℓ_{MAX} bytes;
- (3) apply F4Jumble^{-1} (Proposition 3.9);
- (4) reject a result that does not end in the 16-byte padding of the string’s human-readable part, and remove that padding;
- (5) parse the rest as a raw encoding, rejecting the whole string if any rule of Definition 3.7 fails.

UFVKs and UIVKs use the same procedure under their own human-readable parts (ZIP 316, “Encoding of Unified Addresses”, “Encoding of Unified Full/Incoming Viewing Keys” and “Usage for Unified Addresses, UFKs, and UIVKs”; BIP 350).

The padding supplies the redundancy that defeats the generic nonmalleability attack, since a random decoding must end in the exact 16 bytes, and gives the human-readable part the same

protection against malleation as the payload, which may help against cross-network substitution and against confusion of an address with a viewing key (ZIP 316, “Usage for Unified Addresses, UFVKs, and UIVKs” and “Heuristic analysis”).

Length rules (ZIP 316, “Rationale for length restrictions” and “Efficiency”). The 38-byte minimum is Revision 2’s: one P2PKH item of 22 bytes and the padding. Revision 0 specified 48 bytes. No set of items valid in Revision 0 has an encoding of 22 to 31 bytes (the smallest Revision 0 input is $45 + 16 = 61$ bytes), so the difference is unobservable except in the error reported, and a Consumer supporting Revision 2 MAY apply either minimum to Revision 0 strings. A Consumer that cannot support strings up to ℓ_{MAX} MUST support those with ℓ_M of at least 256 bytes; a full implementation supports ℓ_{MAX} .

Display rule, specified (ZIP 316, “Requirements for both Unified Addresses and Unified Viewing Keys”). If the string of a UA or UVK is shown in an abridged form, at least its first 20 characters MUST be shown. The rationale is the ZIP’s (“Rationale for showing at least the first 20 characters”). After the longest Mainnet human-readable part of Revision 0, `uview`, and the separator 1, 14 characters remain, 70 bits of jumbled data at 5 bits per character, below the target of 2^{125} against classical attacks, and multi-target attacks are possible; 20 is an absolute minimum and a wallet may show more. Only a prefix counts, because displays that show different substrings let a user compare only the characters common to all of them, and because checksum characters complicate the analysis of partial matching.

Item sizes. A Sapling or Orchard receiver item has $1 + 1 + 43 = 45$ bytes (typecode, length, 11-byte diversifier and 32-byte transmission key), a P2PKH receiver item $1 + 1 + 20 = 22$ bytes, and the padding 16 bytes. Hence a UA with Sapling and Orchard receivers has $\ell_M = 106$, and one with P2PKH, Sapling and Orchard receivers $\ell_M = 128$ (ZIP 316, “Efficiency”).

Example 3.11 (A unified address, end to end). Let the seed be the 32 bytes $0x00, 0x01, \dots, 0x1f$, on Mainnet, with the Orchard account 9 at $m_{\text{Orchard}}/32'/133'/9'$ (Definition 2.7), the external scope and the index $j = 1$. Every value below is computed by script from the seed.

Receiver. The diversifier key `dk` is the first half of the $0x82$ expansion of the account’s full viewing key, and FF1 encryption of $\text{LE}_{88}(1)$ gives d_1 (Definition 3.1):

<code>dk</code>	82cc9d79742fe5ae9a142b9336a98677b154fe20401eb18998dbed915b0453ce
$\text{LEBS2OSP}_{88}(\text{LE}_{88}(1))$	01000000000000000000000000000000
d_1	3fadf8edb20a3301e8260a
pk_d^*	a311f4cbd54d7d6a76baac88c244b0b121c6dc22a8bcce15898e267829fc1e01

Decryption $\text{FF1-AES256}.\text{Decrypt}(\text{dk}, "", d_1)$ returns $\text{LE}_{88}(1)$, so $j = 1$, and the ownership test $\text{pk}_d = [\text{ivk}] \text{DiversifyHash}^{\text{Orchard}}(d_1)$ of Lemma 3.3 holds for the external `ivk`. It fails for the internal `ivk`_{int}, although `dk`_{int} also decrypts d_1 , to the index 184814530803446824004762235; recovery alone proves nothing, and the test on pk_d decides. The neighbouring diversifiers are $d_0 = \text{e340636542ece1c81285ed}$ and $d_2 = \text{987fd74a2256c596a66f83}$; that consecutive indices give unrelated strings illustrates the *Ironwood Guide*’s Assumption “FF1-AES256 as a pseudorandom permutation” (§“Diversified addresses”) and is not evidence for it.

Container. The UA has one item, so its raw encoding (Definition 3.7) is the header $\text{compactSize}(0x03) \parallel \text{compactSize}(43) = 032b$ followed by the 43-byte receiver $d_1 \parallel \text{pk}_d^*$, 45 bytes in all. The padding is 75 (u) followed by fifteen zero bytes, so M has $\ell_M = 61$, $\ell_L = 30$ and $\ell_R = 31$. The cells of Construction 3.8:

$a = M[0..30]$	832b3f3adf8edb20a3301e8260aa311f4cbd54d7d6a76baac88c244b0b121
$b = M[30..61]$	c6dc22a8bccce15898e267829cf1e8175000000000000000000000000000000
$x = b \oplus G_0(a)$	e6f1529b83a0ad0a4c209da8e4365ca14a598bbf1d03084a1c326f929c574ca
$y = a \oplus H_0(x)$	73b358eb784bb2315fd35c595628dca9a8045ef217278dd0cbf84dc81a
$d = x \oplus G_1(y)$	ad26b7f9a43e09cbf318ef84f2ad2b423ecba017191704643d64be4c76daab6b
$c = y \oplus H_1(d)$	98979f7b8d2dfc7b161a8d29975f87878171b6cf33b303f770b4aa64

The header, the diversifier and the padding are legible in a and b , and in neither c nor d .

String. The 61 bytes of $c \parallel d$ regroup into 98 five-bit symbols, the last padded with two zero bits. With the human-readable part u , the separator 1 and the 6-symbol checksum, the string has $1 + 1 + 98 + 6 = 106$ characters:

u1nztelxna9h7w0vtpd2xjhxt4lpu8s9cmdl8n8vcr7actf2ny45nd07cy8cyuhuvw3axcp545y0ktq9cezuzx84jyhex8dk4tdvwhu4dl

Decoding reverses each stage of Construction 3.10; $\text{F4Jumble}^{-1}(c \parallel d) = \mathcal{M}$. The string equals the unified-address test vector for account 9 and index 1 of the zcash-test-vectors suite, which ZIP 32, “Test Vectors”, names.

3.5 Unified viewing keys

Definition 3.12 (Unified full and incoming viewing keys). A UFVK and a UIVK are containers of Definition 3.7 under the string encoding of Construction 3.10 with their own human-readable parts; a Consumer **MUST** use the decoding procedure of Construction 3.10. Their items, at the receiver typecodes, are the following.

1. Orchard, $0x03$: the 96-byte raw full viewing key, respectively the 64-byte raw incoming viewing key, of Definition 3.6.
2. Sapling, $0x02$: in a UFVK the 128 bytes

EncodeExtFVKParts(ak, nk, ovk, dk)

$$:= \text{LEBS2OSP}_{256}(\text{repr}_1(\text{ak})) \parallel \text{LEBS2OSP}_{256}(\text{repr}_1(\text{nk})) \parallel \text{ovk} \parallel \text{dk}$$

of ZIP 32, “Sapling helper functions”, which SHOULD be derived from the account-level extended full viewing key; in a UIVK the 64 bytes $dk \parallel I2LEOSP_{256}(ivk)$, valid only if $ivk \in \{1, \dots, 2^{251} - 1\}$ (protocol specification, §“Sapling Incoming Viewing Keys”; ZIP 316 notes the exclusion of 0). The Sapling UIVK item carries dk beyond the specification’s Sapling incoming viewing key, which is ivk alone (protocol specification, §“Sapling Incoming Viewing Keys”); dk maps an index to a diversifier and back, so a UIVK yields addresses at indices (Construction 3.16) and recovers the index of a diversifier (Lemma 3.3).

3. Transparent P2PKH, $0x00$: the 65 bytes $c \parallel \text{ser}_p(\text{pk})$ of a 32-byte BIP 32 chain code and a 33-byte compressed public key, the last 65 bytes of the BIP 32 extended public key serialisation (BIP 32, “Serialization format”). The UFVK item is the key at $m/44'/\text{coin_type}'/a'$, the account level of BIP 44, and the UIVK item its external (non-change) child at $m/44'/\text{coin_type}'/a'/0$.

The current viewing-key types reuse the receiver typecodes; future types MAY use others. The Priority List does not govern viewing keys: a Consumer normally takes the union of the information of all items (ZIP 316, “Encoding of Unified Full/Incoming Viewing Keys”).

The transparent item omits the BIP 32 depth, parent fingerprint and child number; the format is ZIP 316’s. That the child number of the account-level key would reveal the account index is this volume’s inference, not a statement of the ZIP. The Sapling UFVK item carries no chain code, so a UFVK supports no non-hardened Sapling child derivation below the account.

Construction 3.13 (Attenuation of a UFVK to a UIVK). Item by item, with the typecode preserved:

1. a Sapling full viewing key to its incoming viewing key (protocol specification, §“Sapling Key Components”), with dk carried over;
2. an Orchard full viewing key (ak, nk, r_{ivk}) to the external-scope incoming viewing key (dk, ivk) (*Ironwood Guide*, §“Viewing keys”);
3. a transparent P2PKH account key to its child at non-hardened index 0 by BIP 32 public-to-public derivation (BIP 32, “Public parent key $\rightarrow$ public child key”).

Items with unrecognised typecodes, and items of experiments the user has not opted into, MUST be dropped (ZIP 316, “Deriving a UIVK from a UFVK”). The treatment of metadata items is stated with them, in §3.7.

Assumption 3.14 (Independence of the per-protocol key trees). For a wallet seed S (Definition 2.1) and an account index a , the Orchard account key of Definition 2.7, the Sapling extended spending key at $m_{\text{Sapling}}/32'/\text{coin_type}'/a'$ of ZIP 32 and the BIP 32 extended private key at $m/44'/\text{coin_type}'/a'$ are jointly computationally indistinguishable from three independently generated keys of the respective protocols: a uniform Orchard spending key with a uniform chain code, a uniform Sapling master-derived key and a uniform BIP 32 key.

The assumption has a heuristic basis and no proof. It follows if BLAKE2b-512 under ZcashIP32Sapling (the Sapling master key) and HMAC-SHA512 under the key Bitcoin seed (the BIP 32 master key) are modelled as random oracles independent of BLAKE2b-512 under ZcashIP32Orchard, and the Sapling and BIP 32 hardened child derivations as PRFs keyed by the parent chain code (HMAC as a PRF: *Crypto Guide*, §“PRFs from hash functions: length extension and keyed BLAKE2”; ZIP 32, “Sapling master key generation” and “Sapling child key derivation”; BIP 32, “Master key generation” and “Private parent key $\rightarrow$ private child key”). Lemma 2.5 proves the Orchard component and nothing more.

Proposition 3.15 (Capabilities of a unified viewing key). *Let the keys of one account be generated from a seed, with use_qsk = false for the Orchard component, and let Assumption 3.14 hold.*

- (a) *From a UFVK the following are computed: for Orchard, the incoming and outgoing viewing keys of both scopes and nk, hence receipt, spend and outgoing detection (Proposition 2.15);*

for Sapling, the same through ZIP 32’s internal key derivation; for transparent keys, every external and internal address and the transparent outgoing viewing keys (Construction 2.18). No secret that is infeasible to compute from its own component becomes feasible from the whole UFVK: in particular the Orchard ask (Ironwood Guide, §“Key capabilities and scope”, Proposition “Capability separation”, part (a)), the Sapling spend authorizing key and a transparent private key.

- (b) A UIVK gives receipt detection for the external scope in each shielded protocol it carries, and no spend detection for its shielded items: nk is infeasible to compute from an Orchard (dk, ivk) (Ironwood Guide, §“Key capabilities and scope”, Proposition “Capability separation”, part (b)), and internal-scope receipt is neither detected (Proposition 2.17) nor derivable (Proposition 2.20). A transparent UIVK item yields the external addresses $m/44'/\text{coin_type}'/a'/0/j$ and not the internal ones. Spends of transparent funds are public on the transparent chain, so their visibility is a property of the chain, not of the key.

Metadata items confer no viewing authority (§3.7), so the statements hold for containers of either revision.

Proof. Componentwise. The computations of (a) and (b) are those of the cited constructions applied to each item. For the separations: by Assumption 3.14 the components are indistinguishable from independently generated keys, so a reduction given one component generates the others itself. An algorithm that outputs a secret of one protocol from the whole container thus yields an algorithm that outputs it from that protocol’s component alone, with the advantage of Assumption 3.14 added; the success of either is efficiently decidable, by recomputing the public key or viewing key from the output. The per-component statements are Propositions 2.15, 2.17 and 2.20 with the *Ironwood Guide*, §“Key capabilities and scope”, Proposition “Capability separation”, under the assumptions named there, for Orchard; ZIP 32 and the protocol specification, §“Sapling Key Components”, cited, for Sapling; and Assumption 2.19 with BIP 32, “Security”, for the transparent keys. The transparent spend clause is a property of transparent inputs, which name the outpoints they spend in the clear. $\square$

3.6 Deriving unified addresses

Construction 3.16 (Unified address at an index). Let U be a UIVK. Choose one index $j \in \{0, \dots, 2^{88} - 1\}$, which MUST be valid for every item of U :

1. for an Orchard item, always;
2. for a Sapling item, if and only if $\text{DiversifyHash}^{\text{Sapling}}(d_j) \neq \perp$, with d_j the diversifier of the item’s dk at j (Definition 3.1);
3. for a transparent P2PKH item, if and only if $0 \leq j \leq 2^{31} - 1$ and BIP 32 derivation of the child at index j succeeds; it fails with probability below 2^{-127} (BIP 32, “Public parent key $\rightarrow$ public child key”).

Each receiver is derived from its item at the same j : a Sapling or Orchard receiver as the address at index j of (dk, ivk) (protocol specification, §“Sapling Key Components” and §“Orchard Key Components”; *Ironwood Guide*, §“Diversified addresses”, Construction “Diversified address”), and a transparent P2PKH receiver as the hash160 of the non-hardened BIP 32 public child j of the external key, so that its path is $m/44'/\text{coin_type}'/a'/0/j$ (BIP 44). Items with unrecognised typecodes, and items of experiments the user has not opted into, MUST be dropped; metadata items are treated in §3.7 (ZIP 316, “Deriving a Unified Address from a UIVK”).

Proposition 3.17 (Indices valid for a unified key). *Let $V(U)$ be the set of indices valid for every item of a UIVK U .*

- (a) *If U has only an Orchard item, $V(U) = \{0, \dots, 2^{88} - 1\}$.*
- (b) *If U has a transparent item, $V(U) \subseteq \{0, \dots, 2^{31} - 1\}$, a fraction $2^{31}/2^{88} = 2^{-57}$ of the index space, so U has at most 2^{31} unified addresses.*
- (c) *Let U have a Sapling item, and let BLAKE2s-256 in $\text{GroupHash}^{\text{J}(\cdot)*}$ be modelled as a random oracle, the model of the protocol specification, §“Group Hash into Jubjub”, notes. The events that the indices are valid for the Sapling item are independent over distinct indices, each of probability*

$$\pi := \frac{h_J (r_J - 1)}{2^{256}} = \frac{8 (r_J - 1)}{2^{256}} \approx 0.4528,$$

with $h_J = 8$ the cofactor and r_J the prime subgroup order of Jubjub (protocol specification, §“Jubjub”); an index is invalid with probability $1 - \pi \approx 0.5472$. ZIP 32’s “approximately half” and ZIP 316’s probability $1/2$ are this value rounded.

- (d) *With a Sapling and a transparent item, $V(U)$ is the set of indices below 2^{31} that are Sapling-valid and at which BIP 32 derivation succeeds; its expected size lies within $2^{31} \cdot 2^{-127} = 2^{-96}$ of $\pi \cdot 2^{31}$.*

For the Sapling key of account 9 of the seed of Example 3.11, 4527 of the indices $0, \dots, 9999$ are valid, a fraction 0.4527, and index 0 is valid.

Proof. The set $V(U)$ is the intersection of the per-item valid sets of Construction 3.16, which gives (a) and (b), distinct indices giving distinct receivers.

(c) Distinct indices give distinct diversifiers (Proposition 3.2 (a)), hence distinct inputs to BLAKE2s-256, whose outputs h are then independent and uniform on $\{0, 1\}^{256}$. The group hash returns a point if and only if $P := \text{abst}_J(h) \neq \perp$ and $[h_J]P \neq \mathcal{O}$ (protocol specification, §“Group Hash into Jubjub”). The map abst_J reads h as a 255-bit integer $v < q_J$ and a sign bit $\tilde{u}$, rejecting $v \geq q_J$ and v with no curve point (protocol specification, §“Jubjub”). Each point with $u \neq 0$ is the image of exactly one h , since u and $q_J - u$ have different parities, and each of the two points $(0, \pm 1)$ is the image of two. The curve has $h_J r_J$ points and its h_J -torsion subgroup, the kernel of $[h_J]$, has 8 points: $(0, 1)$, $(0, -1)$ and six with $u \neq 0$. The strings h for which the hash succeeds are thus in bijection with the points with $u \neq 0$ outside that subgroup, of which there

are $b_j r_j - 2 - 6 = 8(r_j - 1)$, so $\pi = 8(r_j - 1)/2^{256}$.

(d) Parts (b) and (c) and Construction 3.16 (3) combined by linearity of expectation over the 2^{31} indices below 2^{31} , each of which fails BIP 32 derivation with probability below 2^{-127} . $\square$

Remark 3.18 (Default addresses). ZIP 316 fixes neither an order in which a wallet searches for a valid index nor a default unified address. The only normative defaults are ZIP 32’s per-protocol default diversifiers: for Sapling d_j with j the least valid index, for Orchard d_0 (ZIP 32, “Sapling diversifier derivation” and “Orchard diversifier and OVK derivation”). Issuing the unified address at the least index valid for every item is designed but unspecified (Definition 1.1).

Index 0 and recovery across wallets, specified (ZIP 316, “Deriving a Unified Address from a UIVK”). Index 0 is invalid for a Sapling item with probability $1 - \pi$ (Proposition 3.17), so a wallet that requires a Sapling receiver often issues its first unified address at a later index, while some wallets, earlier than ZIP 316 or without a Sapling component in their unified addresses, always generate a transparent P2PKH address at index 0. Therefore every Zcash wallet, whether or not it supports unified addresses, **MUST** assume that transparent funds may exist at diversifier index 0 of each ZIP 32 account, even where it would never generate a unified address at that index; reliable recovery of funds when key material moves between wallets depends on this. Its use in restoration is stated in §6.2.

3.7 Revision 2 containers

Definition 3.19 (Metadata items). The following is Revision 2 of ZIP 316, specified (Definition 1.1).

1. Human-readable parts. A UA is either *shielded-only*, with human-readable part zu , which **MUST NOT** hold a transparent item (typecode $0x00$ or $0x01$), a Consumer rejecting any that does, or *transparent-enabled*, with tu . A UFVK uses uvf and a UIVK uvi ; on Testnet test is appended. The human-readable part distinguishes a Revision 0 container ($u, uviv, uivk$) from a Revision 2 one.
2. Metadata items, typecodes $0xC0$ to $0xFC$ (Definition 3.7), **MAY** affect the interpretation of the container, **MUST NOT** be selected as receivers and **MUST NOT** confer viewing authority.
3. **MUST-understand** items. Typecodes $0xE0$ to $0xFC$ are **MUST-understand**: a Consumer that does not recognise such an item **MUST** regard the whole container as unsupported and not process it further. This is the exception to the ignore rule of Definition 3.7.
4. Registry. Typecodes $0x04$ to $0xBF$ are unassigned; $0xC0$ to $0xDF$ are metadata that need not be understood, none assigned; $0xE0$ is the Address Expiry Height and $0xE1$ the Address Expiry Time, permitted in zu and tu addresses and not in Revision 0; $0xE2$ to $0xFC$ are unassigned **MUST-understand** metadata; $0xFD$ to $0xFFF9$ are unassigned; and $0xFFFA$ to $0xFFFF$ are experimental. Producers **MUST NOT** include items whose typecodes are unassigned or not permitted for the type of the container.

5. Address expiry items. An item of typecode 0xE0 holds the *Address Expiry Height*, an unsigned 32-bit little-endian integer; a UA carrying it MUST be considered expired when the height of the chain exceeds that value. An item of typecode 0xE1 holds the *Address Expiry Time*, an unsigned 64-bit little-endian Unix time; a UA carrying it MUST be considered expired when the current time is after that value. A Producer MAY include either, both or neither.
6. Retroactive rule. A Revision 0 container MUST NOT hold a MUST-understand metadata item, and a Consumer MUST reject one that does.
7. Derivation. To derive a UIVK from a UFVK, or a UA from a UFVK or UIVK, a Consumer MUST understand every MUST-understand item of the source. The derived container MUST be Revision 2 if the source holds any metadata item, and SHOULD be Revision 2 if the source is. A UIVK derived from a UFVK MUST retain its 0xE0 and 0xE1 items unmodified. A UA derived from a UFVK or UIVK that carries 0xE0 (respectively 0xE1) MUST carry 0xE0 (respectively 0xE1) with a value at most that of the viewing key. Metadata items that need not be understood and are unrecognised are dropped under Constructions 3.13 and 3.16.
8. Containment. Revision 2 drops the rule that a UVK holds a shielded item, and admits a tu UA that consists of one P2PKH receiver (Definition 3.7). Revision 2 also admits a P2SH viewing-key item, which lies outside this volume; Table 3 is that of Revision 0.

Sources: ZIP 316, “Revisions”, “Metadata Items”, “Address Expiration Metadata”, “Typecode Registry”, “Requirements for both Unified Addresses and Unified Viewing Keys”, “Deriving a UIVK from a UFVK” and “Deriving a Unified Address from a UIVK”.

A Consumer that predates Revision 2 does not recognise the Revision 2 human-readable parts. Requiring MUST-understand items to appear only under those parts therefore enforces the MUST-understand rule even against such a Consumer (ZIP 316, “Rationale for making Revision 0 UA/UVKs with MUST-understand Typecodes invalid”).

Sender rules for expiring addresses, specified (ZIP 316, “Address Expiration Metadata” and its “Rationale”). The expiry height of a transaction is its field `nExpiryHeight` (*Ironwood Guide*, §“The transaction format”). A Sender supporting Revision 2 MUST set a nonzero `nExpiryHeight` in a transaction to a UA with an Address Expiry Height, and MUST NOT send the transaction if the `nExpiryHeight` it would otherwise construct exceeds that height; lowering `nExpiryHeight` to the address’s value instead would reveal the destination’s expiry on chain. With only an Address Expiry Time, it SHOULD choose `nExpiryHeight` so that the transaction expires no more than 24 hours after the current time. With both items both restrictions apply, and with several recipient UAs in one transaction all their constraints apply. A wallet MUST NOT send to an address it knows to have expired, and its error SHOULD suggest obtaining a current address. The choice of `nExpiryHeight` is stated in §9.3.

Linkability through metadata (ZIP 316, “Address Expiration Metadata”). Diversified UAs derived from one UIVK that hold only Sapling or Orchard receivers and no metadata are unlinkable (§3.1); with metadata items present this no longer holds, and it is RECOMMENDED that the

expiry height and time of distinct derived addresses vary. Re-issuing the same receivers with different expiry metadata yields a visually distinct address that is directly linkable to the original, since the receivers are shared.

3.8 Recipient addresses

Definition 3.20 (Recipient address). A *recipient address* is one of the following:

1. a transparent address, P2PKH or P2SH, in its Base58Check encoding (Definition 3.6 (f));
2. a Sapling payment address in its Bech32 encoding (Definition 3.6 (e));
3. a unified address (Definition 3.7, Construction 3.10 and Definition 3.19);
4. a *TEX address* (transparent-source-only address): the Bech32m encoding, with human-readable part *tex* on Mainnet and *textest* on Testnet, of the 20-byte validating-key hash of a P2PKH address. It is obtained from a Base58Check P2PKH address whose decoding has 22 bytes beginning with the lead bytes [0x1C, 0xB8] (Testnet [0x1D, 0x25]) by removing them, and it is parsed by requiring the human-readable part and exactly 20 decoded bytes (ZIP 320, “Specification”).

Sprout addresses are not recipient addresses: no transaction adds value to the Sprout pool, and no typecode exists for them (§14.2; ZIP 316, “Requirements for both Unified Addresses and Unified Viewing Keys”). A Sender paying a TEX address, as any output of a transaction, **MUST** spend only transparent (P2SH or P2PKH) outputs in that transaction, and **SHOULD** send only to transparent outputs (ZIP 320, “Specification”; §11.3).

Two predicates on recipient addresses, defined here, are used by payment construction and by ZIP 321 (ZIP 321, “memo”; §14.2).

- (i) An address is *memo-capable* if and only if it is a Sapling payment address or a unified address that contains a Sapling or Orchard receiver, since a memo travels only in a note plaintext (*Ironwood Guide*, §“The note plaintext”). A memo reaches the recipient of a unified address only when the Sender selects a shielded receiver.
- (ii) An address is *transparent-only* if and only if it is a transparent or TEX address, or a unified address whose non-metadata items are all transparent receivers, which is possible only as a Revision 2 *tu* address.

4 Compact blocks (ZIP 307)

This section states the compact block as an abstract message format: the records of a block and of a transaction, the compact outputs, spends and Actions and the fields they omit, the compact variant of trial decryption with the soundness of accepting a plaintext that no tag authenticates, and the placement of notes in the note commitment trees from per-block tree sizes. The service that delivers compact blocks is the subject of “The light-client service” (§5).

4.1 The compact block and the compact transaction

A *compact block* carries only the data of a block that a wallet needs to detect payments to its addresses, to detect spends of its notes and to maintain the witnesses of its notes (ZIP 307, “Compact Stream Format”); for the transparent pool it carries, on request, the data needed to detect spends of the wallet’s transparent coins.

Remark 4.1 (Normative status of the compact format). ZIP 307, “Compact Stream Format”, “Output Compression” and “Spend Compression”, specifies the compact format for the Sapling pool only: a compact block of height, block hash and transactions, and a compact transaction of index, transaction identifier, spends and outputs. It names neither the Orchard pool nor the Ironwood pool, nor per-block tree sizes, nor transparent data. ZIP 318 and ZIP 326 address light wallets and specify no compact format. The remainder of the format below (the parent hash of a block, the tree sizes, the compact Actions of both Orchard-protocol pools, the transparent part and pool selection) is designed but unspecified (Definition 1.1). The volume states the format as an abstract message format, not as a serialisation, and states no field number, field width or encoded size. The status of each ZIP cited is that of Table 1.

Definition 4.2 (Compact block). A *compact block* of height h is a tuple

$$CB = (h, \text{hash}, \text{prevHash}, \text{hdr}, \text{vtx}, S)$$

where

1. $h \in \mathbb{N}$ is the block height;
2. hash and prevHash are the 32-byte hashes of the block and of its parent block (protocol specification, §“Block Header Encoding and Consensus”);
3. hdr is either the encoded block header or empty; the abstract format does not guarantee that it is present;
4. $\text{vtx} = (ctx_0, \dots, ctx_{m-1})$ is the sequence of compact transactions (Definition 4.3) of the block, in block order;
5. $S = (S^{\text{Sapling}}, S^{\text{Orchard}}, S^{\text{Ironwood}})$ gives, for each shielded pool pool, the number S^{pool} of leaves of that pool’s note commitment tree after every transaction of the block has been appended, written $S_{\text{end}}^{\text{pool}}(h)$.

Each S^{pool} is an integer in $[0, 2^{32}]$, where 2^{32} is the leaf capacity of a tree of depth $\text{MerkleDepth}^{\text{Sapling}} = \text{MerkleDepth}^{\text{Orchard}} = 32$ (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Constants”). The width of any serialisation of S^{pool} lies outside the abstract format.

Definition 4.3 (Compact transaction). A *compact transaction* is a tuple

$$ctx = (i, \text{txid}, SSp, SOut, A^{\text{Orchard}}, A^{\text{Ironwood}}, T)$$

where

1. $i \in \mathbb{N}$ is the index of the transaction in its block;
2. txid is the 32-byte transaction identifier in protocol byte order, the form in which it is computed, not the byte-reversed form used for display (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”; protocol specification, §“Transaction Identifiers”);
3. SSp and $SOut$ are the sequences of compact Sapling spends and compact Sapling outputs, and A^{Orchard} and A^{Ironwood} the sequences of compact Actions of the Orchard pool and of the Ironwood pool (Definition 4.5), each in the order of the transaction’s descriptions, on which note positions depend (§4.4);
4. T is an optional *transparent part* ($O_{\text{in}}, O_{\text{out}}$): O_{in} is the sequence of outpoints (txid', n) spent by the transaction’s transparent inputs, in input order, and O_{out} the sequence of pairs (value, script) of its transparent outputs, in output order, with value in zatoshi.

The coinbase transaction of a block is its first transaction and the one whose single transparent input has a null previous outpoint (protocol specification, §“Coinbase Transactions”); it spends no outpoint, its O_{in} is empty, and it is recognised by $i = 0$. No fee field is part of the abstract format.

Definition 4.4 (Pool selection). Let $\mathcal{P} \subseteq \{\text{transparent, Sapling, Orchard, Ironwood}\}$. The compact block *restricted to* $\mathcal{P}$ is obtained from a compact block by keeping, in each compact transaction, only the components of the pools in $\mathcal{P}$ (SSp and $SOut$ for Sapling, the sequence A^{pool} for an Orchard-protocol pool, and T only when $\text{transparent} \in \mathcal{P}$), and by omitting every compact transaction left with no component. The tuple S is kept whole, and a compact block is returned for every block of a requested range, possibly with empty vtx. The default is $\mathcal{P} = \{\text{Sapling, Orchard, Ironwood}\}$. The range query of “The query interface” (§5.2) takes $\mathcal{P}$ as an argument. Pool selection is designed but unspecified (Definition 1.1); ZIP 307 includes by default only the transactions that have Sapling spends or outputs (ZIP 307, “Block header validation”).

For every shielded pool in $\mathcal{P}$ the restriction keeps every commitment of that pool in its order, since an omitted compact transaction has no component of the pool. The per-pool counts and offsets of Definition 4.11 are therefore the same on a restricted block as on the unrestricted one.

4.2 Compact outputs, spends, and Actions

Definition 4.5 (Compact spend, output, and Action). Let C_{enc} be the note ciphertext of a Sapling output or of an Action: 580 bytes, the encryption of the 564-byte note plaintext followed by the 16-byte tag. Its split at the offsets 52 and 564 is that of the *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”.

- (a) A *compact Sapling spend* is the 32-byte nullifier nf of a Spend description.
- (b) A *compact Sapling output* is the triple $(\text{cmu}, \text{ephemeralKey}, C_{\text{enc}}[0:52])$, a payload of $32 + 32 + 52 = 116$ bytes.

- (c) A *compact Action*, of the Orchard pool and of the Ironwood pool alike, is the tuple $(nf, cmx, ephemeralKey, C_{enc}[0:52])$, a payload of $32 + 32 + 32 + 52 = 148$ bytes.

Parts (a) and (b) are specified (ZIP 307, “Output Compression” and “Spend Compression”); part (c) is designed but unspecified (Definition 1.1). The fields are those of the protocol specification, §“Spend Description Encoding and Consensus”, §“Output Description Encoding and Consensus” and §“Action Description Encoding and Consensus”. Only payload sizes are stated; the size of any serialisation lies outside the abstract format.

The nullifier nf of a compact Action has two roles. It detects a spend of the note that the Action consumes, and it is the element ρ of the note that the Action creates (*Ironwood Guide*, §“Nullifier chaining”, Construction “Chaining rule”), an input of esk , ψ and rcm and hence of the commitment that acceptance recomputes (Definition 4.9). A compact Action without nf therefore cannot be accepted.

A compact block omits the memo $C_{enc}[52:564]$, the tag $C_{enc}[564:580]$, the outgoing ciphertext C_{out} , value commitments, randomised validating keys, anchors, proofs and signatures (ZIP 307, “Output Compression”, “Spend Compression” and “Transaction privacy”). Recovery of a memo or of an outgoing plaintext requires the full transaction, fetched by its identifier (“Leakage of transaction queries”, §8.4).

Proposition 4.6 (Sufficiency of the compact fields). *Let CB be a compact block of height h , and let a wallet hold incoming viewing keys and the nullifiers of its notes.*

- (i) *Receipt detection, that is compact decryption and acceptance of an output (§4.3), is a function of the key, of h , and of the record $(cmu, ephemeralKey, C_{enc}[0:52])$ of a Sapling output or $(nf, cmx, ephemeralKey, C_{enc}[0:52])$ of an Action.*
- (ii) *Spend detection is a function of the fields nf of SSp , A^{Orchard} and A^{Ironwood} .*
- (iii) *Note placement is a function of the order of the commitments in vtx and of S (§4.4).*

No field omitted by the compact block enters (i) to (iii).

Proof. (i) Trial decryption and acceptance read the ephemeral key, the ciphertext prefix that encrypts the lead byte, diversifier, value and note seed, the commitment, the height through the admissible lead bytes, and, for an Action, nf as ρ (*Ironwood Guide*, §“Trial decryption and note acceptance”, Constructions “Trial decryption” and “Acceptance of an Ironwood-pool output”; ZIP 307, “Scanning for relevant transactions” for Sapling); the memo and the tag enter only the full decryption. (ii) A spend of a note is detected by membership of its nullifier among the published ones (ZIP 307, “Detecting spends”). (iii) The chain appends each pool’s commitments in block, transaction and description order to the least unused leaf (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Note Commitment Trees”), so positions follow from that order and a tree size. $\square$

The omitted fields serve the verification of the chain, which the client delegates to the server; that delegation and its assumption are stated in “Architecture and authority” (§5.1).

4.3 Compact trial decryption

Trial decryption and acceptance of full outputs are those of the *Ironwood Guide*, §“Trial decryption and note acceptance” (Constructions “Trial decryption” and “Acceptance of an Ironwood-pool output”), cited and not re-derived. This subsection states the compact variant.

Definition 4.7 (Compact note plaintext). The *compact note plaintext* is the tuple $\text{np}_c = (\text{leadByte}, d, v, \text{rseed})$, encoded as

$$\text{leadByte} \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed},$$

of $1 + 11 + 8 + 32 = 52$ bytes: the note plaintext of the *Ironwood Guide*, §“The note plaintext”, Definition “Note plaintext”, without its memo (protocol specification, §“Note Plaintexts and Memo Fields” and §“Encodings of Note Plaintexts and Memo Fields”). The AEAD encryption is a stream-cipher encryption followed by a tag, so $C_{\text{enc}}[0:52]$ is the encryption of exactly these 52 bytes, which are the contents and the opening of the note commitment (ZIP 307, “Output Compression”).

Construction 4.8 (Compact decryption of an output). Input: an incoming viewing key ivk of the pool’s protocol and a compact output or compact Action with fields ephemeralKey and $c = C_{\text{enc}}[0:52]$.

1. Compute $\text{epk} := \text{abst}(\text{ephemeralKey})$ on the pool’s curve, abst_J on Jubjub for Sapling and abst_P on Pallas for either Orchard-protocol pool; return $\perp$ if $\text{epk} = \perp$.
2. Compute $K := \text{KDF}(\text{KA.Agree}(\text{ivk}, \text{epk}), \text{ephemeralKey})$ with the key agreement and key derivation of the pool’s protocol.
3. Compute $\text{np}_c := c \oplus \text{KS}_1(K, 0^{96}, 52)$, where $\text{KS}_1(K, 0^{96}, 52)$ is the first 52 bytes of the ChaCha20 keystream under key K and the all-zero nonce 0^{96} of the protocol specification, §“Symmetric Encryption”, starting at block counter 1, the counter from which ChaCha20-Poly1305 encrypts (block 0 yields the Poly1305 key; *Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Construction “ChaCha20-Poly1305”; RFC 8439, §2.8); equivalently, bytes 64 to 115 of the keystream from counter 0.
4. Return np_c parsed as a compact note plaintext (Definition 4.7).

The tag is never checked. The result is a *candidate*, not an accepted note (ZIP 307, “Scanning for relevant transactions”).

Definition 4.9 (Acceptance of a compact candidate). Let $\text{np}_c = (\text{leadByte}, d, v, \text{rseed})$ be the candidate returned by Construction 4.8 under ivk for an output of pool pool in a compact block of height b . Let H_C be the Canopy activation height and $\text{ZIP212GracePeriod} = 32,256$ blocks

(protocol specification, §“Constants”). The set of admissible lead bytes is

$$\text{allowedLeadBytes}^{\text{pool}}(b) := \begin{cases} \{0x01\} & \text{if } b < H_C, \\ \{0x01, 0x02\} & \text{if } H_C \leq b < H_C + \text{ZIP212GracePeriod}, \\ \{0x02\} & \text{otherwise, if pool} \neq \text{Ironwood}, \\ \{0x03\} & \text{otherwise} \end{cases}$$

(protocol specification, §“Note Plaintexts and Memo Fields”), so that an Orchard-pool note has lead byte 0x02 and an Ironwood-pool note 0x03. The wallet accepts np_c if and only if every check passes:

- (i) $\text{leadByte} \in \text{allowedLeadBytes}^{\text{pool}}(b)$; for Sapling, moreover, $\text{rcm} < \eta_j$ and $\text{g}_d := \text{DiversifyHash}(d) \neq \perp$, with $\text{rcm} := \text{rseed}$ read as an integer for lead byte 0x01;
- (ii) with $\text{pk}_d := [\text{ivk}] \text{g}_d$, and with $\rho := \text{nf}$ of the same Action for either Orchard-protocol pool, the note commitment recomputed from $(\text{g}_d, \text{pk}_d, v, \rho, \psi, \text{rcm})$, where ψ and rcm are derived from rseed as the pool and the lead byte prescribe (ρ and ψ absent for Sapling), is not $\perp$ and extracts to the record’s cmu (Sapling) or cmx (either Orchard-protocol pool);
- (iii) for every lead byte other than 0x01, the field ephemeralKey equals the encoding of $[\text{esk}] \text{g}_d$, with esk re-derived from rseed , and from ρ for either Orchard-protocol pool.

The accepted note is $n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ without memo, with ρ and ψ absent for a Sapling note. The derivations of ψ , rcm and esk are cited and not restated: for Sapling and the Orchard pool to the protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)” (ZIP 212, “Changes to the process of sending Sapling or Orchard notes”); for the Ironwood pool to the *Ironwood Guide*, §“Trial decryption and note acceptance”, Construction “Acceptance of an Ironwood-pool output”, whose checks (i), (ii) with (iv), and (iii) are checks (i), (ii) and (iii) here (ZIP 2005, “Changes to the Protocol Specification”). The method of comparison is not part of the rule.

Check (iii) is a step of the specification’s decryption procedure and of ZIP 307, not an addition (ZIP 212, “Changes to the process of receiving Sapling or Orchard notes”). For an Ironwood-pool note it is made directly after the computation of g_d , in the order of ZIP 2005. Without it a sender could agree the key with one address of a recipient while naming the diversifier of another, and link the two addresses by observing acceptance (*Ironwood Guide*, §“Trial decryption and note acceptance”, Remark “Role of check (iii)”); ZIP 212, “Motivation”). A Sapling note with lead byte 0x01, created under the rules before ZIP 212, skips check (iii), since its esk is not derivable from rseed .

For a note n the *commitment input* is the pair of committed values and trapdoor from which check (ii) computes its commitment: $((\text{g}_d, \text{pk}_d, v), \text{rcm})$ for Sapling and $((\text{g}_d, \text{pk}_d, v, \rho, \psi), \text{rcm})$ for either Orchard-protocol pool. An *address of ivk* is a pair $(d, [\text{ivk}] \text{DiversifyHash}(d))$ with $\text{DiversifyHash}(d) \neq \perp$.

Proposition 4.10 (Soundness of compact acceptance). *Let cm^* be the commitment, cmu or cmx, that a compact output or compact Action carries.*

- (a) *Every note n accepted by Definition 4.9 is an opening of cm^* . For every efficient sender, which chooses ephemeralKey, C_{enc} and the keys under which they are formed, the probability that the wallet accepts n while the sender outputs a note n' whose commitment input differs from that of n and whose commitment extracts to cm^* is negligible, under the binding of the pool's note commitment scheme. The tag of the AEAD gives no integrity against the sender, who holds K , so its omission loses none.*
- (b) *For an output honestly formed for a note n' whose address is not an address of ivk, the probability that ivk accepts a candidate is negligible under the same assumption.*

Whether cm^ is the commitment of an output of the best chain is not a property of decryption; it is settled under the assumption of “Architecture and authority” (§5.1).*

Proof. (a) Check (ii) recomputes the commitment of n deterministically from its commitment input and compares it with cm^* , whatever the ciphertext and whichever key decrypted it, so n opens cm^* . Consider the algorithm that generates the wallet's key, runs the sender and the compact decryption and acceptance, and outputs the commitment inputs of n and n' . When the event of (a) occurs it outputs two distinct commitment inputs whose commitments extract to the same $cm^* \neq \perp$. For either Orchard-protocol pool this is excluded, except with negligible probability, by the argument of the *Ironwood Guide*, §“Trial decryption and note acceptance”, Proposition “Consistency of accepted notes”, part (c), under Proposition “Hiding and binding of note commitments” (§“Note commitments”). That proof uses neither the integrity of ChaCha20-Poly1305 nor key commitment, so it applies unchanged to a candidate decrypted from the 52-byte prefix; and binding holds for every rcm, hence also for the trapdoor derivation of the Orchard pool. For Sapling the named assumption is the computational binding of NoteCommit^{Sapling}, the security requirement of the protocol specification, §“Windowed Pedersen commitments”, together with the injectivity of Extract_{J(r)} on $J^{(r)}$ (protocol specification, §“Coordinate Extractor for Jubjub”).

(b) Let n be accepted. By check (ii) its address is $(d, [ivk] g_d)$ with $g_d = \text{DiversifyHash}(d) \neq \perp$, an address of ivk. Suppose the commitment inputs of n and n' were equal. Then the diversified base $\text{DiversifyHash}(d')$ of the address (d', pk_d') of n' equals g_d , and $pk_d' = [ivk] g_d = [ivk] \text{DiversifyHash}(d')$, so (d', pk_d') is an address of ivk, contrary to the hypothesis. The commitment inputs therefore differ, and the honest sender outputs n' , whose commitment extracts to cm^* ; the honest sender is an efficient sender of (a), which bounds the probability. $\square$

4.4 Note positions and tree sizes

For each shielded pool, every note commitment of a block, not only those of the wallet's notes, is appended to the wallet's copy of that pool's note commitment tree in block order, transaction order and description order: cmx of each Action of the pool and cmu of each Sapling output (ZIP 307, “Creating and updating note witnesses”; *Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Note Commitment Trees”). The

leaf of each accepted note is marked for witness retention. The part of the tree a wallet must retain is fixed in “Commitment-tree synchronisation” (§7).

Definition 4.11 (Note position). Let pool be a shielded pool, b a block height with compact block CB , and $n_{\text{pool}}(b)$ the number of commitments of pool in CB . The *start size* of b is

$$S_{\text{start}}^{\text{pool}}(b) := \begin{cases} S_{\text{end}}^{\text{pool}}(b-1) & \text{if the wallet knows it,} \\ 0 & \text{otherwise, if } b \text{ lies below the activation of pool,} \\ S_{\text{end}}^{\text{pool}}(b) - n_{\text{pool}}(b) & \text{otherwise,} \end{cases}$$

the wallet knowing $S_{\text{end}}^{\text{pool}}(b-1)$ when it holds the compact block of height $b-1$ or a tree state of pool at the end of $b-1$; in the third case the value is taken from CB alone, and the block is rejected if it is negative. For the j -th output o of pool, counted from $j = 0$, of a compact transaction ctx of b ,

$$\text{position}(o) := S_{\text{start}}^{\text{pool}}(b) + \text{off}(ctx) + j,$$

where $\text{off}(ctx)$ is the number of commitments of pool in the compact transactions of b before ctx . Positions are per pool. A block scanned out of order thus places its notes from its own compact block. Note positions are designed but unspecified (Definition 1.1).

For a pool outside the pool set $\mathcal{P}$ of the query that returned b , the count $n_{\text{pool}}(b)$ is not served, and neither position nor check is defined.

The position of a note is the leaf index of its commitment (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”, item 1). Every spend needs it for the authentication path. A Sapling nullifier depends on it, through $\rho = \text{MixingPedersenHash}(\text{cm}, \text{pos})$ (protocol specification, §“Computing ρ values and Nullifiers”), while the nullifier of an Orchard-protocol note does not.

Lemma 4.12 (Consistency of tree sizes). *Let the wallet reject a block b unless, for each shielded pool of the pool set $\mathcal{P}$ of the query that returned b ,*

$$S_{\text{end}}^{\text{pool}}(b) \geq n_{\text{pool}}(b) \quad \text{and} \quad S_{\text{start}}^{\text{pool}}(b) + n_{\text{pool}}(b) = S_{\text{end}}^{\text{pool}}(b).$$

- (a) *If every reported size is the true size, no block is rejected, and every position of Definition 4.11 is the leaf index of the commitment in the pool’s tree.*
- (b) *A reported $S_{\text{end}}^{\text{pool}}(b)$ that is inconsistent with the block’s own commitment count, or with the known end size of $b-1$, is rejected.*
- (c) *A block with $n_{\text{pool}}(b) > 0$ and reported size 0 is rejected. A size read as zero therefore yields a position only where 0 is consistent with the block, and the check needs no means of telling an absent size from zero.*
- (d) *A size that is false but consistent with both conditions is not detected; its exclusion is clause (a) of the server assumption of “Architecture and authority” (§5.1).*

Proof. (a) The chain appends the commitments of a pool in block, transaction and description order, each to the least unused index (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”). The true start size of b is thus the true end size of $b - 1$, which equals $S_{\text{end}}^{\text{pool}}(b) - n_{\text{pool}}(b) \geq 0$, so both conditions hold in every case of the start size; and the j -th commitment of ctx lands at index $S_{\text{start}}^{\text{pool}}(b) + \text{off}(ctx) + j$. (b) A size $S_{\text{end}}^{\text{pool}}(b) < n_{\text{pool}}(b)$ fails the first condition. With a known start size, a size that differs from $S_{\text{start}}^{\text{pool}}(b) + n_{\text{pool}}(b)$ fails the second. (c) A size 0 with $n_{\text{pool}}(b) > 0$ fails the first condition. (d) Let the wallet not know the end size of $b - 1$, and let b lie at or above the activation of pool, so that its start size is that of the third case of Definition 4.11. Adding one integer $\delta \neq 0$ to $S_{\text{end}}^{\text{pool}}(b)$ and to the end sizes of all later scanned blocks, such that every shifted size stays in $[0, 2^{32}]$ and at least the count of its block, preserves both conditions for b , whose start size is computed from its own end size, and for every later block, whose start and end sizes shift by δ together. $\square$

Comparison of the running position with $S_{\text{end}}^{\text{pool}}(b)$ identifies the last commitment of pool in b , at position $S_{\text{end}}^{\text{pool}}(b) - 1$. The tree after that commitment, or for $n_{\text{pool}}(b) = 0$ the tree at the end of $b - 1$, is the tree state at the end of b . Its root is the anchor that a later spend against height b uses (*Ironwood Guide*, §“Anchors”; protocol specification, §“Transactions and Treestates”), and the wallet records it as a candidate anchor.

5 The light-client service

This section states the server from which a light wallet obtains chain data. It fixes the architecture and the one assumption on the server that the volume uses, states which checks a light client can make on the server’s responses and what the server can fabricate or omit undetected, and defines the service as an abstract interface of typed queries on the chain, the note commitment trees, transactions, submission and transparent addresses.

5.1 Architecture and authority

Definition 5.1 (Light-client architecture). A *light-client system* has three components:

1. a *full node*, which validates the chain and is the root of trust for chain state;
2. a *proxy*, which extracts from the node’s view of the best chain the data of compact blocks (Definition 4.2) and answers queries;
3. a *light client*, which holds keys and no copy of the chain, queries the proxy, and maintains its own view of chain state and of its notes.

This volume calls the node and the proxy together the *server*. The wallet of “Parties, pools, and classifications” (§1.2) is the light client. The best chain is that of the *Consensus Guide*, §“Work, and the best chain” (ZIP 307, “High-Level Design”).

Assumption 5.2 (Honest-but-curious server). The server satisfies two clauses.

- (a) *Integrity*. It answers every query from a correct view of the current best chain, each response being the value that the interface of “The query interface” (§5.2) assigns to the query on that chain, and it transmits queries and responses faithfully.
- (b) *Curiosity*. It records every query with its arguments, order and timing, every response, and the transport metadata of every connection, and computes arbitrarily on that record.

Every correctness result of this volume that relies on the server cites clause (a); the privacy results of §8 take clause (b) as the power of the adversary. No other server model is used. ZIP 307, “Security Model”, states the same model: the node and proxy together are honest but curious, and are trusted to provide a correct view of the current best chain and to transmit queries and responses faithfully. Its goal is payment-detection privacy: the proxy does not learn which transactions are addressed to a given wallet and, given network privacy provided separately, does not link different connections or queries to one wallet; the goal is defined in “Payment-detection privacy” (§8.1). The ZIP does not address private spending.

Proposition 5.3 (Light-client checks). *Let a wallet receive responses of the service (§5.2), with compact blocks that carry no header, which the abstract format does not guarantee (Definition 4.2). Then every check the wallet can make compares two responses, or a response with a value the wallet computes from responses. The relations among response fields that the format fixes are the following:*

- (i) *height continuity and prevHash linkage of consecutive compact blocks, and of the first block of a range with the block the wallet has stored, the checks of ZIP 307, “Client-server interaction”, phases B and D, and “Block header validation”;*
- (ii) *consistency of the per-block tree sizes with the commitments served (Lemma 4.12);*
- (iii) *recomputation of the note commitment of each accepted candidate (Definition 4.9, Proposition 4.10);*
- (iv) *agreement of a served tree node (a frontier, a subtree root, a root) with its recomputation from commitments the wallet has scanned;*
- (v) *agreement of a fetched transaction with its queried identifier and with the compact records of that identifier (Remark “Checks of a fetched transaction”);*
- (vi) *equality of a height, block hash or transaction identifier that several responses report.*

No check relates a response to a block header or to proof of work: the wallet verifies no tree root, nullifier, transaction identifier or block hash against a header.

Proof. The fields of a compact block and of its elements (Definitions 4.2, 4.3 and 4.5) and the responses of the service are values of the chain or tree data derived from it; without `hdr` no response carries a header, so no field, not even hash or `prevHash`, which the header determines, can

be checked against one. A check is a predicate on the wallet’s inputs, which are these responses and its keys, so it compares responses with one another or with values computed from them. Items (i) to (vi) relate pairs of fields that the format makes functions of one another: the heights and hashes of adjacent blocks; a block’s commitment count and its sizes; a record’s commitment and its plaintext; a tree node and the leaves below it; a fetched transaction and its identifier and compact records; and a height, hash or identifier repeated across responses. Proof of work is a relation between a header and its solution and target (*Consensus Guide*, §“Equihash and the memoryless model”), which no response carries. $\square$

ZIP 307, “Block header validation”, proposes checks against a header when a compact block carries one; the abstract format does not guarantee a header, and no result of this volume uses them.

Corollary 5.4 (Fabrication of compact outputs).

- (a) *Under clause (a) of Assumption 5.2, every compact candidate accepted by Definition 4.9 is a note of the best chain: its commitment is a leaf of its pool’s note commitment tree at the position that Definition 4.11 assigns. The fields a compact record omits (proofs, signatures, value commitments, anchors, full ciphertexts) are checked by consensus for every transaction of the best chain, so delegating their verification to the server loses no check a light client could make.*
- (b) *Without clause (a), no check authenticates the compact fields against the chain. A server that knows one of the wallet’s addresses can construct a note to it, compute its commitment and note ciphertext, choose the remaining compact fields (nf of an Action, which is p of the note), and serve the result as a compact output or Action, with tree sizes consistent with it; the result passes Definition 4.9. Unless a transaction of the chain appends the same commitment, no authentication path from it to an anchor of the chain can be found except with negligible probability, so the wallet displays a note whose every spend is rejected.*

Proof. (a) By Proposition 4.10(a) an accepted candidate opens the commitment of its compact record. By clause (a) that record is the corresponding field of a transaction of the best chain, whose commitments are appended to the pool’s tree in the order that fixes the position (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; Lemma 4.12(a)). The omitted fields are validated by the consensus rules for every transaction of the chain (*Ironwood Guide*, §“Transaction digests and signatures” and §“Anchors”).

(b) Acceptance checks only the lead byte, the field ranges, the commitment recomputed from the plaintext and $\text{epk} = [\text{esk}] g_d$ (Definition 4.9), and an honestly constructed note to an address of the wallet satisfies all of them; the tree-size checks of Lemma 4.12 pass on sizes the server chooses consistently, by part (d) of that lemma. A spend proves a valid path from the commitment to an anchor of the chain (*Ironwood Guide*, §“Authentication paths”), and a valid path from a value that is not the leaf at its position yields a collision of the Merkle hash, found with at most negligible probability (*Ironwood Guide*, §“Anchors”, Proposition “Membership soundness”; protocol

specification, §“Merkle Path Validity”, for Sapling). ZIP 307, “Output Compression”, grounds the commitment check on the same trust, in the entity that assembles transactions. $\square$

Remark 5.5 (Omission is undetected). Completeness of the served data is trusted, not verified. The size check of Lemma 4.12 detects the omission of a commitment only while the served sizes are correct. A server that omits compact outputs or Actions and serves tree sizes, frontiers, subtree roots and roots consistent with the omission, or omits or alters revealed nullifiers, of which a compact block carries no count, passes every check of Proposition 5.3. Omitted outputs hide receipts. Omitted nullifiers hide spends: the wallet shows spent notes as spendable, and a re-spend of such a note is rejected for a repeated nullifier (*Ironwood Guide*, §“Nullifier sets”) while it discloses to every party that receives it that it spends what an earlier transaction spent, the leak of repeated spends of ZIP 315, “Obtaining user consent for information leakage”. Only clause (a) of Assumption 5.2 excludes omission (ZIP 307, “Security Model”).

Remark 5.6 (Malformed responses). A response with a field outside its type (a nullifier not of 32 bytes, a non-canonical encoding of a field element, a malformed compact output or Action, a block outside the requested range) is an error of the response: the results of the query are not applied, and the query may be repeated. It is not a continuity error and triggers no rollback (“Reorganisation detection and recovery”, §7.6). This rule is designed but unspecified (Definition 1.1).

Remark 5.7 (Checks of a fetched transaction). A full transaction fetched by its identifier SHOULD be checked against the corresponding compact outputs, in addition to verification of its signatures (ZIP 307, “Block header validation”). The checks are: the identifier computed from the fetched transaction equals the queried one (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”; protocol specification, §“Transaction Identifiers”); its outputs and Actions agree with the compact records; and its signatures verify over its signature digest (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Signature digest”). They bind the full transaction to the compact data and its authorisation to its effects. They bind neither to the chain: compact blocks carry too little to check an identifier against the block’s transaction tree, and that binding rests on clause (a) of Assumption 5.2.

5.2 The query interface

A wallet holds keys and no chain state (Definition 5.1), so the interface delivers what the protocols below consume: every compact output and Action of every shielded pool, for trial decryption (*Ironwood Guide*, §“Trial decryption and note acceptance”); every revealed nullifier, for spend detection (*Ironwood Guide*, §“Nullifier sets”); for each pool, tree sizes, frontiers and the roots of complete subtrees, for positions and authentication paths (*Ironwood Guide*, §“Authentication paths”); the full transactions that concern the wallet; submission of its transactions and their status; and, for transparent receivers, the transparent outputs paid to its addresses.

Definition 5.8 (Subtree root and completing height). Let $\text{pool} \in \{\text{Sapling}, \text{Orchard}, \text{Ironwood}\}$, whose note commitment tree has depth 32 and layers numbered from layer 0, the root, to layer 32, the leaves (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Note Commitment Trees”, for Sapling). For $0 \leq i <$

2^{16} , subtree i of the tree is the subtree whose leaves are the positions $[i \cdot 2^{16}, (i + 1) \cdot 2^{16})$; its root is the node M_i^{16} , sixteen layers above the leaves. When the tree holds n leaves, subtree i is complete if $n \geq (i + 1) \cdot 2^{16}$, after which its root no longer changes; the complete subtrees are those with $i < \lfloor n/2^{16} \rfloor$. The *completing height* of a complete subtree i is the height of the block whose transactions append position $(i + 1) \cdot 2^{16} - 1$.

Definition 5.9 (Transaction status). The *status* of a transaction identifier txid on the server’s best chain, with tip height T , is one of:

1. unknown: the server knows no transaction with identifier txid;
2. unmined: the server knows the transaction, in its mempool or in a block off the best chain, and no block of the best chain contains it;
3. mined(h): the block of the best chain at height $h \leq T$ contains it.

The encoding of the status lies outside the abstract interface. The status is designed but unspecified (Definition 1.1).

Definition 5.10 (Light-client query interface). The *light-client service* is the set of typed queries of Table 4, each answered from the server’s best chain under clause (a) of Assumption 5.2. A *block identifier* is a height or a 32-byte block hash. A *pool set* is a set $\mathcal{P}$ of the pool selection of “The compact block and the compact transaction” (§4.1): shielded pools, with transparent data selectable alongside. The service serves every pool set and transparent data. A query whose argument names no block of the best chain, or a transaction that the server does not know, is answered with $\perp$. Queries of the mempool and of server metadata are not part of the interface.

The frontier in a response of TreeState is that of the *Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Frontier”: for a tree of n leaves, the left-sibling nodes along the path of the next leaf, one for each set bit of n , from which the root and every later append are computed. The empty frontier is the state before the pool’s first commitment. The final treestate of a block is that of the protocol specification, §“Transactions and Treestates” (*Ironwood Guide*, §“Anchors”, Definition “Treestate and anchor”). The serialisation of the frontier is fixed by no ZIP and is not restated. ZIP 307, “Proxy operation”, also identifies a transaction by its block and its index in the block; the interface uses the identifier alone.

Remark 5.11 (Classification of the queries). ZIP 307, “Proxy operation”, specifies the queries Tip, Block, Blocks and Tx. No ZIP specifies TreeState, SubtreeRoots, Submit, Status or the transparent query Utxos, and ZIP 307, “Client-server interaction”, leaves the acquisition of a starting tree state open: “Or, it has to set X to the block height at which Sapling activated, so as to be sent the entire commitment tree. [TODO: Decide which to specify.]” These queries, and pool selection in Blocks, are designed but unspecified (Definition 1.1).

Remark 5.12 (Minimal query set). A shielded synchronisation uses Tip, SubtreeRoots for each shielded pool, Blocks and TreeState; the full transactions that concern the wallet add Tx. A wallet

Table 4. Queries, arguments, and responses of the light-client service (Definition 5.10). Each response is the value on the server’s best chain; $\perp$ answers an argument that names nothing on it.

Query	Arguments	Response
<i>Chain</i>		
Tip()	none	(b, hash) of the tip of the best chain; the tip input of every confirmation count (§9)
Block(b)	block identifier b	the compact block of b , with all shielded pools and transparent data
Blocks($s, e, \mathcal{P}$)	heights $s \leq e$; pool set $\mathcal{P}$	the compact blocks at heights $s, s + 1, \dots, e$ in ascending order, restricted to $\mathcal{P}$; every height of $[s, e]$ present, a block left with no transaction carrying an empty list
<i>Trees</i>		
TreeState(b)	block identifier b	height and hash of b , and for each shielded pool the size n and the frontier of its tree in the final treestate of b ; an empty frontier denotes the empty tree
SubtreeRoots(pool, i)	shielded pool pool; start index i	in index order, the pair (root, completing height) of every complete subtree of the tree of pool with index at least i
<i>Transactions</i>		
Tx(txid)	32-byte identifier	the full serialised transaction
Submit(bytes)	serialised transaction	accepted(txid) if the server’s node admits it to its mempool, else rejected
Status(txid)	32-byte identifier	the status of Definition 5.9
<i>Transparent</i>		
Utxos(A)	set A of transparent addresses	the unspent outputs of the best chain paying to A , each with outpoint, value, script and height

that watches transparent receivers adds Utxos, whose argument names its addresses in the clear (“Key independence of queries”, §8.2). This set is the requirement of the synchronisation protocols of “Note discovery” (§6) and “Commitment-tree synchronisation” (§7), not a deployed fact.

6 Note discovery

This section states the protocol by which a wallet reconstructs its notes from the chain. It fixes the chain state and the account birthday that bound the scan, the keys an account is scanned with under the rules of ZIP 326, the note state of a wallet and the scan of one compact block that updates it, the continuity predicate and the conditions under which a scan in any order is correct, the theorems of sound and complete discovery and of recovery from the seed, and the work that scanning costs. The tree information a wallet retains for its witnesses is the subject of “Commitment-tree synchronisation” (§7).

Throughout, P ranges over the shielded pools Sapling, Orchard and Ironwood; H_S is the Sapling activation height, and $H_{\text{NU6.3}}$ the NU6.3 activation height of ZIP 258. For a compact block b , $n_P(b)$ and $S_{\text{end}}^P(b)$ are the commitment count and the end size of Definition 4.11.

6.1 The account birthday

Definition 6.1 (Chain state). For a height b , the *chain state* at b is

$$C(b) := (b, \text{hash}_b, (m_P(b), F_P(b)))_{P \in \{\text{Sapling}, \text{Orchard}, \text{Ironwood}\}},$$

where hash_b is the 32-byte hash of block b ; $m_P(b) \in [0, 2^{32}]$ is the number of leaves of the note commitment tree of pool P in the final treestate of block b ; and $F_P(b)$ is the frontier of that tree (*Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Frontier”): for each set bit t of $m_P(b)$, the root of the completed left subtree of height t on the rightmost filled path. The empty tree has $m_P(b) = 0$ and the empty frontier, and the full tree, $m_P(b) = 2^{32}$, has its root as frontier. The three trees are distinct trees of depth 32 ($\text{MerkleDepth}^{\text{Sapling}} = \text{MerkleDepth}^{\text{Orchard}} = 32$; protocol specification, §“Constants” and §“Note Commitment Trees”): the Sapling tree of the specification, and the Orchard-pool and Ironwood-pool trees, each built with the Orchard Merkle hash (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”). The chain state $C(b)$ is the response of $\text{TreeState}(b)$ (Table 4).

By the *Crypto Guide*’s Definition “Frontier” (§“Append-only and incrementally updatable trees”), the pair $(m_P(b), F_P(b))$ determines the root of the tree of P after block b and suffices to append every later leaf without any leaf appended before block $b + 1$. That it also yields every later authentication path is shown in “Frontiers as scan seeds” (§7.3).

Definition 6.2 (Account birthday). The *birthday* of an account is a height B , known to the wallet, that is a lower bound on the height of the first block in which the account could have received funds (ZIP 326, “Terminology”). With the birthday the wallet stores the chain state $C(B - 1)$, which summarises every note commitment of every pool before B . The scan of the

account starts at block B , and no block below B is scanned for it. The birthday of a wallet is the minimum of the birthdays of its accounts.

- (i) *Added accounts.* Let an account with birthday B' be added while the greatest height of a block the wallet has scanned exceeds B' . Every block of $[B', \text{that height}]$ is then scanned again with the scanning keys of the new account (§6.2), since earlier scans did not try them; the results of the other accounts are unchanged.
- (ii) *No recorded birthday.* A wallet that restores a seed without a recorded birthday takes $B := H_5$: no shielded address of a light client can have received a relevant transaction before Sapling activation (ZIP 307, “Client-server interaction”, “Importing a pre-existing seed”). The bound concerns shielded receipts only; the transparent history of the account’s addresses may lie below it.

Remark 6.3 (Choosing a birthday). Let the tip be at height T when an account is created, and let d be at least the wallet’s assumed maximum reorganisation depth. The birthday of the new account is taken as $B \leq T - d + 1$, with stored chain state $C(B - 1)$ at height $T - d$ or below. A reorganisation that replaces at most d blocks re-mines no transaction below height $T - d + 1$; with B closer to the tip, such a reorganisation could re-mine a transaction paying the account at a height below B , which the wallet never scans, and the payment would be missed. Consensus imposes no maximum reorganisation depth (*Consensus Guide*, §“Rollback limits, coinbase maturity, and finality”), so d is a wallet parameter, not a guarantee. The rollback window of 600 blocks that ZIP 218, “Block-count-based constants”, recommends concerns nodes and bounds no wallet’s choice of d . An earlier birthday is always safe for discovery and costs scanning work (§6.5).

6.2 Scanning keys

Definition 6.4 (Scanning keys of an account). Let an account a have Sapling keys and Orchard-protocol keys derived as in “Hierarchical key derivation (ZIP 32)” (§2), the latter with the flag `use_qlsk` of Construction 2.10. Scopes are $s \in \{\text{ext}, \text{int}\}$.

- (i) *Sapling.* For each scope s , the pair $(\text{ivk}_s^S, \text{nk}_s^S)$ of incoming viewing key and nullifier deriving key (ZIP 32, “Sapling internal key derivation”). Sapling outputs are scanned from B .
- (ii) *Orchard protocol.* For each scope s and each $u \in \{\text{false}, \text{true}\}$, the incoming viewing key $\text{ivk}_{s,u}$ of scope s computed with `use_qlsk = u`, and the nullifier deriving key nk , the same in both branches of Construction 2.10 and in both scopes (Construction 2.14).

A receiver and its ivk are scoped to the Orchard protocol, not to a pool: the same ivk trial-decrypts Orchard-pool and Ironwood-pool note ciphertexts (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”); the pool of an Action is that of the bundle that carries it, and Definition 4.9 admits lead byte $0x02$ in the Orchard pool and $0x03$ in the Ironwood pool. The *use_qlsk set* U_a is $\{u\}$ when the wallet knows the account’s value u and $\{\text{false}, \text{true}\}$ when it does not.

The following rules of ZIP 326, with their normative levels, fix which ivk is tried on which output.

- (R1) *Restoration*. A wallet restoring from a seed MUST take $U_a = \{\text{false}, \text{true}\}$ and scan with both values until rule (R2) narrows it, because `use_qsk` is not part of the recovery information (“Scanning when restoring from seed”).
- (R2) *Narrowing*. Once the wallet sees funds received in either Orchard-protocol pool under a value u , it SHOULD stop scanning with the other value, $U_a := \{u\}$. The evidence stands if the transaction that provided it is later reorganised away, since it was valid on some chain (“Scan ranges”).
- (R3) *Ironwood pool*. The Ironwood pool is scanned from $\max(B, H_{\text{NU6.3}})$ to the tip with $\text{ivk}_{s,u}$ for $u \in U_a$, hence with up to four keys ($\{\text{ext}, \text{int}\} \times \{\text{false}, \text{true}\}$) while `use_qsk` is unknown, subject to rule (R4). The value $u = \text{true}$ is kept for an account whose birthday precedes NU6.3 activation, since a recorded birthday may be conservatively early (“Scan ranges”).
- (R4) *First funds*. ZIP 326 describes this rule as an optimisation and does not require it (“First-funds scanning in the Ironwood pool”). While the wallet knows that the account has received no Ironwood-pool funds in any block below the one scanned, only the external keys $\text{ivk}_{\text{ext},u}$ are needed for an output, except in a transaction whose `valueBalanceIronwood` is negative, where both scopes are used. The knowledge is kept only for the contiguous prefix of heights from $\max(B, H_{\text{NU6.3}})$ scanned without a receipt, and a reorganisation whose last unchanged block is at height h_f cuts the prefix back to $\max(h_f, H_{\text{NU6.3}})$. A compact transaction carries no value balance (Definition 4.3), so a scan of compact blocks applies the exception only where it obtains `valueBalanceIronwood` otherwise; without it, both scopes are used.
- (R5) *Orchard pool* (“Scan ranges”). Orchard-pool note ciphertexts SHOULD NOT be scanned with `use_qsk = true` keys. For an account with birthday after NU6.3 activation they SHOULD NOT be scanned at all. For an account with birthday before NU6.3 activation they SHOULD NOT be scanned with the external `use_qsk = false` key after NU6.3 activation. If the account’s Orchard-pool balance is known to be zero at a block at or after NU6.3 activation, they SHOULD NOT be scanned in any block that descends from it.
- (R6) *Order*. A wallet MAY scan blocks in any order (“First-funds scanning in the Ironwood pool”).
- (R7) *Outgoing notes*. Outgoing notes are detected with the ovk of the same `use_qsk` value, which adds no key dimension (“Outgoing-note detection”).

Under (R5) the Orchard pool is scanned with `use_qsk = false` only: with $\text{ivk}_{\text{int},\text{false}}$ from B until the Orchard-pool balance is known to be zero after NU6.3 activation, and with $\text{ivk}_{\text{ext},\text{false}}$ from B to $H_{\text{NU6.3}} - 1$. The *scanning keys* $K_a(P, b, t)$ of account a for pool P , block b and transaction t

are the triples (ivk, nk, s) that items (i) and (ii) and rules (R1) to (R5) select. Every ivk is labelled (a, s) , which attributes each accepted note to its account and scope.

Transparent receipts are detected at the transparent addresses the wallet watches, which MUST include index 0 of each account (“Deriving unified addresses”, §3.6; ZIP 316, “Deriving a Unified Address from a UIVK”). The Ironwood pool is scanned as a pool of its own: its outputs are accepted under its lead byte, its revealed nullifiers are matched only against nullifiers of Ironwood-pool notes, and its tree is its own (*Ironwood Guide*, §“Nullifier sets”, Remark “Pool locality”).

Remark 6.5 (Omitted keys and conformant senders). Rules (R2), (R4) and (R5) omit a key only where the restrictions of ZIP 326, “Wallet key-generation restrictions”, exclude an output of the account under it. Every key of an account has one use_qsk value (MUST). No wallet sends Orchard-pool funds to $use_qsk = \text{true}$ keys, or to any external receiver after NU6.3 activation (MUST NOT). Internal receivers are exposed to no party outside the wallet (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”), so before the account’s first Ironwood-pool funds a note at its internal Ironwood receiver arises only from the account’s own transaction, funded by a negative value $BalanceIronwood$ (“First-funds scanning in the Ironwood pool”). No value enters the Orchard pool after NU6.3 activation, so a zero Orchard-pool balance stays zero (“Scan ranges”). An output created in violation of these restrictions by a non-conformant sender may be missed; this is the cost of the SHOULD NOT rules, and it is part of the statement of Theorem 6.14.

Proposition 6.6 (Spend detection requires the nullifier key). *The nullifier of an Orchard-protocol note is $DeriveNullifier_{nk}(\rho, \psi, cm)$ (Ironwood Guide, §“The nullifier”, Definition “Nullifier”), and that of a Sapling note is $PRF_{nk}^{nfSapling}(\rho^*)$ with*

$$\rho^* = \text{repr}_J(\text{MixingPedersenHash}(cm, pos)),$$

a function of the nullifier deriving key and of the note’s position (protocol specification, §“Computing ρ values and Nullifiers”).

- (a) *Let an Orchard-protocol account key have $use_qsk = \text{false}$. No efficient algorithm that receives the incoming viewing key (dk, ivk) of one scope and ak^P , but not nk , and is then given notes of its choice with pairwise distinct ρ outputs the nullifier of one of them, except with negligible probability.*
- (b) *Assume that $PRF^{nfSapling}$, keyed by nk^* of a Sapling key generated as the specification prescribes, is pseudorandom to an efficient algorithm that holds the incoming viewing key and the spend validating key of that key: the specification’s requirement that $PRF^{nfSapling}$ be a pseudorandom function (§“Pseudo Random Functions”), applied to this key distribution. Then the statement of (a) holds for the Sapling key.*

A holder of (dk, ivk) therefore detects receipts, since trial decryption needs only ivk (Construction 4.8), and cannot recognise a revealed nullifier as that of its note. A wallet that holds only incoming viewing keys has an empty tracked-nullifier set, and the set of notes it considers unspent

is a superset of the true set. Links through other public data are outside the statement (Ironwood Guide, §“Nullifier uniqueness and unlinkability”, Remark “Limits”). For `use_qls = true` no claim is made (Remark 2.11).

Proof. (a) Let $\mathcal{C}$ be an algorithm that outputs the nullifier of a submitted note with probability ε . It yields a distinguisher for the *Ironwood Guide*’s Proposition “Nullifier unlinkability” (§“Nullifier uniqueness and unlinkability”): the distinguisher runs $\mathcal{C}$ on its inputs (dk, ivk) and ak^p , submits the notes $\mathcal{C}$ is given, and outputs 1 exactly when an answer equals the value $\mathcal{C}$ outputs for that note. In the real game it outputs 1 with probability at least ε . In the ideal game each answer is Extract_p of a point uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, independent of the notes, except with probability $1/p_{\text{Vesta}}$; it equals a given value with probability at most $2/\#\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, since Extract_p is at most two-to-one. Hence ε is bounded by the advantage bound of that proposition plus a negligible term, under the assumptions it names. For the internal scope the proof of that proposition applies with its hybrid H_2 changed: the pair $(ivk_{\text{int}}, dk_{\text{int}})$ is replaced by $(\text{Commit}_{\text{r}}^{\text{ivk}}(ak, nk), u_3)$ by Assumption 2.16, whose reduction, like that of H_2 , draws ak and nk itself and passes on the fourth and fifth components of its tuple. The cost is the advantage against that assumption in place of the term for the *Ironwood Guide*’s Assumption “Joint hiding of the viewing keys” (§“Viewing keys”), and the later hybrids are unchanged, since the game after the hop is that of H_2 . That proposition concerns a key generated from a uniform spending key; for an account key the bound holds up to the loss of Lemma 2.9. (b) An algorithm that outputs $\text{PRF}_{nk}^{\text{nfSapling}}(\rho^*)$ for a submitted ρ^* not queried before distinguishes the function from a random function, which outputs that value with probability 2^{-256} , contradicting the assumption. $\square$

6.3 Receipt and spend detection

Definition 6.7 (Note state of a wallet). The *note state* of a wallet consists of:

- (i) per account a and pool P , the set of *detected notes*, each with its value, scope, position pos (Definition 4.11), creating transaction, its nullifier when derivable, its memo once fetched, and a *spent mark*: either unspent or the triple (height, index in block, txid) of the transaction that revealed its nullifier;
- (ii) per pool P , the *tracked set* W_P of the nullifiers of detected notes of P not marked spent, and the set R_P of *retained unlinked nullifiers*, each with its revealing transaction;
- (iii) for each wallet-relevant transaction, its mined height, or $\perp$ if it is unmined;
- (iv) the chain state (Definition 6.1) of the last block of each maximal interval of scanned heights; for a scan in increasing height, that of the last scanned block.

A transaction is *wallet-relevant* if and only if it creates a detected note or reveals the nullifier of one. A light wallet stores no copy of the chain; its note state is reconstructed from compact blocks (Definition 4.2) and fetched full transactions alone, supplied by a server that the wallet does not trust for privacy (Assumption 5.2, clause (b)). The information retained for witnesses is adjoined in “The information a wallet retains” (§7.4).

Construction 6.8 (Scanning a block). Let b be a compact block that continues the wallet’s chain, in the sense fixed in “Scan order” (§6.4), scanned for a set A of accounts. For each pool P , in block, transaction and description order, the start size of P in b (Lemma 4.12) and Definition 4.11 give each compact output or Action its position.

1. *Receipts*. For each compact Sapling output (for $P = \text{Sapling}$) and each compact Action of P (for an Orchard-protocol pool), in transaction t , and each account $a \in A$, run Construction 4.8 and Definition 4.9 under every $(\text{ivk}, \text{nk}, s) \in K_a(P, b, t)$ (Definition 6.4). Record an accepted note with its value, scope s , position and transaction. When nk is held, compute its nullifier nf (for Sapling from its position), and: if $\text{nf} \in R_p$, mark the note spent at the retained transaction and remove nf from R_p ; otherwise add nf to W_p .
2. *Spends*. For each nullifier revealed in P by b (each compact Sapling spend, and the nf of each compact Action of P): if it lies in W_p , mark the matching note spent at $(\text{height}(b), \text{index}, \text{txid})$ and remove the nullifier from W_p ; otherwise retain it in R_p with its revealing transaction.
3. *Chain state*. Compute $C(\text{height}(b))$ from the stored $C(\text{height}(b) - 1)$ and b : the hash of b , and for each pool the size $S_{\text{end}}^P(b)$ and the frontier obtained by appending the commitments of P in b , in order, to $F_p(\text{height}(b) - 1)$. It replaces $C(\text{height}(b) - 1)$ in the note state.

Step 1 precedes step 2, so a receipt and its spend in one block, or in one range scanned in increasing height, are both found in one pass. A nullifier is compared only with nullifiers of its own pool (*Ironwood Guide*, §“Nullifier sets”). The retention period of R_p is fixed by Lemma 6.12 (§6.4).

Enhancement. For each wallet-relevant transaction the wallet may fetch the full transaction by $\text{Tx}(\text{txid})$ (phase C of ZIP 307, “Client-server interaction”), which yields the memo, by decryption of the full C_{enc} , and the outgoing plaintexts recoverable under the account’s ovk , at the privacy cost stated in “Leakage of transaction queries” (§8.4).

Class (Definition 1.1). Trial decryption, nullifier matching in increasing height and enhancement are specified (ZIP 307, “Local processing”, “Detecting spends” and “Client-server interaction”); the scanning keys are those of ZIP 326. The retained set R_p and the chain state of step 3 are designed but unspecified.

6.4 Scan order

Remark 6.9 (The interaction of ZIP 307). ZIP 307, “Client-server interaction”, divides the interaction between client and server into four phases.

- (A) *First start*. The client obtains the tip height X and the commitment tree state at X , or sets $X := H_S$; the ZIP leaves the choice open. For an imported seed $X := H_S$ (“Importing a pre-existing seed”).
- (B) *First update*. The client obtains the tip Y and the compact blocks $X, \dots, Y$. It checks that the hash of block X equals its stored copy, a mismatch meaning that X was orphaned by a

reorganisation, and discards block X without further processing. For each later block it validates the header if one is present, trial-decrypts for its notes, creates and updates witnesses of detected notes, and scans for their nullifiers.

- (C) *Transactions*. At user direction, the client fetches full transactions.
- (D) *Later updates*. The client obtains the tip Z and the blocks $Y, \dots, Z$, rechecks block Y as in (B), updates witnesses, marks the notes whose nullifiers appear as spent at that height, and scans for new notes as in (B).

In the queries of Table 4 the phases use Tip, TreeState or Block, Blocks and Tx.

Scanning a range $[b, b']$ requires the chain state $C(b - 1)$: its hash for the continuity check of block b , and each pool's size and frontier for positions and authentication paths. The wallet holds it after scanning block $b - 1$, or obtains it by $\text{TreeState}(b - 1)$; a server-supplied frontier is accepted under clause (a) of Assumption 5.2 and is checked against no header (Proposition 5.3).

Definition 6.10 (Continuity predicate). Let the wallet hold the chain state $C(b) = (b, \text{hash}_b, (m_P(b), F_P(b))_P)$. A compact block b *continues the wallet's chain at b* if and only if

- (i) $\text{height}(b) = b + 1$;
- (ii) $\text{prevHash}(b) = \text{hash}_b$; and
- (iii) for every pool P , the start size of P in b , $S_{\text{end}}^P(b) - n_P(b)$ (Lemma 4.12), equals $m_P(b)$.

Within a range each block is checked against the chain state computed from its predecessor, and the first block against the stored chain state. When a range is scanned directly below an already scanned block $b' + 1$, that block is checked again against the chain state $C(b')$ that the new range computes. This extends ZIP 307's single hash comparison at the start of each phase to every block and to both sides of every range boundary.

Rule (verify first). The scan results of a block are applied to the note state only after the block is checked to continue the wallet's chain, and the results of a range only after each of its blocks is. Failure of the predicate is a *continuity error*, handled by rollback ("Reorganisation detection and recovery", §7.6). A malformed or insufficient field is an error of the response, not a continuity error ("Architecture and authority", §5.1). The predicate beyond ZIP 307's comparison is designed but unspecified (Definition 1.1).

Definition 6.11 (Scanned heights). For a wallet with birthday B , the *maximum scanned height* is the greatest height of a scanned block, and the *fully scanned height* f is the greatest $b \geq B - 1$ such that every block of $[B, b]$ has been scanned. Under out-of-order scanning f may lie below the maximum scanned height.

Lemma 6.12 (Out-of-order spend detection). *Fix a chain. Let a note of an account in pool P be created at height h_n and its nullifier revealed at height $h_r \geq h_n$, and let the wallet hold the note's nullifier deriving key. If each nullifier retained in R_P (Construction 6.8) that was revealed at height b is kept until the fully scanned height is at least b , the note is marked spent at the*

revealing transaction whichever of the blocks h_n and h_s is scanned first. The retention bound is the least that guarantees this: if a nullifier revealed at h is discarded while $f < h$, some block below h is unscanned, and a note of the account created there and spent at h is never marked spent.

Proof. If block h_n is scanned no later than block h_s , the note's nullifier is in W_p when block h_s is scanned (steps 1 and 2 of Construction 6.8, in that order when $h_n = h_s$), and it is matched on arrival. Otherwise block h_s is scanned first; the nullifier matches no tracked note and is retained in R_p . Since $B \leq h_n \leq h_s$ (Definition 6.2), f reaches h_s only after block h_n is scanned, so the nullifier is still in R_p when step 1 finds the note, and the note is marked spent. Minimality: block h has been scanned, since it revealed the nullifier, so $f < h$ means that some $h' \in [B, h - 1]$ is unscanned; a note created at h' and spent at h has then lost the only record of its spend. $\square$

Lemma 6.13 (Order independence of scanning). *Fix a chain and a range $[B, T]$. Let a schedule scan every block of $[B, T]$ once, in any order and in ranges of any lengths, each block b with key sets $K_a(P, b, t)$, and let*

- (a) *the results of each range be applied only after continuity is verified (Definition 6.10), and*
- (b) *unlinked nullifiers be retained as Lemma 6.12 requires.*

Then the resulting notes, positions, nullifiers and spent marks equal those of a scan of $[B, T]$ in increasing height that applies to each block the same key sets, and the chain state held for height T is $C(T)$. The key sets of Definition 6.4 depend on the order only through rules (R2), (R4) and the last clause of (R5), whose omitted keys accept no output of the account under the restrictions of ZIP 326; under those restrictions every order of Definition 6.4 yields the note state of the scan in increasing height. The witness half of order independence is stated in “Witness readiness” (§7.5).

Proof. Receipt detection of an output is local to its block: it depends only on the compact output and the key set (Construction 4.8, Definition 4.9). Positions are local: by Lemma 4.12 the start size of each pool in b follows from the compact block b alone, and (a) makes it agree with the end size of the predecessor. Nullifiers are functions of the note, the key and, for Sapling, the position. Spent marks follow from Lemma 6.12 in either order. The chain state at T is computed by step 3 of Construction 6.8 from the chain state below the last range that ends at T , which (a) makes the true one, by induction over the ranges. For the last clause, a key omitted by (R2), (R4) or the last clause of (R5) in one order and tried in another accepts no output of the account (the Remark on omitted keys above; for (R5), no value enters the Orchard pool after NU6.3 activation, so a zero Orchard-pool balance stays zero), so both orders accept the same outputs. $\square$

Theorem 6.14 (Soundness and completeness of note discovery). *Assume clause (a) of Assumption 5.2, and let the server's best chain be unchanged while an account a with birthday B is scanned by Construction 6.8 with the scanning keys of Definition 6.4, under conditions (a) and (b) of Lemma 6.13. Let f be the fully scanned height. For every $h \leq f$, restricted to notes created at heights at most h and to spent marks set by nullifiers revealed at heights at most h :*

- (i) the note state holds a note of a in pool P at position p if and only if the best chain has, at or below h , an output of P at position p that Definition 4.9 accepts under a key of $K_a(P, b, t)$ for its block b and transaction t ;
- (ii) when the wallet holds the note's nullifier deriving key, the note is marked spent if and only if its nullifier is revealed in pool P of the best chain at or below h , and, except with negligible probability, a revealed nullifier equal to it marks a spend of that note and of no other.

Completeness is over accepted outputs under the keys that Definition 6.4 applies, not over all outputs addressed to the account: an output that its sender encrypted malformed, one that acceptance rejects, and one that a non-conformant sender created under a key omitted by the rules of ZIP 326 are excluded. Ranges scanned out of order above f add notes and spent marks above h , which the statement does not cover. A holder of incoming viewing keys alone obtains (i) only (Proposition 6.6). The assumptions of (ii) are those named in the results cited in the proof.

Proof. By clause (a) the scanned compact blocks are those of the best chain, and every output of each is trial-decrypted under the applicable keys (Construction 6.8). Soundness of acceptance is Proposition 4.10, and membership of the accepted note in the best chain is Corollary 5.4(a). Positions are exact by Lemma 4.12(a) and Definition 6.10. The result does not depend on the order, by Lemma 6.13; this gives (i).

For (ii), spent status is nullifier matching within P , with spends found in either order by Lemma 6.12. For Orchard-protocol notes, one nullifier per note and distinct nullifiers for distinct notes are the *Ironwood Guide*'s Lemma "One nullifier per note" (§"Double-spend resistance") and Proposition "Nullifier uniqueness" (§"Nullifier uniqueness and unlinkability"), under the assumptions they name. For Sapling notes they are the Spend statement, which fixes the nullifier of the consumed note (protocol specification, §"Spend Statement (Sapling)"), and the requirement that $\text{PRF}^{\text{nfSapling}}$ be collision-resistant across all keys (§"Pseudo Random Functions"), with ρ^* distinct for distinct positions (§"Computing ρ values and Nullifiers"). Without clause (a), Remark 5.5 defeats completeness and Corollary 5.4(b) defeats soundness. $\square$

Theorem 6.15 (Recovery from the seed). *Let an account have $\text{use_qsk} = \text{false}$ and a valid account key ("Valid account keys", §2.6), so that its keys are determined by the seed S and the account index (Proposition 2.12). Under clause (a) of Assumption 5.2, scanning the best chain from a lower bound on the account's birthday, or from H_5 when none is recorded (Definition 6.2), with the scanning keys of Definition 6.4, in particular with $U_a = \{\text{false}, \text{true}\}$ by (R1) until (R2) narrows it, reconstructs the note state of the account up to the fully scanned height, as characterised by Theorem 6.14. Memos, and the outgoing plaintexts of outputs that the account encrypted under its ovk, are recovered once the wallet-relevant transactions are fetched. Transparent funds are recovered at the addresses the wallet watches: index 0 of each account by obligation (§3.6); how far beyond index 0 a wallet looks is fixed by no ZIP and is designed but unspecified (Definition 1.1). Data that never reached the chain, such as the memos of transactions never mined, is not recoverable. An account with $\text{use_qsk} = \text{true}$, whose keys S does not determine, is outside the theorem, although Definition 6.4 scans with both values when restoring.*

Proof. Proposition 2.12 gives the keys, hence the scanning keys. Definition 6.2 makes the scan start at or below the first receipt. Theorem 6.14 gives the note state. Memos and outgoing plaintexts are functions of the fetched full transactions and the keys, and under clause (a) Tx returns the transactions of the best chain. $\square$

Remark 6.16 (A reference interaction). The following interaction of a wallet that synchronises by ranges out of order is designed but unspecified (Definition 1.1). Per synchronisation the wallet runs:

1. SubtreeRoots($P, 0$) for each pool, storing the roots of the complete subtrees;
2. Tip();
3. selection of ranges: those near the tip, and those completing the subtrees of found notes (§7.5), before historic ranges;
4. for each range $[s, e]$: TreeState($s - 1$) unless $C(s - 1)$ is held, then Blocks(s, e, P), the continuity check and the scan;
5. a return to step 2 when a scan changes the selection: a newly found note, or a rollback after a continuity error.

The order is a wallet policy fixed by no ZIP, and it is observable by the server (“Leakage of range queries”, §8.3).

6.5 The cost of scanning

Trial decryption costs one key agreement per compact output or Action per candidate ivk, for every block from the birthday to the tip, independently of the wallet’s own activity. Compaction (Definition 4.5) reduces the bandwidth, not the number of trials. Restoring an account repeats this work from its birthday. Retrieval of full transactions and the maintenance of authentication paths scale with the wallet’s own notes, the latter linearly in its unspent notes.

Proposition 6.17 (Scanning work per target day). *At NU7’s target spacing of 25 seconds (ZIP 218, “Block target spacing”; Consensus Guide, §“Confirmation times at 25-second spacing”), a target day of 86,400 seconds holds 3456 blocks. A block holds at most 330 Actions of the Orchard pool and the Ironwood pool together (ZIP 218, “Shielded pool action limits”), so a target day holds at most 1,140,480 compact Actions of the two pools, with at most 168,791,040 bytes of their compact payload at 148 bytes each (Definition 4.5), and requires at most 1,140,480 trial decryptions of Actions per candidate ivk; the number of candidate keys of an account is fixed by Definition 6.4. The same bounds hold for Actions, Sapling spends and Sapling outputs together, since the shared budget of the same section limits their sum to 330 per block, and a compact Sapling output (116 bytes) or spend (32 bytes) is smaller than a compact Action. The figures count target blocks: the number of blocks mined in a day varies, and the target spacing is not a rate limit.*

Proof. The counts are $86,400/25 = 3456$, $330 \cdot 3456 = 1,140,480$ and $148 \cdot 1,140,480 = 168,791,040$. The per-block limits are those of ZIP 218, “Shielded pool action limits”: each pool of the Orchard protocol is limited separately by

$$\text{OrchardProtocolBlockActionLimit} = 330,$$

and every Orchard-pool Action, Ironwood-pool Action, Sapling spend and Sapling output consumes one unit of a single shared budget $\text{GlobalShieldedBudget} = 330$, so the two pools of the Orchard protocol together hold at most 330 Actions. Each compact Action, and each compact Sapling output, costs one trial decryption per candidate key (Construction 4.8); a compact Sapling spend costs none. $\square$

7 Commitment-tree synchronisation

This section states the tree information that a wallet retains in order to spend its notes. It shows that the witness maintenance of ZIP 307 forces a scan from each note to its anchor; decomposes each note commitment tree at level 16 into subtrees and a cap built from downloaded subtree roots; shows that a frontier seeds every later witness; defines the information a wallet retains and proves it sufficient; states exactly when a witness is computable; and states the detection of reorganisations and the recovery from them, with the bound on the rollback height, the safety theorem and the correctness theorem of synchronisation.

Throughout, P ranges over the shielded pools Sapling, Orchard and Ironwood, with the chain state $C(b)$ of Definition 6.1, the birthday B of Definition 6.2, and the commitment count $n_P(b)$ and end size $S_{\text{end}}^P(b)$ of a compact block b (Definition 4.11). Assumption 5.2 is the server assumption; only its clause (a) is used here.

7.1 Witness maintenance by linear scanning

Remark 7.1 (What a spend needs). A detected note is not yet spendable. A spend needs, besides the note, its position p and the authentication path of p to the anchor of a block at or after the note’s block (*Ironwood Guide*, §“Authentication paths”, Construction “Authentication path”; anchors as in §“Anchors”, Definition “Treestate and anchor”). By the *Ironwood Guide*’s Lemma “Authentication paths from public data” (§“Authentication paths”), that path is a function of p and of the first n leaves of the pool’s tree, n being the tree size at the anchor. Path construction and anchor selection are cited, not re-derived; the anchor height is fixed in “Anchors, confirmations, and expiry” (§9). The lemma is stated for the trees of the Orchard protocol; its proof uses only the recurrence of the tree and applies unchanged to the Sapling tree, which has the same depth 32 and the same layer convention (protocol specification, §“Note Commitment Trees” and §“Merkle Path Validity”). Everything below applies to each pool P separately.

An *incremental witness* of a note at position p , after $m > p$ leaves have been appended to the pool’s tree, is the information from which the authentication path of p in the tree of size m is computed: the position p , the siblings of the path whose ranges lie below m , and the frontier of the leaves appended after p (ZIP 307, “Local processing”, “Creating and updating note witnesses”; *Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Frontier”).

Construction 7.2 (Linear witness maintenance). ZIP 307, “Local processing”, “Creating and updating note witnesses”, states the protocol for the Sapling pool and its commitments cmu ; it applies to each pool with that pool’s commitments, cmx for Actions. Compact blocks are processed in increasing height, with the order of transactions and outputs preserved. The wallet holds its copy of the pool’s tree as a frontier and one incremental witness per tracked note. For each note commitment c of the block, in order:

- (i) append c to the tree;
- (ii) append c to every held witness;
- (iii) if the output of c is accepted by Definition 4.9, create a witness of it from the current tree.

A witness updated through block X yields the authentication path of its note to the anchor of block X . No ZIP specifies any other tree-maintenance protocol.

Lemma 7.3 (Linear maintenance consumes every commitment). *Under linear witness maintenance, let a note be created in block h_n and let $a \geq h_n$. The wallet holds the authentication path of the note to the anchor of block a only after it has processed, in order, every note commitment of the pool in every block from h_n through a . It creates the note’s witness only after it holds the tree state at the end of block $h_n - 1$, obtained either by processing every earlier block or from a chain state (Definition 6.1). In particular, a wallet restored from a seed spends no note created at h_n before it has scanned forward from h_n to the anchor.*

Proof. By induction on the processed commitments. Steps (i) and (ii) are the only operations on the tree and the witnesses, and each consumes the next commitment in chain order; step (iii) creates a witness from the current tree, which after the commitment of the note is the tree state at the end of block $h_n - 1$ followed by the commitments of h_n up to that of the note. A witness therefore reflects block X only after every commitment through X has passed through step (ii), and it yields no path to the anchor of a before it reflects block a . $\square$

Remark 7.4 (The retained-information method). The method of “Subtree roots and the cap” (§7.2) to “Witness readiness” (§7.5) removes the constraint of Lemma 7.3. It replaces per-note witnesses by subtree roots, obtained through the query `SubtreeRoots` (Definition 5.10), and a frontier, so that the path of a note needs the commitments of its own subtree only. The method is designed but unspecified (Definition 1.1); no ZIP specifies it or the subtree-root query.

7.2 Subtree roots and the cap

Definition 7.5 (Subtrees and the cap). Fix a pool P and its note commitment tree of depth 32, with nodes M_i^b in layer b counted from the root (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”; protocol specification, §“Merkle Path Validity”). Let MerkleCRH_P be its Merkle hash, $\text{MerkleCRH}^{\text{Sapling}}$ for Sapling and $\text{MerkleCRH}^{\text{Orchard}}$ for either Orchard-protocol pool, and Uncommitted_P its unused-leaf value, $\text{Uncommitted}^{\text{Sapling}}$ or

$\text{Uncommitted}^{\text{Orchard}} = 2$ (protocol specification, §“Constants”). For $0 \leq l \leq 32$ and $0 \leq i < 2^{32-l}$, the node at level l and index i is

$$\text{node}(l, i) := \mathcal{M}_i^{32-l},$$

the root over the positions $[i \cdot 2^l, (i+1) \cdot 2^l]$; its *range* is that interval. The nodes satisfy the first line below, and the second defines the values $\text{Empty}_{p,l}$:

$$\text{node}(l+1, i) = \text{MerkleCRH}_p(31-l, \text{node}(l, 2i), \text{node}(l, 2i+1)),$$

$$\text{Empty}_{p,0} := \text{Uncommitted}_p, \quad \text{Empty}_{p,l+1} := \text{MerkleCRH}_p(31-l, \text{Empty}_{p,l}, \text{Empty}_{p,l})$$

for $0 \leq l < 32$. Each $\text{Empty}_{p,l}$ is the root over a range of unused leaves (*Ironwood Guide*, §“Authentication paths”, Lemma “Authentication paths from public data”, for Orchard-protocol trees; the same recurrence for Sapling). *Subtree i* is the node $\text{node}(16, i)$ with its range, and its completeness, root and completing height are those of Definition 5.8. There are $2^{32}/2^{16} = 2^{16} = 65536$ subtrees, with indices 0 to $2^{16}-1$, each over 65536 positions. The subtree of a position p is $\sigma(p) := \lfloor p/2^{16} \rfloor$. The *cap* is the tree formed by the nodes $\text{node}(l, i)$ with $16 \leq l \leq 32$; its leaves are the 2^{16} level-16 nodes, and its root is the root $\text{node}(32, 0)$ of the tree. The same geometry holds for the Sapling, Orchard-pool and Ironwood-pool trees.

Construction 7.6 (The cap over level-16 values). For a pool P and a list $r_0, \dots, r_{2^{16}-1}$ of level-16 values, the *cap over (r_i)* is the append-only Merkle tree of depth 16 (*Crypto Guide*, §“Append-only and incrementally updatable trees”, Definition “Append-only Merkle tree”) whose cap-level-0 labels are the r_i , whose empty-leaf value is $\perp_0 := \text{Empty}_{p,16}$, and whose combination of two cap-level- u labels into a cap-level- $(u+1)$ label is

$$\text{NodeHash}_{u+1} := \text{MerkleCRH}_p(15-u, \cdot, \cdot) \quad (0 \leq u < 16),$$

the tree’s Merkle hash at level $u+16$, that is, in layer $31-(u+16)$. Complete subtree roots obtained from $\text{SubtreeRoots}(P, i)$ enter at cap positions $i, i+1, \dots$ in index order.

Lemma 7.7 (Cap root equals the tree root). *Let the tree of P hold n leaves, and for each i let $r_i := \text{node}(16, i)$ of that tree: the complete root when $(i+1) \cdot 2^{16} \leq n$; the root over the appended leaves of subtree i padded with Uncommitted_p when $i \cdot 2^{16} < n < (i+1) \cdot 2^{16}$; and $\text{Empty}_{p,16}$ when $i \cdot 2^{16} \geq n$. Then the cap-level- u node of index i of the cap over (r_i) equals $\text{node}(16+u, i)$ for $0 \leq u \leq 16$. In particular the root of the cap equals $\text{node}(32, 0)$, the anchor of the pool for that tree (*Ironwood Guide*, §“Anchors”, Definition “Treestate and anchor”).*

Proof. Induction on u . For $u=0$ the claim is the definition of r_i . For the step, the cap combines the cap-level- u nodes of indices $2i$ and $2i+1$ with $\text{MerkleCRH}_p(15-u, \cdot, \cdot)$; by the induction hypothesis they are $\text{node}(16+u, 2i)$ and $\text{node}(16+u, 2i+1)$, and the recurrence of the tree gives $\text{node}(17+u, i) = \text{MerkleCRH}_p(31-(16+u), \dots)$, in the same layer $15-u$. The value $r_i = \text{Empty}_{p,16}$ for an unused subtree follows from the leaf rule, by which unused positions hold Uncommitted_p (*Ironwood Guide*, §“The Merkle hash and the tree”, Definition “Note commitment tree”, item “Leaves”), and from the induction in the proof of the *Ironwood Guide*’s Lemma “Authentication paths from public data” (§“Authentication paths”). The empty labels of the cap,

$\perp_{u+1} := \text{NodeHash}_{u+1}(\perp_u, \perp_u)$, are then $\text{Empty}_{P,16+u}$ by the same induction, so the cap's labels of empty ranges agree with the tree's. $\square$

Lemma 7.8 (Path decomposition at level 16). *Let the tree hold n leaves, let $p < n$ and $\sigma := \sigma(p)$, and let $(s_0, \dots, s_{31})$ be the authentication path of p (Ironwood Guide, §“Authentication paths”, Construction “Authentication path”), $s_t = \text{node}(t, j_t \oplus 1)$ with $j_t := \lfloor p/2^t \rfloor$.*

- (i) *For $t < 16$, the range of s_t lies inside subtree σ , and s_t is a function of the leaves of subtree σ below n ; it is $\text{Empty}_{P,t}$ if its range starts at or beyond n .*
- (ii) *For $t \geq 16$, s_t is the cap node of cap level $t - 16$ and index $j_t \oplus 1$, a function of the level-16 nodes r_i with i in its range and $i \neq \sigma$. These are complete subtree roots; the root over the incomplete subtree $\lfloor (n - 1)/2^{16} \rfloor$ when n is not a multiple of 2^{16} , a function of the frontier of the tree after its n -th leaf; and $\text{Empty}_{P,16}$.*

Hence the leaves of subtree σ , the roots of the complete subtrees other than σ and the frontier of the tree of size n suffice for the path, and the wallet needs the commitments of no subtree other than σ .

Proof. The range of s_t is $[(j_t \oplus 1)2^t, ((j_t \oplus 1) + 1)2^t)$, and s_t is a function of the leaves in its range (Ironwood Guide, §“Authentication paths”, Lemma “Authentication paths from public data”, whose proof shows this for every node). For $t < 16$ the range lies inside $[\sigma 2^{16}, (\sigma + 1)2^{16})$, since j_t and $j_t \oplus 1$ have the same quotient by 2^{16-t} , which is σ . For $t \geq 16$ the range is a union of whole subtree ranges, and it excludes σ because $j_t \oplus 1 \neq j_t$ and $\lfloor \sigma/2^{t-16} \rfloor = j_t$; Lemma 7.7 computes s_t from the level-16 nodes of its range. The root of the incomplete subtree is the fold, with the $\text{Empty}_{P,t}$ values as right siblings, of the carries below level 16 of the frontier of the tree of size n , one for each set bit of $n \bmod 2^{16}$ (Crypto Guide, §“Append-only and incrementally updatable trees”, Proposition “Incremental append in logarithmic time”, proof). $\square$

Construction 7.9 (Acquisition of subtree roots). At each synchronisation the wallet queries $\text{SubtreeRoots}(P, 0)$ for each pool $P \in \{\text{Sapling}, \text{Orchard}, \text{Ironwood}\}$ (Table 4) and records, for each complete subtree i , the pair (root, completing height) of Definition 5.8; the roots enter the cap at index i (Construction 7.6). The roots are trusted under clause (a) of Assumption 5.2: no root is checked against a header, and a root is verified only if the wallet also computes it from scanned commitments (Proposition 5.3, item (iv)). The acquisition is designed but unspecified (Definition 1.1).

7.3 Frontiers as scan seeds

Lemma 7.10 (A frontier suffices to continue the tree). *Let F be the frontier of the tree of pool P after block $h - 1$ (Crypto Guide, §“Append-only and incrementally updatable trees”, Definition “Frontier”), holding m leaves, with carries f_t for the set bits t of m , and let $c_m, c_{m+1}, \dots$ be the note commitments of P in blocks $h, h + 1, \dots$ in chain order. From F and these commitments alone the wallet computes*

- (i) *the root of the tree after every block $b \geq h - 1$, and*

- (ii) for every position $p \geq m$ appended by a block at most b , the authentication path of p to that root.

The commitments at positions below m are not needed. Consequently the chain state $C(B - 1)$ stored with the birthday B of an account (Definitions 6.1 and 6.2) seeds every later witness of the account.

Proof. (i) Repeated application of the *Crypto Guide*’s Proposition “Incremental append in logarithmic time” (§“Append-only and incrementally updatable trees”), each append producing the next frontier and root.

(ii) By the *Ironwood Guide*’s Lemma “Authentication paths from public data” (§“Authentication paths”), a right sibling of p ($b_t = 0$) has a range above $p \geq m$, hence over later leaves or unused positions, and is computed from the c_q with Empty_{p_t} values. A left sibling ($b_t = 1$) has the range $[(j_t - 1) 2^t, j_t 2^t]$. If the range starts at or above m , it is computed from later leaves. If it lies below m , then $j_t 2^t \leq m$ and $j_t = \lfloor p/2^t \rfloor \geq \lfloor m/2^t \rfloor$ give $j_t = \lfloor m/2^t \rfloor$, which is odd since $b_t = 1$; so bit t of m is set and the range is the block of the frontier at level t , whose root is the carry f_t . If the range contains m , its part below m is the union of the frontier’s blocks at the set bits of m below t , since the range is aligned at level t ; its root is the fold of those carries with the later leaves of the range. $\square$

Definition 7.11 (Seeded batch). A batch of consecutive blocks $b_0, \dots, b_k$, of heights increasing by one, is *seeded* with the chain state $C(h_s)$ of the block preceding b_0 (Definition 6.1), with frontiers $F_p(h_s)$ of sizes $m_p(h_s)$. Before any retained information is modified, the wallet accepts the batch only if b_0 continues the seed in the sense of Definition 6.10: $\text{height}(b_0) = h_s + 1$, $\text{prevHash}(b_0) = \text{hash}_{h_s}$, and $m_p(h_s) + n_p(b_0) = S_{\text{end}}^P(b_0)$ for each pool P (Definition 4.2, Lemma 4.12); each later block is checked against its predecessor in the same way. A violation is a continuity error (Definition 6.10), handled in “Reorganisation detection and recovery” (§7.6). Under clause (a) of Assumption 5.2, the root of each seed frontier $F_p(h_s)$ equals the anchor of block b_s in P ; nothing else checks it, since the abstract format does not guarantee a header (Proposition 5.3). The per-block sizes are designed but unspecified (Definition 4.2, Remark 4.1).

7.4 The information a wallet retains

Definition 7.12 (Retained tree information). The *retained tree information* of a wallet is adjoined to its note state (Definition 6.7). It comprises a finite set $\mathcal{H}$ of heights, the *retained block boundaries*, chosen by the wallet and the same for every pool, which contains $B - 1$, and for each pool P :

- (a) for each unspent note of the wallet in P , its position p and its leaf, and each sibling s_t with $t < 16$ of the path of p (Lemma 7.8(i)) from the moment its range lies wholly below the tree size; such a node is a function of appended leaves and does not change thereafter (*Ironwood Guide*, §“Authentication paths”, Lemma “Authentication paths from public data”, items (i) and (ii) and proof);

- (b) the complete subtree roots with their completing heights (Definition 5.8), and the frontier of the tree after the last scanned block;
- (c) for each $r \in \mathcal{H}$, the chain state $C(r)$, whose frontiers are the tree states after block r : the anchors of the blocks in $\mathcal{H}$ are the wallet's candidate anchors, and the heights in $\mathcal{H}$ are its rollback points.

The *rollback window* is the span of $\mathcal{H} \setminus \{B - 1\}$. Because $\mathcal{H}$ is common to the pools, a rollback restores all trees to one block. Any other node may be discarded once no retained path needs it. An anchor older than the window is retained only by explicit choice: ZIP 318, “Anchor-height bucketing and cohorts”, requires the anchor of each migration transaction to be the tree state at a shared network-wide boundary height, so a wallet that follows it retains those boundaries; any other retention of old anchors is wallet policy. The definition fixes information, not a representation, and is designed but unspecified (Definition 1.1).

Proposition 7.13 (Sufficiency of the retained information). *Fix a pool P .*

- (i) *Let p be the position of a retained note, $\sigma := \sigma(p)$, and $A \in \mathcal{H}$ a retained block boundary with tree size $n > p$. If the blocks that hold the commitments of subtree σ below n and at or above the birthday seed are scanned, the authentication path of p to the anchor of A is computable from the retained information.*
- (ii) *For every height r at which the wallet holds a chain state $C(r)$ for all pools, with $n_r(P) := m_p(r)$ the size of the tree of P after r , discard every retained node over a range that meets $[n_r(P), 2^{32})$, every note position at or above $n_r(P)$, every subtree root with completing height above r and every $C(r')$ with $r' > r$, and take the frontier of $C(r)$ as the frontier after the last scanned block. The result is a function of the blocks at heights at most r , and it is information that a wallet which scanned only those blocks retains.*

Proof. (i) By Lemma 7.8, each sibling s_t is of one of the following sorts. A sibling whose range lies below n is the same node at every later tree state: for $t < 16$ it is in item (a), or, where its range lies below the birthday seed, it is a carry of the seed frontier (Lemma 7.10); for $t \geq 16$ it is a cap node over complete subtree roots of item (b) (Lemma 7.7). The sibling ranges are pairwise disjoint, so at most one contains $n - 1$ without lying below n . That sibling is the root over its range, whose part below n is the union of the blocks of the frontier of $C(A)$ at the set bits of n below its level; it is the fold of those carries with Empty_{p_i} values, and the frontier of $C(A)$ is in item (c). A sibling whose range starts at or beyond n is Empty_{p_i} .

(ii) After the discards, every retained node lies over a range below $n_r(P)$ and is a function of the leaves appended through block r (*Ironwood Guide*, §“Authentication paths”, Lemma “Authentication paths from public data”); every retained subtree root has completing height at most r , and is likewise such a function; the retained chain states are those at heights at most r ; and the frontier after r is that of $C(r)$, which the wallet holds for every pool at the same r . A wallet that scanned only the blocks at most r can hold each of these items, and item (a) is satisfied, since a sibling whose range is not below $n_r(P)$ is not yet due. $\square$

7.5 Witness readiness

Theorem 7.14 (Witness from retained information). *Let A be a retained block boundary with tree size n in pool P , let p be a position, $\sigma := \sigma(p)$, and let $(s_0, \dots, s_{31})$ be the authentication path of p to the anchor of A . If $p \geq n$, the Ironwood Guide’s Construction “Authentication path” (§“Authentication paths”) defines no path, and by its Proposition “Membership soundness” (§“Anchors”), item (iii), no valid path to that anchor with position p exists, except with negligible probability, for the Orchard-protocol pools; for Sapling, by the same argument under the collision resistance of PedersenHash that the protocol specification requires of MerkleCRH^{Sapling} (§“MerkleCRH^{Sapling} Hash Function”), since Uncommitted^{Sapling} is not in the range of NoteCommit^{Sapling} (§“Windowed Pedersen commitments”). Otherwise the path is computable from the wallet’s information if every s_t is of one of the following kinds:*

- (a) $t < 16$, and s_t is computable from the commitments of subtree σ below n that the wallet has scanned or holds summarised in a carry, whose range lies below n , of a seed frontier (Lemma 7.10) or in the frontier of $C(A)$;
- (b) $t = 16$, and s_t is a complete subtree root that the wallet holds;
- (c) $t \geq 16$, and s_t is a cap node computed from held complete roots, carries at levels at least 16 of the frontier of $C(A)$ or, with range below n , of a seed frontier, Empty _{$P, 16$} and, where its range meets the incomplete subtree, the root over that subtree’s leaves below n , computed from the frontier of $C(A)$;
- (d) the range of s_t starts at or beyond n , and $s_t = \text{Empty}_{P, t}$;
- (e) $t \geq 16$, and s_t is a carry of the frontier of $C(A)$ or a carry, whose range lies below n , of a seed frontier.

Proof. Kinds (a) to (d) are the sorts of sibling in the proof of Proposition 7.13(i), extended in kind (c) by frontier carries, and a sibling of kind (e) is a retained node. $\square$

Example 7.15 (Kinds of the siblings of one path). Let $p = 65537$, so $\sigma = 1$, and let the anchor’s tree hold $n = 471098 = 7 \cdot 65536 + 12346$ leaves, so that subtrees 0 to 6 are complete and subtree 7 is incomplete. The siblings $s_0, \dots, s_{15}$ are of kind (a); s_{16} , the root of subtree 0, is of kind (b); s_{17} , over subtrees 2 and 3, and s_{18} , over subtrees 4 to 7, which meets the incomplete subtree, are of kind (c); and $s_{19}, \dots, s_{31}$, whose ranges start at $8 \cdot 65536$ or beyond, are of kind (d).

Definition 7.16 (Block extent of a subtree). For pool P and subtree s , let $c_p(s)$ be the completing height of subtree s (Definition 5.8), and let $c_p(-1)$ be the activation height of P . The *block extent* of subtree s is the interval of heights $[\max(c_p(s-1), B), c_p(s)]$, both ends included, the part below B being summarised by the chain state $C(B-1)$ (Lemma 7.10); for the incomplete last subtree the interval ends at the anchor height. After detecting a note at position p , the wallet scans the extent of subtree $\sigma(p)$ beyond the ranges it has already scanned. The incomplete subtree of every pool is covered by the range from $\max(B, h_c)$ to the tip, where h_c is the least,

over the pools, of the completing height of the pool's last complete subtree, or of the pool's activation height if it has none.

The start of an extent is inclusive because the completing block of subtree $s - 1$ may also hold the first commitments of subtree s ; the positions within that block are fixed by Lemma 4.12.

Proposition 7.17 (Readiness of a witness). *Let a note of pool P lie at position p , $\sigma := \sigma(p)$, and let the anchor block A , of height a , have tree size $n > p$ and $\kappa := \lfloor n/2^{16} \rfloor$ complete subtrees. The witness of p at A is computable if*

- (1) *the blocks of subtree σ are scanned from the start of its extent (Definition 7.16) up to a , or through its completing height if $\sigma < \kappa$;*
- (2) *the wallet holds the root of every complete subtree $i < \kappa$ with $i \neq \sigma$, including those after σ ; and*
- (3) *the frontier of $C(A)$ is retained, which scanning the extent of subtree κ through a yields when n is not a multiple of 2^{16} .*

Condition (2) can be weakened through frontier carries: every complete subtree $i \neq \sigma$ lies under a sibling of the path of p , and the root of subtree i is not needed when its range lies inside a carry, at a level at least 16 and at most that of the sibling, of the frontier of $C(A)$ or, provided the carry's range lies below n , of a seed frontier. Note eligibility, anchor choice and confirmation depth are separate conditions ("Spendable notes", §9.5).

Proof. Each condition supplies one kind of Theorem 7.14: condition (1) kind (a); condition (2) kind (b) and the complete part of kind (c); condition (3) the incomplete part of kind (c). Kind (d) needs nothing. For condition (3), the carries of the frontier at levels $t \geq 16$ are cap nodes over complete roots, and those below level 16 are functions of the leaves of subtree κ below n , appended in its extent. For the last claim, the ranges of $s_{16}, \dots, s_{31}$ partition the subtree indices other than σ ; if the range of subtree i lies inside such a carry, which is the same node at A , its range lying below n , the sibling over it is that carry, of kind (e), or a cap node computed from it, of kind (c), without the root of subtree i . $\square$

Remark 7.18 (Scheduling). Scanning the tip range and the extents of the subtrees of found notes before historic ranges is a priority rule of wallet policy, designed but unspecified (Definition 1.1). The order of range requests is observable by the server ("Leakage of range queries", §8.3).

Corollary 7.19 (Witnesses under any scan order). *Under the hypotheses of Lemma 6.13, a schedule of ranges that covers $[B, T]$ in any order yields, for every note and every retained anchor, the witness of a scan in increasing height from B , provided a note enters a witness only once the extent of its subtree (Definition 7.16) is scanned up to the anchor and the conditions of Proposition 7.17 hold.*

Proof. By the *Ironwood Guide's* Lemma "Authentication paths from public data" (§"Authentication paths"), the path is a function of the position and of the leaves below n , which do not depend

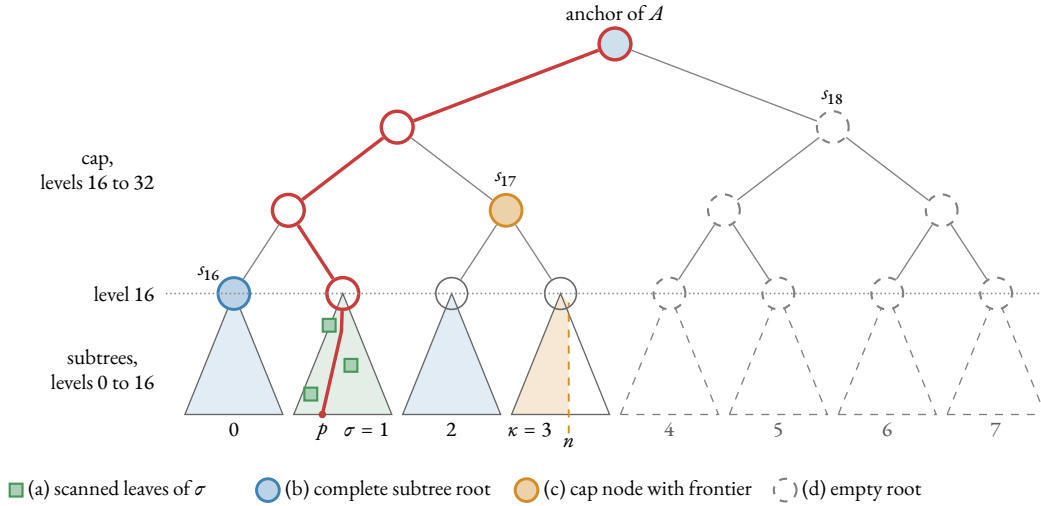


Figure 3. A note commitment tree cut at level 16, drawn with eight subtrees in place of 2^{16} and the cap above them compressed to three levels. Subtrees 0 and 2 are complete, subtree $\sigma = 1$ holds the note at position p , subtree $\kappa = 3$ is incomplete at the anchor's tree size n , and subtrees 4 to 7 are unused. The route of p to the anchor is drawn in red. Its siblings are marked by the kind of Theorem 7.14 and the source that supplies them: the siblings $s_0, \dots, s_{15}$ inside subtree σ , from its scanned leaves (a); s_{16} , a downloaded complete subtree root (b); s_{17} , a cap node over a complete root and the root of subtree κ computed from the frontier after A (c); and s_{18} , the empty root beyond n (d). The split of the path at level 16 is Lemma 7.8.

on the scan order. Positions are local to their blocks by Lemma 4.12, and the detection of the note is order-independent by Lemma 6.13. Once the stated extents are scanned, Theorem 7.14 makes the path computable. $\square$

Remark 7.20 (Cost of a restored wallet). A restored wallet seeds each tree from the chain state $C(B - 1)$ (Lemma 7.10), holds one root per complete subtree (Construction 7.6), and scans for witnesses only the extents of the subtrees of its notes and the tip range, instead of replaying the tree from the pool's first block. By Proposition 7.17, the witness of a found note is ready once those ranges are scanned. The trial decryption of every block from B onward (Proposition 6.17) remains, and so does the scanning of the extents.

7.6 Reorganisation detection and recovery

Remark 7.21 (What a reorganisation invalidates). A reorganisation (*Ironwood Guide*, §“Anchors”, Definition “Reorganisation”) makes three kinds of wallet information wrong: the transactions of the wallet mined in replaced blocks become unmined; the anchors of replaced blocks vanish; and tree information over positions appended in replaced blocks no longer describes the chain.

Construction 7.22 (Detection of a reorganisation). Each synchronisation after the first requests blocks from its previously synchronised height X inclusive and checks that the hash of the received block X equals the stored one; a mismatch implies that block X was orphaned by a reorganisation (ZIP 307, “Client-server interaction”, phases B and D). This volume extends

the check to every block and to both sides of every range boundary (Definition 6.10). Failure of the continuity predicate is a *continuity error* and triggers recovery (Construction 7.24). A malformed field, or a tree size that is absent, outside $[0, 2^{32}]$ or below the block's own commitment count (Lemma 4.12(b)), is an error of the response and triggers none. The extension beyond ZIP 307's comparison is designed but unspecified (Definition 1.1).

Definition 7.23 (Rollback to a height). *Rollback* to a height r at which the wallet holds a chain state $C(r)$ consists of the following changes.

- (i) The chain state of the wallet becomes $C(r)$, and each pool's retained tree information becomes that of Proposition 7.13(ii) for r .
- (ii) Every transaction mined above r is marked unmined. The spend marks that such transactions set, on shielded notes and on transparent outputs, are cleared, and the nullifier of each note so unmarked returns to the tracked set $\mathcal{W}_p$ of its pool.
- (iii) Scan results above r are discarded: the unlinked nullifiers of R_p revealed above r , and the scanned ranges above r , so that the scanned heights of Definition 6.11 are at most r . A note whose creating transaction is now unmined loses its position and, for Sapling, its nullifier, which depends on the position; both are derived again if a later scan detects the note.
- (iv) The knowledge of rule (R4) of Definition 6.4 is cut back as that rule states; the evidence of rule (R2) stands.
- (v) Local-only data is kept: sent and received notes stay recorded, since the memos and transactions that the wallet created may be unrecoverable from the chain, while a note whose transaction is unmined is excluded from balances and from spendability ("Spendable notes", §9.5).

Construction 7.24 (Recovery from a continuity error). Let $k \geq 2$ be an offset, a wallet parameter. On a continuity error at height e , that is, when a block of height e fails Definition 6.10 against the wallet's chain state $C(e - 1)$, the wallet runs the following steps.

- (1) If $e - k < B - 1$, it obtains $C'(B - 1) := \text{TreeState}(B - 1)$, replaces its stored chain state at $B - 1$ by $C'(B - 1)$, rolls back to $B - 1$ (Definition 7.23), and goes to step (4).
- (2) Otherwise let r be the greatest height with $B - 1 \leq r \leq e - k$ that is a retained block boundary ($r \in \mathcal{H}$) or at which the wallet has recorded the hash of its scanned block. The height $B - 1$ qualifies, since $B - 1 \in \mathcal{H}$.
- (3) If $r \in \mathcal{H}$, the wallet rolls back to r . Otherwise it obtains $C'(r) := \text{TreeState}(r)$ (Definition 6.1), trusted under clause (a) of Assumption 5.2. If the hash of $C'(r)$ equals the recorded hash, the wallet takes the frontiers of $C'(r)$ as its tree states after r , discards every retained node over a position at or above their sizes, and rolls back to r . If the hashes differ, its block r is off the server's chain, and it repeats step (1) with $e := r + 1$.

- (4) It restarts synchronisation from the query `Tip()`.

A fetched chain state extends the depth to which the wallet recovers beyond its rollback window, but only at a height whose hash the wallet has recorded: without that comparison, a fetched chain state cannot show that the wallet's blocks at most r lie on the server's chain, and results from replaced blocks below r would survive. The construction is designed but unspecified (Definition 1.1).

Proposition 7.25 (Recovery reaches the fork). *Assume clause (a) of Assumption 5.2, let the server's best chain be unchanged during recovery, and exclude collisions of the block hash. Let the wallet's scanned heights below e form the interval $[B, e - 1]$, as in a scan in increasing height. The wallet's block at a height $h \in [B - 1, e - 1]$ is the block it scanned at h , or for $h = B - 1$ the block of its stored $C(B - 1)$. The fork point h_f is the greatest $h \in [B - 1, e - 1]$ at which the wallet's block is the server's block at h , and $h_f < B - 1$ if there is none. Agreement at a height implies agreement at every lower height of $[B - 1, e - 1]$: the wallet's blocks are linked by the checks of Definition 6.10, and each header contains the hash of its parent (protocol specification, §“Block Header Encoding and Consensus”).*

- (i) *A continuity error at e implies $h_f \leq e - 2$.*
- (ii) *Repeated recovery by Construction 7.24 strictly decreases the rollback height r until $r \leq h_f$, or until step (1) replaces the stored chain state at $B - 1$; after either, no continuity error recurs.*
- (iii) *No offset below 2 guarantees progress: a rollback to $e - 1$ keeps the orphaned block, and the next scan fails again at e .*

Proof. (i) Under clause (a) the server's block e has the requested height, so the failing clause of Definition 6.10 is (ii) or (iii). The server's block e has as parent its chain's block $e - 1$, and its sizes are functions of that chain's prefix (Lemma 4.12(a)). If the wallet's block $e - 1$ were the server's block $e - 1$, its hash and its sizes, which the wallet computed from the same prefix, would satisfy both clauses. So the chains disagree at $e - 1$, and $h_f \leq e - 2$.

(ii) Consider one recovery, with rollback height $r \leq e - k \leq e - 2$. If r is a retained boundary and $r > h_f$, the wallet's block r is off the server's chain, so the server's block $r + 1$, whose `prevHash` is the hash of the server's block r , fails continuity at $e' = r + 1$, and the next rollback height is at most $e' - 2 < r$. If r is a recorded height and the hashes differ, step (3) repeats with $e := r + 1$ and the next height is at most $r + 1 - k < r$. If the hashes agree, $r \leq h_f$ by the definition of the fork point. Heights are bounded below by $B - 1$, and a rollback height that would lie below it is replaced by step (1). Once $r \leq h_f$, the wallet's block r is on the server's chain, and every later block of that chain continues its predecessor. After step (1) the wallet's chain state at $B - 1$ is the server's and every result above it is discarded, so the same holds.

(iii) After a rollback to $e - 1$ the wallet's block $e - 1$ is unchanged, and by (i) it is off the server's chain; the server's block e fails against it again. $\square$

Theorem 7.26 (Reorganisation safety). *Under the hypotheses of Proposition 7.25, recovery by Construction 7.24 ends at a height r such that every block of the wallet at most r is a block of the server's chain: either a retained boundary or a recorded height $r \leq h_p$, the latter with the chain state $\text{TreeState}(r)$, or $r = B - 1$ with the chain state $\text{TreeState}(B - 1)$. Scanning the server's chain from $r + 1$ then yields the note state and the retained tree information of a scan of that chain from the birthday, with local-only data kept.*

Proof. Proposition 7.25 gives the end height r and its property. After the rollback, the retained information is a function of the blocks at most r (Proposition 7.13(ii), Definition 7.23), and those blocks are shared by both chains; where a fetched chain state is used, its frontiers are those of the server's chain under clause (a). Theorem 6.14 and Lemma 7.10 then apply to the server's chain from $r + 1$, seeded with $C(r)$. $\square$

Theorem 7.27 (Correctness of synchronisation). *Assume clause (a) of Assumption 5.2 and exclude collisions of the block hash. Let the server's best chain be unchanged from some time on, the final chain, and let T be the height of a tip of the final chain that the wallet obtains after that time. Let the wallet*

- (1) *check continuity before applying the results of a range (Definition 6.10), on both sides of every range boundary;*
- (2) *retain unlinked nullifiers as Lemma 6.12 requires;*
- (3) *take positions from consistent sizes (Lemma 4.12);*
- (4) *recover from every continuity error by Construction 7.24, scanning a height again only after a rollback below it;*
- (5) *enter a note into a witness only once the extent of its subtree is scanned up to the anchor (Definition 7.16);*
- (6) *scan the range that contains T after obtaining T , with continuity checked at its lower boundary (Definition 6.10), and discard scan results above T ; and*
- (7) *hold, as its complete subtree roots, those returned by a SubtreeRoots query made after obtaining T , restricted to completing height at most T .*

If, after its last rollback, its scanned ranges cover $[B, T]$, in any order, the wallet holds the note state and the retained tree information of a scan of the final chain in increasing height from B .

Proof. The theorem composes earlier results and adds no mechanism. By (6) the range that contains T was requested after the tip query that returned T , so under clause (a) the wallet's block T is that of the final chain. By (1), each block of $[B, T]$ was checked against the wallet's chain state of its predecessor, and by (4) no height below it was scanned again afterwards without a rollback that also discarded the block; so the wallet's blocks of $[B - 1, T]$ are linked by prevHash, and by the hash chain of the headers each of them is the final chain's block at its height. By (4) a height is

scanned again only after a rollback below it, which discards the earlier result at that height (Definition 7.23 (iii)), so every kept result of $[B, T]$ is that of the wallet’s current block; by (6) no result above T is kept. The scan of $[B, T]$ is therefore one of the final chain without a continuity error. By (7) and clause (a), the retained subtree roots are those of the final chain with completing height at most T . Lemma 6.13 gives the note state of the scan in increasing height, Corollary 7.19 the witnesses, and Theorem 6.14 characterises the notes found. $\square$

Remark 7.28 (The rollback window). Consensus bounds no reorganisation depth, and node rollback windows are local policy (*Consensus Guide*, §“Rollback limits, coinbase maturity, and finality”). ZIP 307, “Local processing”, advises caching tree and witness state for 100 recent blocks, on the ground that no full node rolls back the chain by more than 100 blocks. That ground no longer holds: ZIP 218, “Block-count-based constants”, recommends a node rollback limit of 600 blocks at its 25-second target spacing and keeps coinbase maturity at 100 blocks, and neither figure bounds the window of a wallet. The depth that a wallet repairs from its retained information is the span of its retained block boundaries, a wallet parameter; a chain state fetched by `TreeState` at a height whose hash the wallet has recorded extends it under clause (a) of Assumption 5.2 (Construction 7.24).

8 Leakage to the light-client server

This section states what the server of “The light-client service” (§5) learns about a light wallet. It defines payment-detection privacy and unlinkability as indistinguishability games against the server of Assumption 5.2 and the view on which the server decides; proves that the view is a deterministic function of the chain, a tuple of schedule parameters and transport metadata, never of the wallet’s keys; derives the leakage of range queries, transaction queries, transparent-address queries and submission as instances of that theorem; and closes with a table of the server’s observables and the class of each mitigation.

8.1 Payment-detection privacy

The server is the node and the proxy together (Definition 5.1), under Assumption 5.2: clause (a) makes the response to each query a function of the server state and the query, and clause (b) is the power of the adversary. No second adversary is defined. The *server state* Σ_t at time t is the server’s best chain at t together with the state of its node’s transaction pool; the latter enters only the responses of `Submit` and `Status` (Definition 5.9). An *evolution* is a family $\Sigma = (\Sigma_t)_{t \leq t^*}$ up to a horizon t^* . A *wallet configuration* is a pair (K, u) : K is the wallet’s key material, the spending, full viewing, incoming viewing, outgoing viewing and nullifier deriving keys of every account; u is every other input of the wallet, namely the birthdays, the user’s actions and their times, the scheduling policy’s configuration, any state retained from earlier sessions, and the law from which the network endpoint of each connection is drawn.

Definition 8.1 (Server view). The *view* V of a wallet is the record of clause (b) of Assumption 5.2: the transcript

$$((c_j, q_j, a_j, t_j, r_j))_{j=1, \dots, J}$$

of the wallet’s queries in the order issued, each with its connection c_j , its query name q_j of Definition 5.10, its argument tuple a_j , its time t_j and its response r_j ; and, for each connection c , the *transport metadata* $M_c = (E_c, t_c^{\text{open}}, t_c^{\text{close}})$, the network endpoint that the server sees and the open and close times. Times are recorded at a finite resolution, and a run up to a horizon issues finitely many queries, so every view ranges over a countable set.

Definition 8.2 (Payment-detection privacy game). Let π be a scheduling policy, a public randomised rule that emits the wallet’s queries, made precise in “Key independence of queries” (§8.2). The *payment-detection privacy game* PD_π is the following indistinguishability game, with the advantage of a distinguishing game of the *Crypto Guide*, §“Security as a game”, Definition “Security game and advantage”; the adversary is not restricted to polynomial time.

1. The adversary fixes an evolution Σ , whose chain may contain outputs to either key set, and two configurations (K_0, u) and (K_1, u) with the same u .
2. The challenger draws a uniform bit $b \in \{0, 1\}$, runs the wallet (K_b, u) under π against a server with evolution Σ up to t^* , and hands the adversary the view V of that run (Definition 8.1).
3. The adversary outputs $b' \in \{0, 1\}$ and wins if and only if $b' = b$; its advantage is $\text{Adv} = |\Pr[b' = b] - 1/2|$.

Network privacy holds when the endpoint E_c of each connection is drawn afresh, independently of everything else, from a law that does not depend on which wallet opens the connection. The *unlinkability game* UL_π , under network privacy, differs in two points: the adversary fixes Σ and two configurations (K_0, u_0) and (K_1, u_1) ; and the challenger runs two consecutive synchronisation sessions, both of wallet 0 when $b = 0$, the second resuming from the state in which the first ends, and a session of wallet 0 followed by a session of wallet 1 when $b = 1$. The adversary receives the views of both sessions. Policy π provides *payment-detection privacy* (respectively *unlinkability*) if every adversary has advantage zero in PD_π (respectively UL_π), and provides it *within* ε if every advantage is at most ε .

Remark 8.3 (The goal of ZIP 307). ZIP 307, “Security Model”, states the goal informally and with a lowercase “should”: the proxy should not learn which transactions received from the chain are addressed to a given light wallet, and, assuming network privacy (“via Tor or similar”), should not be able to link different connections or queries as deriving from the same wallet. It takes the node and proxy combination to be honest but curious, trusted to provide a correct view of the current best chain and to transmit queries and responses faithfully. Game PD_π formalises the first sentence and game UL_π the second; the trust clause is clause (a) of Assumption 5.2, an assumption distinct from the goal. The chain is common to both worlds of each game. What the chain itself reveals to a party without keys is the subject of the *Ironwood Guide*, §“Privacy”, Theorem “Privacy of an Action”, whose Remark “Non-claims” excludes wallet and network metadata, the subject of this section.

Remark 8.4 (Scope and status). ZIP 307, “Security Model”, does not address how to spend notes privately. Submission through the same server is nevertheless part of the view of Definition 8.1

and is analysed in “The submission channel” (§8.5). The status of ZIP 307 and of ZIP 314 is that of Table 1. ZIP 314, titled “Privacy upgrades to the Zcash light client protocol”, carries no specification text, so a specified successor to the privacy treatment of ZIP 307 is an open problem (Definition 1.1).

ZIP 307 makes two statements about its interaction of phases A to D (“Scan order”, §6.4). First, since synchronisation proceeds over a broadcast medium, it leaks no information about which transactions interest the client (“Transaction privacy”). Second, the start of each of phases B and D reveals the height to which the client last synchronised, an information leak under the security model, assuming network privacy (“Block privacy via bucketing”). The two statements hold together: the first concerns the content of the responses, the second the bounds of the queries, two components of the view that Theorem 8.8 separates. The second is a leak in UL_π , since the resume height links a session to the session that reached it; both are made precise in “Key independence of queries” (§8.2) and “Leakage of range queries” (§8.3).

8.2 Key independence of queries

A light wallet delegates the storage and filtering of the chain to the server and keeps its keys; its note state (Definition 6.7) is rebuilt on the device. What reaches the server is its query schedule, recorded as in Definition 8.1.

Lemma 8.5 (Query arguments carry no key material). *Every argument of every query of Definition 5.10 is one of the following: a block identifier, a height or a block hash (Block, TreeState); a pair of heights $s \leq e$ with a pool set $\mathcal{P}$ (Blocks); a shielded pool with a subtree index (SubtreeRoots); a transaction identifier (Tx, Status); a set A of transparent addresses (Utxos); the bytes of a transaction (Submit). Query Tip takes none. No argument is a spending, full viewing, incoming viewing, outgoing viewing or nullifier deriving key, and no argument other than that of Submit is a nullifier of a note that the wallet holds unspent. The argument of Submit carries the nullifiers of the notes it spends; it is a component of L (“transactions submitted”) whose law depends on K , so submission is not covered by PD_π (“The submission channel”, §8.5). Against adversaries restricted to polynomial time, what the public data of its Ironwood-pool bundle reveal of its keys is bounded by the Ironwood Guide, §“Privacy”, Theorem “Privacy of an Action”, under the preconditions and assumptions named there; that theorem claims nothing for an Orchard-pool bundle (its Remark “Non-claims”) and does not treat Sapling components. A transparent address is a receiver, disclosed by declaration (the Remark “Transparent boundary” below). Which arguments the wallet chooses may depend on its keys; that dependence is carried by the schedule parameters of Theorem 8.8.*

Proof. By inspection of the argument types of Definition 5.10 (Table 4). □

Lemma 8.6 (Local detection and witness assembly). *Trial decryption and acceptance (Construction 4.8, Definition 4.9), nullifier matching and the search of retained unlinked nullifiers (Construction 6.8), and the assembly of authentication paths from retained information (Definition 7.12, Proposition 7.13) take as inputs the wallet’s keys, responses already received and retained data, and issue no query.*

Proof. By inspection of the cited definitions. The underlying procedures are the *Ironwood Guide*’s, §“Trial decryption and note acceptance” (Construction “Trial decryption”) and §“Authentication paths” (Lemma “Authentication paths from public data”), both computed from data already held. $\square$

Definition 8.7 (Schedule parameters). A *scheduling policy* π is a public randomised rule that, from the responses received so far and a tuple L , emits the next query with its argument, its connection and its time. The *schedule parameters* L collect every other input on which the emitted schedule depends: the wallet birthday (Definition 6.2) and the resume heights; the bounds, the pool sets and the order of range queries; the subtree extents requested (Definition 7.16); the transaction identifiers fetched or queried; the transactions submitted; the sets of transparent addresses queried; and the policy’s random coins, a finite bit string that fixes times, orders and connection choices. Every component takes values in a countable set. Keys enter the schedule only through L (Lemmas 8.5 and 8.6). Given Σ , the law of L is induced by the configuration (K, u) . The endpoints $E = (E_c)_c$ are drawn from the law that u fixes, independently of L ; without network privacy that law may be a single point.

For random variables X and Y with values in a countable set, $\Delta(X, Y) := \frac{1}{2} \sum_s |\Pr[X = s] - \Pr[Y = s]|$ is their statistical distance.

Theorem 8.8 (The server’s view of a light wallet). Assume Assumption 5.2, and let π be a scheduling policy. There is a deterministic function F_π with

$$V = F_\pi(\Sigma, L, E).$$

Hence:

- (i) two wallets with equal L and equal endpoints produce the same view, whatever their keys;
- (ii) in PD_π , with L_b the schedule parameters of wallet b and $V_b := F_\pi(\Sigma, L_b, E)$, every adversary that fixes Σ has

$$\text{Adv} \leq \frac{1}{2} \Delta(V_0, V_1) \leq \frac{1}{2} \Delta(L_0, L_1),$$

and the first bound is attained by an adversary that is not restricted in time; in particular π provides payment-detection privacy if and only if, for every Σ , K_0 , K_1 and u , the view has the same law for K_0 and K_1 , which holds whenever the law of L given Σ is the same for both;

- (iii) the bounds of (ii) hold for UL_π with the pair of the two sessions’ parameter tuples in place of L , the endpoints being independent of b under network privacy.

Privacy holds exactly when the view has the same law for both key sets, and in particular whenever the law of L given Σ does not depend on the wallet’s receipts, its detected notes and spends, or the transparent addresses it watches; each leak of “Leakage of range queries” (§8.3) to “The submission channel” (§8.5) is an instance.

Proof. Induction on the transcript. Query j , with its name, argument, connection and time, is

emitted by π from L and responses $r_1, \dots, r_{j-1}$. Response r_j is, by clause (a) of Assumption 5.2, a function of Σ_{t_j} and query j . The metadata M_c is E_c with open and close times that the schedule fixes. So V is a function F_π of the three inputs, which gives (i).

For (ii), fix Σ . An adversary is, after its choice, a randomised decision rule; write $D(v) \in [0, 1]$ for the probability that it outputs 1 on view v . By the *Crypto Guide*'s Definition "Security game and advantage" (§"Security as a game"), $\text{Adv} = \frac{1}{2} |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|$, and

$$\left| \sum_v (\Pr[V_1 = v] - \Pr[V_0 = v]) D(v) \right| \leq \sum_{v \in S} (\Pr[V_1 = v] - \Pr[V_0 = v]) = \Delta(V_0, V_1),$$

where S is the set of v with $\Pr[V_1 = v] > \Pr[V_0 = v]$ (or its complement, for the opposite sign); equality holds for D the indicator of S . For the second bound, E is independent of L_b with the same law in both worlds, so $\Delta((L_0, E), (L_1, E)) = \Delta(L_0, L_1)$; and for a deterministic f , $\Delta(f(X), f(Y)) \leq \Delta(X, Y)$, since $\Pr[f(X) = s]$ is the sum of $\Pr[X = x]$ over the preimage of s and the triangle inequality applies term by term. An adversary that chooses Σ at random has an advantage at most the maximum over its choices. The characterisation follows, since the advantage of the optimal adversary is $\frac{1}{2} \Delta(V_0, V_1)$. For (iii), apply the same argument to the pair of sessions; under network privacy E is independent of b and of the parameter tuples. $\square$

Corollary 8.9 (Compact responses reveal no receipt). *The response to $\text{Blocks}(s, e, \mathcal{P})$, $\text{Block}(b)$, $\text{TreeState}(b)$ and $\text{SubtreeRoots}(\text{pool}, i)$ is the same for every wallet that issues the same query at the same time. Let a policy's shielded synchronisation consist of Tip , $\text{SubtreeRoots}(\text{pool}, 0)$ for each shielded pool, $\text{TreeState}(B - 1)$, Blocks queries that cover $[B, T]$ in increasing height with bounds fixed by B , the resume heights, the returned tips and the policy's coins and one pool set $\mathcal{P}$ in every Blocks query, fixed independently of receipts, and the TreeState queries of Construction 7.24, with the retained boundaries $\mathcal{H}$ and the heights at which block hashes are recorded fixed by B , the resume heights, the responses and the policy's coins; and let its query times be fixed by its coins and the responses, not by the duration of local computation. Its L is then a function of B , the resume heights, the pool set and the coins, none a function of receipts, so its compact-block phase leaks no receipt. The commitment recomputation that replaces the discarded AEAD tag as the integrity check of compact decryption (Definition 4.9, Proposition 4.10) is local by Lemma 8.6. Requesting every complete subtree root from index 0 is independent of the positions of the wallet's notes; requesting a particular extent is not (Proposition 8.15). A policy that drops the Orchard pool from $\mathcal{P}$ once rule (R5) of Definition 6.4 stops its scanning reveals the height at which the Orchard-pool balance became known to be zero; keeping the pool in $\mathcal{P}$ and omitting only the trial decryption avoids this.*

Proof. The first sentence is clause (a) of Assumption 5.2. The rollback heights of Construction 7.24 are functions of $\mathcal{H}$, the stored block hashes and the responses, hence, by the hypothesis on $\mathcal{H}$ and on the recorded heights, of Σ , B , the resume heights and the coins. The remainder is Theorem 8.8(i) and (ii) with the listed L , whose law given Σ does not depend on K . $\square$

The hypothesis on times is necessary: a policy that issues its next query when the processing of a range ends, when that processing takes longer for a range with detected notes, puts a function of receipts into the times of L .

Remark 8.10 (Transparent boundary). Query $\text{Utxos}(A)$ puts the addresses of A into L and into the view in the clear, by declaration, not by inference, and a query with $|A| \geq 2$ presents the members of A as one requester's. The addresses are derived from K , so in PD_π a policy that queries its watched transparent addresses has advantage $\frac{1}{2}$ against the adversary that reads A . The guarantee of Corollary 8.9 is a property of the shielded layer only.

Proposition 8.11 (Transparent-address queries). *Let a wallet watch n transparent addresses and query them one address per query, in rounds that visit every address once in a uniformly random order, at times whose successive gaps are independent draws of law $\text{Exp}(1/T)$ with $T = 86400/n$ seconds, of mean T , one expected query per address per day (Math Guide, §“Beyond finite spaces: countable additivity, limits, and densities”, Example “A jittered schedule”). Then no query argument contains two addresses, and the server cannot link two addresses by co-occurrence in one query. It can still link them through the connection and its endpoint when their queries share either, and through the times in L : per-address randomised timing does not unlink addresses queried over one connection. The schedule removes one linking channel, not linkability. The schedule is designed but unspecified (Definition 1.1); ZIP 318, “Transfer scheduling”, specifies an analogous schedule, a uniformly random shuffle and independent exponential delays, for its migration transactions.*

Proof. The arguments are singletons by construction. The remaining channels are the connection, endpoint and time components of the view, which Theorem 8.8 retains unchanged. The law of the gaps is cited, not derived. $\square$

8.3 Leakage of range queries

Proposition 8.12 (Birthday bound from range queries). *Let m be the least height of any block that the wallet requests, where $\text{Blocks}(s, e, \mathcal{P})$ requests the heights of $[s, e]$, $\text{Block}(h)$ the height h , and $\text{TreeState}(h)$ the height $h + 1$ whose chain state below it is fetched. A policy that requests no block below the wallet birthday B (Definition 6.2, the minimum of its accounts' birthdays) reveals $B \leq m$, and $B = m$ when it requests a range that starts at B , which need not be its first range under a prioritised order. Since B lower-bounds the first block in which any account could have received funds (ZIP 326, “Terminology”), the server learns the upper bound m on the height below which the wallet expects no receipt; once the wallet requests a range that starts at B , so that $m = B$, it learns that bound exactly. Adding an account with an earlier birthday lowers m later. A policy that starts at Sapling activation, as ZIP 307 prescribes for a seed that the client did not generate (“Importing a pre-existing seed”), and as this volume does for a seed restored without a recorded birthday (Definition 6.2), reveals only that constant. The birthday is a component of u , common to both worlds of PD_π ; its disclosure is a disclosure of u and, where birthdays differ, a linking component in UL_π .*

Proof. The birthday B is a component of L and m a function of the view; the statement is an instance of Theorem 8.8. $\square$

Proposition 8.13 (Range bounds and scan state). *In the interaction of ZIP 307, “Client-server interaction”, the range of phase B starts at the height X that the previous synchronisation reached, and that of phase D at the height Y that phase B reached. Each start therefore reveals the previous synchronisation height, the tip at the time of the previous session, and so links consecutive sessions: in UL_π an adversary that chooses u_1 with a resume height other than the tip returned to the first session wins with probability 1. Under a prioritised policy the bounds are those of the policy’s ranges (verification ranges near the stored tip, extents of subtrees, historic ranges); they reveal the scan state, the maximum and fully scanned heights of Definition 6.11, to the extent that the ranges are functions of it, and need not equal the previous synchronisation height.*

Proof. The bounds are components of L , and the statement is an instance of Theorem 8.8(ii) and (iii). For the adversary: when $b = 0$ the second session starts at the tip returned to the first, when $b = 1$ at the resume height of u_1 , and the two differ with probability 1. $\square$

Remark 8.14 (Order of range queries). The order of range queries is a component of L . A policy that orders its ranges by a wallet-dependent priority (“Scan order”, §6.4, the reference interaction; “Witness readiness”, §7.5, scheduling) reveals that order, by Theorem 8.8. Every such priority is designed but unspecified (Definition 1.1).

Proposition 8.15 (Out-of-order subtree completion). *Let a policy, on detecting a note at position p of pool P , always request the block extent of subtree $s = \lfloor p/2^{16} \rfloor$ (Definition 7.16) as a query that the in-order schedule would not issue at that time. The extent starts at $\max(c_p(s-1), B)$ and ends at the completing height $c_p(s)$, or at the anchor height for the incomplete last subtree; the server knows every completing height from the responses to $\text{SubtreeRoots}(P, 0)$, so it learns s and that the wallet holds a detected note among the at most 2^{16} leaves of subtree s . Against such a policy, an adversary in PD_π that places an output to K_0 alone in a subtree outside both wallets’ in-order schedule at that time, and chooses K_1 with no receipt in Σ , wins with probability 1. The statement is an inference over the abstract policy of “Witness readiness” (§7.5), not a measurement of any deployment; no mitigation is specified, and one is an open problem (Definition 1.1).*

Proof. The requested extent is a function of p , a receipt, and a component of L ; the map from extent bounds to s is public. Wallet 0 issues the extent query and wallet 1 does not, so the adversary that outputs $b' = 0$ exactly when the view contains that query wins with probability 1, an instance of Theorem 8.8(ii). $\square$

Construction 8.16 (Bucketing). Fix a bucket size $N \geq 1$ and let $[X]_N := X - (X \bmod N)$. Phase B requests $\text{Blocks}([X]_N, Y, \mathcal{P})$ in place of $\text{Blocks}(X, Y, \mathcal{P})$, and phase D requests $\text{Blocks}([Y]_N, Z, \mathcal{P})$ in place of $\text{Blocks}(Y, Z, \mathcal{P})$. The blocks from $[X]_N$ to X are checked, height by height, to carry the block hash of the wallet’s chain view, and are then discarded without further processing. On a mismatch at height $Q > [X]_N$ the wallet rolls back (Definition 7.23) to $Q - 1$ if it holds $C(Q - 1)$. Otherwise it obtains $C'(Q - 1) := \text{TreeState}(Q - 1)$, compares hashes as in step (3) of Construction 7.24, takes its frontiers and rolls back to $Q - 1$. Both chains then agree at $Q - 1$, and it sets $X := Q - 1$. A mismatch at $Q = [X]_N$ goes to Construction 7.24 with $e := Q$. In phase D, Y replaces X throughout this check. Most reorganisations are thereby

handled within the same range. The construction is specified, as the modification of phases B and D in ZIP 307, “Block privacy via bucketing”, which the ZIP describes as a proposed enhancement.

Proposition 8.17 (Leakage under bucketing). *Under Construction 8.16 the start of a range of phase B or D reveals $\lfloor X \rfloor_N$ and no more of the resume height X : two wallets whose resume heights share a bucket and whose parameter tuples agree elsewhere issue ranges with equal starts. The ends of the ranges, their order and their timing are unchanged, and so is their leakage. The gain is in the set of resume heights consistent with a start, from one height to one bucket of N heights; it is not an advantage bound of UL_π , in which an adversary that chooses u_1 with a resume height in another bucket still wins with probability 1.*

Proof. The start is $\lfloor X \rfloor_N$, a function of X that is constant on buckets; the other components of L are untouched (Theorem 8.8). The last clause is the argument of the proof of Proposition 8.13 with starts in distinct buckets. $\square$

8.4 Leakage of transaction queries

Proposition 8.18 (Uncovered transaction queries). *A query $Tx(txid)$ or $Status(txid)$ issued without cover puts that transaction into L and reveals it as one that the wallet is fetching or tracking: a received transaction whose memo or outgoing plaintext the wallet recovers (the enhancement of Construction 6.8), a sent one, or a transparent dependency. This is the set that payment-detection privacy protects. Against a policy that fetches exactly its wallet-relevant transactions, an adversary in PD_π that places an output to K_0 alone, at a height that the common schedule scans before the horizon, and chooses K_1 with no receipt in Σ , wins with probability 1. A transaction identifier queried in two sessions links them in UL_π .*

Proof. The fetched set is a function of receipts and a component of L . Wallet 0 fetches the transaction of the output and wallet 1 does not; the statement is an instance of Theorem 8.8(ii) and (iii). $\square$

Remark 8.19 (Rules of ZIP 307 for full transactions). ZIP 307, “Transaction privacy”, states the following interoperability rules. A light client needs full transactions for the memo fields of received notes and for outgoing ciphertexts, to recover spend information when a seed is imported. It SHOULD obscure the exact transactions of interest by also downloading numerous uninteresting transactions; it SHOULD download all transactions of any block from which it fetches a single full transaction; and it MUST convey to the user that fetching full transactions reduces privacy. The MUST is unconditional, not a fallback for omitting the SHOULDs. Under the second SHOULD the fetched set reveals the blocks that hold a transaction of interest, not the transactions. The rules are specified (Definition 1.1).

No query of Definition 5.10 returns the full transactions of a block, since Block returns the compact block; the second SHOULD therefore costs one Tx query per transaction of the block.

Remark 8.20 (Timing of transaction queries). A Tx query issued at a time fixed by the completion of the range in which its transaction was detected is correlated in time with that range; under network privacy with separate connections, the times in L still link the two (Theorem 8.8(iii)). A delay drawn independently of that completion time would remove the correlation. No ZIP specifies one for transaction queries, and no stated design fixes its law: an open problem (Definition 1.1).

8.5 The submission channel

Proposition 8.21 (Submission through the server). *Let the wallet submit a transaction by Submit on a connection c to its synchronisation server. The server then holds the complete transaction before it is broadcast, at a time in L , on a connection with endpoint E_c . Without separation at the network layer it links the transaction to that endpoint and to the other queries of the same connection or endpoint, among them the birthday bound, the range bounds and the transaction queries of the session. The disclosure lies outside the goal of ZIP 307 and inside the view of Theorem 8.8.*

Proof. The transaction, its time and its connection are components of the view; the statement is an instance of Theorem 8.8(i). $\square$

The server’s operator terminates the transport, so channel encryption such as TLS protects queries from third parties and not from the server. Every connection exposes to the server its endpoint E_c and its open and close times, the endpoint being the wallet’s network address or that of an exit of an anonymity network or of a proxy. Without network privacy E_c is a component common to all sessions of one wallet, which is why the unlinkability goal of ZIP 307, “Security Model”, assumes network privacy.

Proposition 8.22 (Separate circuits). *Let submission and synchronisation use connections whose endpoints are drawn independently, as separate anonymity-network circuits give; or let submission go to a server that does not share its view with the synchronisation server, in which case the Submit query is absent from that server’s view and the separation rests on that non-collusion assumption. Then the endpoint no longer links the Submit query to the synchronisation session. Correlation remains through the times in L ; through content, the anchor and expiry heights of the transaction against the range bounds of the session; and through application-level identifiers, such as a later Status query for the submitted transaction identifier on the synchronisation connection. For payments the separation is designed but unspecified (Definition 1.1). For migration transactions ZIP 318 specifies the network-privacy step (“Network-layer privacy”: the wallet MUST offer Tor and, where available, Nym, and MAY broadcast through a server other than the synchronisation server) and the separation in time of synchronisation from broadcast (“Decoupling synchronization from broadcast”).*

Proof. The endpoint components of the two connections are independent, so their joint law is the same whether or not the connections belong to one wallet; every other component of L is unchanged, and the remaining channels are those components (Theorem 8.8(iii)). In the second case the synchronisation server’s view contains no Submit query, and under non-collusion the other server’s view is not joined to it. $\square$

Table 5. Observables of the light-client server, what each reveals about the wallet, the mitigation of each, and the class of that mitigation (Definition 1.1). Every row is a component of the schedule parameters or of the transport metadata of Theorem 8.8.

Observable	Reveals	Mitigation	Class
Network endpoint and connection times	network location and activity windows	network anonymity (Proposition 8.22)	designed but unspecified; specified for migration broadcasts (ZIP 318)
Least requested height	an upper bound on the birthday (Proposition 8.12)	a start at Sapling activation, at the cost of Proposition 6.17	specified for an imported seed (ZIP 307)
Range bounds and order	resume height and scan state (Proposition 8.13)	bucketing of the start (Construction 8.16)	specified (ZIP 307)
Out-of-order completion ranges	the subtree of a detected note (Proposition 8.15)	none	open problem; the leak is inferred
Transaction queries	the transactions of interest (Proposition 8.18)	decoys and whole-block fetches (SHOULD)	specified (ZIP 307)
Submission	the transaction before broadcast, linked to its session (Proposition 8.21)	separate circuits	designed but unspecified
Transparent-address queries	the addresses, by declaration (§8.2)	one address per query at exponential times in random order (Proposition 8.11); linkage by connection and timing remains	designed but unspecified
Pool sets of range queries	if narrowed by wallet state, the height at which the Orchard-pool balance became known to be zero (Corollary 8.9)	one pool set fixed independently of receipts; only trial decryption omitted	designed but unspecified

8.6 Summary of server observables

Table 5 lists the observables of the server, what each reveals, its mitigation and the class of that mitigation (Definition 1.1). The interface of Definition 5.10 omits mempool queries, so no row concerns the mempool.

Every row of Table 5 is a component of L or of the transport metadata (Theorem 8.8); none is a function of the content of compact blocks (Corollary 8.9). The size of each leak depends on policies that the protocol leaves open: beyond the rules of ZIP 307 and the migration schedule of ZIP 318, no ZIP fixes the scheduling policy of a wallet.

9 Anchors, confirmations, and expiry

This section fixes the heights and the set of notes from which a wallet builds a transaction. It states the confirmation policy of ZIP 315; the target height and the anchor height, with the lemma that ties the anchor depth to the trusted confirmation depth; the expiry rule of ZIP 203 and the expiry height a wallet sets; the record a wallet keeps of the transactions it creates and the notes those transactions lock; and the spendability categories of ZIP 315, with the spendable set defined from the wallet's state.

9.1 The confirmation policy

An output mined in a block of the best chain leaves the chain when a reorganisation replaces that block (*Ironwood Guide*, §“Anchors”, Definition “Reorganisation”), so a wallet spends an output only once it has a number of confirmations fixed by a policy. The rules of this subsection are those of ZIP 315 and are specified (Definition 1.1); the status of ZIP 315 is that of Table 1.

Definition 9.1 (Confirmation policy). A *confirmation policy* is a triple $(c_{\text{tr}}, c_{\text{untr}}, z)$ of integers $1 \leq c_{\text{tr}} \leq c_{\text{untr}}$, the *trusted* and the *untrusted confirmation depth*, and a flag $z \in \{\text{true}, \text{false}\}$ that permits zero-confirmation shielding. Confirmations are those of the *Consensus Guide*, §“Work, and the best chain”, Definition “Confirmations”: at a tip of height T , an output mined in the block of height $m \leq T$ of the best chain has $T - m + 1$ confirmations, its own block being the first; an output of a transaction that no block of the best chain contains has 0. ZIP 315 RECOMMENDS the depths $c_{\text{tr}} = 3$ and $c_{\text{untr}} = 10$ (“Trusted and untrusted TXOs”) and permits, without recommending, $z = \text{true}$ (“Transparent UTXOs”, MAY); a wallet SHOULD have a policy that it communicates to the user in a clear form (“Trusted and untrusted TXOs”).

Definition 9.2 (Trusted and untrusted outputs). A *TXO* is an output, transparent coin or shielded note, of a transaction of the best chain or of a mempool visible to the wallet (ZIP 315, “Terminology”). A TXO received by the wallet is *trusted* if it is an output of a transaction created by the wallet itself, which a wallet SHOULD treat as trusted, or of an external transaction that the user has marked trusted, a marking that a wallet MAY offer; every other received TXO is *untrusted* (ZIP 315, “Trusted and untrusted TXOs”; ZIP 315, “Prompt accessibility of funds”). A trusted TXO requires c_{tr} confirmations and an untrusted TXO c_{untr} . The output of a shielding transaction is not trusted by virtue of its internal receiver (ZIP 315, “Transparent UTXOs”); its depth rule is stated below.

The rationale of the default depths is ZIP 315's and is cited, not derived (ZIP 315, “Rationale for the given numbers of confirmations”): the trusted depth is 3 because reorganisations are anecdotally shorter than three blocks, and the two depths differ because the value of a trusted TXO is recoverable after a rollback, the wallet re-creating its transaction, whereas an untrusted TXO may need the sender to pay again. The modelled double-spend risk of a given count is that of the *Consensus Guide*, §“Waiting for confirmations” and §“Wallet confirmation policy”: three confirmations give an attacker with 10% of the hash rate a success probability of 1.712%, and ten about 0.0008%. A confirmation count bounds the modelled double-spend probability only for

a stated attacker share q of the hash rate and a tolerated probability ε (*Consensus Guide*, §“Wallet confirmation policy”); it is not a guarantee.

Confirmation counts are in blocks, not in time. At the 25-second target spacing (ZIP 218, “Block target spacing”; *Consensus Guide*, §“Confirmation times at 25-second spacing”), 10 blocks are $10 \times 25 = 250$ s and 3 blocks are $3 \times 25 = 75$ s. ZIP 218 does not argue for reducing confirmation counts (ZIP 218, “Motivation”) and keeps the anchor depth at 3 blocks (ZIP 218, “Anchor selection depth”). Neither an equal count nor an equal elapsed time establishes an equal risk, whose model is that of the *Consensus Guide*.

Two kinds of transaction enter the rules below; both are constructed in “TEX transfers and shielding transactions” (§11.3). A *shielding transaction* spends transparent UTXOs of the wallet and pays their value, less the fee, to an internal shielded receiver of the spending account, with no other recipient. An *ephemeral UTXO* is the transparent output that the first transaction of a transfer to a TEX address creates and its second transaction spends (ZIP 320, “Design considerations for Senders”; ZIP 315, “Trusted and untrusted TXOs”).

The following rules are specified. *Floor*: wallets SHOULD NOT permit the spending of TXOs with fewer than 3 confirmations, except (i) transparent UTXOs spent in shielding transactions and (ii) ephemeral UTXOs of a ZIP 320 transfer, whose second transaction spends an output that may be unmined (ZIP 315, “Trusted and untrusted TXOs”). *Zero-confirmation shielding*: wallets MAY spend received transparent UTXOs in wallet-internal shielding transactions with zero confirmations (ZIP 315, “Transparent UTXOs”); the flag z of Definition 9.1 records whether the wallet does. *Shielding outputs*: a shielded output of a wallet-internal shielding transaction meets its depth requirement if and only if every transparent UTXO that the transaction spent has at least $c_{\text{untr}} - c_{\text{tr}}$ confirmations and the shielding transaction has at least c_{tr} (ZIP 315, “Shielded TXOs”). With h_s the greatest mining height of the spent UTXOs and T the tip, the first condition is $T - h_s + 1 \geq c_{\text{untr}} - c_{\text{tr}}$.

Remark 9.3 (Two readings of the shielding rule). ZIP 315 states the rule for shielding outputs twice, and the two statements differ. The paragraph “Shielded TXOs” requires $c_{\text{untr}} - c_{\text{tr}}$ confirmations of the source UTXOs; the paragraph “Transparent UTXOs” requires c_{untr} confirmations of the transaction that produced them. Both require c_{tr} confirmations of the shielding transaction. Let the sources be mined at greatest height h_s and the shielding transaction at $h_s + d$ with $d \geq 0$. At the policy (3, 10, true), the first reading is met at every tip

$$T \geq h_s + \max(6, 2 + d),$$

and the second at every tip $T \geq h_s + \max(9, 2 + d)$; the two agree for $d \geq 7$. Indeed, a block at height m has $T - m + 1$ confirmations at tip T (Definition 9.1), and

$$\begin{aligned} T - h_s + 1 \geq 7 &\iff T \geq h_s + 6, \\ T - h_s + 1 \geq 10 &\iff T \geq h_s + 9, \\ T - (h_s + d) + 1 \geq 3 &\iff T \geq h_s + 2 + d. \end{aligned}$$

The rule stated above is the first reading, the one inside ZIP 315’s definition of a confirmed-spendable TXO. Which reading the ZIP intends is an open problem (Definition 1.1).

9.2 Target and anchor heights

Definition 9.4 (Target height). Let T be the height of the tip of the wallet’s view of the chain, the height returned by `Tip()` (Definition 5.10). The *target height* of a transaction built at that view is $H := T + 1$, the height of the first block that could include it. Every height-dependent rule of construction is fixed by H : the consensus branch, the fee rule and the expiry height are those of a block of height H , and confirmation counts are taken at the tip $T = H - 1$.

Definition 9.5 (Anchor height). Let $(c_{\text{tr}}, c_{\text{untr}}, z)$ be a confirmation policy (Definition 9.1) and $H \geq c_{\text{tr}}$ a target height. The *anchor height* of a transaction with target height H is

$$A := H - c_{\text{tr}},$$

a function of H and the policy alone. One anchor height serves every anchor that the transaction carries, including that of an Orchard-protocol bundle whose Actions spend no note. For a pool $P \in \{\text{Sapling, Orchard, Ironwood}\}$, the anchor of the transaction’s bundle of P is the root of P ’s note commitment tree in the final treestate of the block at height A (*Ironwood Guide*, §“Anchors”, Definition “Trestate and anchor”), and the authentication path of every note that the transaction spends from P is computed to that root (*Ironwood Guide*, §“Authentication paths”). Since $c_{\text{tr}} \geq 1$, the block at height A precedes the block at height H , as the consensus anchor rules require (*Ironwood Guide*, §“Anchors”; protocol specification, §“Action Transfers and their Descriptions” and §“Spend Transfers, Output Transfers, and their Descriptions”). A transaction therefore carries, for each pool it touches, its own anchor and authentication paths, the Orchard pool and the Ironwood pool of the Orchard protocol separately.

Lemma 9.6 (Anchor height and confirmations). *Let T be the tip, $H = T + 1$ and $A = H - c_{\text{tr}}$. For a note mined at height $m \leq T$ of the best chain, the following are equivalent:*

- (i) *the note’s commitment is a leaf of the tree rooted by the anchor of its pool at A ;*
- (ii) *$m \leq A$;*
- (iii) *the note has at least c_{tr} confirmations at T .*

The anchor depth thus equals the trusted confirmation depth: a note with exactly c_{tr} confirmations lies in the block at height A . At $c_{\text{tr}} = 3$ the anchor height is $A = T - 2$, and a note with at least $c_{\text{untr}} = 10$ confirmations is mined at a height at most $T - 9 < A$.

Proof. The final treestate of the block at height A holds exactly the commitments appended in the blocks at heights $0, \dots, A$ (*Ironwood Guide*, §“Anchors”, Definition “Trestate and anchor” and Remark “Old anchors”), so (i) and (ii) are equivalent. By Definition 9.1 the note has $T - m + 1$ confirmations, and $T - m + 1 \geq c_{\text{tr}}$ if and only if $m \leq T + 1 - c_{\text{tr}} = A$, so (ii) and (iii) are equivalent. The instances follow with $c_{\text{tr}} = 3$, $A = T + 1 - 3$, and $T - m + 1 \geq 10$ if and only if $m \leq T - 9$. $\square$

The rule for the anchor is specified: a wallet SHOULD choose an anchor a number of blocks back from the head of the chain equal to the trusted confirmation depth, the ZIP’s instance being

the final treestate of the block at height $H_{315} - 3$ when the current block is at height H_{315} (ZIP 315, “Anchor selection”). ZIP 218 recommends that the depth stay 3 blocks at the 25-second spacing, $3 \times 25 = 75$ s (ZIP 218, “Anchor selection depth”). The rule is a recommendation to the builder, not a consensus rule: the anchor rules admit the final treestate of any earlier block.

Remark 9.7 (The anchor convention). ZIP 315 counts the anchor depth back from the head of the chain, and ZIP 218, “Anchor selection depth”, reads it as the tip, so that a 4-block reorganisation is needed to remove the anchor; the *Ironwood Guide*, §“Anchors”, Remark “Recommended anchor depth”, follows that reading, which puts the anchor at $T - 3$ with $c_{\text{tr}} + 1$ confirmations. This volume’s convention counts from the block being built, $H = T + 1$, one block shallower, which gives Lemma 9.6: the anchor block has exactly c_{tr} confirmations. Both satisfy the consensus anchor rules.

The anchor depth is c_{tr} whatever the trust of the notes spent. The eligibility of each note is decided by its own depth (Definition 9.2), so an untrusted note spent at depth c_{untr} is proved against the anchor at depth c_{tr} , whose tree contains it by Lemma 9.6. ZIP 315 gives three reasons for the rule, specified and cited (ZIP 315, “Rationale for anchor selection”): a rollback past the anchor block invalidates the transaction, whose revealed nullifiers then link it to any later transaction spending the same notes; too deep an anchor keeps recently received notes from being spent; and a fixed depth, rather than one depending on whether the notes spent are trusted, does not reveal their trust. The anchor is published data (*Ironwood Guide*, §“Anchors”, Remark “Anchor choice as public data”).

Every Orchard-protocol bundle with at least one Action carries an anchor, which must refer to the final treestate of an earlier block in its pool, whether or not any Action spends a note of nonzero value (protocol specification, §“Action Transfers and their Descriptions”; consensus rule). Giving such a bundle the anchor at A makes it indistinguishable by its anchor from a bundle that spends (*Ironwood Guide*, §“Anchors”, Remark “Anchor choice as public data”).

9.3 Expiry (ZIP 203)

Definition 9.8 (Transaction expiry). Every transaction of version at least 3 carries a field $\text{nExpiryHeight} \in \{0, \dots, 2^{32} - 1\}$, a block height and never a timestamp; for a non-coinbase transaction, $\text{nExpiryHeight} \leq 499,999,999$ (consensus rule). A non-coinbase transaction with $N := \text{nExpiryHeight} \neq 0$ is *expired* at height h if and only if $h > N$, and it MUST NOT be mined at a height greater than N (consensus rule), so the block at height N is the last that may include it; the value $N = 0$ disables expiry (protocol specification, §“Transaction Encoding and Consensus”; ZIP 203, “Specification”). Coinbase transactions, which a wallet does not create, are outside the definition.

Lemma 9.9 (Includability at the next block). *Let t be a non-coinbase transaction with expiry height N , and T the tip. The expiry rule permits t in the block at the target height $H = T + 1$ if and only if $N = 0$ or $T < N$; when $N \neq 0$, it permits t in no block of height greater than N . Inclusion requires the other consensus rules as well; the lemma concerns expiry only.*

Proof. By Definition 9.8, the rule permits t at height H if and only if $N = 0$ or $H \leq N$, and $H = T + 1 \leq N$ if and only if $T < N$. The second claim is the rule itself. $\square$

Construction 9.10 (Expiry height of a wallet transaction). A wallet sets $\text{nExpiryHeight} := H + \delta$, where H is the target height (Definition 9.4) and $\delta \geq 1$ is an *expiry delta* chosen by the wallet. Reading ZIP 203’s “current height” as the target height is designed but unspecified (Definition 1.1). Every later rule that depends on the delta, the locking of notes (§9.4) and the polling and re-sending of transactions (§13), uses the δ actually chosen.

ZIP 315 specifies (SHOULD) that a wallet create transactions using the default expiration height of ZIP 203, which it gives as 40 blocks from the current height (ZIP 315, “Expiration height”). ZIP 218 changes the default of node implementations that create transactions to $\text{NU7PoWTargetSpacingRatio} \times 40 = 120$ blocks (SHOULD), to keep an expiry of about 50 minutes, and keeps an explicitly configured delta (ZIP 218, “Default expiry delta”). ZIP 203, “Changes for NU7”, records that change for implementations creating transactions, from 40 to 120 blocks at NU7 activation, and leaves the expiry consensus rule unchanged. ZIP 315 is not updated for the 25-second spacing. At that spacing, ZIP 315’s SHOULD therefore has two readings: its literal 40 blocks, about $40 \times 25 = 1000$ s, and the default of ZIP 203 that it cites, which ZIP 203 gives as 120 blocks from NU7 activation, about $120 \times 25 = 3000$ s. ZIPs 203, 218 and 401 support the second; which one ZIP 315 intends is an open problem (Definition 1.1). Either time is an estimate, since block intervals vary, and not a guaranteed period of validity.

The choice of delta has a privacy consequence, specified. ZIP 315 lists the expiry height, with the anchor position, among the data that reveal to which height the creator was synchronised and some of its policy (ZIP 315, “Kinds of information leakage”), and it specifies a common default (ZIP 315, “Expiration height”). An expiry delta other than the common one therefore distinguishes its creator, as a fee other than the conventional fee does (§10).

The expiry height is fixed by the transaction identifier: nExpiryHeight is effecting data, an input of the header digest (ZIP 244, “T.1: header_digest”), so by Lemma 13.1(ii) a party matches a transaction, and rejects it once expired, by its identifier alone.

When a transaction expires, nodes remove it from their mempools together with every transaction that depends on it (ZIP 203, “Specification”). This is node policy layered on the consensus rule, which constrains only the contents of blocks.

9.4 The transaction record and locked notes

Definition 9.11 (Transaction record). The *transaction record* of a wallet holds, for each transaction t that the wallet creates:

- (i) the identifier $\text{txid}(t)$ and the encoding of t , once the wallet holds it;
- (ii) its fee, its target height H_t and its expiry height N_t ;
- (iii) the set of its inputs: the notes it spends in each shielded pool and the transparent out-points it spends;

- (iv) its mined height, or $\perp$ while it is unmined;
- (v) the earliest height h_t^{obs} at which it was observed: initially H_t for a transaction the wallet builds, and the height of the block in which it was first seen mined for a record entered from compact blocks;
- (vi) the latest tip at which the server reported it unmined.

A record entered without the encoding of the transaction has N_t unknown; such is the record of a transaction learnt from compact blocks, whose compact transaction carries no expiry height (Definition 4.3), and later unmined by a rollback (Definition 7.23). The transitions of the record are those of “Status tracking and terminal states” (§13.2). The record is information that the wallet retains, not a storage layout.

Definition 9.12 (Locked note). Fix an expiry delta $\delta \geq 1$, a parameter of the definition, whose default at the 25-second spacing is the open problem of §9.3. At target height H , a note is *locked* if and only if some recorded transaction t that spends it satisfies one of:

- (a) t is mined, necessarily at a height below H ;
- (b) $N_t = 0$;
- (c) $N_t \geq H$;
- (d) N_t is unknown and $h_t^{\text{obs}} + \delta \geq H$.

A locked note is excluded from selection.

Proposition 9.13 (Locks follow includability). *Let t be a recorded transaction with known expiry height N_t that spends a note. At target height $H = T + 1$, the conditions of Definition 9.12 hold for t if and only if t is mined or the expiry rule permits t at H (Lemma 9.9). Hence, for a note that no mined recorded transaction spends and whose recorded spending transactions all have known expiry heights, being unlocked is the clause of ZIP 315 that the TXO is not committed to be spent in another unexpired transaction created by the wallet (ZIP 315, “Categories of TXOs according to spendability”); a mined spend, which condition (a) locks, makes the note spent, and ZIP 315’s clause that the TXO is unspent excludes it; and an unmined t with $N_t > 0$ releases its inputs, as far as t is concerned, at every target height $H > N_t$.*

Proof. For known N_t , condition (d) of Definition 9.12 is void. By Lemma 9.9, the expiry rule permits t at H if and only if $N_t = 0$ or $H \leq N_t$, which are conditions (b) and (c); with condition (a) this gives the first claim. By Definition 9.8, t is unexpired at H if and only if $N_t = 0$ or $H \leq N_t$, so for an unmined t the conditions hold exactly when t is an unexpired transaction of the wallet that spends the note, and a note is unlocked exactly when no such transaction exists, which is ZIP 315’s clause (ZIP 203, “Specification”). For $H > N_t > 0$ and t unmined, each of (a) to (d) fails. $\square$

Remark 9.14 (Unknown expiry). Condition (d) of Definition 9.12 assumes that t was created with the delta δ at a target height at most h_t^{obs} , so that $N_t \leq h_t^{\text{obs}} + \delta$. If $N_t = 0$, or its creator used a

larger delta with $N_t > h_t^{\text{obs}} + \delta$, the note is released from $H = h_t^{\text{obs}} + \delta + 1$ while t can still be mined (for $H \leq N_t$, or at every H if $N_t = 0$), and a re-spend conflicts with the late mining of t . If its creator used a smaller delta, the note stays locked longer than necessary. Either misjudgement lasts until the wallet sees t mined, or obtains its encoding and hence N_t by the query Tx (Definition 5.10).

Definition 9.15 (Watched nullifiers). The *watched nullifiers* are the set $\mathcal{W} := \bigcup_p \mathcal{W}_p$ of the tracked sets of Definition 6.7: the nullifiers of the wallet’s detected notes not marked spent. A lock (Definition 9.12) removes a note from selection and never removes its nullifier from $\mathcal{W}$. A nullifier leaves $\mathcal{W}$ only when Construction 6.8 marks its note spent, that is, when a transaction revealing it is mined, whatever that transaction’s expiry height, $N = 0$ included; or when a rollback unmines the creating transaction of a Sapling note, whose nullifier is then discarded until a later scan derives it again (Definition 7.23(iii)). A rollback that clears the spent mark returns the nullifier to $\mathcal{W}$.

The mining of any transaction that spends a note of the wallet is therefore detected by scanning through $\mathcal{W}$, independently of status queries (*Ironwood Guide*, §“Nullifier sets”). A rule that removed from $\mathcal{W}$ the nullifiers spent by transactions with $N = 0$, combined with status polling that ends at the first report of an unmined transaction, would leave such a transaction, when it has no output to the wallet, mined and never observed as mined: no block would reveal a watched nullifier, and no note of the wallet would be created by it.

9.5 Spendable notes

Definition 9.16 (Spendability categories). Relative to a chain and a wallet state (ZIP 315, “Categories of TXOs according to spendability”):

- (i) a TXO is *known-spendable* if and only if (1) it is unspent at the wallet’s view of the tip and that view is reasonably up to date, (2) it is not committed to be spent in another unexpired transaction created by the wallet (Proposition 9.13), and (3) the wallet can effect the spend, by holding its spending key or by mediating interaction with a hardware device or another offline signing protocol;
- (ii) a shielded TXO is *confirmed-spendable* if and only if it is known-spendable, the wallet can construct its witness (Theorem 7.14), and either it is not an output of a wallet-internal shielding transaction and has c_{tr} confirmations if trusted, c_{untr} if untrusted (Definition 9.2), or it is such an output and meets the rule for shielding outputs of §9.1 (ZIP 315, “Shielded TXOs”);
- (iii) a known-spendable TXO that is not confirmed-spendable is *unconfirmed-spendable*;
- (iv) a TXO is *watch-only* if and only if the wallet holds its full viewing key, or for a transparent TXO its address, but not its spending key;
- (v) a transparent UTXO is *confirmed-shieldable* if and only if it is known-spendable and the wallet can construct a correct redeem script for it (ZIP 315, “Transparent UTXOs”).

The following rule is specified: a wallet MUST NOT attempt to spend, in a user-initiated transaction, a TXO that is not confirmed-spendable. ZIP 315 notes that the confirmed-spendable balance is 0 until the wallet has synchronised at least the nullifier set to the tip (ZIP 315, “Reporting of balances”).

Definition 9.17 (Spendable note at a target height). Let T be the tip, $H = T + 1$, $A = H - c_{\text{tr}}$ (Definition 9.5), and f the fully scanned height (Definition 6.11). A shielded note n of pool P , mined at height m , is *spendable at H* if and only if:

- (a) the note n is in the note state (Definition 6.7), it is not marked spent, and $f \geq T$;
- (b) it is not locked at H (Definition 9.12);
- (c) the wallet holds the spending key of the account of n , or mediates a signer for it;
- (d) it meets its depth: $T - m + 1 \geq c_{\text{tr}}$ or $T - m + 1 \geq c_{\text{untr}}$ according to its trust (Definition 9.2), or, if n is an output of a wallet-internal shielding transaction, the rule for shielding outputs of §9.1;
- (e) the authentication path of the position of n to the anchor of P at A is computable from retained information (Theorem 7.14, Proposition 7.17).

The *spendable set* at H is the set of notes spendable at H , each with its value, pool, position and authentication path at A .

Condition (d) implies $m \leq A$ in each case, since $c_{\text{untr}} \geq c_{\text{tr}}$ and the shielding transaction has at least c_{tr} confirmations, so n lies in the tree of the anchor by Lemma 9.6. The spendable set at H is the set of confirmed-spendable shielded notes of Definition 9.16, with ZIP 315’s “reasonably up-to-date” instantiated as $f \geq T$; the instantiation is designed but unspecified (Definition 1.1). Clause (a) is clause (1) of known-spendability: with $f \geq T$, every block up to the tip has been scanned, and a note is marked spent exactly when its nullifier is revealed on the best chain at or below T (Theorem 6.14, item (ii)). Clause (b) is clause (2) by Proposition 9.13, with condition (d) of Definition 9.12 in place of the expiry height where that is unknown; clause (c) is clause (3); clauses (e) and (d) are the witness and depth clauses of confirmed-spendability. The spendable set is the input state of construction (“Transaction construction”, §11): it is reconstructed by synchronisation (§6, Theorem 6.14) and taken as given by fee computation and note selection.

Remark 9.18 (Spending before the nullifier set is complete). A policy that admits a note once the block extent of its own subtree is scanned up to A (Definition 7.16), before every range up to T is scanned, departs from clause (a) of Definition 9.17 and from ZIP 315, whose clause on an up-to-date view carries an open TODO in the ZIP. The departure fails as follows. A note whose spend was mined in a range not yet scanned passes the policy; a transaction that spends it again is rejected by the consensus rule that a nullifier must not repeat (*Ironwood Guide*, §“Nullifier sets”), and its repeated nullifier links it to the earlier spend for every party that sees it. The departure is designed but unspecified (Definition 1.1).

10 Fees (ZIP 317)

This section states the fee a wallet pays. It fixes the fee as a consensus quantity, defines the conventional fee of ZIP 317 over logical actions and bounds it before signing; it states the padded Action counts on which the fee is computed; and it states the relay, mempool and block-template rules that apply to a transaction paying less.

10.1 The conventional fee

For a transaction other than a coinbase transaction, the *transaction fee* is the value remaining in its transparent transaction value pool: the sum of its transparent input values and of the balancing values of its shielded pools, minus the sum of its transparent output values (protocol specification, §“Transactions and Treestates”; *Ironwood Guide*, Definition “Transparent transaction value pool” and Definition “Balancing value”, in “The balancing value”). Consensus requires it to be non-negative. No transaction field encodes it, and every term of the sum is public, so the fee is public.

A fee rule for a shielded protocol meets the requirements of ZIP 317, “Requirements”: the fee is computable from the public data of the transaction alone, since a fee that depended on private construction data, such as which notes were spent or what change was returned, would disclose that data; it scales linearly with the numbers of inputs and outputs, which makes transaction-rate denial of service costly (ZIP 317, “Motivation” and “Denial of Service”); it neither penalises padding for privacy nor favours any pool; and it lets a small number of inputs of value below the marginal fee be spent. A fee other than the one every wallet pays distinguishes the wallet that chose it: non-standard fees may reveal specific users, wallets or wallet versions (ZIP 317, “Security and Privacy considerations”).

The conventional fee (Definition 10.2) is not a consensus rule. ZIP 317, “Fee calculation”, states that it is not a consensus requirement that fees follow the formula, and that wallets SHOULD create transactions that pay it, to reduce information leakage, unless overridden by the user. It binds through relay and block-template policy only (§10.3). ZIP 259, “NU7 consensus changes”, changes neither the formula nor its parameters.

In what follows, $\lceil x \rceil$ is the least integer $\geq x$ and $\lfloor x \rfloor$ the greatest integer $\leq x$, for real x .

Definition 10.1 (Logical actions). Let a transaction of version 5 or version 6 have the following public fields of its final encoding (protocol specification, §“Transaction Encoding and Consensus”; ZIP 229, “Transaction Format”), each a non-negative integer:

- $tx_in_total_size$ and $tx_out_total_size$, the byte lengths of the encoded fields tx_in and tx_out , the concatenations of the encoded transparent inputs and outputs;
- $nJoinSplit$, the number of Sprout JoinSplit descriptions;
- $nSpendsSapling$ and $nOutputsSapling$, the numbers of Sapling spend and output descriptions;
- $nActionsOrchard$, the number of Orchard-pool Actions;
- $nActionsIronwood$, the number of Ironwood-pool Actions, 0 in a version 5 transaction.

With the parameters $p2pkh_standard_input_size := 150$ and $p2pkh_standard_output_size := 34$, in bytes, the number of *logical actions* of the transaction is

$$\begin{aligned} logical_actions := & \max(\lceil tx_in_total_size / 150 \rceil, \lceil tx_out_total_size / 34 \rceil) \\ & + 2\ nJoinSplit + \max(nSpendSapling, nOutputsSapling) \\ & + nActionsOrchard + nActionsIronwood, \end{aligned}$$

the sum of the transparent, Sprout, Sapling, Orchard and Ironwood contributions, in that order (ZIP 317, “Fee calculation”, the contributions of Revision 0 and Revision 1).

The Sprout contribution $2\ nJoinSplit$ is 0 in every valid transaction: transactions of version 5 and version 6 have no JoinSplit field, and transactions of version 4 are invalid under NU7 (ZIP 2003; ZIP 259, “NU7 consensus changes”). The term is kept so that the definition is that of ZIP 317. The two divisors are the maximal encoded sizes of a P2PKH input and output (ZIP 317, “Rationale for the chosen parameters”):

$$\begin{aligned} 150 &= 36 \text{ (outpoint: 32-byte txid, 4-byte index)} + 110 \text{ (script)} + 4 \text{ (sequence)}, \\ 110 &= 1 \text{ (length)} + 1 \text{ (signature length)} + 72 \text{ (signature)} + 1 \text{ (sighash type)} \\ &\quad + 1 \text{ (key length)} + 33 \text{ (compressed key)} + 1 \text{ (margin)}, \\ 34 &= 8 \text{ (value)} + 1 \text{ (script length)} + 25 \text{ (P2PKH script)}. \end{aligned}$$

Charging transparent parts by size rather than by count charges a large script in proportion to its size; a 2-of-3 P2SH multisig input, about 70% larger than a P2PKH input, is counted so (same section).

Definition 10.2 (Conventional fee). Let $marginal_fee := 5000$ zatoshi per logical action and $grace_actions := 2$. The *conventional fee* of a transaction with *logical_actions* as in Definition 10.1 is

$$conventional_fee := marginal_fee \cdot \max(grace_actions, logical_actions) \text{ zatoshi}$$

(ZIP 317, “Fee calculation”).

The parameter values and their rationale are ZIP 317’s (“Rationale for the chosen parameters”).

Remark 10.3 (Revisions of the fee rule). ZIP 317, “Fee calculation”, defines *logical_actions* as the sum of the contributions of the applicable revision, and a contribution defined by a revision is included in every later one (ZIP 317, “Revisions”). Revision 0 defines the transparent, Sprout, Sapling and Orchard contributions. Revision 1 adds the Ironwood contribution $nActionsIronwood$: Ironwood-pool Actions have the Action structure of Orchard-pool Actions and are counted identically. Revision 2 adds a contribution for memo bundles (ZIP 231), defined on the transaction format of ZIP 248, which NU7 does not introduce (ZIP 259, “Rationale for no new transaction format”). Definition 10.1 is therefore Revision 0 with the Revision 1 term. The statuses of these ZIPs are those of Table 1.

Corollary 10.4 (The minimum fee). *Every transaction with $\text{logical_actions} \leq 2$ has conventional fee $\text{marginal_fee} \cdot \text{grace_actions} = 5000 \cdot 2 = 10,000$ zatoshi, and no transaction has a smaller conventional fee.*

Proof. By Definition 10.2, since $\max(2, \text{logical_actions}) = 2$ when $\text{logical_actions} \leq 2$, and $\max(2, \text{logical_actions}) \geq 2$ always. $\square$

A wallet computes the fee before it signs, and the encoded size of a transparent input depends on its signature. The following proposition bounds the fee from the unsigned transaction.

Proposition 10.5 (Fee estimate before signing). *Let a wallet fix, before signing, every field of a transaction except the scripts of its transparent inputs $1, \dots, k$, and assign to each input i a bound $b_i \in \mathbb{N}$ on its encoded size after signing, as follows:*

- (i) $b_i = 150$ for a P2PKH input spending a public key with a compressed 33-byte encoding, such as each of the wallet’s own transparent keys;
- (ii) otherwise any b_i known to bound the encoded size of the signed input.

Let fee_{est} be the conventional fee computed with tx_in_total_size replaced by $\sum_{i=1}^k b_i$. Then fee_{est} is at least the conventional fee of the signed transaction.

Proof. Signatures enter only the scripts of the transparent inputs, so the field tx_out and every shielded count of Definition 10.1 are the same in the unsigned and the signed transaction. A DER-encoded signature has at most 72 bytes, so the script of a P2PKH input with a compressed key has at most $1 + 1 + 72 + 1 + 1 + 33 = 109$ bytes and the input at most $36 + 109 + 4 = 149 \leq 150$ bytes; under clause (ii) the bound holds by hypothesis. The signed tx_in_total_size is therefore at most $\sum_i b_i$. The maps $x \mapsto \lceil x/150 \rceil$, $(a, b) \mapsto \max(a, b)$, the addition of the unchanged remaining contributions, and $n \mapsto 5000 \cdot \max(2, n)$ are each non-decreasing in every argument, so their composition is, and fee_{est} is at least the conventional fee of the signed transaction. $\square$

Hypothesis (i) does not extend to a P2PKH input with an uncompressed 65-byte key encoding, which has up to 181 bytes and needs $b_i \geq 181$ under (ii). For an input with no known bound on its signed size, the fee is unknown before signing.

10.2 The fee on padded counts

The fields of Definition 10.1 are those of the final encoded transaction: every bundle padded and every change output included (ZIP 317, “Fee calculation”). A wallet therefore prices a transaction by predicting its padded shape (Definition 10.6) and its change before it selects inputs; a fee computed on unpadded counts underpays whenever padding adds a logical action. This rule is ZIP 317’s and is not a consensus rule.

Definition 10.6 (Padded Action count). Let a bundle of the Orchard protocol, in the Orchard pool or the Ironwood pool, have $s \in \mathbb{N}$ real spends and $o \in \mathbb{N}$ real outputs, and let

$\text{enableCrossAddress} \in \{0, 1\}$ be its flag value (*Ironwood Guide*, §“Bundle flags”, Definition “Flag values and expanded receiver”). Its *requested Action count* is

$$r(s, o) := \begin{cases} \max(s, o) & \text{if } \text{enableCrossAddress} = 1, \\ s + o & \text{if } \text{enableCrossAddress} = 0, \end{cases}$$

and its *padded Action count* is $n(s, o) := 0$ if $s = o = 0$ and $n(s, o) := \max(r(s, o), 2)$ otherwise. The counts $n\text{ActionsOrchard}$ and $n\text{ActionsIronwood}$ of Definition 10.1 are the padded Action counts of the two bundles. A non-empty Sapling bundle with s real spends and o real outputs has $n\text{SpendSapling} = s$ and $n\text{OutputsSapling} = \max(o, 2)$.

The count $s + o$ is that of the pairing a wallet performs when cross-address transfers are disabled: the note an Action creates has the expanded receiver of the note it consumes, so each real spend occupies an Action with a fabricated zero-valued output at its own receiver, and each real output an Action with a fabricated zero-valued spend at its receiver (ZIP 326, “Fabricated same-address outputs and randomized note ciphertexts”; ZIP 374, “Fabricated same-address outputs (NU6.3)”), respectively. Every Orchard-pool bundle has $\text{enableCrossAddress} = 0$ (*Ironwood Guide*, “Chain state and pool rules”, rule (i); ZIP 258, “Consensus rules from NU6.3 activation”); an Ironwood-pool bundle may set it to 1.

The floor 2 and the Sapling padding are a padding policy, designed but unspecified in the sense of Definition 1.1. Their only general ZIP anchor is non-normative: ZIP 317, “Requirements”, notes that standard builders pad to two inputs and two outputs for each shielded pool, and “Rationale for the chosen parameters” calls padding to two actions the standard approach; ZIP 318 fixes the padding of its migration and note-preparation transactions only (below). The Sapling rule departs from ZIP 317’s remark by adding no spend beyond those needed. Dummy Sapling spends, which this policy does not use, are constructed as in protocol specification, §“Dummy Notes (Sapling)”; the dummy Actions of the Orchard protocol are those of §11.7. Logical actions count an Action once: a charge per input plus per output would penalise the padding that the Action structure forces, since each Action consumes one note and creates one, and the maximum in each contribution favours no pool (ZIP 317, “Rationale for logical actions”).

ZIP 318 specifies two exceptions. Under “Canonical migration transaction structure”, a canonical migration transaction has one Orchard-pool spend in an Orchard-pool bundle of exactly two Actions, one spend Action and one output Action that carries either change or a zero-valued dummy output (ZIP 318, “Rationale for the canonical transaction shape”), and one Ironwood-pool output in an unpadded Ironwood-pool bundle of one Action. Its *logical_actions* is 3 and its conventional fee 15,000 zatoshi. The “canonical fee (provisionally the ZIP 317 minimum fee)” of ZIP 318 is read here as this conventional fee: read as 10,000 zatoshi, it leaves one unpaid action (Definition 10.8). A note-preparation transaction SHOULD have an Orchard-pool bundle of exactly 16 Actions, padded as needed (ZIP 318, “Note preparation transactions”), so its count is 16, not $n(s, o)$.

Proposition 10.7 (Fee of a single-pool payment). *Consider a transaction with no transparent inputs or outputs, one input note and one payment output, with or without one change output, whose bundles are padded as in Definition 10.6.*

- (a) If the input note and every output lie in the Ironwood pool, in a bundle with `enableCrossAddress = 1`, or all lie in the Sapling pool, the conventional fee is 10,000 zatoshi.
- (b) If the input is a single Orchard-pool note, the payment output lies in the Ironwood pool, and the change output, if any, lies in the Orchard pool or the Ironwood pool, then the conventional fee is 20,000 zatoshi.

Proof. (a) In the Ironwood pool $s = 1$ and $o \in \{1, 2\}$, so $r = \max(1, o) \leq 2$ and $n = 2$. In the Sapling pool the contribution is $\max(1, \max(o, 2)) = 2$. In either case $\text{logical_actions} = 2$, and Corollary 10.4 gives 10,000 zatoshi.

(b) No value enters the Orchard pool, since $\text{valueBalanceOrchard} \geq 0$ for every transaction (*Ironwood Guide*, “Chain state and pool rules”, rule (ii); ZIP 258, “Consensus rules from NU6.3 activation”), and an Orchard-pool output lies at the receiver of the note consumed in its Action; so a payment to another party cannot lie in the Orchard pool, and the full routing rule is that of §11.2. The payment lies in the Ironwood pool by hypothesis. The Orchard-pool bundle has $s = 1$ and $o \in \{0, 1\}$, so $r = s + o \leq 2$ and $n = 2$. The Ironwood-pool bundle has $s = 0$ and $o \in \{1, 2\}$, so $n = 2$. Hence $\text{logical_actions} = 4$ and the conventional fee is $5000 \cdot 4 = 20,000$ zatoshi. $\square$

10.3 Relay, mempool, and block template

The conventional fee binds through three mechanisms, none of them a consensus rule. Nodes that normally relay transactions are expected to relay those paying at least the conventional fee, unless other reasons of robustness or denial-of-service mitigation apply (ZIP 317, “Transaction relaying”). Mempool size limiting adds a penalty *low_fee_penalty* to the eviction weight of a transaction paying less than the conventional fee (ZIP 401; ZIP 317, “Mempool size limiting”). Block-template construction limits the unpaid actions of a block (Definition 10.8).

Definition 10.8 (Unpaid actions). For a transaction tx with fee $tx.\text{fee}$ as in §10.1 and $tx.\text{logical_actions}$ as in Definition 10.1, the number of *unpaid actions* is $\text{unpaid_actions}(tx) := 0$ if tx is a coinbase transaction, and otherwise

$$\text{unpaid_actions}(tx) := \max(0, \max(\text{grace_actions}, tx.\text{logical_actions}) - \lfloor tx.\text{fee} / \text{marginal_fee} \rfloor).$$

The unpaid actions of a block are the sum over its transactions. The recommended block-template algorithm adds a transaction that pays less than the conventional fee only while that sum stays at most $\text{block_unpaid_action_limit} = 50$ (ZIP 317, “Recommended algorithm for block template construction”).

Lemma 10.9 (Zero unpaid actions). *A transaction other than a coinbase transaction has $\text{unpaid_actions} = 0$ if and only if its fee is at least its conventional fee.*

Proof. Let $m := \max(\text{grace_actions}, \text{logical_actions})$, an integer. By Definition 10.8, $\text{unpaid_actions} = 0$ if and only if $\lfloor \text{fee} / \text{marginal_fee} \rfloor \geq m$. For an integer m and real x ,

$\lfloor x \rfloor \geq m$ if and only if $x \geq m$; and since $\text{marginal_fee} > 0$, $\text{fee}/\text{marginal_fee} \geq m$ if and only if $\text{fee} \geq \text{marginal_fee} \cdot m$, which is the conventional fee of Definition 10.2. $\square$

A transaction with more than $\text{block_unpaid_action_limit} = 50$ unpaid actions is never included by the recommended algorithm, and nodes MAY drop it, or any transaction with more unpaid actions than a configurable limit (ZIP 317, “Transaction relaying”). A relay policy may set that limit to 0; by Lemma 10.9 it then drops exactly the transactions that pay less than the conventional fee. The conventional fee is therefore the least fee that passes every unpaid-action limit, whatever its value. For example, the migration transaction of §10.2 with $\text{logical_actions} = 3$ has one unpaid action at a fee of 10,000 zatoshi.

A logical action is not the unit of the block limits of ZIP 218, “Shielded pool action limits”: these count the sum $n\text{SpendSapling} + n\text{OutputSapling}$ of a block’s Sapling descriptions, not the maximum that Definition 10.1 charges, so paying the conventional fee does not ensure that a transaction fits a given block (*Consensus Guide*, “A block budget for shielded work”). The limits add no transaction field. Nor does the block-level fee contribution of ZIP 259, “Rationale for no new transaction format”: ZIP 235, which NU7 deploys (ZIP 259, “NU7 consensus changes”), removes from circulation $\lfloor 6F/10 \rfloor$ of the total fee F of the non-coinbase transactions of a block, and the coinbase transaction claims only the remaining $F - \lfloor 6F/10 \rfloor$ of it (ZIP 235, “Consensus Rule Changes”). The rule is specified (Definition 1.1); it divides the fees of a block between removal and the miner, and changes neither the fee that a transaction pays nor the conventional fee.

11 Transaction construction

This section states how a wallet turns payments into transactions. It defines the transaction plan and its balance equation; the pool in which each payment and each change output lies; transfers to TEX addresses and shielding transactions; input selection as a fixed point of the fee; change; the outgoing viewing key of every output; the assembly of Orchard-protocol bundles with dummy Actions in uniform order; and the authorisation of a built transaction, with the theorem that the result pays what the plan states. The input state is the spendable set of Definition 9.17; the fee is the conventional fee of §10.

11.1 The transaction plan

Definition 11.1 (Payment and payment request). A *payment* is a transfer of funds implemented by a single output of a transaction: a triple (a, v, m) of a recipient address a (Definition 3.20), a value $v \in \mathbb{N}$ in zatoshi and an optional memo m , a memo being admitted only when a is memo-capable. A *payment request* is a finite map R from indices to payments; it asks for one transaction that pays every payment in its range (ZIP 321, “Terminology”). ZIP 321 encodes payment requests as URIs (§14.1); the label and message parameters of that encoding are display data and enter no transaction.

Construction has two phases. *Planning* maps a payment request and the wallet state, its spendable notes (Definition 9.17) and its tip, to a plan exact to the zatoshi, and uses no spending key. *Execution* builds, proves and authorises the transactions of the plan and records them (§13); submission

is a separate act. Every choice of inputs, fee and change is fixed in the plan before a spending key is used. No ZIP defines plans: the two-phase structure is designed but unspecified (Definition 1.1).

Definition 11.2 (Transaction plan). A *transaction plan* is a non-empty sequence $P = (S_1, \dots, S_\ell)$ of *steps*. Each step describes one transaction by:

- (i) its target height H (Definition 9.4), anchor height A (Definition 9.5) and expiry height N (Definition 9.8);
- (ii) its *spending account*, the account of the wallet to which every input of the step belongs;
- (iii) its inputs, partitioned by pool (transparent, Sapling, Orchard, Ironwood), each either a note of the spending account spendable at H (Definition 9.17), a transparent UTXO of that account that meets its depth (§9.1), or a reference (j, i) to output i of an earlier step S_j ;
- (iv) its payment outputs, each a payment of the request together with the pool assigned to it (§11.2);
- (v) its change outputs, each a value and a pool;
- (vi) its fee $f \in \mathbb{N}$, computed under the conventional fee rule (Definition 10.2).

A transparent output of a step may be marked *ephemeral*: it pays a fresh transparent address of the wallet and exists only to be spent by a later step (§11.3). Each step satisfies the *balance equation*

$$\sum_{\text{inputs}} v = \sum_{\text{payments}} v + \sum_{\text{change}} v + f,$$

the sums taken over all pools, ephemeral outputs counted among the payments of their step. In the built transaction f is the value remaining in the transparent transaction value pool: the sum of the transparent inputs, minus the transparent outputs, plus every shielded balancing value (*Ironwood Guide*, §“The balancing value”, Definitions “Transparent transaction value pool” and “Balancing value”).

Clause (ii) restricts this section to steps funded by one account; ZIP 316 also admits transfers whose sending account is undetermined, a case that enters only the choice of outgoing viewing key (§11.6).

Definition 11.3 (Well-formed plan). A plan $(S_1, \dots, S_\ell)$ is *well formed* if and only if

- (i) every reference (j, i) among the inputs of step S_k has $j < k$;
- (ii) every output of an earlier step that a step spends is ephemeral; no step spends a change output of an earlier step;
- (iii) no output, of the wallet or of a step, is spent twice within the plan;
- (iv) every ephemeral output of a step is spent by exactly one later step.

Proposition 11.4 (Chaining through transparent outputs). *Let step S_k of a plan spend output i of an earlier step S_j , and let t_j be the transaction built for S_j .*

- (a) *If the output is transparent, every field of the transaction of S_k is computable once t_j is built, before t_j is mined.*
- (b) *If the output is a shielded note of non-zero value, the proof that covers its spend (its Sapling spend proof, or the proof of its Orchard-protocol bundle) is not computable before t_j is mined.*
- (c) *If the output is an Orchard-protocol note and the transaction of S_k has version 6, its signature digest, and hence every spend-authorisation and binding signature of it, is computable once t_j is built; only proving waits for the mining of t_j .*
- (d) *If the output is a Sapling note, the signature digest of the transaction of S_k is not computable before t_j is mined.*

A plan whose steps are all completed before any of them is submitted therefore passes value between steps only through transparent outputs, which Definition 11.3 (ii) and the ephemeral outputs of Definition 11.2 realise.

Proof. (a) A transparent input names the outpoint $(\text{txid}(t_j), i)$ and is signed over a digest that commits to the value and script of the spent coin (ZIP 244, “S.2: transparent_sig_digest”). The identifier is a function of the effecting data of t_j (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”), and value and script are fixed by S_j ; none depends on the mining of t_j .

(b) A proof for a spend of a note of non-zero value needs the note’s authentication path to the anchor (*Ironwood Guide*, §“Authentication paths”; condition A3 of the Action statement, which exempts only a consumed note of value 0). The path is a function of the note’s position in its tree, which is fixed only when t_j is mined (Definition 4.11).

(c) The version-6 signature digest commits to no anchor and to no proof (ZIP 229, “Anchor commitment (version 6)”; *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data” and Definition “Signature digest”). The effecting data of an Orchard-protocol spend are computable before mining: its nullifier is $\text{DeriveNullifier}_{\text{rk}}(\rho, \psi, \text{cm})$, a function of note data and not of the position (*Ironwood Guide*, §“The nullifier”), and ρ of the note is fixed by the Action of t_j that creates it; the key rk and the commitment cv^{net} depend on keys, values and fresh randomness. ZIP 374, “Anchors and pre-authorisation”, states this workflow, and ZIP 318, “Note preparation transactions”, uses it.

(d) A Sapling nullifier is computed from

$$\rho = \text{MixingPedersenHash}(\text{cm}, \text{pos}),$$

where pos is the note’s position (protocol specification, §“Computing ρ values and Nullifiers”), and it is effecting data, committed by the signature digest (ZIP 244, “Signature Digest”).

For the final claim: by (b) and (d), a step that spends a shielded note of non-zero value of an earlier step cannot be completed before that step’s transaction is mined, so a plan completed as

a unit passes value only through transparent outputs, which (a) admits; a zero-valued Orchard-protocol note carries no value. $\square$

The partially created form of (c) is that of §12.5. Fee and change are fixed jointly: the change outputs enter the padded counts of Definition 10.6 and hence the conventional fee, and the fee determines the change value; §11.5 resolves the dependence.

Execution is not deterministic. The note seeds $rseed$, hence esk , ψ and rcm , the value-commitment trapdoors rcv , the randomisers α , the proofs and the permutations of §11.7 are drawn fresh (*Ironwood Guide*, §“Construction of a transaction”), so one plan executed twice yields transactions with different effecting data and different transaction identifiers. A plan fixes values and shape, not the transaction identifier.

11.2 Payments to pools

Construction 11.5 (Output pool of a payment). The output of a payment (a, v, m) is determined by a :

- (i) a transparent address yields a transparent output to it;
- (ii) a TEX address yields a transparent P2PKH output of the second transaction of a ZIP 320 transfer (§11.3);
- (iii) a Sapling address yields a Sapling output;
- (iv) a unified address yields an output to its receiver of the most preferred type in the Priority List of §3.3 that the sender supports, which ZIP 316 makes a MUST (ZIP 316, “Encoding of Unified Addresses”); an Orchard receiver is paid by an Ironwood-pool output;
- (v) an address with no receiver that the sender supports is refused, and so is a payment with a memo whose output is transparent.

Case (iv) is sound because both pools of the Orchard protocol share keys, addresses and receivers (*Ironwood Guide*, §“The Orchard protocol and its two pools”; ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes” and “Receiving funds into the Ironwood pool”).

Orchard-pool rules. Consensus admits no value into the Orchard pool: for every transaction, $valueBalanceOrchard \geq 0$; and every Orchard-pool Action has cross-address transfers disabled, so the note it creates lies at the expanded receiver of the note it consumes (ZIP 258, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, §“Chain state and pool rules”, rules (i) and (ii), and §“Bundle flags”). A wallet MUST NOT send funds to any external receiver, its own included, in the Orchard pool (ZIP 326, “Wallet key-generation restrictions”), and MUST NOT use its knowledge of the internal addresses of all its accounts to send funds from one account to an internal address of another (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”). A wallet that generates `use_qsk = true` keys MUST NOT send Orchard-pool funds to them at any point (ZIP 326, “Wallet key-generation restrictions”). This volume reads “external receiver” as every receiver that is not an internal-scope receiver (Construction 2.14) of an account of the wallet.

Change. A change output lies at an internal-scope address of the spending account (§11.5) in the Ironwood pool, in the Sapling pool, or, only as Proposition 11.6 allows, in the Orchard pool. The choice among these is wallet policy, designed but unspecified (Definition 1.1); a change output in a pool other than that of the inputs moves value between pools, and the moved amount is published in the balancing values.

Proposition 11.6 (Orchard-pool change). *In a transaction built by a wallet that follows Construction 11.5, every Orchard-pool output of non-zero value lies at the expanded receiver of the note consumed by its own Action, a real note or a fabricated zero-valued note, and that receiver is an internal-scope receiver of the spending account. Hence no payment output lies in the Orchard pool; non-zero change remains in the Orchard pool only at internal-scope receivers of the spending account, with total at most the value of the Orchard-pool notes spent; any other change lies in the Ironwood or the Sapling pool.*

Proof. With `enableCrossAddress = 0`, condition A10 of the Action statement (*Ironwood Guide*, §“Bundle flags” and §“Chain state and pool rules”, rule (i)) places each Orchard-pool output at the expanded receiver of the note consumed in its Action. By the first rule of ZIP 326 under the reading of Construction 11.5, an output of non-zero value lies at an internal-scope receiver of an account of the wallet, and by the second that account is the spending account. A payment output is routed by cases (i) to (iv) of Construction 11.5, none of which yields an Orchard-pool output. Let V_{in} and V_{out} be the totals of the consumed and created Orchard-pool notes; the balancing value of the Orchard pool is $V_{\text{in}} - V_{\text{out}} \geq 0$ (rule (ii)), so the Orchard-pool change, a part of V_{out} , is at most V_{in} . Every other change output lies, by Construction 11.5, in the Ironwood or the Sapling pool. $\square$

The following consequences are cited later rather than restated. Orchard-pool bundles are priced by Definition 10.6 with the requested count $s + o$; Orchard-pool fabricated outputs and spends are assembled in §11.7; and the canonical migration and note-preparation transactions of ZIP 318 are the exceptions to the padding floor stated in §10.2.

One Orchard-protocol spending key authorises spends in both Orchard-protocol pools. Ironwood-pool and Orchard-pool spends use the same full viewing key and the same spend authorising key ask, each pool’s bundle with its own note commitment tree, anchor and nullifier set (*Ironwood Guide*, §“Key capabilities and scope”, §“The Orchard protocol and its two pools” and §“Chain state and pool rules”); the two bundles of one transaction are built, proved and signed side by side.

Migration is specified. ZIP 2005 states that wallets SHOULD move all funds they control, transparent, Sprout and Sapling funds included, into recoverable Ironwood-pool notes as soon as practically possible, and that other wallets are REQUIRED to be able to receive them (ZIP 2005, “Proactive movement of funds to recoverable notes”). ZIP 318 specifies a scheduled migration from the Orchard pool to the Ironwood pool; its scheduling rules and privacy analysis are cited, not restated (ZIP 318, “Specification” and “Privacy Implications”). Its block-count constants are stated for a 75-second block spacing; their treatment under the 25-second spacing of ZIP 218, which NU7 deploys (ZIP 259, “NU7 consensus changes”), is an open problem (Definition 1.1). The privacy of the choice of source pool is stated in §11.4.

11.3 TEX transfers and shielding transactions

Construction 11.7 (Transfer to a TEX address). A sender paying one or more TEX addresses from shielded funds **MUST** create two transactions (ZIP 320, “Design considerations for Senders”), realised as two steps of one plan.

- (1) The first transaction tr_0 spends shielded notes only and pays, in place of the n TEX payments of values $v_1, \dots, v_n$, one ephemeral transparent output of value

$$v_{\text{eph}} := \sum_{i=1}^n v_i + \text{fee}(tr_1)$$

to a fresh ephemeral transparent address of the wallet.

- (2) The second transaction tr_1 spends exactly that output and pays the n TEX payments as P2PKH outputs (*Crypto Guide*, §“The transparent layer’s hashes”), with no memo and no change, so that its input equals its outputs plus its fee.

Here $\text{fee}(tr_1)$ is the conventional fee of one P2PKH input priced at 150 bytes (Proposition 10.5 (i), the ephemeral key being a compressed key of the wallet) and n P2PKH outputs of 34 bytes each:

$$\text{fee}(tr_1) = 5000 \cdot \max(2, \max(\lceil 150/150 \rceil, \lceil 34n/34 \rceil)) = 5000 \cdot \max(2, n) \text{ zatoshi},$$

which is 10,000 zatoshi for $n = 1$ and $n = 2$ and 15,000 zatoshi for $n = 3$. A single transaction that pays a TEX address and spends shielded inputs is refused: a sender to a TEX address **MUST** spend only transparent (P2SH or P2PKH) UTXOs in that transaction, and **SHOULD** pay only transparent outputs in it (ZIP 320, “Specification”). ZIP 320 does not require that those UTXOs come from a single transparent address.

The plan of Construction 11.7 is well formed (Definition 11.3): its one reference points from tr_1 to the ephemeral output of tr_0 , which is spent exactly once. By Proposition 11.4 (a) both transactions are completed before either is mined, and ZIP 315 exempts the ephemeral UTXO from the confirmation floor (§9.1).

Ephemeral addresses are specified. A wallet based on ZIP 32 **SHOULD** choose ephemeral addresses so that their private keys are recoverable from the seed, **SHOULD NOT** choose them so that they can be linked between transactions without the seed or the relevant transparent viewing keys, and so **SHOULD** avoid collisions with the addresses of earlier outputs, change outputs included; a wallet **MUST** recognise, and be able to spend, funds that a recipient returns to an ephemeral address (ZIP 320, “Design considerations for Senders”). An ephemeral address reserved for a construction that fails is not reused, since reuse would link the two transactions; this rule is designed but unspecified (Definition 1.1).

Construction 11.8 (Shielding transaction). A *shielding transaction* is the transaction of a one-step plan with an empty payment request. It spends transparent UTXOs of the spending account and pays their total, less the conventional fee, to change outputs at an internal-scope shielded receiver of that account (Construction 2.14), with no other recipient and no

transparent output. Its outputs lie in the Ironwood pool, since no value enters the Orchard pool (Construction 11.5) and ZIP 2005 asks for recoverable Ironwood-pool notes (ZIP 2005, “Proactive movement of funds to recoverable notes”); its balancing value in that pool is negative. Its UTXOs may be spent with zero confirmations (§9.1). The value threshold at which a wallet shields is wallet policy, designed but unspecified (Definition 1.1).

Shielding rules are specified. A wallet that receives transparent funds **SHOULD** auto-shield them by default; it **SHOULD NOT** auto-shield from several transparent addresses in one transaction, and **SHOULD NOT** use *opportunistic shielding*, the shielding of received transparent funds within a user-initiated transaction, which would show its recipients the link between the transparent addresses and the payment (ZIP 315, “Long-term storage of funds”). Users **MUST** be able to disable auto-shielding (ZIP 315, “Auto-shielding”). Spent transparent UTXOs **SHOULD** be sent only to an internal shielded receiver of the wallet, except the ephemeral outputs of a ZIP 320 transfer (ZIP 315, “Linkability of transactions or addresses”).

Transparent inputs are public, so a transaction that spends UTXOs of different transparent receivers links those receivers. ZIP 315 states that UTXOs received on different transparent receivers **SHOULD NOT** be shielded in one transaction, and that shielding several UTXOs of one address together leaks nothing further (ZIP 315, “Linkability of transactions or addresses”). The same holds for ephemeral addresses, so a transaction spends the ephemeral output of at most one ZIP 320 transfer.

11.4 Input selection

Definition 11.9 (Covering selection). Fix a step with payments of total out , target height H , anchor height A and a number k of change outputs (§11.5 fixes k). Let U be the finite set of inputs of the spending account that Definition 11.2 (iii) admits at (H, A) , each $u \in U$ with value $v(u)$, and let $\mathcal{S} \subseteq 2^U$ be the family of *admissible* selections, those that meet the pool rules stated at the end of this subsection. For $I \in \mathcal{S}$ let $shape(I)$ be the public counts of Definition 10.1 of the step funded by I , with its payments and k change outputs, after padding (Definition 10.6), and let $fee(I) := \text{conventional_fee}(shape(I))$ (Definition 10.2). A selection $I \in \mathcal{S}$ is *covering* if and only if

$$\sum_{u \in I} v(u) \geq out + fee(I).$$

Adding an input never lowers a count of Definition 10.1, so fee is non-decreasing under inclusion, and a covering selection is a fixed point of the fee: its total must cover a fee that depends on it. A plan requires a covering selection for each step; any procedure that returns one conforms. The following construction is one such procedure.

Construction 11.10 (Fee escalation). Set $I_0 := \emptyset$ and $t_0 := 0$. At round $k \geq 0$:

- (1) compute the balance of the step on I_k (§11.5 gives its exact form);
- (2) if I_k is covering, return I_k ;

- (3) otherwise let $r_k := \text{out} + \text{fee}(I_k) - t_k > 0$ be the shortfall, and choose $I_{k+1} \in \mathcal{S}$ with total $t_{k+1} := \sum_{u \in I_{k+1}} v(u) \geq t_k + r_k$; if no such I_{k+1} exists, fail with insufficient funds.

The choice of I_{k+1} among the admissible sets that step (3) allows is the wallet's.

Proposition 11.11 (Termination of fee escalation). *Fee escalation terminates after finitely many rounds. On success the returned selection is covering, and the fee of the step is the conventional fee of its final padded shape. On failure at round k , no admissible selection has total at least $\text{out} + \text{fee}(I_k)$. No bound on the number of rounds is claimed beyond this finiteness.*

Proof. A round that does not stop has $t_{k+1} \geq t_k + r_k > t_k$, so the totals increase strictly. Every t_k lies in the finite set $\{\sum_{u \in I} v(u) : I \in \mathcal{S}\} \cup \{0\}$, which admits no infinite strictly increasing sequence. On success the returned I_k is covering by step (2), and its fee is $\text{fee}(I_k)$, the conventional fee of $\text{shape}(I_k)$ by Definition 11.9. On failure, step (3) found no admissible set of total at least $t_k + r_k = \text{out} + \text{fee}(I_k)$. $\square$

A failure does not show that no covering selection exists: an admissible set J with $\text{fee}(J) < \text{fee}(I_k)$ and total in $[\text{out} + \text{fee}(J), \text{out} + \text{fee}(I_k))$ covers. Fee escalation is therefore sound, returning only covering selections, and complete only for choice rules that exclude this case.

Definition 11.12 (Uneconomic input). An input u is *uneconomic* if and only if $v(u) < \text{marginal_fee}$, where $\text{marginal_fee} = 5000$ zatoshi (Definition 10.2). The strict inequality is ZIP 317's wording, “below the marginal fee” (ZIP 317, “Requirements” and “Rationale for the chosen parameters”).

Lemma 11.13 (Cost of an additional input). *Let L be the number of logical actions of a step (Definition 10.1) and L' that after adding one input.*

- (i) *A P2PKH input priced at 150 bytes gives $L' - L \in \{0, 1\}$.*
- (ii) *A shielded spend added to a non-empty bundle with s real spends and o real outputs, $s + o \geq 1$, gives $L' - L \in \{0, 1\}$, and leaves the bundle's padded count unchanged if and only if $s + 1 \leq \max(o, 2)$ in an Orchard-protocol bundle with cross-address transfers enabled, $s + o = 1$ in an Orchard-pool bundle, and $s + 1 \leq \max(o, 2)$ in a Sapling bundle.*
- (iii) *A shielded spend that opens an empty bundle gives $L' - L = 2$, the padded count going from 0 to 2 for every kind of bundle.*
- (iv) *The fee rises by $\text{marginal_fee} \cdot (\max(2, L') - \max(2, L)) \leq \text{marginal_fee} \cdot (L' - L)$, and not at all while $L' \leq \text{grace_actions} = 2$.*

Hence an uneconomic input of positive value adds positive net value to the step if and only if $\max(2, L') = \max(2, L)$: when padding absorbs its spend, or while the step stays within the grace window.

Proof. (i) For every $x \in \mathbb{N}$, $\lceil (x + 150)/150 \rceil = \lceil x/150 \rceil + 1$; the transparent contribution is the maximum of this term and the unchanged output term, so it rises by 0 or 1.

(ii) and (iii). By Definition 10.6 the contribution of a non-empty bundle is $\max(\max(s, o), 2)$ with cross-address transfers enabled, $\max(s + o, 2)$ in an Orchard-pool bundle, and $\max(s, \max(o, 2))$ for Sapling, whose outputs are padded to $\max(o, 2)$; an empty bundle contributes 0. Each non-empty form is $\max(x, c)$ with $c \geq 2$ and a count x that rises by exactly one, so it rises by 0 or 1, and by 0 if and only if $x + 1 \leq c$: that is $s + 1 \leq \max(o, 2)$ in the first and third forms, and $s + o + 1 \leq 2$, which with $s + o \geq 1$ is $s + o = 1$, in the second. A single spend in an empty bundle gives $\max(1, 2) = 2$ with cross-address transfers enabled, $\max(1 + o, 2) = 2$ in the Orchard pool and $\max(1, \max(o, 2)) = 2$ for Sapling. No other contribution changes.

(iv) From Definition 10.2, since $n \mapsto \max(2, n)$ is non-decreasing and rises by at most the rise of n . For the final claim: if $\max(2, L') = \max(2, L)$ the fee is unchanged and the input adds $v(u) > 0$; otherwise the fee rises by at least *marginal_fee* $> v(u)$.

The instances $(s, o, \text{rise of the padded count})$ are, with cross-address transfers enabled and for Sapling, $(0, 1, 0)$, $(1, 1, 0)$, $(1, 2, 0)$ and $(2, 2, 1)$; in an Orchard-pool bundle $(0, 1, 0)$, $(1, 1, 1)$, $(1, 2, 1)$ and $(2, 2, 1)$; and $(0, 0, 2)$ for the empty bundle of every kind. $\square$

ZIP 315 states that automatic shielding and automatic or opportunistic migration SHOULD NOT be applied to inputs whose cost of shielding or migrating exceeds their economic value (ZIP 315, “Auto-shielding”); under the conventional fee, Lemma 11.13 decides which inputs these are.

The choice of source pool is constrained by privacy. Value that crosses pools is published in the balancing values (*Ironwood Guide*, §“The balancing value”, Remark “Disclosure by the balancing value”), so selecting inputs in the pool of the payment outputs keeps the amount private, and ZIP 315 states the aim of minimising pool crossing (ZIP 315, “Linkability of transactions or addresses”). Wallets MUST NOT automatically combine funds across pools to satisfy a transfer, since that could reveal the total the user holds in a pool (ZIP 315, “Information leakage for transfers between pools”). One policy serving these rules is designed but unspecified (Definition 1.1): order the shielded pools with the pool of the outputs first, then the other Orchard-protocol pool, then Sapling, or, for Sapling outputs, Sapling, then Ironwood, then Orchard; remove the pools that the user’s policy forbids; and spend from the first pool whose notes alone cover the payment.

Transparent sources and recipients are specified. A wallet MUST NOT send funds to a transparent address unless all of the source funds come from shielded pools, and this SHOULD be a single shielded pool (ZIP 315, “Linkability of transactions or addresses”); the ZIP 320 transfer of Construction 11.7 meets this rule, its first transaction spending only shielded notes. A wallet SHOULD NOT spend funds from a transparent address in a transaction with an external recipient unless the user gives explicit consent for that transaction, and MAY restrict transfers further (ZIP 315, “Allowed transfers”). The admissible family $\mathcal{S}$ of Definition 11.9 consists of the selections that meet the rules of this and the preceding paragraph, those of §11.3, and the wallet’s policy.

11.5 Change

Construction 11.14 (Balance and change). *Change* is the excess of a step’s inputs over its payments and fee, returned to the wallet in $k \geq 0$ outputs whose pool follows Construction 11.5. Let *in* be the total of the inputs, over all pools and ephemeral inputs included, *out* the total of

the payment outputs, and f_k the conventional fee of the padded shape with k change outputs, which is non-decreasing in k .

- (1) If $in < out + f_0$, fail with shortfall $out + f_0 - in$, the requirement fed back to selection (Construction 11.10).
- (2) If $in = out + f_0$, the step has no shielded input or output and no change memo is requested, set $k := 0$ and fee f_0 .
- (3) Otherwise let k be the least number of change outputs, 1, or a larger number chosen by the wallet to split change across notes, a choice designed but unspecified (Definition 1.1), subject to $in \geq out + f_k$. The change is $c := in - out - f_k \geq 0$, divided among the k outputs, each carrying the change memo if one is requested. If $out + f_0 \leq in < out + f_1$, no $k \geq 1$ qualifies: the step fails with requirement $out + f_1 - in$, or, if $in > out + f_0$, may instead set $k := 0$ and fold the positive remainder $in - out - f_0$ into the fee; the plan records which.

Balance invariant. Before building, the transparent, Sapling, Orchard and Ironwood balances of the step, each the value of the step's inputs in that pool minus the value of its outputs in that pool, sum to the planned fee. A sum above the fee means that change was omitted; a sum below it, insufficient funds.

Change goes to an internal-scope address of the spending account (ZIP 32, "Orchard internal key derivation"; Construction 2.14; ZIP 316, "Deriving Internal Keys"), its diversifier index a wallet policy; non-zero change remains in the Orchard pool only at an internal-scope receiver of the spending account, the receiver of the note consumed in its Action (Proposition 11.6). A zero-valued fabricated output at the receiver of a spent note is not change.

The balance invariant is the balance equation of Definition 11.2 with every term moved to one side, summed pool by pool.

Proposition 11.15 (Zero-valued change). *Let a step have a shielded input or output, and let Construction 11.14 complete it with $k \geq 1$ change outputs, of total 0 when $in = out + f_k$.*

- (i) *The public counts of the step are the same function of its inputs, payments and k whether the change is 0 or positive.*
- (ii) *If change were instead omitted at exact balance, the counts at exact balance would differ from those with change exactly when one more output of the change pool raises that pool's padded count or opens a bundle (Definition 10.6). An observer who knows the payment outputs, such as a party that receives every payment, and sees the counts would then infer that the inputs sum exactly to the payments plus the fee.*
- (iii) *In the Ironwood pool, bundles with equal public leakage that differ only in whether the change is 0 or positive are computationally indistinguishable.*

The zero-valued output also carries a requested change memo. Where padding already absorbs the change output, as in the two-Action Orchard-pool bundle of ZIP 318, the omission of (ii) reveals nothing (ZIP 318, "Canonical migration transaction structure").

Proof. (i) and (ii) follow from Construction 11.14 and Definition 10.6: the counts depend on k , which is at least 1 whatever the value of c , and omission replaces k by 0 at exact balance only. (iii) is *Ironwood Guide*, §“Privacy”, Theorem “Privacy of an Action”, under its preconditions (P1) to (P4) and its assumptions: Actions whose private inputs have equal public leakage are indistinguishable, and a created note’s value is a private input, not public leakage. No claim is made for Sapling outputs beyond (i). $\square$

A zero-valued note needs no witness. Condition A3 of the Action statement, $v^{\text{old}} \cdot (\text{root} - \text{rt}) = 0$, checks the authentication path only of a consumed note of non-zero value (*Ironwood Guide*, §“The Action statement”), so a zero-valued change note can later be spent without tracking its path.

Only a step with no shielded input or output and no change memo omits change at exact balance. The second transaction tr_1 of Construction 11.7 is the instance, and a change memo is not carried there. A change note of value $v < \text{marginal_fee}$ is an uneconomic input when it is later spent (Definition 11.12), and adds positive net value only where Lemma 11.13 gives no rise of the fee. The wallet’s policy for such change is designed but unspecified (Definition 1.1), constrained by the following proposition.

Proposition 11.16 (Fee uniformity). *The fee of a transaction is public, the value remaining in its transparent transaction value pool, and its conventional fee is a function of public fields (Definitions 10.1 and 10.2), so every observer computes both. A transaction whose fee differs from the conventional fee of its shape is distinguished from every transaction of that shape that pays the conventional fee. Folding a remainder into the fee (Construction 11.14, case (3)) produces exactly such a fee whenever the remainder is positive. The folding threshold is the wallet’s policy, designed but unspecified (Definition 1.1).*

Proof. The fee is the sum of the transparent input values and the balancing values, minus the transparent output values, each a public field (*Ironwood Guide*, §“The balancing value”); the transparent sizes and the shielded counts of Definition 10.1 are public fields. The predicate “the fee differs from the conventional fee of the shape” is therefore a function of the public data; it holds for a transaction with a folded remainder $\text{in} - \text{out} - f_0 > 0$, whose fee exceeds f_0 , and fails for every transaction of the same shape that pays the conventional fee. ZIP 317 states that non-standard fees may reveal specific users or wallets (ZIP 317, “Security and Privacy considerations”). $\square$

11.6 Outgoing viewing keys

Definition 11.17 (Outgoing viewing key of an output). For a transfer, the sender determines:

- (a) the *sending account*: the account that the transfer names, if it names one; otherwise, if every fund used comes from addresses of one account, that account, a rule that ZIP 316 makes a SHOULD; otherwise the sending account is undetermined;
- (b) the *preferred sending protocol*, the most preferred receiver type (the Priority List of §3.3) among the funds used: Orchard if any Orchard-protocol note, of either pool, is spent, else Sapling, else transparent.

The *outgoing viewing key of an output* is, with the sending account determined, that account’s external or internal ovk of the preferred protocol at the account level, external for a payment and internal for a wallet-internal output such as change or a shielding output (SHOULD); with the sending account undetermined, the external or internal ovk of the full viewing key of one address of the preferred protocol from which funds are sent, for example the first (SHOULD) (ZIP 316, “Usage of Outgoing Viewing Keys”). The Orchard external ovk is that of the *Ironwood Guide*, Construction “Diversifier key and outgoing viewing key” in §“Viewing keys”, and the internal one that of Construction 2.14; the transparent pair is that of Construction 2.18, the two halves of $I_{\text{ovk}} = \text{PRFexpand}_c([\text{0x00}] \parallel \text{ser}_P(\text{pk}))$; the Sapling keys are those of ZIP 32, “Sapling internal key derivation”. The choice governs the outgoing ciphertext C_{out} of every payment and change output. Its construction, which also admits $\text{ovk} = \perp$ with a uniformly random ock, is cited (*Ironwood Guide*, §“The outgoing ciphertext”, Construction “Outgoing ciphertext”; protocol specification, §“Sending Notes (Sapling)” and §“Sending Notes (Orchard)”).

For change the rule gives the internal ovk. The specification also permits no ovk: with $\text{ovk} = \perp$ the sender draws ock and the outgoing plaintext uniformly, no viewing key recovers the outgoing plaintext of the change, and the wallet finds its change by trial decryption under the internal ivk (*Ironwood Guide*, §“Trial decryption and note acceptance”; protocol specification, §“Sending Notes (Orchard)”).

Encrypting change with no ovk departs from the SHOULD of ZIP 316. The holder of an account’s external ovk therefore recovers the recipient, value and memo of the account’s Ironwood-pool payment outputs (*Ironwood Guide*, §“The outgoing ciphertext”, Construction “Recovery with ovk”), and those of its change only with the internal ovk.

11.7 Padding, dummies, and shuffling

Construction 11.18 (Assembly of an Orchard-protocol bundle). Fix an Orchard-protocol bundle with s real spends and o real outputs, and let n be its padded Action count (Definition 10.6), or $n = 1$ for the Ironwood-pool bundle of a canonical migration transaction, or $n = 16$ for the Orchard-pool bundle of a note-preparation transaction (§10.2). The wallet sets $\text{enableSpend} = \text{enableOutput} = 1$, the flags that ZIP 315 notes depend under normal circumstances only on whether a transaction is a coinbase transaction (ZIP 315, “Kinds of information leakage”).

- (A) *Cross-address transfers enabled*, an Ironwood-pool bundle with $\text{enableCrossAddress} = 1$, the normal case of ZIP 229 (ZIP 229, “enableCrossAddress polarity”), which the wallet uses for every Ironwood-pool bundle. Pad the list of real spends with $n - s$ dummy consumed notes and the list of real outputs with $n - o$ dummy created notes (*Ironwood Guide*, §“Actions, bundles, and dummy notes”, Definition “Dummy notes”), each at an independent fresh random address; draw independent uniform permutations σ and τ of $\{1, \dots, n\}$; place spend j in Action $\sigma(j)$ and output j in Action $\tau(j)$.
- (B) *Orchard pool*, cross-address transfers disabled (Construction 11.5). Form
 - (i) for each real spend, one Action whose output is a fabricated zero-valued note at the spent note’s address, with its C_{enc} filled with random bytes (MUST), since a

real encryption would let a holder of that receiver's ivk link the spend's nullifier to the address (ZIP 326, "Fabricated same-address outputs and randomized note ciphertexts");

- (ii) for each real output, one Action whose spend is a fabricated zero-valued note at that output's address, a real spend signed with the account's ask for that receiver, carrying no dummy spending key (ZIP 374, "Fabricated same-address outputs (NU6.3)");
- (iii) $n - s - o$ fully dummy Actions, whose dummy spend and dummy output share one common fresh random address, the only Actions that carry a dummy spending key;

then draw a uniform permutation π of $\{1, \dots, n\}$ and place the j -th Action so formed at position $\pi(j)$.

The second Action of the canonical Orchard-pool bundle of ZIP 318 is of kind (B)(iii) when that bundle carries no change, and of kind (B)(ii) when it does (ZIP 318, "Canonical migration transaction structure"). A dummy output at an address independent of its Action's consumed note exists only in bundles with cross-address transfers enabled; in kind (B)(iii) the dummy output shares the fresh random address of its dummy spend. A dummy spend coexists with any anchor, since condition A3 of the Action statement checks no path for a zero-valued consumed note. Per Action, rcv , α and the created note's rseed are drawn fresh, and ρ of the created note is the Action's nf^{old} (*Ironwood Guide*, §"Construction of a transaction"). A Sapling bundle samples rcv and α per spend and rcv per output, pads its outputs as in Definition 10.6, and its binding signing key is the sum of the spends' rcv minus the sum of the outputs' (protocol specification, §"Balance and Binding Signature (Sapling)"). The builder keeps the permutations, which are not part of the transaction, so that later roles (§12) attach per-output data to the right Action.

Proposition 11.19 (Uniform order and pairing of Actions). *In a bundle with cross-address transfers enabled built by Construction 11.18, the position of each spend and the pairing of spends with outputs are uniform and independent of the order of the plan: the map from spend index to Action position is a uniform permutation, and the spend-to-output pairing is a uniform permutation independent of it. In an Orchard-pool bundle the order of the Actions is a uniform permutation independent of the order of the plan. Positions therefore carry no information about the plan beyond the Action count; for an Ironwood-pool bundle, the public data of the Actions reveal no more than their public leakage.*

Proof. With σ and τ independent uniform permutations of $\{1, \dots, n\}$ applied to the padded spend and output lists, Action i pairs spend $\sigma^{-1}(i)$ with output $\tau^{-1}(i)$, so spend j is paired with output $\pi(j)$ for $\pi := \tau^{-1} \circ \sigma$. The map $(\sigma, \tau) \mapsto (\sigma, \pi)$ is a bijection of pairs of permutations, since $\tau = \sigma \circ \pi^{-1}$; it maps the uniform distribution to the uniform distribution, so π is uniform and independent of σ . The plan's order enters only as the indexing of the lists, which both permutations randomise. The Orchard-pool case is the single uniform permutation π . The last clause is *Iron-*

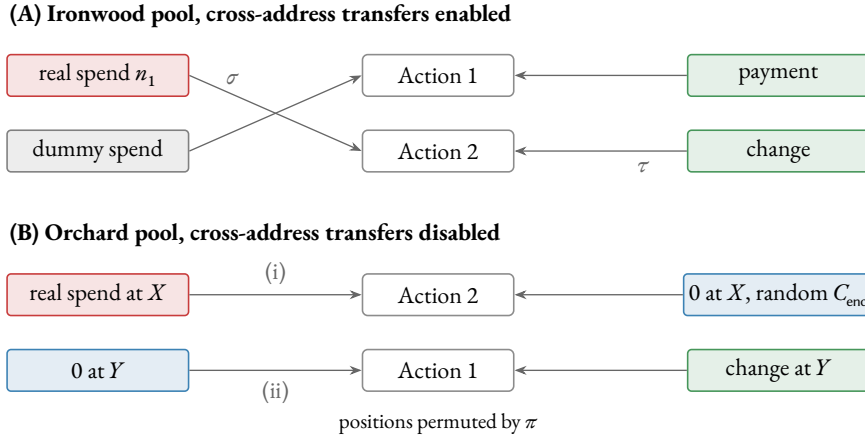


Figure 4. Assembly of Orchard-protocol bundles with $n = 2$ Actions (Construction 11.18). (A) An Ironwood-pool bundle with one real spend and two real outputs: the padded spend list and the output list are permuted independently by σ and τ , so any spend may pair with any output. (B) An Orchard-pool bundle with one real spend at receiver X and one change output at an internal receiver Y of the spending account: each real side is paired with a fabricated zero-valued side at its own receiver, the one beside the real spend with a random C_{enc} , and only the order of the Actions is permuted, by π . Red marks real spends, green real outputs, blue fabricated sides and grey dummies.

wood Guide, §“Privacy”, Theorem “Privacy of an Action”, under its preconditions (P1) to (P4) and its assumptions, applied to the bundle as a whole. No indistinguishability is claimed between the kinds (B)(i), (B)(ii) and (B)(iii) of Orchard-pool Actions, whose bundles that theorem does not cover. $\square$

11.8 Authorisation of a transaction

The transaction carries the consensus branch identifier in force at its target height H (*Ironwood Guide*, §“The transaction format”). Transaction versions 5 and 6 are valid and version 4 is not (ZIP 2003, “Specification”; ZIP 259, “Abstract”); a transaction with an Ironwood-pool component is of version 6. The lock time is specified: the field `nLockTime` is public and may distinguish a transaction if used, and SHOULD be zero (ZIP 315, “Kinds of information leakage”); the wallet sets it to 0, and ZIP 318 lists `lock_time = 0` among the SHOULDs of its canonical migration transaction structure (ZIP 318, “Canonical migration transaction structure”). The reason is the one proved for the fee in Proposition 11.16: a public field set otherwise than by every other wallet distinguishes the transaction. The expiry height is that of §9.3.

Construction 11.20 (Authorising a built transaction). Let a transaction be built from one step by Construction 11.18, every field of its effecting data fixed.

- (1) Compute its signature digest `SigHash` over the effecting data (ZIP 244, “Signature Digest”, as amended by ZIP 229, “Anchor commitment (version 6)”, which omits the anchors from the version-6 digest; *Ironwood Guide*, §“Transaction digests and signatures”). Every spend-authorisation signature and every binding signature signs `SigHash`.

- (2) For each Orchard-protocol Action of either pool, with the Action's randomiser α , set $\text{rsk} := \text{ask} + \alpha$ and $\text{spendAuthSig} := \text{Sign}_{\text{rsk}}(\text{SigHash})$ (*Ironwood Guide*, §“Randomised validating keys”). Every Action carries one: a fully dummy Action's spend is signed with its own random key, and a fabricated zero-valued spend beside a real Orchard-pool output with the account's ask, not with a dummy key. Each Sapling spend likewise signs SigHash under its randomised key (protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”).
- (3) For each bundle, set $\text{bsk} := \sum \text{rcv} \bmod p_{\text{vesta}}$ over its Actions, check it against bvk , and sign SigHash under bsk (*Ironwood Guide*, §“The binding signature”); the Sapling bsk is that of Construction 11.18.
- (4) Sign each transparent input under the key of its address, by the scheme of the *Crypto Guide*, §“The transparent layer's schemes: ECDSA and BIP-340 over secp256k1”, over the signature digest of that input (ZIP 244, “S.2: transparent_sig_digest”).

Keys. One Orchard-protocol ask serves both scopes and both pools (Construction 2.14; ZIP 32, “Orchard internal key derivation”). One Sapling ask serves both scopes, the internal scope differing in nsk , which enters the proof and the nullifier, not the signature (ZIP 32, “Deriving a Sapling internal spending key”), so a Sapling spend is added under the scope of its note. Each transparent input needs the key of its own address.

Lemma 11.21 (Scope of the signature digest). *The signature digests of version-5 and version-6 transactions commit to the effecting data and, for transparent inputs, to the values and scripts of the coins spent (ZIP 244, “S.2: transparent_sig_digest”), excluding proofs and signatures, and the version-6 digest also excludes the anchors. Hence steps (2) to (4) of Construction 11.20 need no proof; proofs and signatures can be computed in either order; in version 6, signing can precede the choice of anchor; and the transaction identifier is unchanged by proofs, by signatures and, in version 6, by anchors.*

Proof. By the *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Signature digest”, SigHash has the header and shielded children of txid and the transparent signature digest as its children, all computed over effecting data and, for the transparent signature digest, the values and scripts of the coins spent; proofs, signatures and transparent scripts are authorising data, under the authorising-data digest (Definition “Authorising-data digest; effecting and authorising data”). ZIP 229, “Anchor commitment (version 6)”, moves the anchors to the authorising data in version 6; in version 5 they are effecting data (ZIP 244, “Signature Digest”). A step of Construction 11.20 computes a signature over SigHash from keys and randomisers, and the binding key from the rcv values, none of which is a proof, so the order of proving and signing is free. The transaction identifier is a function of the effecting data alone (Definition “Transaction identifier”). The multi-party form of the lemma is that of §12.9. $\square$

Theorem 11.22 (Payment integrity). *Let a well-formed plan (Definition 11.3) be executed by Construction 11.18 and Construction 11.20, with Orchard-protocol keys generated with $\text{use_qsk} = \text{false}$. In each resulting transaction:*

- (i) each payment is paid by exactly one output, in the pool that Construction 11.5 assigns, to the routed receiver, of the payment's value and with its memo, none for a transparent output;
- (ii) every other output is a change output of Construction 11.14, an ephemeral output of the step, or a zero-valued dummy or fabricated output;
- (iii) the fee equals the planned fee: the conventional fee of the final shape, or, where Construction 11.14 folds a remainder into it, that fee plus the remainder.

Moreover, for a version-6 transaction whose signing keys meet hypotheses (H) and (S) of Ironwood Guide, §“Transaction digests and signatures”, Proposition “Bundle binding”, and under the assumptions of that proposition and of its Theorem “Spend authority” (§“Authorisation of spends”), no other party produces, except with negligible probability, an accepted transaction that carries any of the transaction's Orchard-protocol randomised validating keys rk with different effecting data. The theorem claims content, not inclusion: inclusion further requires that no spent nullifier be revealed first, that the anchor block remain on the best chain and that inclusion occur by the expiry height, none of which the wallet controls (§13).

Proof. (i) and (ii). By Definition 11.2 each payment is one output of its step, placed by Construction 11.5, and the only other outputs of a step are its change and ephemeral outputs. Construction 11.18 adds only notes of value 0: dummy created notes, fabricated outputs and the outputs of fully dummy Actions. The padding therefore creates no output of non-zero value, and each real output keeps the receiver, value and memo that the plan gives it.

(iii) In the built transaction the fee is the value remaining in the transparent transaction value pool, the transparent inputs minus the transparent outputs plus the balancing value of each shielded pool, and each balancing value is the value consumed minus the value created in its pool. Since dummy and fabricated notes have value 0, the remaining value is the total of the step's inputs minus the total of its payment, ephemeral and change outputs, which is the planned fee by the balance equation of Definition 11.2 and the balance invariant of Construction 11.14; the planned fee is f_k , or f_0 plus the folded remainder.

Binding. Every signature of the transaction signs SigHash (Construction 11.20), which covers the effecting data (Lemma 11.21). By Ironwood Guide, §“Transaction digests and signatures”, Proposition “Bundle binding”, an accepted transaction that carries some rk_i of the transaction carries the same effecting data, except with negligible probability; Theorem “Spend authority” excludes a valid signature under a key of the wallet on a digest that the wallet did not sign. For transparent and Sapling signatures the commitment to the effecting data is that of ZIP 244; no lower volume proves the unforgeability of those schemes, and none is claimed here. $\square$

The wallet records each of its shielded outputs from the data it built the output with: the address, the value, rseed and the memo, and, for an Orchard-protocol output, ρ , the nullifier of the spend of its Action. For such an output,

$$esk = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0x04] \parallel \text{I2LEOSP}_{256}(\rho))),$$

and the note plaintext follow without any viewing key (Ironwood Guide, §“Encryption to the recipient”, and §“The outgoing ciphertext”, Construction “Recovery with okv”). A fabricated

zero-valued Orchard-pool output at a spent note’s receiver carries a random C_{enc} , which trial decryption under any ivk rejects except with negligible probability (*Ironwood Guide*, §“Encryption to the recipient”, Assumption “Confidentiality and integrity of ChaCha20-Poly1305”; ZIP 326, “Fabricated same-address outputs and randomized note ciphertexts”). Non-zero Orchard-pool change lies only at internal-scope receivers (Proposition 11.6), so scanning finds it under the internal ivk (Definition 6.4), and no external ivk is needed for it (ZIP 326, “Wallet key-generation restrictions”).

12 Partially created transactions (PCZT)

This section states the protocol by which the construction, proving and authorisation of one transaction are divided among several parties (ZIP 374). It defines the partially created transaction, its versions and the kinds of its fields; the roles that act on it and the modification state they maintain; its fields, its canonical empty bundles and its encoding; the version-2 rules for deferred anchors and the Ironwood bundle; construction and IO finalisation; signing, with the theorem that a Signer’s signatures authorise exactly what it checked; proving; combining and redaction, with the order independence of proving and signing and the data a PCZT discloses; and spend finalisation and extraction.

12.1 The PCZT and its requirements

Definition 12.1 (Partially created transaction). A *partially created transaction* (PCZT) of one transaction is a value of the typed record of §12.3 for one PCZT version, exchanged among the parties to the creation of that transaction. It carries the data each role needs for its step (adding inputs and outputs, proving, signing, extracting) and accumulates proofs and signatures until the transaction is complete; since it carries all data a signer needs, a signer can be offline. A PCZT consists of a *Global* record and one *bundle* per protocol: transparent, Sapling, Orchard and, from version 2, Ironwood. Every bundle is present in every PCZT, empty bundles included, because metadata of a protocol may have to be held before it is known whether that protocol has inputs or outputs (ZIP 374, “Abstract”, “Specification” and “Structure”).

The PCZT protocol is specified (Definition 1.1) in ZIP 374, Revision 0, whose status is that of Table 1; no consensus rule depends on it. It covers transactions of versions 5 and 6 only: it supports neither Sprout nor version 4 or earlier, since version 4 predates the ZIP 244 digest and several of the role separations of this section, the independence of proving from signing in particular, fail for it (ZIP 374, “Specification”). The protocol serves signing devices that hold spending keys but can neither compute proofs nor hold a whole transaction; proof delegation, the prover needing note data and the full viewing key but no spending key; transactions with several signers (threshold signing, the transparent multisignature wallets of ZIP 48) or several contributors of inputs, whose parallel contributions are merged; and, for version 6, pre-authorisation, in which the signatures precede the choice of anchors, the witnesses and the proofs (ZIP 374, “Motivation”).

ZIP 374, “Requirements”, states six requirements, recorded here as the ZIP’s and not proved as such:

- (R1) every intermediate state of the creation of a version-5 or version-6 transaction is representable, and adding inputs and outputs, finalising the input and output set, attaching lookup data, proving, signing, merging parallel contributions and extracting are each performable by distinct entities, in an order constrained only by data dependencies;
- (R2) no spending key material is required, the keys of dummy spends excepted, which authorise nothing of value and are removable before any Signer sees the PCZT;
- (R3) a Signer determines from the PCZT alone what it authorises (amounts, recipients, and the correspondence between the fields it signs and the values shown to its user), and data a Signer does not need is removable;
- (R4) merging two PCZTs of the same transaction is deterministic and fails on any conflict;
- (R5) for version 6, signatures can be produced while anchors, witnesses and proofs are absent, and supplying them later leaves the signatures valid;
- (R6) the encoding is compact and strictly versioned, so that no data is read under another version's rules.

Requirement (R3) is made a theorem in §12.7, (R5) a proposition in §12.5 and (R4) a construction in §12.9.

Definition 12.2 (PCZT versions). Version 1 represents transactions of version 5 (ZIP 225); version 2 represents transactions of versions 5 and 6 (ZIP 229). The PCZT version does not fix the transaction version: a version-2 PCZT carries a transaction of version 5 or 6, subject to the per-field rules, and a version-1 PCZT only one of version 5. Relative to version 1, version 2 adds the Ironwood bundle; a note version in {V2, V3} per Orchard-protocol bundle; anchors that may be absent; and an encoding that writes a bundle equal to its slot's canonical empty bundle as absent (ZIP 374, "Versioning"). The fields are made exact in §12.3 and the encoding in §12.4.

No field can be added within a version: the encoding is positional, and a parser cannot skip a field it does not know. A PCZT has no non-proprietary key-value maps, so every field of a version is known in advance and a Signer's view of a PCZT is complete; the proprietary maps are the only extension point within a version, and MUST be used for internal processes only. The format evolves by new versions, which the four-byte version of the header distinguishes. Implementations SHOULD parse every specified version and SHOULD emit the oldest version able to represent a PCZT's state when interoperating with entities that may lack the newest; conversion between versions is lossless exactly when the target version represents the state (ZIP 374, "Extensibility" and "Proprietary Use fields"). The states that version 1 cannot represent are listed in §12.4.

Definition 12.3 (Field kinds of a PCZT). Every field of a PCZT is of one of three kinds (ZIP 374, "Structure"):

- (i) *effecting data*, required fields that are part of the final transaction and are always present, committed to by the signature digest (*Ironwood Guide*, §"Transaction digests and signatures");

- (ii) *authorising data*, initially absent and filled as roles act: proofs, signatures, binding signing keys and, in a transaction of version 6, the anchors;
- (iii) *context data*, which enables roles to act and never enters the final transaction: note openings, keys, witnesses, derivation records and address strings.

In a transaction of version 5 the anchors are effecting data (ZIP 374, “Anchors and pre-authorization”).

The signature digest commits to every non-transparent effecting field as soon as any input is signed, so no shielded analogue of the transparent hash types that leave inputs or outputs open exists. The transparent and the shielded modifiable flags of §12.2 are therefore kept apart, so that a purely transparent PCZT behaves as a partially signed Bitcoin transaction of BIP 174 and BIP 370 does (ZIP 374, “Separate Shielded Modifiable Flag from Transparent Modifiable Flags”).

12.2 Roles and the modification state

Definition 12.4 (Roles). A *role* is a partial map from PCZTs to PCZTs, undefined where the role fails; a role that draws randomness is a family of such maps indexed by its random coins, and the Combiner maps a finite set of PCZTs to one. The roles, with the parties that may execute them (ZIP 374, “Specification” and “Roles”), are:

- (1) *Creator* (a single entity): chooses the transaction version and consensus branch identifier, and sets the Global fields and the empty bundles;
- (2) *Constructor*: adds transparent inputs and outputs, Sapling spends and outputs, and Orchard-protocol Actions;
- (3) *IO Finalizer* (anyone): declares the input and output set final, computes the binding signing keys and signs dummy spends;
- (4) *Updater* (anyone): adds data later roles need, such as derivation records, full viewing keys, witnesses and, for version 6, anchors;
- (5) *Redactor* (anyone): removes data later roles do not need;
- (6) *Prover* (a holder of the note data): adds proofs;
- (7) *Signer* (a holder of a spending key): adds signatures;
- (8) *Combiner* (anyone): merges PCZTs;
- (9) *Spend Finalizer* (anyone): assembles transparent unlocking scripts;
- (10) *Transaction Extractor* (anyone): checks completeness and produces the transaction.

The lifecycle runs the Creator, the Constructors, the IO Finalizer and the Updaters, then the Provers and the Signers, the Combiners and the Redactors, and last the Spend Finalizer and the

Transaction Extractor. A *Verifier* is not a role: it is an inspection of a PCZT’s internal consistency, for example the relations of §12.7, performed on behalf of an entity, such as a hardware signer, that trusts it and cannot perform the checks itself (ZIP 374, “Verifier”).

Definition 12.5 (Modification state). The Global field `tx_modifiable` $\in \{0, 1\}^8$, bits numbered from the least significant, has the following bits (ZIP 374, “Global” and “Combiner”):

- (i) bit 0, transparent inputs modifiable, and bit 1, transparent outputs modifiable: each set by the Creator, checked, and possibly cleared, by a Constructor before it adds an input or output of its kind, cleared by the IO Finalizer when shielded spends or outputs exist, and merged by AND;
- (ii) bit 2, a signature with hash type `SIGHASH_SINGLE` is present: cleared by the Creator, set by a Signer that adds such a signature, read by a Constructor, which then preserves the pairing of each such input with its output, and merged by OR;
- (iii) bits 3 to 6: zero, which ZIP 374 requires in version 1 and the Combiner requires of every PCZT it merges;
- (iv) bit 7, shielded modifiable: set by the Creator, checked, and possibly cleared, by a Constructor before it adds a shielded spend or output, cleared by the IO Finalizer when shielded spends or outputs exist and by every Signer, and merged by AND.

The Creator’s value is therefore 1000 0011 in binary.

Signing freezes the effects. After adding a shielded signature a Signer clears bits 0, 1 and 7: every shielded signature, spend-authorisation and binding alike, is on the one signature digest, which commits to all effecting data (*Ironwood Guide*, §“Transaction digests and signatures”, Proposition “Bundle binding”), and no shielded hash type leaves an input or output open. After adding a transparent signature it clears bit 0 unless the hash type has `ANYONECANPAY`, clears bit 1 unless its base type is `NONE`, sets bit 2 if its base type is `SINGLE`, with or without `ANYONECANPAY`, and clears bit 7 (ZIP 374, “Global” and “Signer”).

A wallet executing a plan (Definition 11.2) acts as Creator, Constructor, IO Finalizer and Updater. It builds the bundles by Construction 11.18 and then, as Updater, attaches to each non-dummy Orchard-pool, Ironwood-pool and Sapling spend the derivation record of §12.3, with the seed fingerprint of Construction 2.2 and the path $(32', \textit{coin_type}', \textit{account}')$ of Definition 2.7; to each transparent input the BIP 44 path $(44', \textit{coin_type}', \textit{account}', \textit{scope}, \textit{index})$, the last two components non-hardened; and to each output that pays a payment of the plan (Definition 11.1) that payment’s address string as `user_address`, which a Signer checks (§12.7) (ZIP 374, “Updater” and “Zip32Derivation”; ZIP 32; BIP 44).

12.3 Fields

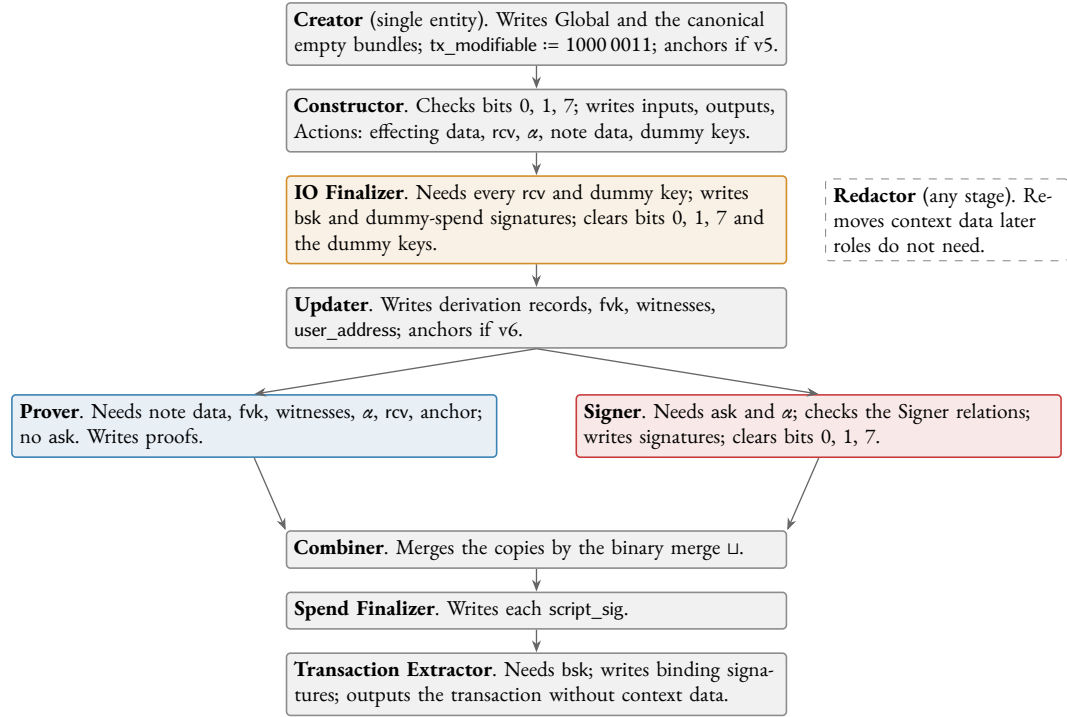


Figure 5. The roles of a PCZT (Definition 12.4), with the fields each writes, the secrets it needs and the bits of tx_modifiable it clears (Definition 12.5). Red marks the one role that needs the spend authorising key ask, amber the role that needs the value-commitment trapdoors and the dummy keys, blue the role that needs the note data and fvk but no ask, and grey the roles that need no secret. The Prover and the Signer run in either order or in parallel on copies joined by the Combiner; what each recipient learns is stated in §12.9.

Definition 12.6 (PCZT fields). *Types.* The types $u8$, $u32$ and $u64$ are the unsigned integers of that width, $i128$ the signed integers of 128 bits and $bool = \{false, true\}$; $\mathbb{B}^n$ is the set of byte strings of length n , T^n the sequences of n elements of T , and $T_1 \times T_2$ the pairs; $Opt(T)$ is T together with an absent value $\perp$; $List(T)$ is the finite sequences over T , $Map(K, V)$ the finite maps from K to V , and Str the UTF-8 strings; $Enum\{A_0, A_1, \dots\}$ is a set of tags, A_i with index i . A *derivation record* is a value of $D := \mathbb{B}^{32} \times List(u32)$, a seed fingerprint (Construction 2.2) and a derivation path, an index with bit 31 set being hardened. Let $P := Map(Str, List(u8))$, the type of a proprietary map. Each record below lists its fields in order, the order of the encoding (§12.4); a field marked (v1) or (v2) is present only in that version (ZIP 374, “Structure” and the sections of each record).

Global. $tx_version \in u32$, which is 5 in version 1 and 5 or 6 in version 2;
 $version_group_id \in u32$, consistent with it (ZIP 225, ZIP 229); $consensus_branch_id \in u32$, which for $tx_version = 6$ names an upgrade at which version 6 is supported, NU6.3 or later, that of NU7 being $8x77190AD9$ (ZIP 259, “NU7 network constants”);
 $fallback_lock_time \in Opt(u32)$, absence meaning 0; $expiry_height \in u32$ (ZIP 203);
 $coin_type \in u32$, the SLIP 44 coin type, which is not part of the transaction and against which roles check network-specific data; $tx_modifiable \in u8$ (Definition 12.5); $proprietary \in P$.

Transparent bundle. $inputs \in List(TI)$ and $outputs \in List(TO)$. An input TI :
 $prevout_txid \in \mathbb{B}^{32}$; $prevout_index \in u32$; $sequence \in Opt(u32)$, absence meaning $2^{32} - 1$;
 $required_time_lock_time \in Opt(u32)$, at least 500,000,000;
 $required_height_lock_time \in Opt(u32)$, in $\{1, \dots, 499,999,999\}$; $script_sig \in Opt(List(u8))$;
 $value \in u64$; $script_pubkey \in List(u8)$; $redeem_script \in Opt(List(u8))$;
 $partial_signatures \in Map(\mathbb{B}^{33}, List(u8))$; $sighash_type \in u8$; $bip32_derivation \in Map(\mathbb{B}^{33}, D)$;
four preimage maps, for RIPEMD160, SHA256, HASH160 and HASH256, of types
 $Map(\mathbb{B}^{20}, List(u8))$, $Map(\mathbb{B}^{32}, List(u8))$, $Map(\mathbb{B}^{20}, List(u8))$ and $Map(\mathbb{B}^{32}, List(u8))$ in that
order; $proprietary \in P$. An output TO : $value \in u64$; $script_pubkey \in List(u8)$;
 $redeem_script \in Opt(List(u8))$; $bip32_derivation \in Map(\mathbb{B}^{33}, D)$; $user_address \in Opt(Str)$;
 $proprietary \in P$.

Sapling bundle. $spends \in List(SS)$; $outputs \in List(SO)$; $value_sum \in i128$, the net value of the spends minus the outputs; $anchor \in \mathbb{B}^{32}$ (v1) or $Opt(\mathbb{B}^{32})$ (v2); $bsk \in Opt(\mathbb{B}^{32})$. A spend SS : $effecting\ cv, nf, rk \in \mathbb{B}^{32}$; $zkproof \in Opt(\mathbb{B}^{192})$; $spend_auth_sig \in Opt(\mathbb{B}^{64})$;
 $recipient \in Opt(\mathbb{B}^{43})$; $value \in Opt(u64)$; $rcm \in Opt(\mathbb{B}^{32})$ and $rseed \in Opt(\mathbb{B}^{32})$, at most one set, by whether the note predates ZIP 212; $rcv \in Opt(\mathbb{B}^{32})$; the proof generation key
 $(ak, nsk) \in Opt(\mathbb{B}^{32} \times \mathbb{B}^{32})$; $witness \in Opt(u32 \times (\mathbb{B}^{32})^{32})$; $\alpha \in Opt(\mathbb{B}^{32})$; a derivation record in
 $Opt(D)$; $dummy_ask \in Opt(\mathbb{B}^{32})$; $proprietary \in P$. An output SO : $effecting\ cv, cmu, epk \in \mathbb{B}^{32}$
and the ciphertexts $C_{enc}, C_{out} \in List(u8)$, whose lengths depend on the transaction version;
 $zkproof \in Opt(\mathbb{B}^{192})$; $recipient \in Opt(\mathbb{B}^{43})$; $value \in Opt(u64)$; $rseed, rcv, ock \in Opt(\mathbb{B}^{32})$; a
derivation record in $Opt(D)$; $user_address \in Opt(Str)$; $proprietary \in P$. The Sapling
constructions are those of the protocol specification and are cited, not restated.

Orchard bundle. $actions \in List(OA)$; $flags \in u8$, with bit 0 enableSpends, bit 1
enableOutputs, bit 2 enableCrossAddress (v2; zero in version 1 and in the Orchard bundle of

every transaction mined under NU6.3 or later, the rule of Construction 11.5), and bits 3 to 7 zero; $\text{value_sum} \in \text{u64} \times \text{bool}$; $\text{anchor} \in \mathbb{B}^{32}$ (v1) or $\text{Opt}(\mathbb{B}^{32})$ (v2); $\text{note_version} \in \text{Enum}\{\text{V2}, \text{V3}\}$ (v2), where V2 is the note plaintext of ZIP 212, lead byte $0x02$, and V3 that of ZIP 2005, lead byte $0x03$, which MUST be V2 in the Orchard bundle and is implicitly V2 in version 1; $\text{zkproof} \in \text{Opt}(\text{List}(\text{u8}))$, one proof for the bundle; $\text{bsk} \in \text{Opt}(\mathbb{B}^{32})$. An Action OA: $\text{cv}^{\text{net}} \in \mathbb{B}^{32}$; a spend OS; an output OO; $\text{rcv} \in \text{Opt}(\mathbb{B}^{32})$, one per Action, since an Action has one net value commitment (*Ironwood Guide*, §“Value commitments”). A spend OS: effecting $\text{nf}, \text{rk} \in \mathbb{B}^{32}$; $\text{spend_auth_sig} \in \text{Opt}(\mathbb{B}^{64})$; $\text{recipient} \in \text{Opt}(\mathbb{B}^{43})$, a raw Orchard address; $\text{value} \in \text{Opt}(\text{u64})$; $\rho, \text{rseed} \in \text{Opt}(\mathbb{B}^{32})$; $\text{fvk} \in \text{Opt}(\mathbb{B}^{96})$, the encoding of $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$; $\text{witness} \in \text{Opt}(\text{u32} \times (\mathbb{B}^{32})^{32})$, a position and a 32-level authentication path; $\alpha \in \text{Opt}(\mathbb{B}^{32})$; a derivation record in $\text{Opt}(D)$; $\text{dummy_sk} \in \text{Opt}(\mathbb{B}^{32})$; $\text{proprietary} \in P$. In a transaction of version 6 the witness may be absent until proving (§12.5). An output OO: effecting $\text{cmx}, \text{epk} \in \mathbb{B}^{32}$ and $C_{\text{enc}}, C_{\text{out}} \in \text{List}(\text{u8})$; $\text{recipient} \in \text{Opt}(\mathbb{B}^{43})$; $\text{value} \in \text{Opt}(\text{u64})$; $\text{rseed}, \text{ock} \in \text{Opt}(\mathbb{B}^{32})$; a derivation record in $\text{Opt}(D)$; $\text{user_address} \in \text{Opt}(\text{Str})$; $\text{proprietary} \in P$.

Ironwood bundle (v2). The fields of the Orchard bundle, with $\text{note_version} = \text{V3}$ and its anchor a root of the Ironwood note commitment tree (§12.5).

Which role sets, requires or clears each field is as ZIP 374 states per field; this section restates it where it defines a role.

Every transparent input’s value and script_pubkey are required, not optional, because the signature digest commits to the value and script of every coin a transaction spends (ZIP 244, as amended by ZIP 229; *Ironwood Guide*, §“Transaction digests and signatures”), and the binding signatures are on that digest (ZIP 374, “TransparentInput”).

Remark 12.7 (The sign of an Orchard-protocol value sum). The pair $\text{value_sum} = (m, b)$ denotes the net value of the bundle’s spends minus its outputs, but ZIP 374 does not state the meaning of b : a gap in the specification. The reference implementation of ZIP 374 reads $b = \text{true}$ as the net value $-m$ and $b = \text{false}$ as $+m$, and the canonical empty bundle has $(0, \text{false})$. This volume adopts that reading, and writes v^{balance} for the integer that a bundle’s value_sum denotes; for Sapling it is the i128 value itself.

A Signer MUST parse the user_address of each output that carries one and confirm that it contains the output’s recipient, directly or as a receiver of a unified address (ZIP 374, “TransparentOutput”, “SaplingOutput” and “OrchardOutput”; Definition 3.7 and Construction 3.10). The anchors, witnesses and commitments that these fields carry are those of the *Ironwood Guide*, §“The note commitment tree”, with its subsections “Anchors” and “Authentication paths”; they are cited, not rebuilt.

Definition 12.8 (Canonical empty bundles). The *canonical empty bundle* of each slot is (ZIP 374, “v2 Encoding”):

- (i) transparent: empty input and output lists;
- (ii) Sapling: empty spend and output lists, $\text{value_sum} = 0$, $\text{anchor} = \perp$, $\text{bsk} = \perp$;

- (iii) Orchard: no Actions, flags = 0000 0011 in binary (spends and outputs enabled), value_sum = (0, false), anchor = $\perp$, note_version = V2, zkproof = bsk = $\perp$;
- (iv) Ironwood: no Actions, flags = 0000 0111 in binary (spends, outputs and cross-address transfers enabled), value_sum = (0, false), anchor = $\perp$, note_version = V3, zkproof = bsk = $\perp$.

A bundle that differs from it in any field, its flags, note version or anchor included, is not canonical empty.

12.4 Encoding

Construction 12.9 (PCZT encoding). The encoding of a PCZT of version n is

$$50\ 43\ 5a\ 54 \parallel \text{l2LEOSP}_{32}(n) \parallel \text{body}_n,$$

the first four bytes being the ASCII string PCZT. A string that does not begin with this 8-byte header is not a PCZT encoding, and a parser that meets an unknown version **MUST NOT** parse the remainder (ZIP 374, “Encoding”). The body is built from the following encodings of the types of Definition 12.6 (ZIP 374, “Data type encodings”):

- (i) u8: one raw byte;
- (ii) u32 and u64: $\text{varint}(x)$, the digits of x in base 128, least significant first, one per byte, with bit 7 set on every byte but the last. Encoders **MUST** emit the shortest form. A k -bit integer takes at most $\lceil k/7 \rceil$ bytes, 5 for u32 and 10 for u64, and in an encoding of that length the last byte contributes only the high $k \bmod 7$ bits, so the fifth byte of a u32 is at most $0x0f$; decoders **MUST** reject encodings that exceed either bound and **MAY** accept non-minimal encodings within them;
- (iii) i128: $\text{varint}(\text{zigzag}(x))$, with $\text{zigzag}(x) = 2x$ for $x \geq 0$ and $2|x| - 1$ for $x < 0$;
- (iv) bool: the byte $0x00$ for false or $0x01$ for true, other bytes rejected;
- (v) $\mathbb{B}^n$, T^n and pairs: the concatenation of the elements, with no length prefix, so that a witness path is 1024 raw bytes;
- (vi) $\text{Opt}(T)$: the byte $0x00$ for $\perp$, and $0x01$ followed by the value otherwise, other tag bytes rejected;
- (vii) $\text{List}(T)$ and Str : the varint of the number of elements, of bytes for a string, followed by the elements; invalid UTF-8 is rejected;
- (viii) $\text{Map}(K, V)$: the varint of the number of entries, followed by each key and its value. Encoders **MUST** emit the entries in strictly ascending lexicographic byte order of the unencoded keys, without duplicates; decoders **MAY** accept other orders and duplicates;
- (ix) Enum: the varint of the tag’s index;

- (x) a record: the concatenation of its fields in the order of Definition 12.6, without tags or padding, a field absent from version n being omitted entirely.

Encoders MUST NOT emit data after the last field; decoders MAY ignore or reject it. *Layout* (ZIP 374, “v1 Encoding” and “v2 Encoding”): $body_1$ is the Global record followed by the transparent, Sapling and Orchard bundles, each Action encoded as cv^{net} , spend, output, rcv. The body $body_2$ is the Global record followed by the four bundles, transparent, Sapling, Orchard and Ironwood, each as a value of Opt; a bundle equal to its slot’s canonical empty bundle (Definition 12.8) MUST be encoded as absent and any other as present, and a parser MUST restore the canonical empty bundle from absence. Encoding in version 1 MUST fail when $tx_version \neq 5$, when the Ironwood bundle is not canonical empty, when the Orchard note version is not V2, or when an anchor required before signing a transaction of version 5 is absent: these are the states version 1 cannot represent.

Lemma 12.10 (Canonicity of the encoding). *For every version n and every PCZT P representable in version n , exactly one byte string is an output for P permitted to an encoder that obeys the MUST rules of Construction 12.9, and distinct PCZTs have distinct encodings. Canonicity is a property of encoder output: since decoders may accept non-minimal variants, unordered or repeated map entries and trailing data, several strings may decode to one PCZT.*

Proof. By induction on the type structure. *Uniqueness.* The permitted encoding of each primitive is a function of its value: the shortest base-128 representation is unique, its last byte non-zero unless the value is 0; the map zigzag is a bijection from the integers onto the naturals; the tags of options, booleans and enumerations are fixed; the counts of lists and strings are determined by the value. A finite map admits exactly one strictly ascending order of its entries without duplicates, since lexicographic order on byte strings is total. A record concatenates uniquely encoded fields in a fixed order. In version 2 the presence of each bundle is determined by equality with Definition 12.8, and no trailing data is permitted. *Injectivity.* Each encoding is self-delimiting given its type, being of fixed length or a count followed by self-delimiting elements, so a decoder that reads the fields in order recovers every value; an absent version-2 bundle is restored as the canonical empty bundle, the only bundle encoded as absent. The header fixes n , so encodings of distinct versions differ as well. $\square$

The canonical-empty rule is exact equality so that two copies of one PCZT encode identically whatever software produced them, and re-encoding cannot move a bundle between its present and absent forms (ZIP 374, “Omitting empty bundles in PCZT v2”).

Example 12.11 (A minimal version-2 PCZT). The Creator’s PCZT (§12.6) for a transaction of version 6, with version group identifier $0xD884B698$ (ZIP 229), consensus branch identifier $0x77198AD9$ (NU7, ZIP 259), expiry height 10,000,000, coin type 133 (Mainnet), no fallback lock time, no proprietary entries and every bundle canonical empty, encodes in version 2 as the 32 bytes of Table 6. With coin type 1 the coin type takes the one byte 01 , the total is 31 bytes, and every later field moves one byte earlier: the position of a field depends on the values before it, which is why no field can be added within a version.

Table 6. The minimal version-2 PCZT of Example 12.11: offset, bytes and field, in the encoding of Construction 12.9.

Offset	Bytes	Field
0	50 43 5a 54	header, PCZT
4	02 00 00 00	version 2
8	06	tx_version = 6
9	98 ed 92 c4 0d	version_group_id
14	d9 95 e4 b8 07	consensus_branch_id
19	00	fallback_lock_time = $\perp$
20	80 ad e2 04	expiry_height
24	85 01	coin_type = 133
26	83	tx_modifiable, bits 0, 1, 7
27	00	empty proprietary map
28 to 31	00 00 00 00	four absent bundles

12.5 PCZT v2: deferred anchors and the Ironwood bundle

In version 2 the Ironwood bundle carries the Ironwood-pool component of a transaction of version 6, with the structure, field meaning and role interactions of the Orchard bundle, the Ironwood pool being a second value pool of the Orchard protocol (ZIP 229; *Ironwood Guide*, §“The Orchard protocol and its two pools”). It differs in three respects: its enableCrossAddress bit is meaningful and normally 1; its anchor is a root of the Ironwood note commitment tree, distinct from the Orchard tree; and its note version **MUST** be V3. Every role **MUST** reject a PCZT with tx_version = 5 whose Ironwood bundle is not canonical empty (Definition 12.8) (ZIP 374, “IronwoodBundle (PCZT v2)”).

Definition 12.12 (Anchors and witnesses in version 2). The following rules hold (ZIP 374, “Anchors and pre-authorization”).

- (i) For tx_version = 5, every shielded bundle’s anchor is set when the bundle’s first spend is added and never changes, since the version-5 digest commits to it.
- (ii) For tx_version = 6, which requires version 2, an anchor **MAY** be absent until proving; an Updater **MAY** set an absent anchor, or replace a set one, at any time before the Prover runs for that bundle; setting or replacing an anchor **MUST** clear every proof already in that bundle, the Sapling spends’ proofs or the Orchard-protocol bundle’s proof, since a proof takes the anchor as a primary input; and the witness of a Sapling or Orchard-protocol spend **MAY** be absent until proving, and is set by an Updater once known.
- (iii) Whenever a bundle has both an anchor and witnesses, the Updater that sets either **MUST** check that every witness of a spend of non-zero value roots to the anchor, and the Prover **MUST** check the same before proving.

Deferral is governed by the transaction version, not by the PCZT version: a transaction of version 5 in a version-2 PCZT fixes its anchors as in version 1. Absence is an absent option, not an

all-zero string, because zero is an element of $\mathbb{F}_{p_{\text{Pallas}}}$ and so a syntactically valid anchor (ZIP 374, “Optional anchors in PCZT v2”).

Proposition 12.13 (Deferred anchors). *Let P be a PCZT of a transaction of version 6.*

- (i) *If every spend of P is signed, replacing the anchor rt of a bundle and its proof π by any anchor rt' and a proof π' valid for it leaves every signature valid and the transaction identifier unchanged: re-anchoring needs only re-proving.*
- (ii) *A spend of an Orchard-protocol note whose creating transaction is not yet mined can be fully signed before that note is in the tree, and its signatures remain valid once an anchor, a witness and a proof are supplied: every field that the signature digest commits to is computable from the note data, and the anchor is not among them.*

Neither holds for a transaction of version 5, and (ii) fails for a Sapling spend in version 6.

Proof. (i) The version-6 digest and transaction identifier omit the anchors, which are authorising data (ZIP 229, “Anchor commitment (version 6)”), and by *Ironwood Guide*, §“Transaction digests and signatures”, Proposition “Bundle binding”, replacing (rt, π) by (rt', π') changes neither the transaction identifier nor any signed message, so every signature remains valid.

(ii) The effecting fields of an Orchard-protocol spend are nf , rk and its Action’s cv^{net} . The nullifier is $\text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm})$ (*Ironwood Guide*, §“The nullifier”), a function of the key and of note data, with ρ fixed by the creating Action and ψ and cm derived from rseed , ρ , the address and the value; it does not depend on the note’s position. The key rk and the commitment cv^{net} depend on keys, values and fresh randomness only.

For version 5 the digest commits to the anchors (ZIP 244, “Signature Digest”), so replacing an anchor invalidates every signature, which refutes (i); and an anchor fixed before the note is in the tree is the root of a tree state that does not contain the note, for which no authentication path of the note exists except with negligible probability (*Ironwood Guide*, §“Anchors”, Proposition “Membership soundness”), so a spend of non-zero value signed then admits no valid proof, so no signature made before the note is in the tree survives completion, which refutes (ii). A Sapling nullifier depends on the note’s position in the tree (protocol specification, §“Computing ρ values and Nullifiers”), which is unknown until the note is mined, and is effecting data; for Sapling only (i) holds. $\square$

ZIP 374 calls the two workflows re-anchoring and the spending of unmined notes; Proposition 11.4 (c) is the latter for the steps of a plan.

12.6 Construction and IO finalisation

The transaction of a PCZT is of version 5, with version group identifier $0x26A7270A$ (protocol specification, §“Transaction Consensus Rules”), or of version 6, with $0x0884B698$ (ZIP 229), whose consensus branch identifier names NU6.3 or a later upgrade; version 4 is not representable, and is invalid from NU7 on (ZIP 2003, deployed by ZIP 259). An Ironwood component requires version 6, the mixed case being rejected by the rule of §12.5. The consensus branch identifier fixes

the rules in force, and a role that does not recognise it cannot check the PCZT against them (ZIP 374, “Global”).

Creator. The Creator chooses tx_version , version_group_id and $\text{consensus_branch_id}$, sets the other Global fields with $\text{tx_modifiable} = 1000\ 0011$ in binary, and sets every bundle to its canonical empty bundle (Definition 12.8); for $\text{tx_version} = 5$ it sets the Sapling and Orchard anchors, and for $\text{tx_version} = 6$ it MAY leave them to an Updater (Definition 12.12) (ZIP 374, “Creator”).

Constructor. Before adding anything the Constructor MUST check the bit of tx_modifiable for the kind added (Definition 12.5), and it updates the bundle’s value_sum . Per Orchard-protocol Action, in the notation of the *Ironwood Guide*, it sets $\text{cv}^{\text{net}} := \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{old}} - v^{\text{new}})$ for a fresh rcv (§“Value commitments”); the nullifier nf^{old} of the consumed note (§“The nullifier”); a uniform α and $\text{rk} := \text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}}$ (§“Randomised validating keys”); and the created note with $\rho^{\text{new}} := \text{nf}^{\text{old}}$, its cmx (§“Note commitments”), epk and C_{enc} (§“Encryption to the recipient”) and C_{out} (§“The outgoing ciphertext”); together with the context fields of the spend and the output. Dummy and fabricated Actions and their order are those of Construction 11.18, only fully dummy Actions carrying dummy_sk . Spends and outputs MUST agree with the bundle flags. When the bundle’s anchor is set, each spend added carries a witness rooting to it, or, for a note of value zero, whose path the Action statement does not check, any placeholder; for $\text{tx_version} = 6$ a spend MAY be added without a witness, even of a note not yet in the tree; for $\text{tx_version} = 5$ every non-dummy spend is of a note in the tree, with a witness to the anchor. A transparent input with a lock-time requirement is added only if its lock type is compatible with those of the other inputs and, when inputs are already signed, it leaves the lock time unchanged (§12.10) (ZIP 374, “Constructor” and “OrchardBundle”).

Construction 12.14 (IO finalisation). Once the input and output set is final, the IO Finalizer performs the following steps, failing where stated (ZIP 374, “IO Finalizer”).

- (1) If shielded spends or outputs exist, it clears bits 0, 1 and 7 of tx_modifiable .
- (2) For each Orchard-protocol bundle with Actions $1, \dots, n$, it requires every rcv_i , sets $\text{bsk} := \sum_i \text{rcv}_i \in \mathbb{F}_{p_{\text{Vesta}}}$, computes

$$\text{bvk} := \sum_i \text{cv}_i^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}},$$

with v^{balance} the integer that the bundle’s value_sum denotes (§12.3), and fails unless $[\text{bsk}] R^{\text{Orchard}} = \text{bvk}$ (*Ironwood Guide*, §“The binding signature”). The Orchard and the Ironwood bundles are finalised separately, each by this computation on its own Actions.

- (3) For the Sapling bundle, it sets bsk to the sum of the spends’ rcv minus the sum of the outputs’ rcv , checked in the same way against the Sapling binding validating key (protocol specification, §“Balance and Binding Signature (Sapling)”).
- (4) It computes the signature digest over the effecting data (*Ironwood Guide*, §“Transaction digests and signatures”), signs each dummy spend under the spend authorising key derived from its dummy_sk or given by its dummy_ask , randomised by the spend’s α , and clears that key.

- (5) For each bundle with `enableCrossAddress = 0`, it MUST verify that each Action’s output recipient equals its spend recipient, and fail otherwise.

The condition of step (5) is condition A10 of the Action statement (*Ironwood Guide*, §“The Action statement”), so no valid proof exists for an Action that violates it; the step reports the violation at the role responsible for it.

Lemma 12.15 (Value-sum consistency). *Let an Orchard-protocol bundle have Actions with $cv_i^{\text{net}} = \text{ValueCommit}_{\text{rcv}_i}^{\text{Orchard}}(v_i^{\text{net}})$, and let v^{balance} be the integer that its `value_sum` denotes. Step (2) of Construction 12.14 passes if and only if $\sum_i v_i^{\text{net}} \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$, and `bsk` is then a binding signing key whose validating key is `bvk`. The check reads no value v_i^{net} , which may therefore have been redacted.*

Proof. With $\text{bsk} = \sum_i \text{rcv}_i$,

$$\text{bvk} - [\text{bsk}] R^{\text{Orchard}} = \left[\sum_i v_i^{\text{net}} - v^{\text{balance}} \right] V^{\text{Orchard}},$$

which is $\mathcal{O}$ if and only if the coefficient vanishes modulo p_{Vesta} , since V^{Orchard} is a non-identity point of the group of prime order p_{Vesta} . This is *Ironwood Guide*, §“The binding signature”, Proposition “Balance and key knowledge”, part (i). No independence of V^{Orchard} and R^{Orchard} is used. The check computes `bvk` from the commitments and `value_sum` alone. $\square$

The binding signing key cannot wait for the Transaction Extractor, since every `rcv` would then travel to it and open every value commitment to every intermediate party; nor can Constructors maintain it incrementally, since serial Constructors would have to update it and parallel ones, whose results a Combiner merges, must not. One role run once the set is final removes both problems, and the same role signs dummy spends so that no Signer parses spending-key material (ZIP 374, “Adding an IO Finalizer role”).

12.7 Signing

Digests. The Signer derives the effecting data from the PCZT and computes the shielded signature digest `SigHash` itself (ZIP 244 for version 5, as amended by ZIP 229 for version 6; *Ironwood Guide*, §“Transaction digests and signatures”), neither proofs nor signatures entering it; for each transparent input it signs, it computes that input’s digest under the input’s `sighash_type` (ZIP 244, “S.2: transparent_sig_digest”) and signs it by the scheme of the *Crypto Guide*, §“The transparent layer’s schemes: ECDSA and BIP-340 over secp256k1”. A Signer that cannot sign under an input’s hash type MUST NOT sign that input (ZIP 374, “TransparentInput”).

Orchard-protocol spends. For an Action it signs, with α present, the Signer sets $\text{rsk} := \text{ask} + \alpha$, fails unless $\text{rk} = [\text{rsk}] G^{\text{SpendAuth}}$, and sets `spend_auth_sig` to the RedPallas signature on `SigHash` under `rsk` (*Ironwood Guide*, §“Randomised validating keys”); Ironwood-pool Actions are signed in the same way in their own bundle. A signature produced elsewhere, for example by a hardware device, is attached if and only if it is valid under the Action’s `rk` on `SigHash`. The Signer then updates `tx_modifiable` (Definition 12.5) (ZIP 374, “Signer” and “OrchardSpend”).

Definition 12.16 (Signer relations). For an Orchard-protocol Action whose explicit data are present, with v^{old} and v^{new} the values of its spend and its output, the *Signer relations* are, in the notation of the *Ironwood Guide*:

- (a) $\text{cv}^{\text{net}} = \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{old}} - v^{\text{new}})$ (§“Value commitments”);
- (b) the consumed note rebuilt from (recipient, v^{old} , ρ , rseed) has nullifier nf under the nk of fvk (§“The nullifier”), and fvk owns the recipient: $\text{pk}_d = [\text{ivk}] \text{g}_d$ for the ivk that fvk determines (§“Diversified addresses”);
- (c) $\text{rk} = \text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}}$ for the ak of fvk (§“Randomised validating keys”);
- (d) cmx is the commitment of the created note rebuilt from (recipient, v^{new} , rseed) with $\rho^{\text{new}} = \text{nf}$ (§“Note commitments”);
- (e) for an output that is not fabricated, esk derived from rseed and ρ^{new} gives epk and a C_{enc} that decrypts to a plaintext with the explicit recipient, value and rseed (§“Encryption to the recipient”), and, when ock is present, C_{out} decrypts under it to $(\text{pk}_d, \text{esk})$ (§“The outgoing ciphertext”).

A fabricated zero-valued output at a spent note’s receiver carries a random C_{enc} (Construction 11.18) and is exempt from (e): a Signer MUST NOT reject it on that ground alone, and SHOULD classify it as a tolerable dummy, checking (d) if it wishes (ZIP 374, “Signer”, “Verifier” and “Fabricated same-address outputs (NU6.3)”).

Obligations before signing (ZIP 374, “Signer”). A Signer MUST reject a PCZT that carries any dummy_ask or dummy_ask: the IO Finalizer signs dummy spends and clears these keys, so their presence asks the Signer to parse spending-key material. It MUST, for each output with a user_address, parse it and confirm that it contains the output’s recipient (§12.3). It SHOULD verify that the explicit values and note data it shows its user are consistent with the effecting data it signs, through the relations of Definition 12.16; a field absent because it was redacted is not an inconsistency, and only a present field that contradicts the effecting data is. The relations need the value and note commitments, BLAKE2b and the functions built on it, nullifier derivation, key agreement, the key derivation and the AEAD of note encryption, the signature digest and RedPallas signing (RedJubjub for Sapling), and no proof system.

Theorem 12.17 (Signer safety). *Let a Signer hold the ask of an Orchard-protocol key generated with use_qsk = false, and suppose that for every PCZT P it signs it computes $m_P := \text{SigHash}(T_P)$ from the effecting data T_P of P , performs the obligations above, verifies relations (a) to (e) of Definition 12.16, with all their fields present, for every Action whose spend it signs, displays the values, recipients and address strings so checked, and signs under keys $\text{ask} + \alpha$ no message other than such digests m_P . Under the assumptions of Ironwood Guide, §“Authorisation of spends”, Theorem “Spend authority”, and Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”), except with negligible probability:*

- (i) every accepted Action that consumes a note of the key carries a signature on some m_P ;

- (ii) every accepted transaction that carries an Action with a randomised validating key rk under which the Signer signed has the effecting data T_Q of some PCZT Q that the Signer signed with an Action of key rk ; in it the Actions whose spends the Signer signed consume and create the notes, with the values and recipients, that the Signer displayed for Q , and every other effecting field, balancing values and transparent data included, equals that of Q .

The theorem concerns content, not inclusion.

Proof. (i) The Signer is the signing oracle of the *Ironwood Guide*, §“Authorisation of spends”, Definition “Spend-authority experiment”, queried with (α, m_p) for α taken from the PCZT and so possibly chosen by an adversary, which that experiment admits; the full viewing key that the PCZT discloses is given to the adversary there. Theorem “Spend authority” bounds by a negligible function the probability of an accepted Action that consumes a note of the key without a queried digest.

(ii) Let T' be accepted and carry an Action with key rk and a signature σ' valid under rk on $\text{SigHash}(T')$. The reduction of the proof of Theorem “Spend authority”, which simulates the key holder from ak^P and answers queries with adversarial randomisers, turns a triple $(rk, \text{SigHash}(T'), \sigma')$ that is not a triple of an answer of the Signer into a forgery in *Ironwood Guide*, §“RedPallas unforgeability”, Proposition “Unforgeability under re-randomisation”; hence, except with negligible probability, $\text{SigHash}(T') = m_Q$ for a PCZT Q that the Signer signed under rk . The collision case of the proof of Proposition “Bundle binding” uses only that each node of the digest tree hashes a string from which its children and fields are recovered uniquely, which holds equally for the version-5 tree (ZIP 244, “TxId Digest” and “Signature Digest”), where the anchors are among the hashed fields and are effecting data (Definition 12.3); it therefore applies to both transaction versions and gives $T' = T_Q$ as effecting data, except with negligible probability under Assumption “Collision resistance of BLAKE2b”. The digest covers the value and script of each transparent input (§12.3), so a false value in Q yields a digest that validation does not compute and a signature that is not accepted. For each Action whose spend the Signer signed, relations (a) and (d) make the displayed data an opening of cv^{net} and of cmx , and by Lemma “Hiding and binding of net value commitments” and Proposition “Hiding and binding of note commitments” of the *Ironwood Guide* no other opening is feasible; relation (b) identifies the consumed note, whose commitment fixes its value; relation (e) makes the created note decryptable by its recipient (Proposition “Decryption correctness”); and the check of `user_address` ties each recipient to the displayed string. $\square$

Pre-authorisation (ZIP 374, “Signer” and “Anchors and pre-authorization”). For a transaction of version 6 a Signer MAY sign while the shielded anchors, witnesses or proofs are absent, since signatures commit to none of them (Proposition 12.13). A Signer asked to do so SHOULD be aware, and where applicable make its user aware, that a spend without a witness may be of a note that never exists on chain: the signature authorises the spend whether or not the note is created. For a transaction of version 5 the anchors are in the digest and MUST be present before signing.

Lemma 12.18 (Binding key after IO finalisation). *If IO finalisation (Construction 12.14) succeeded for an Orchard-protocol bundle, and its commitments cv_i^{net} and its value_sum are unchanged afterwards, then a binding signature under bsk on the signature digest is valid under the bvk that validation derives from the transaction’s cv_i^{net} and balancing value v^{balance} .*

Proof. Validation computes bvk from the same commitments and balancing value (*Ironwood Guide*, §“The binding signature”, Definition “Binding validating key”), and IO finalisation checked that it equals $[\text{bsk}] R^{\text{Orchard}}$ (Lemma 12.15). Correctness of RedPallas in the binding-signature instance (*Ironwood Guide*, §“The binding signature”, Construction “Binding signature”) gives validity. $\square$

The hypothesis of Lemma 12.18 holds for every PCZT reached by the roles after finalisation: cv^{net} is required effecting data, which no later role changes and the merge of §12.9 keeps equal, and a finalised bundle merges only with a bundle of equal Action count and value_sum.

12.8 Proving

Per Orchard-protocol Action the Prover requires the spend’s recipient, value, ρ , rseed, fvk, witness and α , the output’s recipient, value and rseed, and the Action’s rcv; the bundle’s anchor MUST be set, and before proving the Prover MUST verify that the path of the witness of every spend of non-zero value roots to it, and fail otherwise. For Sapling it requires per spend the recipient, the value, rcm or rseed, rcv, the proof generation key and the witness, and per output the recipient, the value, rseed and rcv. The Prover needs no spending key; a delegated Prover learns the note data of the bundle but gains no spending authority (§12.9) (ZIP 374, “Prover”).

Construction 12.19 (Proof creation from a PCZT). For an Orchard-protocol bundle with no Actions the Prover does nothing. Otherwise, for each Action i , it rebuilds the consumed note from its explicit data and the created note from its explicit data with $\rho_i^{\text{new}} = \text{nf}_i$; forms the primary input

$$(rt, cv_i^{\text{net}}, \text{nf}_i, rk_i, \text{cmx}_i, \text{enableSpend}, \text{enableOutputs}, \text{disableCrossAddress})$$

with rt the bundle’s anchor, and the auxiliary input from the witness, the two notes, α_i , the components of fvk_i and rcv_i (*Ironwood Guide*, §“The Action statement”); and fails if the pair does not satisfy the Action statement, in particular if fvk_i does not own the consumed note or cmx_i differs from the commitment of the rebuilt created note. It then creates one Halo 2 proof for all Actions of the bundle (*Ironwood Guide*, §“The Action circuit and the Halo 2 proof”; *Halo 2 Guide*, §“The Halo 2 proof system”) and sets zkproof. The Ironwood bundle is proved in the same way with its own anchor. The primary input consists of effecting data and the anchor only.

An Action whose rk is the identity point $\mathcal{O}$ is invalid (protocol specification, §“Action Descriptions”; ZIP 256, “Orchard identity rk ”), and the Transaction Extractor MUST refuse it (ZIP 374, “Transaction Extractor”). A Prover that refuses it as well spares a proof of a transaction that can-

not be valid; ZIP 374 does not state that check for the Prover, and it is designed but unspecified (Definition 1.1).

12.9 Combining, redaction, and privacy

Construction 12.20 (Binary merge). The *merge* $P \sqcup Q$ of two PCZTs is defined, or fails, as follows (ZIP 374, “Combiner”).

- (1) *Global*. The fields `tx_version`, `version_group_id`, `consensus_branch_id`, `fallback_lock_time`, `expiry_height` and `coin_type` MUST be equal. The field `tx_modifiable` merges bitwise, bits 0, 1 and 7 by AND and bit 2 by OR, bits 3 to 6 being required zero on both sides.
- (2) *Lists of inputs, outputs, spends and Actions*. If either side’s bundle is IO-finalised, its `bsk` being present, both sides have equally many entries and equal `value_sum`. Otherwise the entries at the positions common to both lists merge pairwise, and the extra entries of the longer list are appended provided the shorter side’s `tx_modifiable` permits additions of that kind, the merge failing otherwise. The merged shielded bundle’s `value_sum` is that of the side each of whose lists of the bundle is at least as long as the other side’s, the merge failing if neither side is; with lists of equal lengths the two MUST be equal. ZIP 374 leaves this unstated, so it is designed but unspecified (Definition 1.1).
- (3) *Required fields* of merged entries, and the bundle flags, `note_version` and, in version 1, `anchor`, MUST be equal.
- (4) *Optional fields* merge as $x \sqcup \perp = \perp \sqcup x = x$, and $x \sqcup y = x$ if $x = y$, failing otherwise; in version 2 this rule covers the anchors.
- (5) *Maps* merge by union of their keys, the values of a common key being required equal.

The Combiner of k PCZTs merges them into one that contains every field of each, or fails. It MUST NOT combine PCZTs that do not represent the same transaction, that is, PCZTs not derived by roles from a common ancestor, which rules (1) to (3) check structurally. Once the input and output set is final, the effecting data determine the transaction identifier, which then identifies the PCZT; for version 6, re-anchored copies keep that identifier.

BIP 174 lets a combiner keep one of two conflicting values and discard the other. The PCZT merge requires equality instead, which is always possible because every field is typed and known in advance, so every conflict fails at the Combiner and none diverges later (ZIP 374, “Combiner rejecting non-identical duplicates”).

Lemma 12.21 (Commutativity of the merge). *For PCZTs P and Q , the merge $P \sqcup Q$ is defined if and only if $Q \sqcup P$ is, and then the two are equal.*

Proof. By cases on the rules of Construction 12.20. The equality tests of rules (1) and (3) are symmetric; AND and OR are commutative; the rule for optional fields and the union of maps with

equal values on common keys are symmetric; the condition on IO-finalised bundles is symmetric; in rule (2) the longer list and the side whose permission is consulted are determined by the lengths, not by the order of the arguments, and so is the merged value_sum, which is required equal at equal lengths; and the pairwise merges of common entries commute by induction on the structure of the entries. $\square$

ZIP 374 requires further that combining SHOULD be functionally order-independent, $\text{Combine}(f_A(p), f_B(p)) = f_A(f_B(p)) = f_B(f_A(p))$ for participants that apply f_A and f_B to p . That is a requirement on roles, not a property proved here for all of them; Proposition 12.22 proves it for the Prover and the Signer.

Proposition 12.22 (Order independence of proving and signing). *Let p be an IO-finalised PCZT on which the Prover (Construction 12.19) and a Signer (§12.7) are both defined, the anchors being set for proving and, for a transaction of version 5, fixed before signing (Definition 12.12). Fix the random coins of both roles. Then $\text{Prove}(\text{Sign}(p))$ and $\text{Sign}(\text{Prove}(p))$ are defined and both equal $\text{Prove}(p) \sqcup \text{Sign}(p)$: the two roles run in either order, or in parallel on copies whose outputs are merged by Construction 12.20. For a transaction of version 6 signing may moreover precede the setting of the anchors (Proposition 12.13). If a copy is redacted before either role, and retains the fields that role reads, the equality holds up to the redacted fields, which the merge restores from the other side.*

Proof. Signing. The signed digest is computed from effecting data only, excluding proofs and signatures (Lemma 11.21), and the relations of Definition 12.16 read effecting and context data; Prove writes neither, so Sign reads the same values on p and on $\text{Prove}(p)$ and, with its coins fixed, writes the same signatures and bits. *Proving.* The primary input of the Action statement consists of effecting data and the anchor, and the auxiliary input of context data (Construction 12.19; *Ironwood Guide*, §“The Action statement”); Sign writes none of them, and the Prover reads no spending key, so Prove writes the same proofs on p and on $\text{Sign}(p)$. *Merge.* The role Prove sets only proof fields, absent in p , and Sign only signatures, absent in p , and bits of tx_modifiable, which it only clears, or sets in the case of bit 2; the bundle of p is finalised on both sides with equal counts and value sums. Neither role writes a required field, so rule (3) holds, and a transparent signature is a new key of partial_signatures, merged by rule (5). Rules (1) to (5) of Construction 12.20 therefore give $\text{Prove}(p) \sqcup \text{Sign}(p) = \text{Prove}(\text{Sign}(p)) = \text{Sign}(\text{Prove}(p))$. ZIP 374, “Prover”, states the same independence. $\square$

Redactor (ZIP 374, “Redactor” and the field descriptions). Anyone may clear fields that later roles do not need: per Orchard-protocol Action the spend’s spend_auth_sig, recipient, value, ρ , rseed, fvk, witness, α , derivation record, dummy key and proprietary entries, the output’s recipient, value, rseed, ock, derivation record, user_address and proprietary entries, and rcv; the corresponding Sapling fields; and the transparent and Global lookup data. A PCZT with inputs of several independent Signers is sent to each with only what that Signer needs, for example without the other Signers’ values of α .

Proposition 12.23 (Disclosure by a PCZT). *A PCZT carries strictly more than its transaction: for each shielded spend or output it may carry the counterparty address, the value, the note randomness, rcv , which opens cv^{net} , the full viewing key of the spending account, derivation records, ock , which opens C_{out} , and address strings. Every recipient of a PCZT at a stage learns all data present at that stage; a delegated Prover learns the note data of the spends and outputs but gains no spending authority; the extracted transaction contains none of these data. Participants SHOULD send a PCZT only to entities needed for the remaining roles and SHOULD redact what those do not need, and wallets SHOULD protect stored PCZTs as they protect their note records (ZIP 374, “Privacy Implications”).*

Proof. The listed fields are context data (Definition 12.3), which extraction omits (§12.10); a recipient reads every field present. The Prover’s data (Construction 12.19) contain fvk but no ask , and an accepted spend of a note of the key without a signature under $ask + \alpha$ on a digest that the key holder signed is excluded, except with negligible probability, by *Ironwood Guide*, §“Authorisation of spends”, Theorem “Spend authority”, under its assumptions, whose adversary holds the full viewing key. $\square$

12.10 Spend finalisation and extraction

Spend Finalizer (ZIP 374, “Spend Finalizer” and “TransparentInput”; BIP 11; BIP 16). For each transparent input with enough data, the Spend Finalizer builds `script_sig`: for P2PKH, the push of the one signature, with its hash-type byte, under the key that hashes to the address, followed by the push of that key; for P2SH multisignature, the opcode `OP_0`, the pushes of m signatures in the key order of the redeem script and the push of the redeem script, failing with fewer than m signatures. It MUST fail on a signature whose hash type is not the input’s `sighash_type`. It SHOULD then clear the input’s redeem script, partial signatures, derivation records and preimages. It keeps value and `script_pubkey`, required fields that the digest needs, and sequence and the lock-time requirements, which the digest and Definition 12.24 read at extraction; ZIP 374’s “all other optional data” would include these last fields, a gap in the specification.

Definition 12.24 (Lock-time determination). If no transparent input has a required time or height lock, the lock time is `fallback_lock_time`, or 0 if it is absent. Otherwise the lock type is one supported by every input that states a requirement, an input that states both types supporting both; an input that states neither supports both. If both types remain possible, the height type is chosen; if none remains, the requirements are unsatisfiable. The lock time is the maximum required value of the chosen type (ZIP 374, “Determining Lock Time”).

Construction 12.25 (Transaction extraction). The Transaction Extractor checks, failing on any violation (ZIP 374, “Transaction Extractor”), that

- (i) `tx_version`, `version_group_id` and `consensus_branch_id` name a supported format, and the lock-time requirements are satisfiable (Definition 12.24);
- (ii) for `tx_version = 5` the Ironwood bundle is canonical empty;

- (iii) every `script_sig` is set;
- (iv) every Sapling spend has its proof and signature, and every Sapling output its proof;
- (v) every Orchard-protocol bundle with $n \geq 1$ Actions has a zkproof of the canonical length of $2720 + 2272 n$ bytes (*Ironwood Guide*, §“The transaction format”), and every Action a `spend_auth_sig`;
- (vi) every non-empty shielded bundle has its anchor and `bsk`;
- (vii) no `rk` is $\mathcal{O}$.

It then sets each bundle’s balancing value to v^{balance} , failing if it lies outside the signed 64-bit range of the transaction field; sets the lock time by Definition 12.24; computes `SigHash`; for each bundle derives `bvk` from its value commitments and balancing value, fails unless it matches `bsk`, and signs `SigHash` under `bsk`, by `RedPallas` for the Orchard and Ironwood bundles and by `RedJubjub` for Sapling (Lemma 12.18); and outputs the transaction, without any context data (ZIP 229). It SHOULD then verify every proof and signature of that transaction, each `spend_auth_sig` under its `rk` on `SigHash` included, so that an invalid transaction is found before submission.

A party that holds an Orchard-protocol output’s recipient, value and `rseed`, with ρ^{new} the nf of its Action’s spend, recovers the full note plaintext, memo included, without `ock` or any viewing key, by the derivation of `esk` stated with Theorem 11.22 (§11.8; *Ironwood Guide*, §“Encryption to the recipient”, Proposition “Decryption correctness”; ZIP 212). The random C_{enc} of a fabricated output decrypts to nothing, as the construction intends.

13 The transaction lifecycle

This section states what a wallet does with a transaction between its creation and a terminal state: the encoding it submits and the identifier by which it tracks the transaction; the rule that a transaction is recorded before it is released, with the completeness of the record that follows; submission and rebroadcast; the status observations that update the record; and the terminal states, re-sending and the consensus branch. A transaction is *created* when it is built, proved and authorised from one step of a plan (Definition 11.2; Constructions 11.18 and 11.20). The record is that of Definition 9.11.

13.1 Submission and the transaction identifier

The wallet serialises a created transaction in the encoding of its version: version 5 (ZIP 225, “Transaction Format”) or version 6 (ZIP 229, “Transaction Format”), the only versions in force; a transaction with Ironwood-pool Actions has version 6 (*Ironwood Guide*, §“The transaction format”, Construction “Version-6 transaction”; protocol specification, §“Transaction Encoding and Consensus”). The argument of `Submit` (Definition 5.10) is this byte string. The encoding is cited, not restated.

The wallet tracks a transaction t by its *transaction identifier* $\text{txid}(t)$: the root of the BLAKE2b-256 digest tree of ZIP 244 over the effecting data of t ,

$$\text{txid}(t) = h(\text{ZcashTxHash_} \parallel \beta, d_{\text{hdr}} \parallel d_{\text{tr}} \parallel d_{\text{Sap}} \parallel d_{\text{Orc}} \parallel d_{\text{lrw}}),$$

where $h(P, x)$ is BLAKE2b-256 of x under the 16-byte personalisation P , β is the 4-byte little-endian consensus branch identifier, and the child d_{lrw} is absent in version 5 (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Transaction identifier”, for version 6, and Remark “Version 5”; ZIP 244, “txid_digest”; ZIP 229, “Transaction Identifiers, Auth Digests, and Signature Digests”; protocol specification, §“Transaction Identifiers”). The header digest d_{hdr} hashes, under ZTxIdHeadersHash , the 4-byte little-endian encodings of header, nVersionGroupId , $\text{nConsensusBranchId}$, lock_time and nExpiryHeight (ZIP 244, “T.1: header_digest”), so the expiry height N of Definition 9.8 is effecting data. The wallet computes $\text{txid}(t)$ from t and needs no response of the server to know it.

Lemma 13.1 (Non-malleability of the identifier). *Let t be a transaction of version 5 or 6.*

- (i) *Replacing the authorising data of t , namely its proofs, signatures and transparent scripts and, in version 6, its anchors (anchorSapling, anchorOrchard and anchorIronwood, ZIP 229, “Anchor commitment (version 6)”;* *Ironwood Guide*, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data”), leaves $\text{txid}(t)$ unchanged.
- (ii) *Under the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b” (§“Transaction digests and signatures”), no efficient algorithm outputs, except with negligible probability, two transactions with different effecting data and equal identifiers.*
- (iii) *Under the same assumption, a transaction built for one consensus branch and the same payment built for another have different identifiers, except with negligible probability.*

Proof. (i) Authorising data lie under the authorising-data digest and under no node of the tree whose root is txid (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Authorising-data digest; effecting and authorising data”). (ii) Each node of the tree hashes an injective encoding (fixed-size fields and length-prefixed scripts, concatenated in order) of effecting data or of the digests of its children (ZIP 244, “txid_digest”). If the effecting data differ, the inputs of some node differ. A node whose inputs differ either has equal digests, which is a collision, or passes different digests to its parent, whose inputs then differ; at the root, equal identifiers from different inputs are a collision. (iii) The branch identifier enters both the root personalisation $\text{ZcashTxHash_} \parallel \beta$ and the header digest through $\text{nConsensusBranchId}$, so the two root pairs differ, and equal identifiers are a collision (*Ironwood Guide*, §“Transaction digests and signatures”, Remark “Consensus-branch binding”). $\square$

By (i) and (ii), the identifier tracks a pending transaction from its creation on, before and after any re-signing or re-proving. The identifier of versions up to 4, the double SHA-256 of the encoding (protocol specification, §“Transaction Identifiers”), is malleable; it belongs to no version in force and is not used.

Construction 13.2 (Recording and submission). The following rules are designed but unspecified (Definition 1.1).

- (L1) *Recording before release.* The wallet enters each transaction t it creates into its transaction record (Definition 9.11), which locks the notes t spends (Definition 9.12), in storage that survives restarts and rollbacks (Definition 7.23, item (v)), before it submits t or releases its bytes to any other party. It never removes an entry of the record.
- (L2) *Multi-step plans.* For a plan of $\ell > 1$ steps (Definitions 11.2 and 11.3), the wallet creates every step before it records any, records all ℓ transactions in one update of the record, and submits them after that update, in step order. A construction that fails at some step leaves no recorded, hence by (L1) no transmittable, prefix. Every step can be created before any is mined, since a well-formed plan chains steps only through transparent outputs (Proposition 11.4); the transfer to a TEX address of Construction 11.7 is the instance (ZIP 320, “Design considerations for Senders”).
- (L3) *Submission.* The wallet submits t by `Submit(bytes)`, whose response is `accepted(txid)` or `rejected` (Definition 5.10). Neither response changes the record. It submits at a tip T that leaves a margin of blocks, a wallet parameter, before the last includable height N (Definition 9.8), rather than at $T = N - 1$.
- (L4) *Rebroadcast.* While t is pending and unmined (Definition 13.8), the wallet resubmits it at intervals whenever, at its tip T , $N = 0$ or $T < N$ (Lemma 9.9) and no transaction included in its view of the chain spends an input of t . For $N = 0$ this continues until t is mined or an input of t is spent by a mined transaction. The rule is motivated by ZIP 203, “Specification”, under which nodes drop a transaction from their mempools at its expiry.

The semantics of the two responses of `Submit` are those of the server’s node and are fixed by no ZIP: `accepted` means that the node admitted t to its mempool, `rejected` that it refused it. Under clause (a) of Assumption 5.2, acceptance implies neither propagation to other nodes nor inclusion in a block, since relay policy is local to each node and may refuse a transaction that consensus still admits in the next block (Lemma 9.9); and rejection does not imply that t is invalid. Rule (L3) keeps a margin for this reason. Each submission is an observation of the server (Proposition 8.21).

Proposition 13.3 (Completeness of the transaction record). *Let the wallet follow rule (L1), and let no party other than the wallet hold the bytes of a created transaction t before the wallet submits them. Then, if a block of any chain includes a transaction with the effecting data of t , the record contains t , and the included transaction has identifier $\text{txid}(t)$, except with negligible probability. The converse fails: a recorded transaction that the network never received is stale, not lost, and, for $N \neq 0$, reaches the terminal state expired unmined of Definition 13.8; for $N = 0$ it stays pending by item (c) until the end of its consensus branch is known, and then reaches expired unmined with the bound N^* (“Consensus-branch bound” below).*

Proof. A block includes a transaction only if the party that assembled the block held its bytes.

The first party other than the wallet to hold bytes of t received them by the wallet’s submission, by hypothesis; by (L1) the wallet recorded t before that event, and the entry survives restarts and rollbacks and is never removed. A valid transaction with the effecting data of t carries, for each spend of t other than a dummy, a signature under a key of the wallet on the signature digest those data fix; except with negligible probability no party obtains such a signature except from bytes the wallet released (*Ironwood Guide*, §“RedPallas unforgeability”, Proposition “Unforgeability under re-randomisation”, for Orchard-protocol spends; for transparent and Sapling spends the unforgeability of those schemes is assumed, as in §11.8, where no lower volume proves it). Bytes derived from those of t by another party with the effecting data of t differ from t at most in authorising data, so the included transaction has identifier $\text{txid}(t)$ by part (i) of the lemma above. The converse fails because recording does not imply delivery. $\square$

Where a partially created transaction is extracted by another party (“Spend finalisation and extraction”, §12.10), that party holds the bytes of t before any submission, and the hypothesis is arranged differently: the wallet records $\text{txid}(t)$, N and the inputs of t , locking them, before it releases the partially created transaction to any party that may extract it, and at a point where the effecting data of t are final. After IO finalisation (Construction 12.14) of a partially created transaction with shielded spends or outputs, or, for one that is transparent-only, once signatures without SIGHASH_ANYONECANPAY and without SIGHASH_NONE have cleared bits 0 and 1 of tx_modifiable (ZIP 374, “Global”), the inputs and outputs are final, the lock time is determined by Definition 12.24, and the proofs and signatures still to be added are authorising data; by part (i) of the lemma the extracted transaction then has the recorded identifier, and the proof above applies with release in place of submission. The entry acquires the encoding of t when the wallet obtains it.

Remark 13.4 (Retained data). From creation until it is mined, t exists only in the wallet’s record, in remote mempool and, for an extracted partially created transaction, at the extracting party. Once the wallet holds the encoding of t (Definition 9.11, item (i)) the record keeps it, and rule (L4) needs it; before that the wallet does not rebroadcast t . It may hold data that the chain never will: a transaction that ends expired and unmined lies on no chain that agrees with the server’s up to N (Lemma 13.7), and its entry may be the only trace of it.

13.2 Status tracking and terminal states

Remark 13.5 (Scope of scanning). Scanning detects a transaction only through a nullifier of the watched set $\mathcal{W}$ (Definition 9.15) or an output accepted under the wallet’s scanning keys (Construction 6.8). A recorded transaction that spends no note of $\mathcal{W}$ and has no such output, every fully transparent transaction among them, is invisible to scanning. Its fate is learnt only by queries $\text{Status}(\text{txid})$ (Definition 5.9), each of which puts its identifier into the view of the server (Proposition 8.18).

For a recorded transaction t write N for its expiry height, h_t for its mined height ($\perp$ while unmined), h_t^{obs} for its earliest observation height, and u_t for the latest tip at which the server reported it unmined ($\perp$ initially): items (ii), (iv), (v) and (vi) of Definition 9.11. A *status observation* (s, T) is a response s of $\text{Status}(\text{txid}(t))$ received while the wallet’s view of the chain has tip T .

Construction 13.6 (Status transitions). The record entry of t changes only as follows.

- (i) On an observation (s, T) with $s \in \{\text{unknown}, \text{unmined}\}$: $u_t := T$.
- (ii) On an observation $(\text{mined}(b'), T)$: $h_t := b'$, $h_t^{\text{obs}} := \min(h_t^{\text{obs}}, b')$ and $u_t := \perp$.
- (iii) Scanning a block of height b' in which t is wallet-relevant (Construction 6.8) has the effect of (ii).
- (iv) A rollback to r (Definition 7.23) sets $h_t := \perp$ if $h_t > r$, and $u_t := \min(u_t, r)$ if $u_t \neq \perp$, since a report at a tip above r concerned blocks that the rollback discards.

Query termination. The wallet issues $\text{Status}(\text{txid}(t))$ for a pending transaction t with $h_t = \perp$: if $N \neq 0$, until $u_t \geq N$, that is, until t is reported unmined at a tip of at least N ; if $N = 0$, for as long as t is pending and unmined. No rule stops at a first report of unmined or at a fixed depth after first observation. A rollback that lowers u_t below N , or sets $h_t := \perp$, resumes the queries. The transitions and the termination rule are designed but unspecified (Definition 1.1); the termination rule rests on the consensus rule of Definition 9.8 (ZIP 203, “Specification”). For an entry with N_t unknown (Definition 9.11), the wallet obtains the encoding by $\text{Tx}(\text{txid})$ (Definition 5.10); until then it queries as for $N = 0$, does not rebroadcast, and its inputs are locked by condition (d) of Definition 9.12.

Lemma 13.7 (Expired unmined is final on its chain). *Assume clause (a) of Assumption 5.2. Let t be a transaction created by the wallet, with expiry height $N \neq 0$, and let (s, T) be an observation with $s \in \{\text{unknown}, \text{unmined}\}$ and $T \geq N$, answered from a best chain C of the server (Consensus Guide, §“Work, and the best chain”) whose blocks at heights at most N are those of the wallet’s view. Then no chain whose blocks at heights at most N are those of C includes t .*

Proof. The wallet’s view has tip $T \geq N$, so C has blocks at every height up to N . By clause (a), s is the status of $\text{txid}(t)$ on C (Definition 5.9): no block of C contains t , so no block of height at most N of a chain that agrees with C up to N does. The transaction t is not a coinbase transaction, and by Lemma 9.9 the expiry rule permits it in no block of height greater than N (protocol specification, §“Transaction Consensus Rules”; ZIP 203, “Specification”). $\square$

A reorganisation with fork point below N reopens the question, since the new chain need not agree with C up to N . The wallet meets it as a continuity error (Definition 6.10), and the rollback of Construction 7.24 to some $r < N$ sets $u_t \leq r < N$ by transition (iv), which resumes the queries.

Let B be the birthday of the wallet (Definition 6.2) and $\mathcal{H}$ its retained block boundaries (Definition 7.12). Write $r_{\min}$ for the least element of $\mathcal{H} \setminus \{B - 1\}$, the lowest rollback point of the rollback window, and $r_{\min} := B - 1$ when that set is empty.

Definition 13.8 (Terminal states). A recorded transaction t is *pending* from its recording until it reaches one of the following states.

- (a) *Mined*: $h_t \neq \perp$ and $h_t \leq r_{\min}$, so that no rollback to a point of the window unmines t . The state is terminal relative to the rollback window only, since consensus bounds no

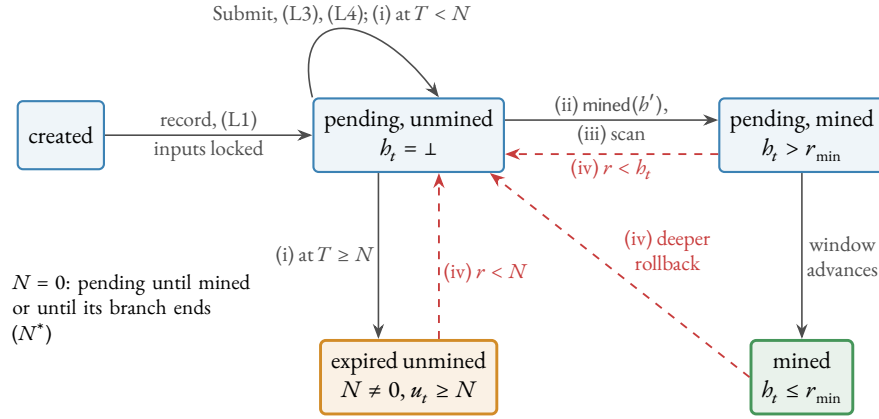


Figure 6. The states of a recorded transaction t (Definition 13.8). Solid edges are transitions of the record: recording by rule (L1), which locks the inputs of t ; submission, rebroadcast and reports unmined or unknown at a tip below N , which leave t pending; a report mined(b') or the scan of a block containing t , transitions (ii) and (iii); burial below the lowest rollback point $r_{\min}$ of the window; and, for $N \neq 0$, a report of an unmined t at a tip $T \geq N$, after which Lemma 13.7 excludes t from every chain that agrees with the server's up to N . Dashed red edges are rollbacks, transition (iv): below b_t from either mined state, from the terminal one only by a rollback deeper than the window, and below N from expired unmined. The green and amber states are terminal; a transaction with $N = 0$ stays pending until mined, unless its consensus branch ends.

reorganisation depth (Remark 7.28); a deeper rollback, to $B - 1$ or to a fetched chain state in steps (1) and (3) of Construction 7.24, returns t to pending by transition (iv).

- (b) *Expired unmined*: $N \neq 0$, $h_t = 1$ and $u_t \geq N$, which gives the conclusion of Lemma 13.7 when the server's chain agrees with the wallet's view up to N ; a disagreement surfaces as a continuity error (Definition 6.10), whose rollback to $r < N$ returns t to pending by transition (iv). Its inputs are released (Proposition 9.13, every later target height exceeding N), its entry is kept, and the user is notified.
- (c) For $N = 0$, a value chosen explicitly: t remains pending until it is mined, rebroadcast by rule (L4), or, once the end of its consensus branch is known, until it reaches state (b) with the bound N^* of that branch ("Consensus-branch bound" below). Its mining is detected by scanning through W , from which a spend, $N = 0$ included, removes a nullifier only once mined (Definition 9.15), or by continued status queries.

The states and their transitions are designed but unspecified (Definition 1.1). The notification is specified: the wallet should notify the user of expired transactions that must be re-sent, and user interfaces and services must never rely on zero-confirmation transactions (ZIP 203, "Wallet behavior and UI").

Remark 13.9 (Re-sending). Re-sending an expired payment usually spends the same TXOs, and ZIP 315 lists which spends of given TXOs are repeated across transactions among the kinds of information leakage, the case arising when a previous transaction expired (ZIP 315, "Kinds of information leakage"): the repeated spends link the expired transaction to its replacement. Selecting

other notes avoids that link, but the nullifiers that the expired transaction revealed still link it to any later transaction that spends those notes (ZIP 315, “Rationale for anchor selection”). A wallet that re-sends accepts one of the two links. ZIP 315, “Note selection”, records as a TODO, not as a rule, that such revealed nullifiers be spent in a note-management transaction; how to avoid both links is an open problem (Definition 1.1).

Consensus-branch bound. The consensus branch bounds a pending transaction as well as its expiry height does. An authorisation binds the consensus branch identifier through β and `nConsensusBranchId`, which must equal the identifier of the branch under which t is validated, so a transaction authorised for one branch is invalid under another whatever its expiry height (*Ironwood Guide*, §“Transaction digests and signatures”, Remark “Consensus-branch binding”, under its Assumption “Collision resistance of BLAKE2b”; ZIP 244, “txid_digest” and “T.1: header_digest”; ZIP 259, “Backward compatibility”; protocol specification, §“Transaction Consensus Rules”). A pending transaction built for a branch that ends at a height X , with $N = 0$ or $X \leq N$, can be mined only below X . Once X is known, write $N^* := X - 1$ if $N = 0$ or $N \geq X$, and $N^* := N$ otherwise. Rule (L4), the query termination of Construction “Status transitions”, state (b) of Definition 13.8 and conditions (b) and (c) of Definition 9.12 apply with N^* in place of N , and Lemma 13.7 holds with N^* , by the branch binding in place of the expiry rule. Thus t becomes expired unmined from a report at a tip of at least N^* , and its inputs are released for the rebuild at every target height $H \geq X$. This rule is designed but unspecified (Definition 1.1). To pay across the activation of the next branch, the wallet rebuilds and re-authorises the payment for the branch in force at its target height (Definition 9.4): a new transaction, with a new identifier by part (iii) of the lemma of §13.1 and a new entry of the record.

14 Payment requests (ZIP 321)

This section states the encoding by which a payee states payments to a payer. It gives the URI grammar of ZIP 321, the grouping of parameters into payments and the conditions under which a parser accepts a URI; then the meaning of the address, amount, memo, label and message parameters, the capacity of one request, and the passage from an accepted request to a transaction plan.

14.1 The URI grammar

A payment request precedes planning (§11.1): the payee states to the payer, in a form that a wallet reads mechanically, the recipient addresses, amounts and memos of the payments it asks for, and ZIP 321 fixes this encoding as a URI scheme. Its terms *payment* and *payment request* (ZIP 321, “Terminology”) are those of Definition 11.1. Everything that follows once the recipients are fixed is constructed in the earlier sections of this volume and in the *Ironwood Guide*, §“Note encryption”, §“Trial decryption and note acceptance”, §“Authentication paths”, §“Anchors” and §“Transaction digests and signatures”; this section re-derives none of it. The status of ZIP 321 is that of Table 1.

Definition 14.1 (Payment request URI). Let L be the set of strings derived from `zcashuri` in the following grammar in ABNF (RFC 5234), reproduced from ZIP 321, “URI Syntax”, with its longest rule continued on an indented line:

```

zcashurn      = "zcash:" ( zcashaddress [ "?" zcashparams ]
                        / "?" zcashparams )
zcashaddress  = 1*( ALPHA / DIGIT )
zcashparams   = zcashparam [ "&" zcashparams ]
zcashparam    = [ addrparam / amountparam / assetparam / memoparam
                  / messageparam / labelparam / otherreqparam / otherparam ]
NONZERO       = %x31-39
DIGIT         = %x30-39
paramindex    = "." NONZERO 0*3DIGIT
addrparam     = "address" [ paramindex ] "=" zcashaddress
amountparam   = "amount" [ paramindex ] "=" 1*DIGIT [ "." 1*8DIGIT ]
assetparam    = "req-asset" [ paramindex ] "=" *base64url
labelparam    = "label" [ paramindex ] "=" *qchar
memoparam     = "memo" [ paramindex ] "=" *base64url
messageparam  = "message" [ paramindex ] "=" *qchar
paramname     = ALPHA *( ALPHA / DIGIT / "+" / "-" )
otherreqparam = "req-" paramname [ paramindex ] [ "=" *qchar ]
otherparam    = paramname [ paramindex ] [ "=" *qchar ]
qchar         = unreserved / pct-encoded / allowed-delims / ":" / "@"
allowed-delims = "!" / "$" / "'" / "(" / ")" / "*" / "+" / "," / ";"

```

Here ALPHA, unreserved and pct-encoded are those of RFC 3986, Appendix A, and base64url is the alphabet of RFC 4648, Section 5, in which the values 62 and 63 are written - and _, with padding omitted. The grammar is read as follows: a parameter whose name is address, amount, label, memo, message or req-asset is derived only by its own production, and otherreqparam and otherparam derive only parameters with other names. A *payment request URI* is a string of L that meets the acceptance conditions (a) to (f) stated below in this subsection.

Without the reading, otherparam would derive amount=1%30, with name amount, no index and a qchar value; ZIP 321, “Invalid Examples”, rejects a URI with this parameter.

Surface forms. The grammar derives no // after the scheme, so a payment request URI has no authority component and is not hierarchical in the sense of RFC 3986, Section 1.2.3; ZIP 321, “Invalid Examples”, rejects zcash:// followed by an address. A URI zcash: $A?P$, with A a zcashaddress, MUST be treated as zcash:?address= $A&P$ (ZIP 321, “URI Semantics”). A request for a single payment SHOULD place its address in the hier-part; several payments are written with addrparam at distinct indices (same section). By this equivalence and condition (b) below, a URI zcash: $A?address=B\&...$ is not accepted: after the rewriting it carries address twice at the empty index.

Indices and grouping. Identify the empty index with 0 and let $J := \{0, 1, \dots, 9999\}$. The *rendering* ι maps 0 to the empty string and $i \geq 1$ to $\cdot \parallel \text{dec}(i)$, where $\text{dec}(i)$ is the decimal representation of i without leading zeros. The strings that match [paramindex] are the empty string and the strings $\cdot d_1 \dots d_k$ with $1 \leq k \leq 4$, $d_1 \in \{1, \dots, 9\}$ and $d_2, \dots, d_k$ decimal digits; these are exactly the decimal representations without leading zeros of the integers 1 to 9999. Hence ι is a bijection from J onto the index suffixes, and its inverse is parsing. The strings $\cdot 0$, $\cdot 01$ and $\cdot 10000$ match no production, and ZIP 321, “Invalid Examples”, rejects address. $\cdot 0=$ and amount. $\cdot 0=$. Parameters whose suffixes denote the same index $i \in J$, the empty index included, belong to one payment P_i ; the order of the parameters is insignificant, and indices need not be consecutive (ZIP 321, “URI Semantics”).

A URI therefore decodes to a finite map $i \mapsto P_i$ on a subset of J , and carries at most $|J| = 10,000$ payments: 9999 at indices 1 to 9999 and one at the empty index.

Lexical rules. Percent encoding (RFC 3986, Section 2.1) occurs only inside `qchar`, that is, only in the values of `label`, `message`, `otherreqparam` and `otherparam`. Addresses, amounts, indices and parameter names are literal, so `amount=1%30`, `%61mount=1` and an address beginning with `%74` are not in L (ZIP 321, “URI Syntax” and “Invalid Examples”). The values of `memo` and `req-asset` are `base64url` without padding, and implementations **MUST NOT** accept the characters `+`, `/` and `=`, which occur only in the standard base64 alphabet (ZIP 321, “URI Syntax”; RFC 4648, Section 5).

Acceptance. A parser accepts a string u as a payment request URI only if all of the following hold (ZIP 321, “URI Syntax”, “URI Semantics” and “Forward compatibility”):

- (a) $u \in L$ under the reading of Definition 14.1; a purported ZIP 321 URI that cannot be so parsed **MUST NOT** be accepted;
- (b) after the hier-part address is rewritten as address at the empty index, no pair of a parameter name and an index occurs twice (**MUST NOT**);
- (c) every index that carries a parameter other than address carries an address (**MUST**);
- (d) u names no parameter with prefix `req-` that the parser does not recognise, since such a parameter makes the entire URI invalid (**MUST**); other unrecognised parameters are ignored (**SHOULD**);
- (e) the address, amount and memo rules of §14.2 hold;
- (f) u decodes to at least one payment, a payment request containing one or more payments (ZIP 321, “Terminology”).

The prefix `req-` is part of the name of a parameter defined by a later revision of ZIP 321, not a modifier of an arbitrary parameter; the parameters `address`, `amount`, `label`, `memo` and `message` never carry it, since every conformant parser is required to understand them (ZIP 321, “Forward compatibility”). An accepted URI asks for one transaction: implementations **SHOULD** construct a single transaction that pays every address of the request (ZIP 321, “URI Semantics”).

Custom assets. The parameter `req-asset` is specified in ZIP 321, “Custom Assets”, with an encoding defined by ZIP 227, which the protocol described in this volume does not include. A parser of this protocol does not recognise `req-asset`, so by condition (d) a URI that carries it is invalid to that parser, and the further rule that `amount` and `req-asset` exclude each other at one index (ZIP 321, “URI Semantics”) never applies.

Remark 14.2 (Case of literals). Quoted strings in ABNF are case-insensitive (RFC 5234, Section 2.3), so L contains strings beginning `ZCASH:` and strings with the parameters `AMOUNT=1` and `REQ-ASSET=`; the value ranges `NONZERO` and `DIGIT`, given as `%x` ranges, and the `base64url` alphabet are case-sensitive. ZIP 321 does not address case: it states neither whether such URIs are to be accepted nor whether `amount` and `AMOUNT` at one index are the same parameter for condition (b). The volume states the grammar as written and resolves neither question; no parser’s behaviour is normative. Class: open problem (Definition 1.1).

14.2 Parameter semantics

Addresses. Implementations SHOULD check that each `zcashaddress` is a valid encoding of an address other than a Sprout address under protocol specification, §“Encodings of Addresses and Keys”: a transparent address (§“Transparent Addresses”, Base58Check), a Sapling payment address (§“Sapling Payment Addresses”, Bech32, ZIP 173) or a unified address (§“Unified Payment Addresses and Viewing Keys” and ZIP 316, Bech32m, BIP 350). Address formats later added to that section SHOULD be supported; where the network of the request is known, each address SHOULD be checked to belong to it; and every requirement of ZIP 316 applies to a unified address (ZIP 321, “URI Semantics”). Sprout addresses MUST NOT be supported (same section). ZIP 321 gives as its reason the restriction of ZIP 211 on adding value to the Sprout pool; in the protocol described, version-4 transactions are invalid, and Sprout transfers exist only in transactions of versions 2 to 4, so Sprout funds are unspendable (ZIP 2003, “Abstract”; ZIP 259, “Abstract”; *Ironwood Guide*, §“The transaction format”). A TEX address (ZIP 320) is in no section of the specification, and ZIP 321 names none; its reading as the recipient address of item 4 of Definition 3.20 is designed but unspecified (Definition 1.1). The addresses of an accepted request are thus recipient addresses in the sense of Definition 3.20.

Construction 14.3 (Amount codec). Let $\text{COIN} = 10^8$ zatoshi per ZEC (*Ironwood Guide*, Definition “Unit of value” in §“Notes”) and $\text{MAX_MONEY} = 2.1 \cdot 10^{15}$ zatoshi (*Ironwood Guide*, §“The transaction format”), so that 21,000,000 ZEC is MAX_MONEY zatoshi. For a string x of decimal digits, let $\text{int}(x)$ be the natural number it denotes, leading zeros ignored, and let $\emptyset^k$ be the string of k characters $\emptyset$.

- (1) *Parse.* Let s match `1*DIGIT [ "." 1*8DIGIT ]`, and write $s = c$ or $s = c \parallel . \parallel f$ with c its whole part and f its fraction. Set $z := \text{int}(f \parallel \emptyset^{8-|f|})$, or $z := 0$ without a fraction, and

$$v := \text{int}(c) \cdot \text{COIN} + z.$$

Accept s if and only if $v \leq \text{MAX_MONEY}$; an accepted s denotes v zatoshi, and $\text{parse}(s) := v$.

- (2) *Render.* For $v \in \{0, \dots, \text{MAX_MONEY}\}$ write $v = c \cdot \text{COIN} + z$ with $0 \leq z < \text{COIN}$. Let $\text{render}(v)$ be $\text{dec}(c)$ if $z = 0$, and otherwise $\text{dec}(c) \parallel . \parallel f$, where f is the 8-digit zero-padded decimal representation of z with its trailing characters $\emptyset$ removed.

Leading zeros of the whole part and trailing zeros of the fraction are ignored, the bound MAX_MONEY is ZIP 321’s “MUST NOT be greater than 21000000 ZEC”, and a string with a comma or other grouping separator, with an empty whole part (`.5`), an empty fraction (`50.`) or more than 8 fraction digits matches no production (ZIP 321, “ZEC Transfer Amount”). ZIP 321 fixes only the value that an amount denotes; the rendering is canonical, not required.

Lemma 14.4 (Round trip of amounts). *For every integer v with $0 \leq v \leq \text{MAX_MONEY}$, the string $\text{render}(v)$ matches `1*DIGIT [ "." 1*8DIGIT ]` and $\text{parse}(\text{render}(v)) = v$. The map parse is not injective. For every accepted s , $\text{render}(\text{parse}(s))$ is the unique string of value $\text{parse}(s)$ whose whole part has no leading zero unless it is the single character $\emptyset$, whose fraction has no trailing*

| *zero, and which has no fraction when $\text{parse}(s)$ is a multiple of COIN.*

Proof. Write $v = c \cdot \text{COIN} + z$ with $0 \leq z < \text{COIN}$. The string $\text{dec}(c)$ is non-empty. If $z = 0$, then $\text{render}(v) = \text{dec}(c)$ matches the production, and parse returns $c \cdot \text{COIN} = v$. If $z \neq 0$, the 8-digit form of z has a non-zero digit, so removing its trailing zeros leaves a fraction f with $1 \leq |f| \leq 8$, and the production matches. Appending $8 - |f|$ zeros to f restores the 8-digit form of z , so parse returns $c \cdot \text{COIN} + z = v$, and accepts since $v \leq \text{MAX_MONEY}$. The strings 50, 050 and 50.00 all denote 5,000,000,000 zatoshi, so parse is not injective. Finally, an accepted s determines $(\text{int}(c), z)$, which are the quotient and remainder of $\text{parse}(s)$ by COIN since $z < \text{COIN}$. A string of the stated form is fixed by these two numbers: its whole part is $\text{dec}(\text{int}(c))$, and its fraction is absent if $z = 0$ and otherwise the 8-digit form of z without trailing zeros. That string is $\text{render}(\text{parse}(s))$. $\square$

Examples. On the examples of ZIP 321, “ZEC Transfer Amount” and “Examples”, Construction 14.3 gives the following. The parameter $\text{amount}=1$ denotes 100,000,000 zatoshi, $\text{amount}=123.456$ denotes 12,345,600,000 zatoshi, and $\text{amount}.1=0.789$ denotes 78,900,000 zatoshi for the payment at index 1. The values 50, 050 and 50.00 denote 5,000,000,000 zatoshi, and 0.5 and 00.500 denote 50,000,000 zatoshi. The value 21000000 denotes $\text{MAX_MONEY} = 2,100,000,000,000,000$ zatoshi and is accepted, while 21000000.00000001 exceeds MAX_MONEY and is rejected. The values 50,000.00, 50,00, 50., .5 and 0.123456789 are invalid. The parameters address.0= and amount.0= are invalid, their index having a leading zero, and so is any parameter with the five-digit suffix .10000.

Absent amount. ZIP 321 gives a payment a value only “if an amount parameter is provided” (ZIP 321, “ZEC Transfer Amount”). A payment without one has no value under ZIP 321, while Definition 11.1 requires a value, so the payer fixes the value before a plan is formed. Class: designed but unspecified (Definition 1.1).

Construction 14.5 (Memo codec). A *memo* is a 512-byte string M , the memo field of a note plaintext (*Ironwood Guide*, §“The note plaintext”; protocol specification, §“Encodings of Note Plaintexts and Memo Fields”). Let b64u be the encoding of RFC 4648, Section 5, with padding omitted, and let $\text{strip}(M)$ be M with its trailing $0x00$ bytes removed.

- (1) *Encode.* $\text{Enc}(M) := \text{b64u}(\text{strip}(M))$, a string of at most $\lceil 512 \cdot 8/6 \rceil = 683$ characters.
- (2) *Decode.* Reject a value that contains a character outside the base64ur1 alphabet, in particular $+$, $/$ or $=$, or whose length is congruent to 1 modulo 4, a length to which no octet string encodes. Otherwise let b be its decoding under RFC 4648, Section 5, and require $|b| \leq 512$ (MUST); then $\text{Dec}(\text{value}) := b \parallel 0x00^{512-|b|}$ (ZIP 321, “Query Keys”).

A payment without a *memo* parameter has no memo (Definition 11.1); its output, if shielded, carries the memo field that ZIP 302 designates as supplying no memo, $0xF6 \parallel 0x00^{511}$, whose encoding is 9g (ZIP 302, “Specification”). A parameter memo= with the empty value differs from an absent parameter: it decodes to $0x00^{512}$, which a reader under ZIP 302 reads as the empty text. If the address at the index of a *memo* parameter does not permit memos, that is, is not *memo-capable* in the sense of predicate (i) of Definition 3.20, as for a transparent or TEX address, parsers MUST consider the entire URI invalid (ZIP 321, “Query Keys”), since a memo travels only in a note plaintext.

| **Lemma 14.6** (Round trip of memos). *For every memo M , $\text{Dec}(\text{Enc}(M)) = M$.*

Proof. The string $\text{Enc}(M)$ uses only the base64url alphabet, and an unpadded encoding of n octets has length $\lceil 8n/6 \rceil$, never congruent to 1 modulo 4, so decoding does not reject it. The encoding b64u is injective with inverse the decoding of RFC 4648, so $b = \text{strip}(M)$, and $|b| \leq 512$. The bytes that strip removes are zeros, so appending $512 - |b|$ zeros to b returns M . $\square$

ZIP 321 does not say whether a value whose final character carries non-zero unused bits (RFC 4648, Section 3.5) is accepted; the encoding Enc produces none. Stripping trailing zeros gives the shortest encoding and is not required: $\text{b64u}(M)$ decodes to the same memo. As an instance, ZIP 321’s example value `VGhpcyBpcyBhIHNPbXBsZSBtZW1vLg`, of 30 characters, decodes to the 22 bytes of the text “This is a simple memo.”, padded with zeros to 512 bytes.

Labels and messages. The label at an index names its address, for example by the name of the recipient; a client that renders the payment for inspection SHOULD display the label together with the address, and a displayed label MUST be identifiable as distinct from the address. The message at an index is descriptive text that a client may display for that payment (ZIP 321, “Query Keys”). Both values are qchar strings, percent-decoded to octet strings; an octet outside qchar reaches them only by percent encoding. Neither enters the transaction (Definition 11.1).

Capacity. A request carries at most 10,000 payments (§14.1), and the one transaction it asks for is bounded further by consensus; the bound that binds is the least of the following.

- (i) *Size.* A block has at most 2,000,000 bytes (protocol specification, §“Block Header Encoding and Consensus”), and a Sapling output contributes at least 948 bytes to a transaction, so a transaction has at most $\lfloor 2,000,000/948 \rfloor = 2109$ Sapling outputs (protocol specification, §“Balance and Binding Signature (Sapling)”). This is the bound that ZIP 321, “URI Semantics”, states.
- (ii) *Shielded budget.* ZIP 218, part of NU7, requires of every block, with S the number of Sapling spends plus outputs, A_O and A_I the numbers of Orchard-pool and Ironwood-pool Actions, and n_{JS} the number of JoinSplit descriptions, each summed over the block’s transactions,

$$\begin{aligned} S \leq 300, \quad A_O \leq 330, \quad A_I \leq 330, \quad n_{JS} \leq 0, \\ A_O + A_I + S + 2 n_{JS} \leq 330 \end{aligned}$$

(ZIP 218, “Shielded pool action limits”); versions 5 and 6 carry no JoinSplit description in any case (*Ironwood Guide*, §“The transaction format”). Each of these sums over a block is at least the term of any one of its transactions. A transaction that pays n payments by Sapling outputs has $o \geq n$ Sapling outputs; with s Sapling spends and a Actions of the two pools it is mined only if $s + o \leq 300$ and $a + s + o \leq 330$, hence only if

$$n \leq o \leq \min(300, 330 - a) - s.$$

The all-Sapling bound is therefore $\min(2109, 300) = 300$ outputs, fewer with spends or Actions, and the bound of (i) never binds; one transaction carries at most 330 units of the shared budget.

- (iii) *Orchard pool*. No payment output lies in the Orchard pool (Construction 11.5, Proposition 11.6), so the Orchard-pool Actions of the transaction only consume the shared budget of (ii).
- (iv) *Ironwood pool*. An Action creates one note, so a transaction that pays n payments by Ironwood-pool outputs has $a_1 \geq n$ Ironwood-pool Actions; with a_0 Orchard-pool Actions, s Sapling spends and o Sapling outputs it is mined, by (ii), only if

$$n \leq a_1 \leq 330 - a_0 - s - o.$$

An Action contributes at least $820 + 2272 + 64 = 3156$ bytes to a transaction: its description, its share of the proof and its spend authorisation signature (ZIP 229, “Transaction Format”). The size limit alone admits $\lfloor 2,000,000/3156 \rfloor = 633$ Actions, so the shared budget binds, at 330 payments by Ironwood-pool outputs.

- (v) *Transparent outputs*. ZIP 218 does not apply its limits to the transparent components of transactions (same section), so payments by transparent outputs are bounded by the size limit alone.

From request to plan. Each payment $P_i = (a, v, m)$ of an accepted request, its value fixed by the payer if absent, is a payment in the sense of Definition 11.1 and becomes one output of the plan: its pool is that of Construction 11.5, a TEX address is paid by Construction 11.7, and its memo, if any, is the memo field of that output’s note plaintext (Construction 14.5). A TEX payment funded from shielded notes needs the two transactions of Construction 11.7, so the MUST of ZIP 320, “Design considerations for Senders”, and not the SHOULD of ZIP 321 that asks for a single transaction, fixes the shape of such a plan. From there construction is that of §11.1 and Constructions 11.18 and 11.20: an Orchard-protocol output is sealed as in the *Ironwood Guide*, §“Encryption to the recipient”, a Sapling output as in the protocol specification, §“Sending Notes (Sapling)”, and the *Ironwood Guide*, §“Transaction digests and signatures”, gives the digest that every signature signs.